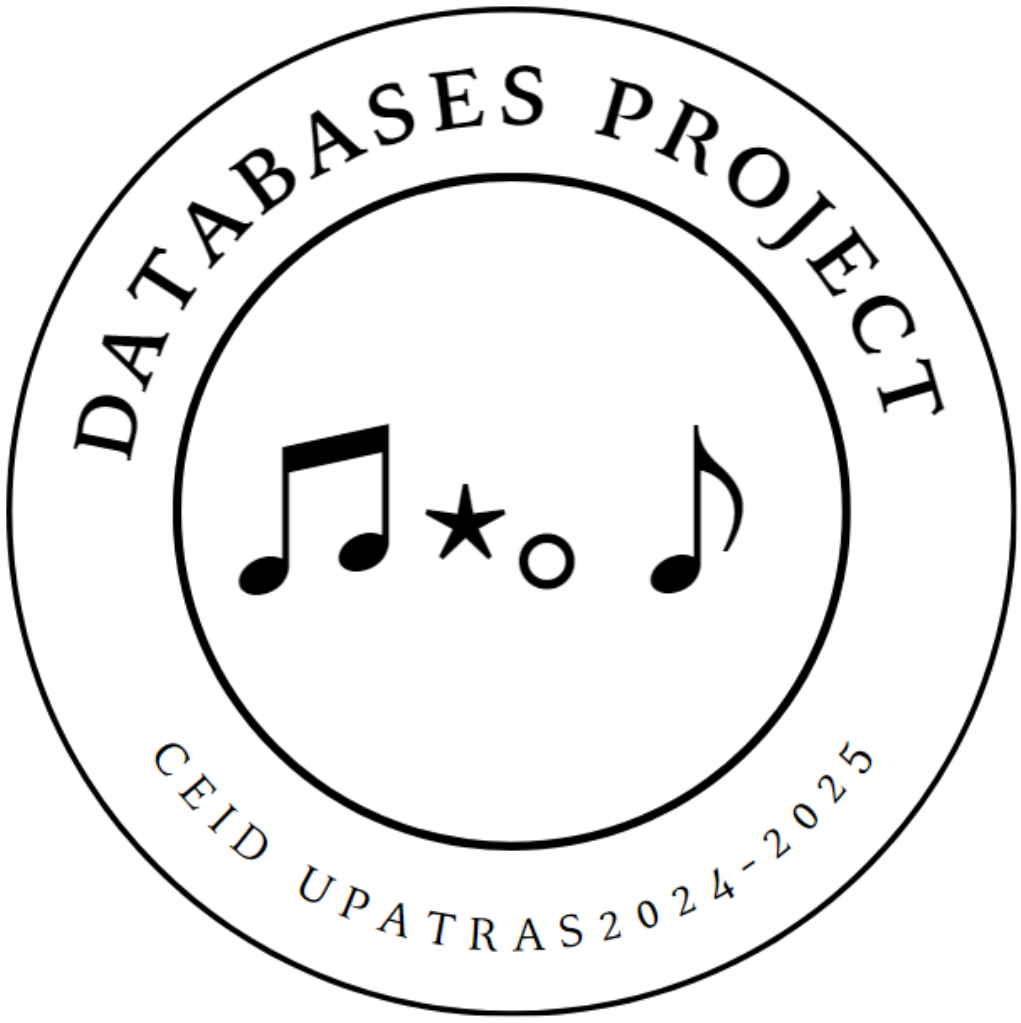
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ , ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ



Εργαστήριο Βάσεων Δεδομένων

Χειμερινό Εξάμηνο 2024-2025

Εργαστηριακή Άσκηση

Υπεύθυνοι Καθηγητές : Β. Μεγαλοοικονόμου , Ε. Βογιατζάκη , Ι .Βασιλόπουλος

ΑΜ	**Ονοματεπώνυμο**	**Έτος σπουδών**
1084963	Τσάλα Ζαφειρία	3ο (έτος εισαγωγής 2022)

Πάτρα, Σεπτέμβριος 2025

Περιεχόμενα αναφοράς

Μέρος Α' – SQL 4

Κεφάλαιο 1: Σχεδιασμός ΒΔ 4

3.1.1. Προπαρασκευαστική Φάση - Δημιουργία της ΒΔ 4

Επιπλέον πίνακες 6

UPDATE-DELETE-ALTER Παραδείγματα 27

Κεφάλαιο 2 – Stored Procedures 32

Απαίτηση 3.1.3.1 - Υπολογισμός Βαθμού Venue 32

Απαίτηση 3.1.3.2 – Προγραμματισμός Συναυλιών 34

Απαίτηση 3.1.3.3 - Υπολογισμός Βαθμού Venue 39

Απαίτηση 3.1.3.4 – Ευρετήρια (Indexes) 42

Κεφάλαιο 3 - Triggers 45

Απαίτηση 3.1.4.1 45

Απαίτηση 3.1.4.2 48

Απαίτηση 3.1.4.3 49

Μέρος Β' – Java GUI 51

1. **DatabaseUI()** 51
2. **SetupGUI()** 51
3. **connectToDB()** 55
4. **setSessionDBA()** 56
5. **authenticateDBA ()** 56
6. **changeDBA()** 58
7. **manageDBAUsers()** 58
8. **showVenuesWithScores()** 62

Εμφάνιση πινάκων με συσχετίσεις 65

9. **showBandmembers()** 65
10. **showArtistCompanyRelations()** 66
11. **showAlbumArtistRelations ()** 68
12. **showAlbumReleases()** 70
13. **showAlbumTracks ()** 70
14. **showProducerCompanyRelations ()** 71

Συναρτήσεις που αφορούν τους τύπους δεδομένων : 73

15. **getEnumValues(String tableName, String columnName)** 73
16. **isDateColumn(String tableName, String columnName)** 74
17. **isAutoIncrement(String tableName, String columnName)** 74

Συναρτήσεις που εμφανίζουν τους πίνακες της ΒΔ : 75

18. **loadTableData(String tableName)** 75
19. **extractTableName(String Selection)** 75

Συναρτήσεις insert , update , delete 76

20. **insertRow()** 76
21. **updateRow()** 81
22. **deleteRow()** 85

Μέρος Γ' – Παράρτημα 87

Συγκεντρωτικός κώδικας μέρους Α' 87

Συγκεντρωτικός κώδικας μέρους Β' 97

Βάση Δεδομένων για το GUI 97

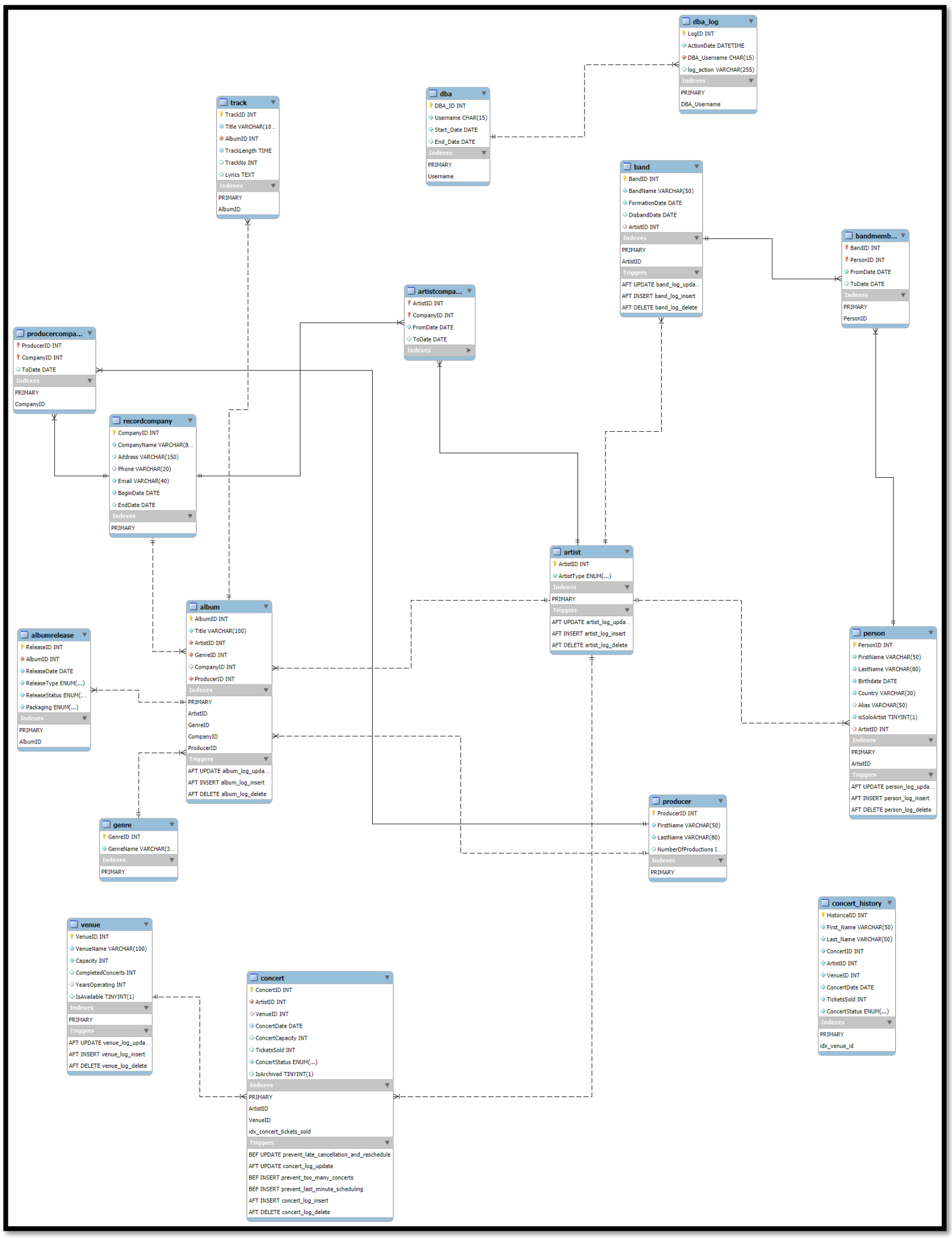
Κώδικας Java 108

Μέρος Α' – SQL

Κεφάλαιο 1: Σχεδιασμός ΒΔ

3.1.1. Προπαρασκευαστική Φάση - Δημιουργία της ΒΔ

Έχοντας πάντα ως γνώμονα το ER της προπαρασκευαστικής φάσης που έχει δοθεί στο E-Class, δημιουργήσαμε την Βάση Δεδομένων (εφ'εξής συμβ. ΒΔ) με όνομα *project*2024 . Παρακάτω φαίνεται το τελικό EER διάγραμμά του project2024 , ακριβώς όπως έγινε export μέσα από το περιβάλλον του MySQL Workbench.



Φιγούρα 1 : EER Διάγραμμα ΒΔ project2024

Στην αρχική φάση υλοποίησης, δημιουργήθηκαν οι πίνακες που είχαν οριστεί κατά την προπαρασκευαστική διαδικασία , με τους οποίους μάλιστα δουλεύαμε σε κάθε εργαστηριακή άσκηση κατά το χειμερινό εξάμηνο.

Σε αρκετά από τα πεδία των πινάκων πραγματοποιήθηκαν τροποποιήσεις όσον αφορά τον τρόπο εισαγωγής των δεδομένων, και έτσι ορίστηκαν ως *NOT NULL*. Η επιλογή αυτή έγινε με σκοπό να αποφευχθεί η καταχώριση κενών τιμών σε πεδία που θεωρούνται κρίσιμα για τη λειτουργία της βάσης. Για παράδειγμα, δεν είναι επιτρεπτό να εισαχθούν κενές τιμές στα πεδία FirstName και LastName ενός καλλιτέχνη, καθώς αποτελούν βασικές και απαραίτητες πληροφορίες σε ένα τέτοιο σενάριο χρήσης της ΒΔ.

Επιπλέον, χρησιμοποιήθηκε η εντολή *ON DELETE CASCADE ON UPDATE CASCADE*, ώστε να διασφαλίζεται η ακεραιότητα των δεδομένων. Με αυτόν τον τρόπο, όταν διαγράφεται μια εγγραφή, διαγράφονται αυτόματα και οι σχετικές εγγραφές από τους πίνακες που συνδέονται με αυτήν μέσω FOREIGN KEY. Αντίστοιχα, σε περίπτωση ενημέρωσης μιας τιμής-κλειδιού, η αλλαγή μεταφέρεται αυτόματα και στους πίνακες που την χρησιμοποιούν. Έτσι, η ΒΔ μας είναι πάντα up-to-date μετά από κάθε αλλαγή, δίχως να χρειάζεται να επέμβει ο χρήστης (ή ας πούμε ο DBA) και να αλλάξει αρκετά πεδία manually. Παρακάτω φαίνεται αναλυτικά και ο κώδικας που αφορά το κομμάτι της προπαρασκευαστικής διαδικασίας:

```
DROP DATABASE IF EXISTS project2024;
CREATE DATABASE project2024;
USE project2024;

-- Table: artist
CREATE TABLE artist (
    ArtistID INT AUTO_INCREMENT PRIMARY KEY,
    ArtistType ENUM('PERSON','BAND','ORCHESTRA','CHOIR','VIRTUAL','OTHER') NOT NULL
);

-- Table: person
CREATE TABLE person (
    PersonID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(80) NOT NULL,
    Birthdate DATE NOT NULL,
    Country VARCHAR(30) NOT NULL,
    Alias VARCHAR(50),
    isSoloArtist TINYINT(1) NOT NULL,
    ArtistID INT,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: band
CREATE TABLE band (
    BandID INT AUTO_INCREMENT PRIMARY KEY,
    BandName VARCHAR(50) NOT NULL,
    FormationDate DATE NOT NULL,
    DisbandDate DATE,
    ArtistID INT,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: bandmember
CREATE TABLE bandmember (
    BandID INT,
    PersonID INT,
    FromDate DATE NOT NULL,
    ToDate DATE,
    PRIMARY KEY (BandID , PersonID),
    FOREIGN KEY (BandID)
        REFERENCES band (BandID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (PersonID)
        REFERENCES person (PersonID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: recordcompany
CREATE TABLE recordcompany (
    CompanyID INT AUTO_INCREMENT PRIMARY KEY,
    CompanyName VARCHAR(80) NOT NULL,
    Address VARCHAR(150),
    Phone VARCHAR(20),
    Email VARCHAR(40) NOT NULL,
    BeginDate DATE NOT NULL,
    EndDate DATE
);

-- Table: artistcompany
-- This table right here explains to which record company an artist is assigned
CREATE TABLE artistcompany (
    ArtistID INT NOT NULL,
    CompanyID INT NOT NULL,
    FromDate DATE NOT NULL,
```

```
    ToDate DATE,
    PRIMARY KEY (ArtistID , CompanyID),
    FOREIGN KEY (ArtistID) REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CompanyID) REFERENCES recordcompany (CompanyID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: producer
CREATE TABLE producer (
    ProducerID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(80) NOT NULL,
    NumberOfProductions INT
);

-- Table: producercompany
-- This table right here explains to which record company a producer is assigned
CREATE TABLE producercompany (
    ProducerID INT NOT NULL,
    CompanyID INT NOT NULL,
    ToDate DATE,
    PRIMARY KEY (ProducerID , CompanyID),
    FOREIGN KEY (ProducerID)
        REFERENCES producer (ProducerID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CompanyID)
        REFERENCES recordcompany (CompanyID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: genre
CREATE TABLE genre (
    GenreID INT AUTO_INCREMENT PRIMARY KEY,
    GenreName VARCHAR(30) NOT NULL
);

-- Table: album
CREATE TABLE album (
    AlbumID INT AUTO_INCREMENT PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    ArtistID INT NOT NULL,
    GenreID INT NOT NULL,
    CompanyID INT NULL,
    ProducerID INT NOT NULL,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (GenreID)
        REFERENCES genre (GenreID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CompanyID)
        REFERENCES recordcompany (CompanyID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (ProducerID)
        REFERENCES producer (ProducerID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: track
CREATE TABLE track (
    TrackID INT AUTO_INCREMENT PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    AlbumID INT NOT NULL,
    TrackLength TIME NOT NULL,
    TrackNo INT,
    Lyrics TEXT,
    FOREIGN KEY (AlbumID)
        REFERENCES album (AlbumID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: albumrelease
CREATE TABLE albumrelease (
    ReleaseID INT AUTO_INCREMENT PRIMARY KEY,
    AlbumID INT NOT NULL,
    ReleaseDate DATE NOT NULL,
    ReleaseType ENUM('LP', 'CD', 'MP3') NOT NULL,
    ReleaseStatus ENUM('OFFICIAL', 'PROMOTION', 'BOOTLEG', 'WITHDRAWN', 'CANCELLED') NOT NULL,
    Packaging ENUM('BOOK', 'CARDBOARD SLEEVE', 'DIGIPAK', 'JEWEL CASE', 'NOT APPLICABLE') NOT NULL,
    FOREIGN KEY (AlbumID)
        REFERENCES album (AlbumID)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

Επιπλέον πίνακες

Πέρα από τους βασικούς πίνακες, δημιουργήθηκαν και ορισμένοι επιπλέον, με σκοπό να υποστηρίξουν την υλοποίηση των ερωτημάτων (Stored Procedures, Triggers) που απαιτούνται για την εργαστηριακή άσκηση.

Πιο αναλυτικά προστέθηκαν:

- Ο πίνακας **venue**, όπου αποθηκεύονται όλα τα στοιχεία που αφορούν τον χώρο διεξαγωγής συναυλιών. Αυτά είναι:
    - το όνομα του χώρου (VenueName) ,
    - το ID του χώρου (VenueID) που αρχικά είναι AUTO INCREMENT για να αποφεύγονται λάθη, αλλά και PRIMARY KEY του πίνακα μιας και είναι το πιο σημαντικό χαρακτηριστικό του
    - η χωρητικότητα του (Capacity) που προφανώς δεν μπορεί να είναι NULL ,
    - τον αριθμό των συναυλιών που έχουν πραγματοποιηθεί σε αυτόν τον χώρο (πεδίο πλήρως απαραίτητο και αναγκαίο για την απαίτηση 3.1.2.2 και την 3.1.3.1, η οποία μάλιστα αφορά τον υπολογισμό της βαθμολογίας του χώρου σύμφωνα με τον πίνακα 2 της εκφώνησης) ,
    - το πόσα χρόνια λειτουργεί (YearsOperating) που μπορεί να είναι και NULL και τέλος
    - το αν είναι διαθέσιμος (IsAvailable) , ώστε να ξέρουμε αν μπορεί να γίνει χρήση ή όχι από κάποιον καλλιτέχνη. Το IsAvailable λειτουργεί σαν μια validator μεταβλητή που παίρνει τις τιμές 1 και 0 , όπου το 1 είναι TRUE («ΝΑΙ ,ο χώρος είναι available») και το 0 FALSE («ΟΧΙ , ο χώρος δεν είναι available»).

- Ο πίνακας **concert** που αφορά τις συναυλίες που προγραμματίζονται. Σε αυτόν τον πίνακα, έχουν προστεθεί τα παρακάτω πεδία , σύμφωνα με τις απαιτήσεις της άσκησης:
    - Το ID της συναυλίας (ConcertID) που είναι AUTO INCREMENT και PRIMARY KEY , μιας και είναι το βασικό αναγνωριστικό του πίνακα και το πιο σημαντικό χαρακτηριστικό του,
    - Το ID του καλλιτέχνη (ArtistID) που πραγματοποιεί την συναυλία. Αυτό το πεδίο έχει οριστεί ως FOREIGN KEY μιας και παραπέμπει στον πίνακα 'artist' και ξέρουμε εύκολα σε ποιον αναφερόμαστε,
    - Το ID του χώρου διεξαγωγής (VenueID) , το οποίο είναι και αυτό FOREIGN KEY για τον ίδιο λόγο που αναφέρθηκε στην προηγούμενη σειρά,
    - Η χωρητικότητα (ConcertCapacity) της συναυλίας , πεδίο αναγκαίο για την εύρεση του επιθυμητού χώρου διεξαγωγής σε επόμενο ερώτημα,
    - Η ημερομηνία διεξαγωγής (ConcertDate) , το οποίο προσωπικά επιλέχθηκε να είναι NOT NULL ακόμα και όταν μιλάμε για μια ακυρωμένη συναυλία (ακόμα και η ακυρωμένη κάποτε ήταν 'SCHEDULED' για συγκεκριμένη ημερομηνία),
    - Το πόσα εισιτήρια πωλήθηκαν (TicketsSold) (και αυτό αναγκαίο για τα επόμενα ερωτήματα) , το οποίο έχει και default τιμή 0 αν δεν ξέρουμε πόσα sales υπήρχαν,
    - Η κατάσταση της συναυλίας (ConcertStatus) που παίρνει 3 τιμές (SCHEDULED, CANCELLED, COMPLETED) και οπωσδήποτε NOT NULL . Το status είναι χρήσιμο σε επόμενο ερώτημα για να ξέρουμε αν μπορούμε να προγραμματίσουμε κι άλλες συναυλίες για τον καλλιτέχνη κλπ.
    - Το IsArchived που λειτουργεί σαν validator μεταβλητή (αντίστοιχο με το IsAvailable του πίνακα venue) που χρησιμοποιείται για να εμφανίζουμε το πλήρες ιστορικό των συναυλιών (απαίτηση 3.1.2.2 , 2η κουκκίδα: *Ο συναυλίες οι οποίες έχουν ολοκληρωθεί ή ακυρωθεί χωρίς δυνατότητα επεναπρογραμματισμού τους, διαγράφονται από τον πίνακα που περιέχει τις προγραμματισμένες συναυλίες και μεταφέρονται στο ιστορικό συναυλιών (Απαίτηση 3.1.2.3). Η κατάστασή τους στο ιστορικό θα πρέπει να μεταφραπεί από προγραμματισμένη ή ακυρωμένη σε ολοκληρωμένη. Αν η αίτηση ήταν ακυρωμένη θα καταχωρίζεται ως ημερομηνία διεξαγωγής NULL, αλλιώς θα καταχωρίζεται η ημερομηνία διεξαγωγής της.)*

- Ο πίνακας **concerthistory** που περιέχει όλα τα πεδία για την εμφάνιση του πλήρους ιστορικού συναυλιών. Ο πίνακας κατά βάση είναι ίδιος με τον πίνακα **concert**, ωστόσο τώρα κοιτάμε μόνο τα COMPLETED και τα CANCELLED concerts (ConcertStatus)

- Ο πίνακας **dba** που περιέχει τα εξής πεδία:
    - Το ID του Διαχειριστή της ΒΔ (DBA_ID) που είναι AUTO INCREMENT για να αποφευχθούν λάθη, και PRIMARY KEY μιας και είναι το βασικό χαρακτηριστικό του πίνακα,
    - το Username του εκάστοτε Διαχειριστή (Username) που το έχουμε ορίσει UNIQUE , ώστε να μην δημιουργούνται χρήστες με ίδιο username και υπάρχει σύγχυση (ακόμα και σε real life εφαρμογές , δεν γίνεται να έχουμε το ίδιο username/email με κάποιον άλλον),
    - η ημερομηνία έναρξης ως Διαχειριστής ΒΔ (Start_Date) που είναι οπωσδήποτε NOT NULL,
    - η ημερομηνία λήξης συνεργασίας του DBA (End_Date) που προφανώς παίρνει τιμή NULL αν ο DBA είναι ακόμα ενεργός
- Ο πίνακας **dba_log** που έχει τα εξής :

- Το ID του Log (LogID) που είναι PRIMARY KEY του πίνακα,
- η ημερομηνία που έγινε κάποια αλλαγή/εισαγωγή (ActionDate) που είναι της μορφής DATETIME και το DEFAULT είναι η συνάρτηση NOW() , ώστε να κρατάει real-time τις αλλαγές ο πίνακας ,
- το Username αυτού που έκανε την αλλαγή (DBA_Username),
- το log_action που είναι ένα πεδίο στο οποίο περιγράφεται ουσιαστικά το τι αλλαγή έγινε ακριβώς και σε ποιον πίνακα (απαίτηση 3.1.4.1)

Αξίζει να αναφερθεί πως στο Β' Μέρος της Εργαστηριακής άσκησης έχουν προστεθεί μερικά πεδία ακόμα στον πίνακα **dba** , τα οποία είναι πιο applicable σε real life scenarios (π.χ. είναι χρήσιμο να κρατάμε το ονοματεπώνυμο , το mail κλπ.)

Παρακάτω εμφανίζεται ο κώδικας που χρησιμοποιήθηκε για την δημιουργία αυτών των πινάκων:

```
CREATE TABLE venue (
    VenueID INT AUTO_INCREMENT PRIMARY KEY,
    VenueName VARCHAR(100) NOT NULL,
    Capacity INT NOT NULL,
    CompletedConcerts INT,
    YearsOperating INT,
    IsAvailable TINYINT(1) DEFAULT 1
);

-- Table: concert
CREATE TABLE concert (
    ConcertID INT AUTO_INCREMENT PRIMARY KEY,
    ArtistID INT NOT NULL,
    VenueID INT NOT NULL,
    ConcertDate DATE NOT NULL,
    ConcertCapacity INT,
    TicketsSold INT DEFAULT 0,
    ConcertStatus ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED') NOT NULL,
    IsArchived TINYINT(1) DEFAULT 1,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (VenueID)
        REFERENCES venue (VenueID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: concert_history
CREATE TABLE concert_history (
    HistoricalID INT AUTO_INCREMENT PRIMARY KEY,
    ConcertID INT NOT NULL,
    ArtistID INT NOT NULL,
    VenueID INT NOT NULL,
    ConcertDate DATE NOT NULL,
    TicketsSold INT NOT NULL,
    ConcertStatus ENUM('COMPLETED', 'CANCELLED') NOT NULL,
);

-- Table: dba
-- This table is used to keep track of who has access to the DB and manages tables and records
CREATE TABLE dba (
    DBA_ID INT AUTO_INCREMENT PRIMARY KEY,
    Username CHAR(15) UNIQUE NOT NULL,
    Start_Date DATE NOT NULL,
    End_Date DATE DEFAULT NULL
);

-- Table: dba_log
-- This is used to keep a log of all the actions made on the DB
-- We keep a track of the logged in user and log exactly what they change , where they make that change and the time they do.
-- (afto to erwthma einai akoma kalutero sto Java implementation)
CREATE TABLE dba_log (
    LogID INT AUTO_INCREMENT PRIMARY KEY,
    ActionDate DATETIME NOT NULL DEFAULT NOW(),
    DBA_Username CHAR(15) NOT NULL,
    log_action VARCHAR(255),
    FOREIGN KEY (DBA_Username)
        REFERENCES dba (Username)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

Πριν προχωρήσουμε στην εισαγωγή δεδομένων στους πίνακες, είναι σημαντικό να σημειωθεί ότι οποιαδήποτε τροποποίηση της βάσης δεδομένων πρέπει να γίνεται από χρήστη με δικαιώματα διαχειριστή (DBA). Αυτό όπως έχουμε ήδη αναφέρει και παραπάνω, εξασφαλίζει την ορθή διαχείριση των δεδομένων και την ασφάλεια της βάσης. Συνεπώς, πριν από κάθε εισαγωγή δεδομένων, πρέπει να καταχωρούμε τον χρήστη DBA στον πίνακα **dba** και να τον ενεργοποιούμε ως τρέχοντα χρήστη.

Ο σχετικός κώδικας SQL είναι ο εξής:

```
INSERT INTO dba (DBA_ID, Username,Start_Date,End_Date) VALUES (1,'admin','2015-01-01' , NULL);
SET @current_dba := 'admin';
```

- Για τον πίνακα artist :

```
INSERT INTO artist (ArtistType) VALUES ('BAND');
INSERT INTO artist (ArtistType) VALUES ('BAND');
INSERT INTO artist (ArtistType) VALUES ('BAND');
INSERT INTO artist (ArtistType) VALUES ('BAND');
INSERT INTO artist (ArtistType) VALUES ('BAND');
INSERT INTO artist (ArtistType) VALUES ('BAND');
INSERT INTO artist (ArtistType) VALUES ('BAND');
INSERT INTO artist (ArtistType) VALUES ('BAND');
INSERT INTO artist (ArtistType) VALUES ('BAND');
INSERT INTO artist (ArtistType) VALUES ('BAND');
-- ================================================
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
INSERT INTO artist (ArtistType) VALUES ('PERSON');
-- ================================================
INSERT INTO artist (ArtistType) VALUES ('ORCHESTRA');
INSERT INTO artist (ArtistType) VALUES ('CHOIR');
INSERT INTO artist (ArtistType) VALUES ('VIRTUAL');
INSERT INTO artist (ArtistType) VALUES ('OTHER');
INSERT INTO artist (ArtistType) VALUES ('ORCHESTRA');
INSERT INTO artist (ArtistType) VALUES ('CHOIR');
INSERT INTO artist (ArtistType) VALUES ('VIRTUAL');
INSERT INTO artist (ArtistType) VALUES ('OTHER');
INSERT INTO artist (ArtistType) VALUES ('ORCHESTRA');
INSERT INTO artist (ArtistType) VALUES ('CHOIR');
INSERT INTO artist (ArtistType) VALUES ('VIRTUAL');
INSERT INTO artist (ArtistType) VALUES ('OTHER');
INSERT INTO artist (ArtistType) VALUES ('ORCHESTRA');
INSERT INTO artist (ArtistType) VALUES ('CHOIR');
INSERT INTO artist (ArtistType) VALUES ('VIRTUAL');
INSERT INTO artist (ArtistType) VALUES ('OTHER');
```

Με χρήση της εντολής *select * from artist*; έχουμε το παρακάτω αποτέλεσμα:

	ArtistID	ArtistType
▶	1	BAND
	2	BAND
	3	BAND
	4	BAND
	5	BAND
	6	BAND
	7	BAND
	8	BAND
	9	BAND
	10	BAND
	11	PERSON
	12	PERSON
	13	PERSON
	14	PERSON
	15	PERSON
	16	PERSON
	17	PERSON
	18	PERSON

Φιγούρα 2 : Περιεχόμενα artist

- Για τον πίνακα band:

```
      select * from dba;
      INSERT INTO dba (DBA_ID, Username,Start_Date,End_Date) VALUES (1,'admin','2015-01-01' , NULL);
      SET @current_dba := 'admin';


      INSERT INTO band (BandName, FormationDate, DisbandDate, ArtistID) VALUES ('Arctic Monkeys', '2002-01-01', NULL , 1);
      INSERT INTO band (BandName, FormationDate, DisbandDate, ArtistID) VALUES ('The Stone Roses' ,'1983-01-01' ,'2019-09-16',2);
      INSERT INTO band (BandName, FormationDate, DisbandDate, ArtistID)VALUES  ('The Smiths' ,'1982-01-01' , '1987-01-01', 3);
      INSERT INTO band (BandName, FormationDate, DisbandDate, ArtistID)VALUES  ('Radiohead', '1985-01-01', NULL, 4);
      INSERT INTO band (BandName, FormationDate, DisbandDate, ArtistID)VALUES  ('The Rolling Stones', '1962-07-12', NULL, 5);
      INSERT INTO band(BandName, FormationDate, DisbandDate, ArtistID) VALUES     ('Green Day', '1987-01-01', NULL, 6);
      INSERT INTO band (BandName, FormationDate, DisbandDate, ArtistID)VALUES  ('Pulp' ,'1978-01-01' , NULL, 7);
      INSERT INTO band (BandName, FormationDate, DisbandDate, ArtistID)VALUES  ('Suede' ,'1989-01-01' , NULL, 8);
      INSERT INTO band (BandName, FormationDate, DisbandDate, ArtistID)VALUES  ('ABBA' ,'1972-01-01' , '1982-01-01', 9);
      INSERT INTO band (BandName, FormationDate, DisbandDate, ArtistID)VALUES  ('The Kinks' ,'1963-01-01' , '1997-01-01', 10);


select * from band;
```

Με χρήση της εντολής *select * from band*; έχουμε το παρακάτω αποτέλεσμα:

Result Grid | Filter Rows: | Edit: | Export/Import:

BandID	BandName	FormationDate	DisbandDate	ArtistID
1	Arctic Monkeys	2002-01-01	NULL	1
2	The Stone Roses	1983-01-01	2019-09-16	2
3	The Smiths	1982-01-01	1987-01-01	3
4	Radiohead	1985-01-01	NULL	4
5	The Rolling Stones	1962-07-12	NULL	5
6	Green Day	1987-01-01	NULL	6
7	Pulp	1978-01-01	NULL	7
8	Suede	1989-01-01	NULL	8
9	ABBA	1972-01-01	1982-01-01	9
10	The Kinks	1963-01-01	1997-01-01	10
NULL	NULL	NULL	NULL	NULL

Φιγούρα 3 : Περιεχόμενα band

- Για τον πίνακα person που αφορά τα φυσικά πρόσωπα της μουσικής βιομηχανίας

```
SET @current_dba := 'admin'; -- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se pinakes

INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Alex David' , 'Turner' , '1986-
01-06' , 'United Kingdom' , NULL , 0 , 11);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Matthew' , 'Helders' , '1986-
05-07' , 'United Kingdom' , 'Agile Beast' , 0 , 12);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('John Martin' , 'Maher' , '1966-
10-31' , 'United Kingdom' , 'Johnny Marr' , 0 , 13);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Steven Patrick' , 'Morrissey' ,
'1959-05-22' , 'United Kingdom' , 'Morrissey' , 0 , 14);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Brett Lewis' , 'Anderson' ,
'1967-09-29' , 'United Kingdom' , NULL , 0 , 15);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Billie Joe' , 'Armstrong' ,
'1972-02-17' , 'USA' , NULL , 0 , 16);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Elizabeth' , 'Grant' , '1985-
06-21' , 'USA' , 'Lana Del Rey' , 1 , 17);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Agneta' , 'Fältskog' , '1950-
04-05' , 'Sweden' , 'Agnetha' , 0 , 18);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Peter' , 'Kinnes' , '1943-12-
31' , 'United Kingdom' , 'Pete Quaife' , 0 , 19);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('David' , 'Jones' , '1947-01-08'
, 'United Kingdom' , 'David Bowie' , 1 , 20);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Adrianne' , 'Lenker' , '1991-
07-09' , 'USA' , NULL , 1 , 21);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Miles' , 'Kane' , '1986-03-17'
, 'United Kingdom' , NULL , 1 , 22);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Ian' , 'Brown' , '1963-02-20' ,
'United Kingdom' , NULL , 0 , 23);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('John ' , 'Mellor' , '1952-08-
21' , 'United Kingdom' , 'Joe Strummer' , 0 , 24);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Damon' , 'Albarn' , '1968-03-
23' , 'United Kingdom' , NULL , 0 , 25);
INSERT INTO person (FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID)VALUES ('Madonna' , 'Turner' , '1958-08-
16' , 'USA' , 'Madonna' , 1 , 26);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Angela' , 'Trimble' , '1945-06-
01' , 'USA' , 'Debbie Harry' , 0 , 27);
INSERT INTO person (FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID)VALUES ('Nelly Kim' , 'Furtado' , '1978-
12-02' , 'Canada' , NULL , 1 , 28);
INSERT INTO person(FirstName, LastName, Birthdate, Country, Alias, isSoloArtist, ArtistID) VALUES ('Steve Thomas' , 'Lacy-Moya' ,
'1998-05-23' , 'USA' , 'Steve Lacy' , 1 , 29);


SELECT
    *
FROM
    person
```

Με χρήση της εντολής *select * from person*; έχουμε το παρακάτω αποτέλεσμα:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

PersonID	FirstName	LastName	Birthdate	Country	Alias	isSoloArtist	ArtistID
1	Alex David	Turner	1986-01-06	United Kingdom	NULL	0	11
2	Matthew	Helders	1986-05-07	United Kingdom	Agile Beast	0	12
3	John Martin	Maher	1966-10-31	United Kingdom	Johnny Marr	0	13
4	Steven Patrick	Morrissey	1959-05-22	United Kingdom	Morrissey	0	14
5	Brett Lewis	Anderson	1967-09-29	United Kingdom	NULL	0	15
6	Billie Joe	Armstrong	1972-02-17	USA	NULL	0	16
7	Elizabeth	Grant	1985-06-21	USA	Lana Del Rey	1	17
8	Agneta	Fältskog	1950-04-05	Sweden	Agnetha	0	18
9	Peter	Kinnes	1943-12-31	United Kingdom	Pete Quaife	0	19
10	David	Jones	1947-01-08	United Kingdom	David Bowie	1	20
11	Adrianne	Lenker	1991-07-09	USA	NULL	1	21
12	Miles	Kane	1986-03-17	United Kingdom	NULL	1	22
13	Ian	Brown	1963-02-20	United Kingdom	NULL	0	23

Φιγούρα 4 : Περιεχόμενα person

- Για τον πίνακα <u>bandmember</u> :

```
SET @current_dba := 'admin'; -- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se pinakes


INSERT INTO bandmember (BandID, PersonID, FromDate, ToDate) VALUES (1,1,'2002-01-01' , NULL); -- alex turner
INSERT INTO bandmember (BandID, PersonID, FromDate, ToDate) VALUES (1,2,'2002-01-01' , NULL); -- matt helders
INSERT INTO bandmember (BandID, PersonID, FromDate, ToDate) VALUES (3,3,'1982-01-01' , '1987-01-01'); -- johnny marr
INSERT INTO bandmember (BandID, PersonID, FromDate, ToDate) VALUES (3,4,'1982-01-01' , '1987-01-01'); -- mozz
INSERT INTO bandmember (BandID, PersonID, FromDate, ToDate) VALUES (8,5,'1989-01-01' , NULL); -- brett anderson
INSERT INTO bandmember (BandID, PersonID, FromDate, ToDate) VALUES (6,6,'1987-01-01' , NULL); -- Billie Joe

select * from bandmember;
```

Με χρήση της εντολής *select * from bandmember*; έχουμε το παρακάτω αποτέλεσμα:

Result Grid | Filter Rows: | Export:

BandID	PersonID	FromDate	ToDate
1	1	2002-01-01	NULL
1	2	2002-01-01	NULL
3	3	1982-01-01	1987-01-01
3	4	1982-01-01	1987-01-01
6	6	1987-01-01	NULL
8	5	1989-01-01	NULL

Φιγούρα 5 : Περιεχόμενα bandmember

- Για τον πίνακα recordcompany:

```
SET @current_dba := 'admin'; -- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se pinakes

INSERT INTO recordcompany (CompanyName, Address, Phone, Email, BeginDate, EndDate) VALUES
('Atlantic Records', '1633 Broadway, New York, NY 10019', '+1-212-707-2000', 'info@atlantic.com', '1947-10-01', NULL),
('Columbia Records', '550 Madison Ave, New York, NY 10022', '+1-212-833-8000', 'contact@columbia.com', '1888-01-15', NULL),
('Universal Music', '2220 Colorado Ave, Santa Monica, CA 90404', '+1-310-865-5000', 'info@universal.com', '1934-09-01', NULL),
('Warner Bros Records', '3300 Warner Blvd, Burbank, CA 91505', '+1-818-846-9090', 'contact@warnerbros.com', '1958-03-19', NULL),
('Sony Music', '25 Madison Ave, New York, NY 10010', '+1-212-833-8000', 'info@sonymusic.com', '1929-01-01', NULL),
('Capitol Records', '1750 Vine St, Los Angeles, CA 90028', '+1-323-462-6252', 'info@capitol.com', '1942-04-06', NULL),
('RCA Records', '1540 Broadway, New York, NY 10036', '+1-212-930-4000', 'contact@rca.com', '1901-01-01', NULL),
('Def Jam', '1755 Broadway, New York, NY 10019', '+1-212-333-8000', 'info@defjam.com', '1984-01-01', NULL),
('Interscope', '2220 Colorado Ave, Santa Monica, CA 90404', '+1-310-865-1000', 'contact@interscope.com', '1990-01-01', NULL),
('Elektra Records', '75 Rockefeller Plaza, New York, NY 10019', '+1-212-275-4000', 'info@elektra.com', '1950-01-01', NULL),
('Geffen Records', '2220 Colorado Ave, Santa Monica, CA 90404', '+1-310-278-9010', 'contact@geffen.com', '1980-01-01', NULL),
('A&M Records', '2220 Colorado Ave, Santa Monica, CA 90404', '+1-310-865-1000', 'info@amrecords.com', '1962-01-01', NULL),
('Island Records', '825 Eighth Ave, New York, NY 10019', '+1-212-333-8000', 'contact@island.com', '1959-01-01', NULL),
('Mercury Records', '1755 Broadway, New York, NY 10019', '+1-212-333-8000', 'info@mercury.com', '1945-01-01', NULL),
('Arista Records', '1540 Broadway, New York, NY 10036', '+1-212-930-4000', 'contact@arista.com', '1974-01-01', NULL),
('Reprise Records', '3300 Warner Blvd, Burbank, CA 91505', '+1-818-846-9090', 'info@reprise.com', '1960-01-01', NULL),
('Virgin Records', '150 5th Ave, New York, NY 10011', '+1-212-786-8300', 'contact@virgin.com', '1972-01-01', NULL),
('Motown Records', '2648 W Grand Blvd, Detroit, MI 48208', '+1-313-875-7100', 'info@motown.com', '1959-01-12', NULL),
('Blue Note Records', '1290 Avenue Americas, New York, NY 10019', '+1-212-603-7931', 'contact@bluenote.com', '1939-01-01',
NULL),
('Chess Records', '2120 S Michigan Ave, Chicago, IL 60616', '+1-312-842-1824', 'info@chess.com', '1947-01-01', '1975-12-31'),
('Stax Records', '926 E McLemore Ave, Memphis, TN 38106', '+1-901-946-2535', 'contact@stax.com', '1957-01-01', '1975-12-31'),
('Sun Records', '706 Union Ave, Memphis, TN 38103', '+1-901-521-0664', 'info@sun.com', '1952-01-01', NULL),
('4AD Records', '17-19 Alma Rd, London SW18 1AA, UK', '+44-20-8870-9912', 'contact@4ad.com', '1980-01-01', NULL),
('Rough Trade', '66 Golborne Rd, London W10 5PS, UK', '+44-20-8960-9888', 'info@rought.com', '1978-01-01', NULL),
('Domino Records', '2 Goldhawk Rd, London W12 8DH, UK', '+44-20-8749-7766', 'contact@domino.com', '1993-01-01', NULL),
('XL Recordings', '1 Codrington Mews, London W11 2EH, UK', '+44-20-7471-4300', 'info@xl.com', '1989-01-01', NULL),
('Warp Records', '14 Ladbroke Hall, London W11 3BG, UK', '+44-20-7266-0707', 'contact@warp.com', '1989-01-01', NULL),
('Ninja Tune', '80 The Broadway, London SW19 1RH, UK', '+44-20-8540-3838', 'info@ninja.com', '1990-01-01', NULL),
('Matador Records', '304 Hudson St, New York, NY 10013', '+1-212-995-5882', 'contact@matador.com', '1989-01-01', NULL),
('Sub Pop Records', '2013 4th Ave, Seattle, WA 98121', '+1-206-441-8441', 'info@subpop.com', '1986-01-01', NULL),
('Epitaph Records', '2798 Sunset Blvd, Los Angeles, CA 90026', '+1-213-413-7353', 'contact@epitaph.com', '1980-01-01', NULL),
('Fat Wreck Chords', '2196 Palou Ave, San Francisco, CA 94124', '+1-415-648-3924', 'info@fatwreck.com', '1990-01-01', NULL),
('Dischord Records', '3819 Beecher St NW, Washington, DC 20007', '+1-202-363-5757', 'contact@dischord.com', '1980-01-01', NULL),
('Touch and Go', '1430 E 53rd St, Chicago, IL 60615', '+1-773-324-7524', 'info@tng.com', '1981-01-01', NULL),
('SST Records', '1250 6th St, Santa Monica, CA 90401', '+1-310-393-7774', 'contact@sst.com', '1978-01-01', NULL),
('Merge Records', '1009 E Main St, Durham, NC 27701', '+1-919-688-9969', 'info@merge.com', '1989-01-01', NULL),
('Thrill Jockey', '4023 N Ravenswood, Chicago, IL 60613', '+1-773-525-7400', 'contact@thrill.com', '1992-01-01', NULL),
('Kranky Records', '724 Seward St, Los Angeles, CA 90038', '+1-323-644-2554', 'info@kranky.com', '1993-01-01', NULL),
('Drag City', '2114 W 21st Pl, Chicago, IL 60608', '+1-773-645-0576', 'contact@drag.com', '1990-01-01', NULL),
('Jade Tree', '2310 Kennwynn Rd, Wilmington, DE 19810', '+1-302-994-4904', 'info@jade.com', '1991-01-01', NULL),
('Victory Records', '346 N Justine St, Chicago, IL 60607', '+1-312-666-8661', 'contact@victory.com', '1989-01-01', NULL),
('Metal Blade', '5737 Kanan Rd, Agoura Hills, CA 91301', '+1-818-735-1185', 'info@metal.com', '1982-01-01', NULL),
('Nuclear Blast', '99 Wall St, New York, NY 10005', '+1-212-575-3700', 'contact@nuclear.com', '1987-01-01', NULL),
('Century Media', '99 Wall St, New York, NY 10005', '+1-212-575-3700', 'info@century.com', '1988-01-01', NULL),
('Roadrunner Records', '902 Broadway, New York, NY 10010', '+1-212-333-8000', 'contact@roadrunner.com', '1980-01-01', NULL),
('Candlelight Records', '99 Wall St, New York, NY 10005', '+1-212-575-3700', 'info@candle.com', '1993-01-01', NULL),
('Relapse Records', '5951 Canning St, Philadelphia, PA 19143', '+1-215-726-4043', 'contact@relapse.com', '1990-01-01', NULL),
('Southern Lord', '2132 Central Ave, Los Angeles, CA 90021', '+1-323-585-2555', 'info@southern.com', '1998-01-01', NULL),
('Ipecac Records', '2132 Central Ave, Los Angeles, CA 90021', '+1-323-585-2555', 'contact@ipecac.com', '1999-01-01', NULL),
('Absolutely Kosher', '1412 18th St, San Francisco, CA 94107', '+1-415-401-4300', 'info@kosher.com', '1998-01-01', NULL);


select * from recordcompany;
```

Με χρήση της εντολής *select * from recordcompany*; έχουμε το παρακάτω αποτέλεσμα:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CompanyID	CompanyName	Address	Phone	Email	BeginDate	EndDate
1	Atlantic Records	1633 Broadway, New York, NY 10019	+1-212-707-2000	info@atlantic.com	1947-10-01	NULL
2	Columbia Records	550 Madison Ave, New York, NY 10022	+1-212-833-8000	contact@columbia.com	1888-01-15	NULL
3	Universal Music	2220 Colorado Ave, Santa Monica, C...	+1-310-865-5000	info@universal.com	1934-09-01	NULL
4	Warner Bros Records	3300 Warner Blvd, Burbank, CA 91505	+1-818-846-9090	contact@warnerbros.com	1958-03-19	NULL
5	Sony Music	25 Madison Ave, New York, NY 10010	+1-212-833-8000	info@sonymusic.com	1929-01-01	NULL
6	Capitol Records	1750 Vine St, Los Angeles, CA 90028	+1-323-462-6252	info@capitol.com	1942-04-06	NULL
7	RCA Records	1540 Broadway, New York, NY 10036	+1-212-930-4000	contact@rca.com	1901-01-01	NULL
8	Def Jam	1755 Broadway, New York, NY 10019	+1-212-333-8000	info@defjam.com	1984-01-01	NULL
9	Interscope	2220 Colorado Ave, Santa Monica, C...	+1-310-865-1000	contact@interscope.com	1990-01-01	NULL
10	Elektra Records	75 Rockefeller Plaza, New York, NY 1...	+1-212-275-4000	info@elektra.com	1950-01-01	NULL
11	Geffen Records	2220 Colorado Ave, Santa Monica, C...	+1-310-278-9010	contact@geffen.com	1980-01-01	NULL
12	A&M Records	2220 Colorado Ave, Santa Monica, C...	+1-310-865-1000	info@amrecords.com	1962-01-01	NULL
13	Island Records	825 Eighth Ave, New York, NY 10019	+1-212-333-8000	contact@island.com	1959-01-01	NULL
14	Mercury Records	1755 Broadway, New York, NY 10019	+1-212-333-8000	info@mercury.com	1945-01-01	NULL
15	Arista Records	1540 Broadway, New York, NY 10036	+1-212-930-4000	contact@arista.com	1974-01-01	NULL
16	Reprise Records	3300 Warner Blvd, Burbank, CA 91505	+1-818-846-9090	info@reprise.com	1960-01-01	NULL
17	Virgin Records	150 5th Ave, New York, NY 10011	+1-212-786-8300	contact@virgin.com	1972-01-01	NULL
18	Motown Records	2648 W Grand Blvd, Detroit, MI 48208	+1-313-875-7100	info@motown.com	1959-01-12	NULL
19	Blue Note Records	1290 Avenue Americas, New York, N...	+1-212-603-7931	contact@bluenote.com	1939-01-01	NULL
20	Chess Records	2120 S Michigan Ave, Chicago, IL 60...	+1-312-842-1824	info@chess.com	1947-01-01	1975-1...
21	Stax Records	926 E McLemore Ave, Memphis, TN 3...	+1-901-946-2535	contact@stax.com	1957-01-01	1975-1...
22	Sun Records	706 Union Ave, Memphis, TN 38103	+1-901-521-0664	info@sun.com	1952-01-01	NULL
23	4AD Records	17-19 Alma Rd, London SW18 1AA, UK	+44-20-8870-9912	contact@4ad.com	1980-01-01	NULL
24	Rough Trade	66 Golborne Rd, London W10 5PS, UK	+44-20-8960-9888	info@rought.com	1978-01-01	NULL
25	Domino Records	2 Goldhawk Rd, London W12 8DH, UK	+44-20-8749-7766	contact@domino.com	1993-01-01	NULL

Φιγούρα 6 : Περιεχόμενα recordcompany

- Για τον πίνακα genre:

```
SET @current_dba := 'admin'; -- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se
pinakes

INSERT INTO genre (GenreName) VALUES
('Rock'), ('Pop'), ('Jazz'), ('Classical'), ('Hip Hop'),
('Country'), ('Blues'), ('Electronic'), ('Folk'), ('R&B'),
('Reggae'), ('Punk'), ('Metal'), ('Alternative'), ('Indie'),
('Soul'), ('Funk'), ('Disco'), ('House'), ('Techno'),
('Trance'), ('Dubstep'), ('Ambient'), ('New Age'), ('World'),
('Latin'), ('Salsa'), ('Bossa Nova'), ('Flamenco'), ('Celtic'),
('Gospel'), ('Spiritual'), ('Opera'), ('Symphony'), ('Chamber'),
('Bluegrass'), ('Rockabilly'), ('Swing'), ('Bebop'), ('Fusion'),
('Progressive'), ('Psychedelic'), ('Grunge'), ('Emo'), ('Hardcore'),
('Ska'), ('Reggaeton'), ('Trap'), ('Lo-Fi'), ('Experimental');

select * from genre;
```

Με χρήση της εντολής *select * from genre*; έχουμε το παρακάτω αποτέλεσμα:

GenreID	GenreName
1	Rock
2	Pop
3	Jazz
4	Classical
5	Hip Hop
6	Country
7	Blues
8	Electronic
9	Folk
10	R&B
11	Reggae
12	Punk
13	Metal
14	Alternative
15	Indie
16	Soul
17	Funk
18	Disco
19	House
20	Techno
21	Trance
22	Dubstep
23	Ambient
24	New Age
25	World

Φιγούρα 7 : Περιεχόμενα genre

Για τον πίνακα producer:

```
SET @current_dba := 'admin'; -- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se pinakes

INSERT INTO producer (FirstName, LastName, NumberOfProductions) VALUES
('Rick', 'Rubin', 150),
('Quincy', 'Jones', 200),
('George', 'Martin', 180),
('Dr.', 'Dre', 120),
('Max', 'Martin', 95),
('Timbaland', '', 110),
('Pharrell', 'Williams', 85),
('Mark', 'Ronson', 75),
('Jack', 'White', 60),
('Danger', 'Mouse', 55),
('Butch', 'Vig', 90),
('Steve', 'Albini', 105),
('Bob', 'Rock', 88),
('Alan', 'Parsons', 70),
('Tony', 'Visconti', 82),
('Mutt', 'Lange', 45),
('Trevor', 'Horn', 65),
('Flood', '', 78),
('Steve', 'Lillywhite', 67),
('Daniel', 'Lanois', 52),
('Brian', 'Eno', 95),
('T Bone', 'Burnett', 48),
('Jerry', 'Wexler', 125),
('Phil', 'Spector', 88),
```

```
('Berry', 'Gordy', 92),
('Sam', 'Phillips', 67),
('Ahmet', 'Ertegun', 78),
('Chris', 'Thomas', 56),
('Glyn', 'Johns', 64),
('Eddie', 'Kramer', 72),
('Roy', 'Thomas Baker', 58),
('Arif', 'Mardin', 69),
('Tommy', 'LiPuma', 54),
('Don', 'Was', 61),
('Mitchell', 'Froom', 47),
('Tchad', 'Blake', 52),
('John', 'Leckie', 68),
('Nigel', 'Godrich', 45),
('Dave', 'Sardy', 38),
('Rick', 'Ocasek', 42),
('Brendan', "O'Brien", 65),
('Scott', 'Litt', 58),
('Andy', 'Wallace', 72),
('Bob', 'Clearmountain', 85),
('Tom', 'Lord-Alge', 67),
('Chris', 'Lord-Alge', 61),
('Bob', 'Power', 44),
('Sylvia', 'Massy', 39),
('Jacknife', 'Lee', 35),
('James', 'Blake', 28);

select * from producer;
```

Με χρήση της εντολής *select * from producer*; έχουμε το παρακάτω αποτέλεσμα

ProducerID	FirstName	LastName	NumberOfProductions
1	Rick	Rubin	150
2	Quincy	Jones	200
3	George	Martin	180
4	Dr.	Dre	120
5	Max	Martin	95
6	Timbaland		110
7	Pharrell	Williams	85
8	Mark	Ronson	75
9	Jack	White	60
10	Danger	Mouse	55
11	Butch	Vig	90
12	Steve	Albini	105
13	Bob	Rock	88
14	Alan	Parsons	70
15	Tony	Visconti	82
16	Mutt	Lange	45
17	Trevor	Horn	65
18	Flood		78
19	Steve	Lillywhite	67
20	Daniel	Lanois	52
21	Brian	Eno	95
22	T Bone	Burnett	48
23	Jerry	Wexler	125
24	Phil	Spector	88
25	Berry	Gordy	92
26	Sam	Phillips	67

Φιγούρα 8 : Περιεχόμενα producer

- Για τον πίνακα <u>producercompany</u> :

```
SET @current_dba := 'admin'; -- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se pinakes

INSERT INTO producercompany (ProducerID, CompanyID, ToDate) VALUES
(1, 1, NULL),
(2, 2, NULL),
(3, 3, NULL),
(4, 4, NULL),
(5, 5, NULL),
(6, 6, NULL),
(7, 7, NULL),
(8, 8, NULL),
(9, 9, NULL),
(10, 10, NULL),
(11, 11, NULL),
(12, 12, NULL),
(13, 13, NULL),
```

```
(14, 14, NULL),
(15, 15, NULL),
(16, 16, NULL),
(17, 17, NULL),
(18, 18, NULL),
(19, 19, NULL),
(20, 20, NULL),
(21, 21, NULL),
(22, 22, NULL),
(23, 23, '2003-12-31'),
(24, 24, '1980-12-31'),
(25, 25, '1988-06-30'),
(26, 26, '2006-12-31'),
(27, 27, '2015-12-31'),
(28, 28, '2020-12-31'),
(29, 29, NULL),
(30, 30, NULL),
(31, 31, NULL),
(32, 32, NULL),
(33, 33, NULL),
(34, 34, NULL),
(35, 35, NULL),
(36, 36, NULL),
(37, 37, NULL),
(38, 38, NULL),
(39, 39, NULL),
(40, 40, NULL),
(41, 41, NULL),
(42, 42, NULL),
(43, 43, NULL),
(44, 44, NULL),
(45, 45, NULL),
(46, 46, NULL),
(47, 47, NULL),
(48, 48, NULL),
(49, 49, NULL),
(50, 50, NULL);

select * from producercompany;
```

Με χρήση της εντολής *select * from producercompany*; έχουμε το παρακάτω αποτέλεσμα

	ProducerID	CompanyID	ToDate
	1	1	NULL
	2	2	NULL
	3	3	NULL
	4	4	NULL
	5	5	NULL
	6	6	NULL
	7	7	NULL
	8	8	NULL
	9	9	NULL
	10	10	NULL
	11	11	NULL
	12	12	NULL
	13	13	NULL
	14	14	NULL
	15	15	NULL
	16	16	NULL
	17	17	NULL
	18	18	NULL
	19	19	NULL
	20	20	NULL
	21	21	NULL
	22	22	NULL
	23	23	2003-12-
	24	24	1980-12-
	25	25	1988-06-
	26	26	2006-12-

Φιγούρα 9 : Περιεχόμενα producercompany

- Για τον πίνακα artistcompany:

```
SET @current_dba := 'admin'; -- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se pinakes

INSERT INTO artistcompany (ArtistID, CompanyID, FromDate ,ToDate) VALUES
(1, 25,'2005-01-01', NULL),  -- arctic monkeys - domino records
(2, 11, '1991-01-01', '2019-09-16'), -- stone roses  , geffen
(3, 4, '1982-01-01','1987-01-01'),  -- smiths , warner
(4, 9, '1985-01-01', NULL), -- radiohead , random
(5, 2, '1983-01-01', '1993-01-01'), -- rolling stones , columbia
(6, 16, '1994-01-01', NULL ), -- green day , reprise
(7, 24, '2024-12-12', NULL), -- pulp, rough trade
(8, 31, '1994-01-01' , '1999-01-01'), -- suede , random
(9, 48, '1972-05-31' , '1981-10-25'), -- abba , random
(10, 32, '1965-09-03' , '1969-02-17'), -- the kinks , random
 -- ----------------------------------------
(11, 25,'2005-01-01', NULL), -- alex turner , arctic monkeys
(12, 25,'2005-01-01', NULL), -- matt helders , arctic monkeys
(13, 4, '1982-01-01','1987-01-01'), -- johnny marr  , smiths
(14,4, '1982-01-01','1987-01-01'), -- mozz , smiths
(15, 48, '1994-01-01' , '1999-01-01'), -- brett anderson , suede
(16, 16, '1994-01-01', NULL), -- billie joe , green day
(17, 43, '2011-01-01' ,  NULL), -- lana del rey , solo
(18, 48, '1972-05-31' , '1981-10-25'), -- agnetha  , abba
(19, 32, '1965-09-03' , '1969-02-17'), -- pete quaife , kinks
(20, 20, '1977-01-01', '2015-01-01'), -- bowie , solo
(21, 21, '2014-01-01',NULL), -- adrianne lenker , solo
(22, 22, '2011-01-01', '2023-01-01'), -- miles kane , solo
(23, 11, '1991-01-01', '2019-09-16'), -- ian brown , stone roses
(24, 24, '1980-01-01' , '1990-08-13'), -- joe strummer , solo
(25, 25, '1988-06-30' , '1996-01-21'), -- damon albarn , solo
(26, 26, '1976-12-31' , '2002-08-05'), -- madonna , solo
(27, 27, '2015-12-31', NULL), -- debbie harry , solo
(28, 28, '2005-03-07' , NULL); -- nelly furtado , solo



select * from artistcompany;
```

Με χρήση της εντολής *select * from artistcompany*; έχουμε το παρακάτω αποτέλεσμα

Result Grid | Filter Rows: | Export:

ArtistID	CompanyID	FromDate	ToDate
4	9	1985-01-01	NULL
5	2	1983-01-01	1993-01-01
6	16	1994-01-01	NULL
7	24	2024-12-12	NULL
8	31	1994-01-01	1999-01-01
9	48	1972-05-31	1981-10-25
10	32	1965-09-03	1969-02-17
11	25	2005-01-01	NULL
12	25	2005-01-01	NULL
13	4	1982-01-01	1987-01-01
14	4	1982-01-01	1987-01-01
15	48	1994-01-01	1999-01-01
16	16	1994-01-01	NULL
17	43	2011-01-01	NULL
18	48	1972-05-31	1981-10-25
19	32	1965-09-03	1969-02-17
20	20	1977-01-01	2015-01-01
21	21	2014-01-01	NULL
22	22	2011-01-01	2023-01-01
23	11	1991-01-01	2019-09-16
24	24	1980-01-01	1990-08-13
25	25	1988-06-30	1996-01-21
26	26	1976-12-31	2002-08-05
27	27	2015-12-31	NULL
28	28	2005-03-07	NULL

Φιγούρα 10 : Περιεχόμενα artistcompany

- Για τον πίνακα album :

```
SET @current_dba := 'admin'; -- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se
pinakes

INSERT INTO album (Title, ArtistID, GenreID , CompanyID , ProducerID ) VALUES
('Whatever People Say I Am , That\'s What I\'m Not' , 1 , 1 , 25 , 38),
('Favourite Worst Nightmare' , 1 , 1, 25 , 48),
('Humbug' , 1,1 , 25 , 25),
('AM' , 1, 1, 25, 9),
('The Stone Roses', 2 , 11 , 6 , 9),
('Meat Is Murder' , 3 , 14, 4, 7),
('Kerplunk' , 6 , 12, 45 , 30),
('Ultraviolence' , 17 , 14, 7 , 15),
('Sumbarine' , 11 , 14 , 1 , 41),
('Suede' , 15 , 7, 12 , 7),
('Every Day Is Like Sunday' , 14, 18, 45 , 27),
('American Idiot' , 26 , 38 , 45 , 19),
('Kaiti Garbi - Best Of' , 29 , 3, 6 , 18),
('H atmosfaira hlektrismenh' , 7 , 8 , 17 , 38),
('Trifulla Tetrafulla' , 20 , 8, 19 , 25),
('Dominicana Latina' , 1 , 1 , 37 , 1),
('Oso Zw Mazw' , 7 , 12, 8 , 10),
('Mia Thalassa Galazia' , 12, 36 , 38 , 49);


select * from album;
```

Με χρήση της εντολής *select * from album*; έχουμε το παρακάτω αποτέλεσμα:

	AlbumID	Title	ArtistID	GenreID	CompanyID	ProducerID
▶	1	Whatever People Say I Am , That's ...	1	1	25	38
	2	Favourite Worst Nightmare	1	1	25	48
	3	Humbug	1	1	25	25
	4	AM	1	1	25	9
	5	The Stone Roses	2	11	6	9
	6	Meat Is Murder	3	14	4	7
	7	Kerplunk	6	12	45	30
	8	Ultraviolence	17	14	7	15
	9	Sumbarine	11	14	1	41
	10	Suede	15	7	12	7
	11	Every Day Is Like Sunday	14	18	45	27
	12	American Idiot	26	38	45	19
	13	Kaiti Garbi - Best Of	29	3	6	18
	14	H atmosfaira hlektrismenh	7	8	17	38
	15	Trifulla Tetrafulla	20	8	19	25
	16	Dominicana Latina	1	1	37	1
	17	Oso Zw Mazw	7	12	8	10
	18	Mia Thalassa Galazia	12	36	38	49

Φιγούρα 11 : Περιεχόμενα album

- Για τον πίνακα track :

```
SET @current_dba := 'admin'; -- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se
pinakes


INSERT INTO track (Title , AlbumID, TrackLength, TrackNo , Lyrics) VALUES
('Brianstorm' , 2, '00:03:03' , 1, 'Brian
Top marks for not trying
So kind of you to bless us
With your effortlessness
We\'re grateful and so strangely comforted
And I wonder
Are you putting us under
\'Cause we can\'t take our eyes
Off the t-shirt and ties combination?
Well, see you later, innovator
Some want to kiss, some want to kick you
There\'s not a net you couldn\'t slip through
Or at least that\'s the impression I get
\'Cause you\'re smooth, and you\'re wet
And she\'s not aware yet
But she\'s yours
She\'ll be saying, "use me
Show me the jacuzzi"
I imagine that it\'s there on a plate
Your whole rendezvous rate
Means that you\'ll never be frightened
To make them wait for a while
I doubt it\'s your style
```

```
Not to get what you set
Out to acquire, the eyes are on fire
You are the unforecasted storm
calm, collected and commanding
(Top marks for not trying)
You leave the other stories standing
With your renditions and jokes
Bet there\'s hundreds of blokes
That have wept \'cause you\'ve stolen their
Thunder
Are you putting us under
\'Cause we can\'t take our eyes
Off the t-shirt and ties combination?
Well, see you later, innovator'
),

('I Bet You Look Good On the Dancefloor' , 1 , '00:02:56' , 7 , 'Stop making the eyes at me
I\'ll stop making the eyes at you
What it is that surprises me
Is that I don\'t really want you to
And your shoulders are frozen (cold as the night)
Oh, but you\'re an explosion (you\'re dynamite)
Your name isn\'t Rio, but I don\'t care for sand
And lighting the fuse might result in a bang, b-b-bang-oh
I bet that you look good on the dance floor
I don\'t know if you\'re looking for romance or
I don\'t know what you\'re looking for
I said, I bet that you look good on the dance floor
Dancing to electro-pop like a robot from 1984
Well, from 1984
I wish you\'d stop ignoring me
Because you\'re sending me to despair
Without a sound, yeah, you\'re calling me
And I don\'t think it\'s very fair
That your shoulders are frozen (cold as the night)
Oh, but you\'re an explosion (you\'re dynamite)
Your name isn\'t Rio, but I don\'t care for sand
And lighting the fuse might result in a bang, b-b-bang-oh
I bet that you look good on the dance floor
I don\'t know if you\'re looking for romance or
I don\'t know what you\'re looking for
I said, I bet that you look good on the dance floor
Dancing to electro-pop like a robot from 1984
Well, from 1984
Oh, there ain\'t no love, no Montagues or Capulets
Just banging tunes and DJ sets and
Dirty dance floors and dreams of naughtiness
Well, I bet that you look good on the dance floor
I don\'t know if you\'re looking for romance or
I don\'t know what you\'re looking for
I said, I bet that you look good on the dance floor
Dancing to electro-pop like a robot from 1984
Said, from 1984'),

('Do I Wanna Know?' , 4, '00:04:04',  1 , 'Have you got colour in your cheeks?
Do you ever get that fear that you can\'t shift
The type that sticks around like summat in your teeth?
Are there some aces up your sleeve?
Have you no idea that you\'re in deep?
I dreamt about you nearly every night this week

How many secrets can you keep?
\'Cause there\'s this tune I found
That makes me think of you somehow
And I play it on repeat
Until I fall asleep
Spilling drinks on my settee

(Do I wanna know)
If this feeling flows both ways?
(Sad to see you go)
Was sorta hoping that you\'d stay
(Baby, we both know)
That the nights were mainly made
For saying things that you can\'t say tomorrow day

Crawling back to you
Ever thought of calling when you\'ve had a few?
\'Cause I always do
Maybe I\'m too busy being yours to fall for somebody new
Now, I\'ve thought it through
Crawling back to you

So, have you got the guts?
Been wondering if your heart\'s still open
And if so, I wanna know what time it shuts
Simmer down and pucker up
I\'m sorry to interrupt, it\'s just I\'m constantly on the cusp
Of trying to kiss you
I don\'t know if you feel the same as I do
But we could be together if you wanted to

(Do I wanna know)
If this feeling flows both ways?
```

```
(Sad to see you go)
Was sorta hoping that you\'d stay
(Baby, we both know)
That the nights were mainly made
For saying things that you can\'t say tomorrow day

Crawling back to you (crawling back to you)
Ever thought of calling when you\'ve had a few? (You\'ve had a few?)
\'Cause I always do (\'cause I always do)
Maybe I\'m too (maybe I\'m too busy)
Busy being yours (being yours)
To fall for somebody new
Now, I\'ve thought it through
Crawling back to you

(Do I wanna know)
If this feeling flows both ways?
(Sad to see you go)
Was sorta hoping that you\'d stay
(Baby, we both know)
That the nights were mainly made
For saying things that you can\'t say tomorrow day

(Do I wanna know?)
Too busy being yours to fall
(Sad to see you go)
Ever thought of calling, darling?
(Do I wanna know?)
Do you want me crawling back to you?'),

('2,000 Light Years Away' , 7 , '00:02:56' , 1 , '… I sit alone in my bedroom
Staring at the walls
I\'ve been up all damn night long
My pulse is speeding, my love is yearning
I hold my breath and close my eyes
And dream about her
\'Cause she\'s 2000 light years away
She holds my malachite so tight
So never let go
\'Cause she\'s 2000 light years away
Years away
I sit outside and watch the sunrise
Look out as far as I can
I can\'t see her, but in the distance
I hear some laughter, we laugh together
I hold my breath and close my eyes
And dream about her
\'Cause she\'s 2000 light years away
She holds my malachite so tight
So never let go
\'Cause she\'s 2000 light years away
Years away
I sit alone in my bedroom
Staring at the walls
I\'ve been up all damn night long
My pulse is speeding, my love is yearning
I hold my breath and close my eyes
And dream about her
\'Cause she\'s 2000 light years away
She holds my malachite so tight
Oh, never let go
\'Cause she\'s 2000 light years away
Years away
'),

('Kalimera Kenouria Mou Agapi' , 8 , '00:02:35' , 7 , 'Kai tora pou xwrisame ti thes;
Ti vgainei pia an ftaio i an ftais;
Kainourgia agapi irthe stin kardia
Kai niwthw n\' agapw proti fora
Kalimera, kainourgia mou agapi
Kalimera, kainourgia zoi
Apo simera telos ta lathi
Ftanei na \'maste panta mazi
Apo simera telos ta lathi
Ftanei na \'maste panta mazi
Kalimera, kainourgia mou agapi
Kalimera, kainourgia zoi
Kai tora pou xwrisame mou les
Ta lathi mas n\' afisoume sto xtes
Fovamai, omws, pia pos einai arga
O xronos tora piso de gyrna
Kalimera, kainourgia mou agapi
Kalimera, kainourgia zoi
Apo simera telos ta lathi
Ftanei na \'maste panta mazi
Apo simera telos ta lathi
Ftanei na \'maste panta mazi
Kalimera, kainourgia mou agapi
Kalimera, kainourgia zoi
Apo simera telos ta lathi
Ftanei na \'maste panta mazi
Kalimera, kainourgia mou agapi
Kalimera, kainourgia zoi
'),
```

```
('Fyge se parakalw' , 12 , '00:03:48' , 10 , 'Ti allo prepei na sou po
Lypamai pia de s\' agapo
Ein\' ametrita ta lathi sou
Kai ateliota ta pathi sou
Kourastika na s\' agapo
Kai na se sygxoro
Fyge se parakalw
Fyge se parakalw
Tora irthe i seira sou
Gia na klapsei i kardia sou
Min ta thes ola dika sou
M\' agapas de s\' agapo
Allazoun omos oi kairoi
Den to katalaves kai sy
Osa ta panta exoun oria
Terma pia sta perithoria
Kai gia sygnomi min milas
Ta logia sou halas
Fyge se parakalw
Fyge se parakalw
Tora irthe i seira sou
Gia na klapsei i kardia sou
Min ta thes ola dika sou
M\' agapas de s\' agapo
'),

('Ierosylia' , 5, '00:05:05' , 8 , 'Imoun sti zoi sou
Kati san thriskeia
Tora an fygeis tha \'nai
Ierosylia
Skepsou ksana
Pia kathara
Ti pas kaneis
Ti grothia kano egw siderenia
Tin psyhi san petra skliri
Gia n\' antekso tin pikra pou pira
Otan mou \'pes pos fevgeis esi
Den zitao apo sena symponia
Oute thelw na me lypitheis
Mono ligo prin fygeis zitao na statheis
Kai na thymitheis
Imoun sti zoi sou
Kati san thriskeia
Tora, an fygeis, tha \'nai
Ierosylia
Isoun sti zoi sou
Kati san thriskeia
Tora, an fygeis, tha \'nai
Ierosylia
Ela tora piso
Gyrise piso
Ton pono pou exw
Na stamatiso
Kai ela tora piso,
Gyrise piso
Na s\' agkaliwso
Na se filiso
Vazo stoixima tora mazi sou
Gia osa kaneis kai osa mou les
Kathe vrady pou tha \'rhete tha \'xeis polles
Typses ki enoxes
Imoun sti zoi sou
Kati san thriskeia
Tora, an fygeis, tha \'nai
Ierosylia
Ela tora piso
Gyrise piso
Ton pono pou exw
Na stamatiso
Ki ela tora piso
Gyrise piso
Na s\' agkaliwso
Na se filiso
Skepsou ksana
Pio kathara
Ti pas na kaneis
Ela tora piso
Gyrise piso
Ton pono pou exw
Na stamatiso
Ki ela tora piso
Gyrise piso
Na s\' agkaliwso
Na se filiso
Piso
'),

('Judas' , 17 , '00:05:00' , 5 , 'Oh, whoa-whoa, oh-oh
I\'m in love with Judas, Judas
Oh-oh-oh-oh-oh
I\'m in love with Judas, Judas
Judas, Juda-ah-ah, Judas, Juda-ah-ah
Judas, Juda-ah-ah, Judas, Gaga
```

```
        Judas, Juda-ah-ah, Judas, Juda-ah-ah
        Judas, Juda-ah-ah, Judas, Gaga
        When he calls to me, I am ready
        I\'ll wash his feet with my hair if he needs
        Forgive him when his tongue lies through his brain
        Even after three times, he betrays me
        Ah-ah, ah-ah-ah-ah, ah-ah, ah-ah-ah-ah
        I\'ll bring him down, bring him down, down
        Ah-ah, ah-ah-ah-ah, ah-ah, ah-ah-ah-ah
        A king with no crown, king with no crown
        I\'m just a Holy Fool, oh, baby, it\'s so cruel
        But I\'m still in love with Judas, baby
        I\'m just a Holy Fool, oh, baby, it\'s so cruel
        But I\'m still in love with Judas, baby
        Oh, whoa-whoa, oh-oh
        I\'m in love with Judas, Judas
        Oh-oh-oh-oh-oh
        I\'m in love with Judas, Judas
        Judas, Juda-ah-ah, Judas, Juda-ah-ah
        Judas, Juda-ah-ah, Judas, Gaga
        I couldn\'t love a man so purely
        Even prophets forgave his goofy way
        I\'ve learned love is like a brick, you can
        Build a house or sink a dead body
        Ah-ah, ah-ah-ah-ah, ah-ah, ah-ah-ah-ah
        I\'ll bring him down, bring him down, down
        Ah-ah, ah-ah-ah-ah, ah-ah, ah-ah-ah-ah
        A king with no crown, king with no crown
        I\'m just a Holy Fool, oh, baby, it\'s so cruel
        But I\'m still in love with Judas, baby
        I\'m just a Holy Fool, oh, baby, it\'s so cruel
        But I\'m still in love with Judas, baby
        Oh, whoa-whoa, oh-oh
        I\'m in love with Juda-as, Juda-as
        Oh-oh-oh-oh-oh
        I\'m in love with Juda-as, Juda-as
        Ew
        In the most Biblical sense, I am beyond repentance
        Fame hooker, prostitute, wench vomits her mind
        But in the cultural sense, I just speak in future tense
        Judas, kiss me if offensed, or wear ear condom next time
        I wanna love you
        But something\'s pulling me away from you
        Jesus is my virtue
        And Judas is the demon I cling to
        I cling to
        Just a Holy Fool, oh, baby, it\'s so cruel
        But I\'m still in love with Judas, baby
        I\'m just a Holy Fool, oh, baby, it\'s so cruel
        But I\'m still in love with Judas, baby
        Oh, whoa-whoa, oh-oh
        I\'m in love with Judas, Judas
        Oh-oh-oh-oh-oh
        I\'m in love with Judas, Judas
        Judas, Juda-ah-ah, Judas, Juda-ah-ah
        Judas, Juda-ah-ah, Judas, Gaga'),

        ('Ma Zw' , 7, '00:02:02' , 5 , 'Mesa mou niwthw paraksena ithela na ksera pou tha me vrw
        Me exasa kapou anamesa ston katw kosmo kai ton ourano
        Soma keno
        Poulisa pali psyxi mes sti zali tou erota misotimis
        Ax eafte mou exthre kai aderfe mou pote den ta vrikame emeis
        Den iremeis
        Den iremeis
        Ola mou ta thelw mia zoi
        Me ena prepei ta skotwnw
        Katafernei kapoio na swthei
        Kai me typseis to plirwnw
        Enoxi mou se pinw methaw kai afinomai ginomai liwma
        Xwma
        Ma zw akoma
        Gyro mou ola perierga swmata aniera dixws kardies
        Favlos o kyklos mou mavros o kyknos mou
        Limnes sta pathi vathies
        Pese ki des
        Poios tha mou pei ti einai lathos kai ti amartia na kserw kai egw
        Aggelous exw mazi mou ap tin kolasi mou palevw na vgw
        Prin na pnigw
        Prin na pnigw
        Ola mou ta thelw mia zoi
        Me ena prepei ta skotwnw
        Katafernei kapoio na swthei
        Kai me typseis ta plirwnw
        Enoxi mou se pinw methaw kai afinomai ginomai liwma
        Xwma
        Ma zw akoma
        Ola mou ta thelw mia zoi
        Me ena prepei ta skotwnw
        Katafernei kapoio na swthei
        Kai me typseis ta plirwnw
        Enoxi mou se pinw methaw kai afinomai ginomai liwma
        Ola mou ta thelw mia zoi
        Me ena prepei ta skotwnw
        Katafernei kapoio na swthei
```

```
Kai me typseis ta plirwnw
Enoxi mou se pinw methaw kai afinomai ginomai liwma
Xwma
Ma zw akoma
Xwma
Ma zw akoma
Ma zw akoma
'),

('Ola Gia Ola' , 5 , '00:08:09' , 12 ,   NULL),
('Electricity' , 10 , '00:04:50' , 3 , 'We got a love thats cold as stone
We got a love from our violent homes
We got a love and it got no name
We kiss our love with lips like pain
We got a lotta electricity
We got a love like AC DC
We got a love and it got no shame
We kiss our love with lips like pain
Kissing our love with lips like pain

Oh its bigger than the universe
Its bigger than the universe
Its bigger than the two of us
Oh its bigger than you and me
We got a love between us and its like electricity

We got a love like a violent mind
We get our love from white white lines
We got a love and it got no name
We kiss our love with lips like pain
We got a love from nowhere towns
We got a love like electric sounds
We got a love and it got no shame
We kiss our love with lips like pain
Kissing our love with lips like pain

I said, oh its bigger than the universe
Its bigger than the universe
Its bigger than the two of us
Oh its bigger than you and me
We got a love between us and its like electricity

Oh its bigger than the universe
Its bigger than the universe
Its bigger than the two of us
Oh its bigger than you and me
We got a love between us thats like electricity
'),

('Never Had No One Ever' , 12 , '00:02:33' , 6 , 'When you walk without ease
On these streets where you were raised
I had a really bad dream
It lasted 20 years, 7 months and 27 days
And I know alone that I never, ever oh
Had no one ever
Now Im outside your house
Im alone
And Im outside your house
I hate to intrude
Oh, Im alone, Im alone
Im alone, Im alone, Im alone, Im alone
And I never, never oh
Had no one ever
I never had no one ever
I never had no, no one ever
Had no one never
never, no, oh
'),

('Bad Habit' , 3, '00:04:04' , 10 , 'wanted me
I wish I knew, I wish I knew you wanted me
I wish I knew, I wish I knew you wanted me
What you, ooh, uh, what you do?
Made a move, couldve made a move
If I knew Id be with you
Is it too late to pursue?
I bite my tongue, its a bad habit
Kinda mad that I didnt take a stab at it
Thought you were too good for me, my dear
Never gave me time of day, my dear
Its okay, things happen for
Reasons that I think are sure, yeah
I wish I knew, I wish I knew you wanted me (I wish I knew)
I wish I knew (oh), I wish I knew you wanted me
I wish I knew (yeah), I wish I knew you wanted me (oh)
I wish I knew, I wish I knew you wanted me
Please say to me (please just say to me)
If you still want it
I wish you wouldnt play with me
I wanna know (oh, no)
Uh, can I bite your tongue like my bad habit?
Would you mind if I tried to make a pass at it?
No, youre not too good for me, my dear
Funny you come back to me, my dear
```

```
Its okay, things happen for
Reasons that I cant ignore, yeah
I wish I knew, I wish I knew you wanted me
I wish I knew (wish I knew), I wish I knew you wanted me (oh)
'),

('Ksexasmeni Enilikiosi' , 11, '00:02:11' , 7 , 'Soulja Boy Tell Em
Hopped up out the bed, turn my swag on
Took a look in the mirror, said, "Whats up?"
Yeah, Im gettin money, oh
I hopped up out the bed, turn my swag on
Took a look in the mirror, said, "Whats up?"
Yeah, Im gettin money, oh
Turn my swag on
Its my turn, now turn it up
Yeah, yeah
I put my team on, then my theme song
Now its time to turn it up
Yeah, yeah
I got a question, why they hatin on me?
I got a question, why they hatin on me?
I aint did nothin to Em, but count this money
And put my team on, got my whole clique stuntin
Boy, whats up? Yeah
Boy, whats up? Yeah
When I was nine years old
I put it in my head Ima die for this gold (Soulja Boy Tell Em)
Boy, whats up?
Hopped up out the bed, turn my swag on
Took a look in the mirror, said, "Whats up?"
Yeah, Im gettin money, oh
I hopped up out the bed, turn my swag on
Took a look in the mirror, said, "Whats up?"
Yeah, Im gettin money, oh
Im back again
I know a lot of yall thought I wasnt coming back
Yeah, yeah
I had to prove Em wrong
Got back in the studio and came up with another hit
Yeah, yeah
I told the world my story, the world where Im from
Souljaboy X L dot com, boy, whats up?
Yeah, yeah
Now every time you see me spit
Every time you hear me rhyme
Every time you see me in your state or town, say, "Whats up?"
Yeah, yeah (Soulja Boy Tell Em)
Hopped up out the bed, turn my swag on
Took a look in the mirror, said, "Whats up?"
Yeah, Im gettin money, oh
I hopped up out the bed, turn my swag on
Took a look in the mirror, said, "Whats up?"
Yeah, Im gettin money, oh
'),

-- 14
('Falling Stars (Chugu Bam Bam)' , 15, '00:01:11' , 1 ,NULL),

('Straight To Hell' , 13, '00:06:56' , 14, 'Tracksuit lefka
Pan ap ola kathari psyxi
Metraw mavra lefta
Poios den agapaei to xarti, huh
Cest la vie (Cest la vie)
Pow-pow-pow-pow, BCC
Cest la vie (Cest la vie)
Bcc, Rack attack
Tracksuit lefka
Pan ap ola kathari psyxi
Metraw mavra lefta
Poios den agapaei to xarti, huh
Cest la vie (Cest la vie)
Pow-pow-pow-pow, BCC
Cest la vie (Cest la vie)
'),

('Only Friend' , 16, '00:03:00' , 1 , 'Weathered and fading
Who feels alone in the world?
You do
Too complicated
Its not the same anymore
Youre through
Youre signing off again
But youre my only friend
I dont know what to say
I feel the ending close
Pick up the pieces
Finding a place in the world
To be
Too many reasons
To keep an ear at the door
For me
Will you come back again?
I only need a friend
When its all said and done
```

```
Will you need me too, girl?
Girl
Girl
Oh, girl
'),

('Crying in C Sharp' , 18 , '00:15:00' , 4, NULL),

('Fading Lights', 12, '00:04:22', 2, NULL),
('Silent Echoes', 18, '00:03:58', 5, NULL),
('Midnight Whispers', 4, '00:05:10', 1, NULL),
('Shattered Dreams', 14, '00:04:45', 7, NULL),
('Lost in Time', 9, '00:03:36', 3, NULL),
('Neon Skies', 2, '00:04:12', 1, 'Dancing under neon skies, chasing dreams that never die'),
('Echoes of You', 8, '00:03:59', 2, 'Whispers in the dark, echoes of your heart'),
('Falling Down', 5, '00:04:45', 3, 'Counting every falling star, wishing you were not so far'),
('Broken Compass', 9, '00:05:01', 4, 'Lost without direction, my broken compass leads me home'),
('Silver Rain', 15, '00:03:50', 5, 'Silver rain falls on my face, washing away the pain'),
('Velvet Night', 17, '00:04:33', 6, 'Velvet night surrounds me, shadows dance silently'),
('Crimson Horizon', 16, '00:05:10', 7, 'Crimson horizon sets the sky on fire, igniting desire'),
('Glass Heart', 6, '00:04:05', 8, 'Fragile as glass, my heart beats for you'),
('Midnight Drive', 13, '00:03:48', 9, 'Racing through the midnight drive, feeling so alive'),
('Lost Echo', 18, '00:04:22', 10, 'A lost echo calls my name, but nothing feels the same');



SELECT
    *
FROM
    track
```

Με χρήση της εντολής *select * from track*; έχουμε το παρακάτω αποτέλεσμα:

TrackID	Title	AlbumID	TrackLength	TrackNo	Lyrics
100	Brianstorm	2	00:03:03	1	Brian Top marks for not trying So kind of you to bless us With your effortlessness We're grateful and so strangely comforted And I wonder Are you putting us under 'Cause we can't take our eyes Off the t-shirt and ...
101	I Bet You Look Good On the Dancefl...	1	00:02:56	7	Stop making the eyes at me I'll stop making the eyes at you What it is that surprises me Is that I don't really want you to And your shoulders are frozen (cold as the night) Oh, but you're an explosion (you're dynam...
102	Do I Wanna Know?	4	00:04:04	1	Have you got colour in your cheeks? Do you ever get that fear that you can't shift The type that sticks around like summat in your teeth? Are there some aces up your sleeve? Have you no idea that you're in deep...
103	2,000 Light Years Away	7	00:02:56	1	... I sit alone in my bedroom Staring at the walls I've been up all damn night long My pulse is speeding, my love is yearning I hold my breath and close my eyes And dream about her 'Cause she's 2000 light years awa...
104	Kalimera Kenouria Mou Agapi	8	00:02:35	7	Kai tora pou xwrisame ti thes; Ti vgainei pia an ftaio i an ftais; Kainourgia agapi irthe stin kardia Kai niwthw n' agapw proti fora Kalimera, kainourgia mou agapi Kalimera, kainourgia zoi Apo simera telos ta lathi Ftanei na '...
105	Fyge se parakalw	12	00:03:48	10	Ti allo prepei na sou po Lypamai pia de s' agapo Ein' ametrita ta lathi sou Kai ateliota ta pathi sou Kourastika na s' agapo Kai na se sygxoro Fyge se parakalw Fyge se parakalw Tora irthe i seira sou Gia na klapsei i kardia...
106	Ierosylia	5	00:05:05	8	Imoun sti zoi sou Kati san thriskeia Tora an fygeis tha 'nai Ierosylia Skepsou ksana Pia kathara Ti pas kaneis Ti grothia kano egw siderenia Tin psyhi san petra skliri Gia n' antekso tin pikra pou pira Otan mou 'pes pos fev...
107	Judas	17	00:05:00	5	Oh, whoa-whoa, oh-oh I'm in love with Judas, Judas Oh-oh-oh-oh-oh I'm in love with Judas, Judas Judas, Juda-ah-ah, Judas, Juda-ah-ah Judas, Juda-ah-ah, Judas, Gaga Judas, Juda-ah-ah, Judas, Juda-ah-ah Juda...
108	Ma Zw	7	00:02:02	5	Mesa mou niwthw paraksena ithela na ksera pou tha me vrw Me exasa kapou anamesa ston katw kosmo kai ton ourano Soma keno Poulisa pali psyxi mes sti zali tou erota misotimis Ax eafte mou exthre kai aderfe mo...
109	Ola Gia Ola	5	00:08:09	12	NULL
110	Electricity	10	00:04:50	3	We got a love thats cold as stone We got a love from our violent homes We got a love and it got no name We kiss our love with lips like pain We got a lotta electricity We got a love like AC DC We got a love and it g...
111	Never Had No One Ever	12	00:02:33	6	When you walk without ease On these streets where you were raised I had a really bad dream It lasted 20 years, 7 months and 27 days And I know alone that I never, ever oh Had no one ever Now Im outside your...
112	Bad Habit	3	00:04:04	10	wanted me I wish I knew, I wish I knew you wanted me I wish I knew, I wish I knew you wanted me What you, ooh, uh, what you do? Made a move, couldve made a move If I knew Id be with you Is it too late to pu...
113	Ksexasmeni Enilikiosi	11	00:02:11	7	Soulja Boy Tell Em Hopped up out the bed, turn my swag on Took a look in the mirror, said, "Whats up?" Yeah, Im gettin money, oh I hopped up out the bed, turn my swag on Took a look in the mirror, said, "Wha...
114	Falling Stars (Chugu Bam Bam)	15	00:01:11	1	NULL
115	Straight To Hell	13	00:06:56	14	Tracksuit lefka Pan ap ola kathari psyxi Metraw mavra lefta Poios den agapaei to xarti, huh Cest la vie (Cest la vie) Pow-pow-pow-pow, BCC Cest la vie (Cest la vie) Bcc, Rack attack Tracksuit lefka Pan ap ola kathari p...
116	Only Friend	16	00:03:00	1	Weathered and fading Who feels alone in the world? You do Too complicated Its not the same anymore Youre through Youre signing off again But youre my only friend I dont know what to say I feel the ending clos...
117	Crying in C Sharp	18	00:15:00	4	NULL
118	Fading Lights	12	00:04:22	2	NULL
119	Silent Echoes	18	00:03:58	5	NULL
120	Midnight Whispers	4	00:05:10	1	NULL
121	Shattered Dreams	14	00:04:45	7	NULL

Φιγούρα 12 : Περιεχόμενα track

- Για τον πίνακα <u>albumrelease:</u>

```
SET @current_dba := 'admin'; -- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se pinakes

INSERT INTO albumrelease (AlbumID, ReleaseDate, ReleaseType, ReleaseStatus, Packaging) VALUES
(1, '2022-01-15', 'LP', 'OFFICIAL', 'JEWEL CASE'),
(2, '2021-06-20', 'CD', 'PROMOTION', 'DIGIPAK'),
(3, '2020-09-05', 'MP3', 'OFFICIAL', 'NOT APPLICABLE'),
(4, '2022-11-12', 'CD', 'BOOTLEG', 'CARDBOARD SLEEVE'),
(5, '2021-03-25', 'LP', 'WITHDRAWN', 'BOOK'),
(6, '2020-12-01', 'MP3', 'CANCELLED', 'NOT APPLICABLE'),
(7, '2022-07-30', 'CD', 'OFFICIAL', 'DIGIPAK'),
(8, '2021-10-18', 'LP', 'PROMOTION', 'JEWEL CASE'),
(9, '2020-05-22', 'MP3', 'OFFICIAL', 'NOT APPLICABLE'),
(10, '2022-02-14', 'CD', 'WITHDRAWN', 'CARDBOARD SLEEVE'),
(1, '1985-04-23', 'LP', 'OFFICIAL', 'BOOK'),
(2, '1992-09-15', 'CD', 'OFFICIAL', 'JEWEL CASE'),
(3, '2000-06-01', 'MP3', 'PROMOTION', 'NOT APPLICABLE'),
(4, '1988-11-12', 'LP', 'OFFICIAL', 'CARDBOARD SLEEVE'),
(5, '1999-03-05', 'CD', 'BOOTLEG', 'DIGIPAK'),
(6, '2005-07-20', 'MP3', 'OFFICIAL', 'NOT APPLICABLE');


select * from albumrelease;
```

Με χρήση της εντολής *select * from albumrelease*; έχουμε το παρακάτω αποτέλεσμα:

ReleaseID	AlbumID	ReleaseDate	ReleaseType	ReleaseStatus	Packaging
1	1	2022-01-15	LP	OFFICIAL	JEWEL CASE
2	2	2021-06-20	CD	PROMOTION	DIGIPAK
3	3	2020-09-05	MP3	OFFICIAL	NOT APPLICABLE
4	4	2022-11-12	CD	BOOTLEG	CARDBOARD SLEEVE
5	5	2021-03-25	LP	WITHDRAWN	BOOK
6	6	2020-12-01	MP3	CANCELLED	NOT APPLICABLE
7	7	2022-07-30	CD	OFFICIAL	DIGIPAK
8	8	2021-10-18	LP	PROMOTION	JEWEL CASE
9	9	2020-05-22	MP3	OFFICIAL	NOT APPLICABLE
10	10	2022-02-14	CD	WITHDRAWN	CARDBOARD SLEEVE
11	1	1985-04-23	LP	OFFICIAL	BOOK
12	2	1992-09-15	CD	OFFICIAL	JEWEL CASE
13	3	2000-06-01	MP3	PROMOTION	NOT APPLICABLE
14	4	1988-11-12	LP	OFFICIAL	CARDBOARD SLEEVE
15	5	1999-03-05	CD	BOOTLEG	DIGIPAK
16	6	2005-07-20	MP3	OFFICIAL	NOT APPLICABLE
NULL	NULL	NULL	NULL	NULL	NULL

Φιγούρα 13 : Περιεχόμενα albumrelease

UPDATE-DELETE-ALTER Παραδείγματα

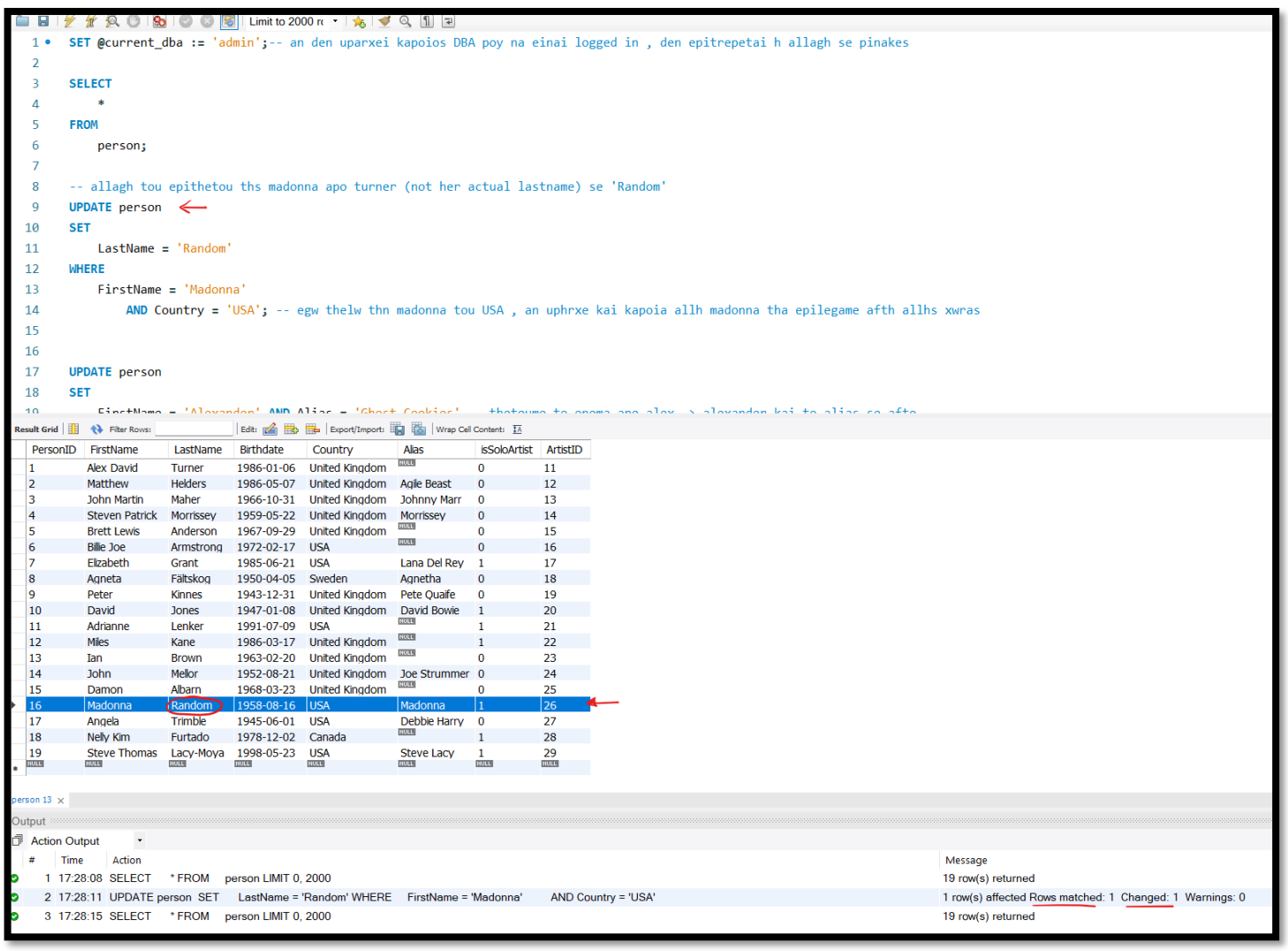
- **Παράδειγμα 1**

Όπως φαίνεται στο κομμάτι κώδικα που παρατίθεται και στα screenshot , στόχος μας είναι να κάνουμε αλλαγή στο επίθετο της Madonna από 'Turner' σε 'Random'.

```
SET @current_dba := 'admin';-- an den uparxei kapoios DBA poy na einai logged in , den epitrepetai h allagh se pinakes

SELECT
    *
FROM
    person;

-- allagh tou epithetou ths madonna apo turner (not her actual lastname) se 'Random'
UPDATE person
SET
    LastName = 'Random'
WHERE
    FirstName = 'Madonna'
        AND Country = 'USA'; -- egw thelw thn madonna tou USA , an uphrxe kai kapoia allh madonna tha epilegame afth allhs xwras
```



Φιγούρα 14 : Επιτυχής αλλαγή του LastName της εγγραφής με FirstName = 'Madonna'

- **Παράδειγμα 2**

Με την εντολή *select * from artist*; φέρνουμε στην οθόνη όλες τις εγγραφές που περιέχονται μέσα στον πίνακα. Προκύπτει το εξής :

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ArtistID	ArtistType
1	BAND
2	BAND
3	BAND
4	BAND
5	BAND
6	BAND
7	BAND
8	BAND
9	BAND
10	BAND
11	PERSON
12	PERSON
13	PERSON
14	PERSON
15	PERSON
16	PERSON
17	PERSON
18	PERSON
19	PERSON
20	PERSON
21	PERSON

Φιγούρα 15: Εγγραφές του πίνακα artist πριν την αλλαγή

Με την εντολή *select * from band*; φέρνουμε όλα τα συγκροτήματα που περιέχει η ΒΔ:

Result Grid | Filter Rows: | Edit: | Export/Import:

BandID	BandName	FormationDate	DisbandDate	ArtistID
1	Arctic Monkeys	2002-01-01	NULL	1
2	The Stone Roses	1983-01-01	2019-09-16	2
3	The Smiths	1982-01-01	1987-01-01	3
4	Radiohead	1985-01-01	NULL	4
5	The Rolling Stones	1962-07-12	NULL	5
6	Green Day	1987-01-01	NULL	6
7	Pulp	1978-01-01	NULL	7
8	Suede	1989-01-01	NULL	8
9	ABBA	1972-01-01	1982-01-01	9
10	The Kinks	1963-01-01	1997-01-01	10
NULL	NULL	NULL	NULL	NULL

Φιγούρα 16: Εγγραφές του πίνακα band πριν την αλλαγή

Ομοίως, φέρνουμε τον πίνακα album στην οθόνη:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

AlbumID	Title	ArtistID	GenreID	CompanyID	ProducerID
1	Whatever People Say I Am , That's ...	1	1	25	38
2	Favourite Worst Nightmare	1	1	25	48
3	Humbug	1	1	25	25
4	AM	1	1	25	9
5	The Stone Roses	2	11	6	9
6	Meat Is Murder	3	14	4	7
7	Kerplunk	6	12	45	30
8	Ultraviolence	17	14	7	15
9	Sumbarine	11	14	1	41
10	Suede	15	7	12	7
11	Every Day Is Like Sunday	14	18	45	27
12	American Idiot	26	38	45	19
13	Kaiti Garbi - Best Of	29	3	6	18
14	H atmosfaira hlektrismenh	7	8	17	38
15	Trifulla Tetrafulla	20	8	19	25
16	Dominicana Latina	1	1	37	1
17	Oso Zw Mazw	7	12	8	10
18	Mia Thalassa Galazia	12	36	38	49
NULL	NULL	NULL	NULL	NULL	NULL

Φιγούρα 17: Εγγραφές του πίνακα album πριν την αλλαγή

Με το παρακάτω code snippet διαγράφουμε την εγγραφή με ArtistID = 1, η οποία αντιστοιχεί στους 'Arctic Monkeys'.

Επειδή στον κώδικα της ΒΔ έχουμε ορίσει περιορισμούς ON UPDATE CASCADE και ON DELETE CASCADE, η διαγραφή του συγκεκριμένου καλλιτέχνη θα οδηγήσει αυτόματα και στη διαγραφή όλων των σχετικών εγγραφών σε άλλους πίνακες που εξαρτώνται από αυτό το ArtistID.

```
DELETE FROM artist
WHERE
    ArtistID = 1;-- diagrafoume to band me id = 1 opou einai oi Arctic Monkeys sto band
```

Αποτέλεσμα εκτέλεσης του script :

BandID	BandName	FormationDate	DisbandDate	ArtistID
2	The Stone Roses	1983-01-01	2019-09-16	2
3	The Smiths	1982-01-01	1987-01-01	3
4	Radiohead	1985-01-01	NULL	4
5	The Rolling Stones	1962-07-12	NULL	5
6	Green Day	1987-01-01	NULL	6
7	Pulp	1978-01-01	NULL	7
8	Suede	1989-01-01	NULL	8
9	ABBA	1972-01-01	1982-01-01	9
10	The Kinks	1963-01-01	1997-01-01	10
NULL	NULL	NULL	NULL	NULL

Φιγούρα 18: Εγγραφές του πίνακα band μετά την αλλαγή

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cel

AlbumID	Title	ArtistID	GenreID	CompanyID	ProducerID
5	The Stone Roses	2	11	6	9
6	Meat Is Murder	3	14	4	7
7	Kerplunk	6	12	45	30
8	Ultraviolence	17	14	7	15
9	Sumbarine	11	14	1	41
10	Suede	15	7	12	7
11	Every Day Is Like Sunday	14	18	45	27
12	American Idiot	26	38	45	19
13	Kaiti Garbi - Best Of	29	3	6	18
14	H atmosfaira hlektrismenh	7	8	17	38
15	Trifulla Tetrafulla	20	8	19	25
17	Oso Zw Mazw	7	12	8	10
18	Mia Thalassa Galazia	12	36	38	49
NULL	NULL	NULL	NULL	NULL	NULL

Φιγούρα 19: Εγγραφές του πίνακα album μετά την αλλαγή

Παρατηρούμε λοιπόν ότι, μετά τη διαγραφή της εγγραφής με **ArtistID = 1** (Arctic Monkeys), διαγράφηκαν αυτόματα και όλες οι σχετικές εγγραφές σε άλλους πίνακες που εξαρτώνται από αυτό το πεδίο. Αυτό συμβαίνει επειδή έχουμε ορίσει τον περιορισμό ON DELETE CASCADE, ο οποίος διασφαλίζει ότι όταν διαγράφεται ένας καλλιτέχνης (ή απλά parent key), καθαρίζονται ταυτόχρονα και όλα τα δεδομένα που συνδέονται μαζί του (π.χ. album, track κλπ.) (ή απλά child key). Με αυτόν τον τρόπο, η βάση δεδομένων διατηρείται συνεπής και δεν μένουν 'ορφανές' εγγραφές.

- **Παράδειγμα 3**

Παρακάτω βλέπουμε τον πίνακα track καθώς και τις στήλες που έχουμε δημιουργήσει (χρησιμοποιούμε την εντολή *describe track*; για να φέρουμε αυτή την λίστα στην οθόνη)

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
TrackID	int	NO	PRI	NULL	auto_increment
Title	varchar(100)	NO		NULL	
AlbumID	int	NO	MUL	NULL	
TrackLength	time	NO		NULL	
TrackNo	int	YES		NULL	
Lyrics	text	YES		NULL	

Φιγούρα 20: Δομή του track

```
describe track; -- perigrafh tou pinaka gia na kanw allages
alter table track drop column TrackNo;-- diagrafh ths sthlhs TrackNo
SELECT
    *
FROM
    track;
```

Με την εντολή DROP COLUMN αφαιρούμε τη στήλη 'TrackNo' από τον πίνακα. Η χρήση της εντολής αυτής είναι ιδιαίτερα χρήσιμη, καθώς μας δίνει τη δυνατότητα να τροποποιούμε εύκολα τη δομή ενός πίνακα, αφαιρώντας πεδία που δεν χρειάζονται πλέον, χωρίς να επηρεάζεται η υπόλοιπη λειτουργία της ΒΔ και έτσι αποφεύγονται κρίσιμα σφάλματα.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
TrackID	int	NO	PRI	NULL	auto_increment
Title	varchar(100)	NO		NULL	
AlbumID	int	NO	MUL	NULL	
TrackLength	time	NO		NULL	
Lyrics	text	YES		NULL	

Φιγούρα 21: Δομή του track μετά την αφαίρεση της στήλης trackno

- **Παράδειγμα 4**

Σε αυτό το παράδειγμα χρησιμοποιούμε την εντολή

```
alter table track rename column Title to Track_Title;-- allagh tou onomatos ths sthlhs
```

για να αλλάξουμε το όνομα της στήλης από ‘title’ σε ‘track_title’

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

TrackID	Title	AlbumID	TrackLength	TrackNo	Lyrics
4	2,000 Light Years Away	7	00:02:56	1	... I sit alone in my bedroom Staring at the walls I've been up all damn night long My pulse is speeding, my love is yearning I hold my breath and close my eyes And dream about her 'Cause she's 2000 light years awa...
5	Kalimera Kenouria Mou Agapi	8	00:02:35	7	Kai tora pou xwrisame ti thes; Ti vgainei pia an ftaio i an ftais; Kainourgia agapi irthe stin kardia Kai niwthw n' agapw proti fora Kalimera, kainourgia mou agapi Kalimera, kainourgia zoi Apo simera telos ta lathi Ftanei na '...
6	Fyge se parakalw	12	00:03:48	10	Ti allo prepei na sou po Lypamai pia de s' agapo Ein' ametrita ta lathi sou Kai ateliota ta pathi sou Kourastika na s' agapo Kai na se sygxoro Fyge se parakalw Fyge se parakalw Tora irthe i seira sou Gia na klapsei i kardia...
7	Ierosylia	5	00:05:05	8	Imoun sti zoi sou Kati san thriskeia Tora an fygeis tha 'nai Ierosylia Skepsou ksana Pia kathara Ti pas kaneis Ti grothia kano egw siderenia Tin psyhi san petra skliri Gia n' antekso tin pikra pou pira Otan mou 'pes pos fev...
8	Judas	17	00:05:00	5	Oh, whoa-whoa, oh-oh I'm in love with Judas, Judas Oh-oh-oh-oh-oh I'm in love with Judas, Judas Judas, Juda-ah-ah, Judas, Juda-ah-ah Judas, Juda-ah-ah, Judas, Gaga Judas, Juda-ah-ah, Judas, Juda-ah-ah Juda...
9	Ma Zw	7	00:02:02	5	Mesa mou niwthw paraksena ithela na ksera pou tha me vrw Me exasa kapou anamesa ston katw kosmo kai ton ourano Soma keno Poulisa pali psyxi mes sti zali tou erota misotimis Ax eafte mou exthre kai aderfe mo...
10	Ola Gia Ola	5	00:08:09	12	NULL
11	Electricity	10	00:04:50	3	We got a love thats cold as stone We got a love from our violent homes We got a love and it got no name We kiss our love with lips like pain We got a lotta electricity We got a love like AC DC We got a love and it g...
12	Never Had No One Ever	12	00:02:33	6	When you walk without ease On these streets where you were raised I had a really bad dream It lasted 20 years, 7 months and 27 days And I know alone that I never, ever oh Had no one ever Now Im outside your...
14	Ksexasmeni Enilikiosi	11	00:02:11	7	Soulja Boy Tell Em Hopped up out the bed, turn my swag on Took a look in the mirror, said, "Whats up?" Yeah, Im gettin money, oh I hopped up out the bed, turn my swag on Took a look in the mirror, said, "Wha...
15	Falling Stars (Chugu Bam Ba...	15	00:01:11	1	NULL
16	Straight To Hell	13	00:06:56	14	Tracksuit lefka Pan ap ola kathari psyxi Metraw mavra lefta Poios den agapaei to xarti, huh Cest la vie (Cest la vie) Pow-pow-pow-pow, BCC Cest la vie (Cest la vie) Bcc, Rack attack Tracksuit lefka Pan ap ola kathari p...
18	Crying in C Sharp	18	00:15:00	4	NULL
19	Fading Lights	12	00:04:22	2	NULL
20	Silent Echoes	18	00:03:58	5	NULL
22	Shattered Dreams	14	00:04:45	7	NULL
23	Lost in Time	9	00:03:36	3	NULL
25	Echoes of You	8	00:03:59	2	Whispers in the dark, echoes of your heart
26	Falling Down	5	00:04:45	3	Counting every falling star, wishing you were not so far
27	Broken Compass	9	00:05:01	4	Lost without direction, my broken compass leads me home
28	Silver Rain	15	00:03:50	5	Silver rain falls on my face, washing away the pain

Φιγούρα 22: Δομή του track πριν τις αλλαγές

Μετά την εκτέλεση της εντολής ALTER έχουμε το αποτέλεσμα παρακάτω:

Result Grid | Filter Rows:

TrackID	Track_Title
4	2,000 Light Years Away
5	Kalimera Kenouria Mou Agapi
6	Fyge se parakalw
7	Ierosylia
8	Judas
9	Ma Zw
10	Ola Gia Ola
11	Electricity
12	Never Had No One Ever
14	Ksexasmeni Enilikiosi
15	Falling Stars (Chugu Bam Ba...
16	Straight To Hell
18	Crying in C Sharp
19	Fading Lights
20	Silent Echoes
22	Shattered Dreams
23	Lost in Time
25	Echoes of You
26	Falling Down
27	Broken Compass
28	Silver Rain

Φιγούρα 23: Δομή του track μετά τις αλλαγές στο όνομα της στήλης

Κεφάλαιο 2 – Stored Procedures

Απαίτηση 3.1.3.1 - Υπολογισμός Βαθμού Venue

Λαμβάνοντας υπόψιν τις επιπρόσθετες απαιτήσεις που υπήρχαν , δημιουργήθηκαν το table που διατηρεί τα χαρακτηριστικά των venues. Οι στήλες που προστέθηκαν αναλύθηκαν στο κεφάλαιο 1 (Βλ. **Επιπλέον πίνακες**) μαζί με τον λόγο που επιλέχθηκε αυτός ο σχεδιασμός και σχετική επεξήγησή.

Πριν προχωρήσουμε στην εκτέλεση παραδειγμάτων καθώς και στην παρουσίαση του κώδικα για το stored procedure , γεμίζουμε τον πίνακα venue με μερικά venues (3 στον αριθμό , αρκετά ώστε να κάνουμε έναν μικρό έλεγχο σε αυτό το σημείο).

```
SET @current_dba := 'admin';
-- eisagwgh venues
INSERT INTO venue (VenueName, Capacity , CompletedConcerts, YearsOperating , IsAvailable) VALUES ('Old Trafford' , 150000 , 150,
100 , 1);
INSERT INTO venue (VenueName, Capacity , CompletedConcerts, YearsOperating , IsAvailable) VALUES ('MSG' , 200000 , 500, 75 , 1);
INSERT INTO venue (VenueName, Capacity , CompletedConcerts, YearsOperating , IsAvailable) VALUES ('Manchester Apollo' , 3500 , 600
, 120,  1);
```

Εμφανίζουμε τον πίνακα με τις εγγραφές:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content

VenueID	VenueName	Capacity	CompletedConcerts	YearsOperating	IsAvailable
1	Old Trafford	150000	150	100	1
2	MSG	200000	500	75	1
3	Manchester Apollo	3500	600	120	1
NULL	NULL	NULL	NULL	NULL	NULL

Φιγούρα 24: Εγγραφές venue

Ο κώδικας για το stored procedure είναι ο παρακάτω :

```
-- Απαίτηση 3.1.3.1
-- Stored procedure η οποία παίρνει ως όρισμα το id ενός χώρου διεξαγωγής
-- και θα υπολογίζει και θα επιστρέφει το βαθμό του με βάση τον Πίνακα 2 της
-- Απαίτησης 3.1.2.2.

DROP PROCEDURE IF EXISTS CalculateVenueScore$
DELIMITER $
CREATE PROCEDURE CalculateVenueScore(IN venue_id INT) -- here, we use a variable named 'venue_id' , we type in the exact id we
wanna search for
                                                                                                    -- and then we fetch the results
where venue_id = VenueID from the venue table
BEGIN -- fetch the exact info the venue table holds for this specific 'venue_id' we are searching for
    SELECT
        VenueID,
        VenueName,
        Capacity,
        CompletedConcerts,
        YearsOperating,
        IsAvailable,

        -- Calculate the exact score as the table explains
        (FLOOR(Capacity / 1000) * 1 +
        FLOOR(CompletedConcerts / 100) * 3 +
        YearsOperating * 2) AS VenueScore
    FROM venue
    WHERE VenueID = venue_id;
END$
DELIMITER ;
```

Η υλοποίηση όσον αφορά τον προγραμματιστικό τρόπο σκέψης δεν ήταν περίπλοκη. Δημιουργούμε το procedure με όνομα *CalculateVenueScore* που δέχεται ως όρισμα το επιθυμητό ID του χώρου (VenueID) για το οποίο θέλουμε να υπολογίσουμε το σκορ.

Κριτήρια	Βαθμολογία
Για κάθε 1.000 άτομα χωρητικότητα	1
Για κάθε 100 πραγματοποιημένες συναυλίες	3
Για κάθε χρόνο λειτουργίας	2

ΠΙΝΑΚΑΣ 2. ΒΑΘΜΟΛΟΓΙΑ ΑΝΑ ΚΡΙΤΗΡΙΟ

Στην συνέχεια κάνουμε fetch όλα τα στοιχεία από τον πίνακα venue και τα χρησιμοποιούμε σύμφωνα με τον Πίνακα 2 . Ορίζουμε ως **VenueScore** αυτό το code block:

```
(FLOOR(Capacity / 1000) * 1 +
        FLOOR(CompletedConcerts / 100) * 3 +
        YearsOperating * 2) AS VenueScore
```

το οποίο αυτό που κάνει ουσιαστικά είναι να μεταφράσει σε συνάρτηση τον πίνακα 2.

Προσοχή όμως.. Χρησιμοποιούμε το FLOOR διότι δεν θέλουμε να υπολογίζουμε δεκαδικούς αριθμούς. Ψάχνουμε πάντοτε τον κοντινότερο ακέραιο και χρησιμοποιούμε εκείνον.

Score για VenueID = 1 ('Old Trafford')

Capacity	CompletedConcerts	YearsOperating	Αναμενόμενο Score
150000	150	100	353

Καλούμε το Stored Procedure με την εντολή

```
CALL CalculateVenueScore(1);
```

Και προκύπτει ότι :

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

VenueID	VenueName	Capacity	CompletedConcerts	YearsOperating	IsAvailable	VenueScore
1	Old Trafford	150000	150	100	1	353

Φιγούρα 25: VenueScore του Old Trafford

Score για VenueID = 2 ('MSG')

Capacity	CompletedConcerts	YearsOperating	Αναμενόμενο Score
200000	500	75	365

Καλούμε το stored procedure με την εντολή:

```
CALL CalculateVenueScore(2);
```

Και προκύπτει ότι

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

VenueID	VenueName	Capacity	CompletedConcerts	YearsOperating	IsAvailable	VenueScore
2	MSG	200000	500	75	1	365

Φιγούρα 26: VenueScore του MSG

Score για VenueID = 3 ('Manchester Apollo)

Capacity	CompletedConcerts	YearsOperating	Αναμενόμενο Score
3500	600	120	261

Καλούμε το stored procedure με την εντολή:

```
CALL CalculateVenueScore(3);
```

Και προκύπτει ότι

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

VenueID	VenueName	Capacity	CompletedConcerts	YearsOperating	IsAvailable	VenueScore
3	Manchester Apollo	3500	600	120	1	261

Φιγούρα 27: VenueScore του Apollo

Απαίτηση 3.1.3.2 – Προγραμματισμός Συναυλιών

Stored procedure στην οποία θα δίνονται ως ορίσματα εισόδου το id ενός καλλιτέχνη, μια ημερομηνία και ένας χαρακτήρας (‘i’, ‘c’ ή ‘a’).

Κώδικας για το stored procedure :

```
-- Απαίτηση 3.1.3.2
-- Stored procedure στην οποία θα δίνονται ως ορίσματα εισόδου το id ενός
-- καλλιτέχνη, μια ημερομηνία και ένας χαρακτήρας (‘i’, ‘c’ ή ‘a’). (πιο αναλυτικά στην εκφώνηση)

DELIMITER //

CREATE PROCEDURE ConcertManagement (
    IN p_ArtistID INT,
    IN p_ConcertDate DATE,
    IN p_Action CHAR(1)
)
BEGIN
    DECLARE v_ConcertID INT;
    DECLARE v_Status ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED');

SELECT
    ConcertID, ConcertStatus
INTO v_ConcertID , v_Status FROM
    concert
WHERE
    ArtistID = p_ArtistID
        AND ConcertDate = p_ConcertDate
LIMIT 1;

    -- Actions based on our needs
    IF p_Action = 'i' THEN
        IF v_ConcertID IS NOT NULL THEN
            IF v_Status = 'SCHEDULED' THEN
                SELECT 'There is a concert already schedule for this date' AS Message;
            ELSEIF v_Status = 'CANCELLED' THEN
                SELECT 'This date"s concert has already been cancelled before' AS Message;
            ELSE
                SELECT 'Concert is completed' AS Message;
            END IF;
        ELSE
            -- Schedule new concert (VenueID = 1, ConcertCapacity = 1000)
            -- Check if venue exists and is available
            IF EXISTS(SELECT 1 FROM venue WHERE VenueID = 1 AND IsAvailable = 1) THEN
                INSERT INTO concert (ArtistID, VenueID, ConcertDate, ConcertCapacity, ConcertStatus)
                VALUES (p_ArtistID, 1, p_ConcertDate, 1000, 'SCHEDULED');

                SELECT 'Concert has been scheduled' AS Message;
            ELSE
                SELECT 'Venue is not available for scheduling' AS Message;
            END IF;
        END IF;

    ELSEIF p_Action = 'c' THEN
        IF v_ConcertID IS NULL THEN
            SELECT 'No concert found for this date.' AS Message;
        ELSE
            IF v_Status = 'CANCELLED' THEN
                SELECT 'Concert already cancelled' AS Message;
            ELSEIF v_Status = 'SCHEDULED' THEN
                UPDATE concert
                SET ConcertStatus = 'CANCELLED'
                WHERE ConcertID = v_ConcertID;
SELECT 'Concert cancelled successfully' AS Message;
            ELSE
                SELECT 'Concert already completed and cant be cancelled.' AS Message;
            END IF;
        END IF;

    ELSEIF p_Action = 'a' THEN
        IF v_ConcertID IS NULL THEN
            SELECT 'There is no cancelled concert' AS Message;
        ELSE
            IF v_Status = 'CANCELLED' THEN
                UPDATE concert
                SET ConcertStatus = 'SCHEDULED'
                WHERE ConcertID = v_ConcertID;
SELECT 'Cancelled concert is back up again' AS Message;
            ELSEIF v_Status = 'SCHEDULED' THEN
                SELECT 'Concert is active' AS Message;
            ELSE
                SELECT 'Concert is already completed' AS Message;
            END IF;
        END IF;

    ELSE
        SELECT 'Invalid type of action. Please only use:  ''i'', ''c'' or ''a''.' AS Message;
    END IF;
END //

DELIMITER ;
```

Προχωρώντας στο testing :

Αρχικά ο πίνακας concert περιέχει τα εξής δεδομένα:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ConcertID	ArtistID	VenueID	ConcertDate	ConcertCapacity	TicketsSold	ConcertStatus	IsArchived
6	1	3	2027-10-07	1500	0	CANCELLED	1
7	1	1	2025-09-03	2500	0	SCHEDULED	1
8	1	2	2025-09-30	15000	0	SCHEDULED	1
9	1	3	2025-10-17	2500	0	SCHEDULED	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Φιγούρα 28: Περιεχόμενα του πίνακα concert

Όπως παρατηρούμε, το ArtistID = 1 έχει **ήδη** 3 συναυλίες SCHEDULED

Για την περίπτωση με 'i' :

- **Schedule συναυλία για ArtistID = 1 που έχει ήδη 3 προγραμματισμένες**

Τρέχουμε την εντολή:

```
CALL ConcertManagement(1, '2025-09-10', 'i');
```

Παρατηρούμε ότι η SQL πετάει error λόγω του trigger που έχουμε ορίσει στην Απαίτηση 3.1.4.2 (Βλ. **Απαίτηση 3.1.4.2**) η οποία δεν επιτρέπει ένας καλλιτέχνης να έχει πάνω από 3 συναυλίες προγραμματισμένες.

```
25  18:21:35  select * from concert LIMIT 0, 2000                    4 row(s) returned
26  18:21:59  CALL ConcertManagement(1, '2025-09-10', 'i')           Error Code: 1644. Cannot schedule a new concert: Artist already has 3 scheduled concerts.
27  18:23:21  select * from concert LIMIT 0, 2000                    4 row(s) returned
```

Φιγούρα 29: Αποτέλεσμα SQL

- **Προγραμματισμός συναυλίας την ίδια μέρα που έχει συναυλία ο ArtistID = 1**

Τρέχουμε την εντολή :

```
CALL ConcertManagement(1,'2025-09-03' ,'i');
```

Αποτέλεσμα:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Message
There is a concert already schedule for this date

Φιγούρα 30: Αποτέλεσμα SQL

- **Προγραμματισμός συναυλίας για τον καλλιτέχνη με ArtistID = 2 .**

Επειδή ο ArtistID = 2 δεν έχει καμία συναυλία προγραμματισμένη , καλούμε το stored procedure με την εντολή:

```
CALL ConcertManagement(2,'2025-09-03' ,'i');
```

Το αποτέλεσμα είναι το εξής :

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Message
Concert has been scheduled

Φιγούρα 31: Αποτέλεσμα SQL

Συγκεντρωτικά ο πίνακας concert μετά την εκτέλεση των παραπάνω:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ConcertID	ArtistID	VenueID	ConcertDate	ConcertCapacity	TicketsSold	ConcertStatus	IsArchived
6	1	3	2027-10-07	1500	0	CANCELLED	1
7	1	1	2025-09-03	2500	0	SCHEDULED	1
8	1	2	2025-09-30	15000	0	SCHEDULED	1
9	1	3	2025-10-17	2500	0	SCHEDULED	1
10	12	1	2025-09-30	1000	0	SCHEDULED	1
11	2	1	2025-09-03	1000	0	SCHEDULED	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Φιγούρα 32: Πίνακας concert μετά την εκτέλεση του script

Για την περίπτωση με 'c':

Ο πίνακας είναι ήδη κάπως έτσι :

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ConcertID	ArtistID	VenueID	ConcertDate	ConcertCapacity	TicketsSold	ConcertStatus	IsArchived
6	1	3	2027-10-07	1500	0	CANCELLED	1
7	1	1	2025-09-03	2500	0	SCHEDULED	1
8	1	2	2025-09-30	15000	0	SCHEDULED	1
9	1	3	2025-10-17	2500	0	SCHEDULED	1
10	12	1	2025-09-30	1000	0	SCHEDULED	1
11	2	1	2025-09-03	1000	0	SCHEDULED	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Φιγούρα 33: Πίνακας concert την δεδομένη χρονική στιγμή

- **Ακύρωση συναυλίας του ArtistID = 1**

Εκτελούμε το παρακάτω CALL :

```
- -- Case 3: Akurwsh sunaulias
- CALL ConcertManagement(1, '2025-09-03', 'c'); -- akurwsh sunaulias tou ArtistID = 1 me
  concertdate = '2025-09-05'
```

Αποτέλεσμα μετά την κλήση της συνάρτησης :

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Message
Concert cancelled successfully

Φιγούρα 34: Αποτέλεσμα SQL

Εμφανίζουμε τον πίνακα concert ξανά και παρατηρούμε ότι όντως ακυρώθηκε η συναυλία

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ConcertID	ArtistID	VenueID	ConcertDate	ConcertCapacity	TicketsSold	ConcertStatus	IsArchived
6	1	3	2027-10-07	1500	0	CANCELLED	1
7	1	1	2025-09-03	2500	0	CANCELLED	1
8	1	2	2025-09-30	15000	0	SCHEDULED	1
9	1	3	2025-10-17	2500	0	SCHEDULED	1
10	12	1	2025-09-30	1000	0	SCHEDULED	1
11	2	1	2025-09-03	1000	0	SCHEDULED	1

Φιγούρα 35: Πίνακας concert μετά την αλλαγή

- **Ακύρωση ήδη ακυρωμένης συναυλίας του ArtistID = 1**

Εκτελούμε το παρακάτω script:

```
-- Case 4: Akurwsh hdh akurwmenhs sunaulias (prepei na leei "already cancelled")
   CALL ConcertManagement(1, '2025-09-03', 'c');
   CALL ConcertManagement(1, '2027-10-07', 'c');
```

Αποτέλεσμα:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Message
Concert already cancelled

Φιγούρα 36: Αποτέλεσμα SQL

και στα δυο CALL.

Για την περίπτωση με 'a':

Ο πίνακας concert είναι ήδη έτσι:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ConcertID	ArtistID	VenueID	ConcertDate	ConcertCapacity	TicketsSold	ConcertStatus	IsArchived
6	1	3	2027-10-07	1500	0	CANCELLED	1
7	1	1	2025-09-03	2500	0	CANCELLED	1
8	1	2	2025-09-30	15000	0	SCHEDULED	1
9	1	3	2025-10-17	2500	0	SCHEDULED	1
10	12	1	2025-09-30	1000	0	SCHEDULED	1
11	2	1	2025-09-03	1000	0	SCHEDULED	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Φιγούρα 37: Πίνακας concert την δεδομένη χρονική στιγμή

Τρέχουμε την εντολή (δεν χρειάζεται να εξηγήσουμε κάτι , υπάρχουν και τα comments μέσα στο script)

```
-- Case 5: Reschedule thn akurwmenh sunaulia
CALL ConcertManagement(1, '2025-09-03', 'a'); -- prin eixame akurwsei thn sunaulia ekeinh thn hmera ,
kai twra prospathoume na thn energopoihsoume ksana
```

Αποτέλεσμα :

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Message
Cancelled concert is back up again

Φιγούρα 38: Αποτέλεσμα SQL

Με κλήση του πίνακα concert παρατηρούμε ότι:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ConcertID	ArtistID	VenueID	ConcertDate	ConcertCapacity	TicketsSold	ConcertStatus	IsArchived
6	1	3	2027-10-07	1500	0	CANCELLED	1
7	1	1	2025-09-03	2500	0	SCHEDULED	1
8	1	2	2025-09-30	15000	0	SCHEDULED	1
9	1	3	2025-10-17	2500	0	SCHEDULED	1
10	12	1	2025-09-30	1000	0	SCHEDULED	1
11	2	1	2025-09-03	1000	0	SCHEDULED	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Φιγούρα 39: Αλλαγή του status από Cancelled σε Scheduled

```
-- Case 6: Energopoihsh hdh energh sunaulia
CALL ConcertManagement(1, '2025-09-03', 'a'); -- energopoioume to parapanw concert ksana
```

Αποτέλεσμα μετά την εκτέλεση :

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Message
Concert is active

Φιγούρα 40: Αποτέλεσμα SQL

Παραδείγματα Error Handling :

- **Ακύρωση συναυλίας που δεν υπάρχει**

```
-- Case 7: Cancel sunaulia pou den uparxei
CALL ConcertManagement(101, '2025-09-20', 'c');
```

Αποτέλεσμα εκτέλεσης:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Message
There is no cancelled concert

Φιγούρα 41: Αποτέλεσμα SQL

- **Ενεργοποίηση ακυρωμένης συναυλίας που δεν υπάρχει καν σαν εγγραφή**

```
-- Case 8: Activate akurwmenh sunaulia pou den uparxei
ousiastika
CALL ConcertManagement(101, '2025-09-20', 'a');
```

Αποτέλεσμα ίδιο με παραπάνω.

- **Λάθος επιλογή**

```
-- Case 9: Lathos epilogh
CALL ConcertManagement(101, '2025-09-10', 'x');
```

Αποτέλεσμα:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Message
Invalid type of action. Please only use: 'i', 'c' or 'a'.

Φιγούρα 42: Αποτέλεσμα SQL

Απαίτηση 3.1.3.3 - Υπολογισμός Βαθμού Venue

- Για αυτή την απαίτηση αρχικά δημιουργούμε stored procedure ***GetConcertVenueInfo*** που δέχεται ως όρισμα το ID της συναυλίας (ConcertID) που θέλουμε να αναζητήσουμε , την απαιτούμενη χωρητικότητα για να γίνει η συναυλία και επιστρέφουμε το ID του venue (VenueID) καθώς και το capacity που έχει ο χώρος.
- Μέσα σε αυτό το procedure έχουμε βάλει IF conditions που ικανοποιούν την απαίτηση που ζητάει να ελέγξουμε αν δεν υπάρχει συναυλία με το συγκεκριμένο id ή είναι ακυρωμένη. Αν ισχύει , τότε επιστρέφεται ως id χώρου διεξαγωγής (VenueID) η τιμή NULL και η χωρητικότητα γίνεται 0.
- Επίσης υπάρχει condition για το αν έχει κλειστεί χώρος διεξαγωγής για τη συγκεκριμένη συναυλία. Αν αληθεύει, τότε επιστρέφει το id και την χωρητικότητα του χώρου διεξαγωγής.
- Αν όμως δεν ικανοποιούνται τα παραπάνω 2 conditions , στρεφόμαστε στο 3ο το οποίο είναι η αναζήτηση χώρου διεξαγωγής που είναι διαθέσιμος και έχει χωρητικότητα 10% μεγαλύτερης από την απαιτούμενη (δηλαδή απαιτούμενη * 1.1)
- Γι' αυτό, δημιουργούμε βοηθητική stored procedure με όνομα ***AssignBestVenueForConcert*** που ουσιαστικά ψάχνει το καλύτερο score ανάμεσα στα venues σύμφωνα με τον Πίνακα 2 , ώστε από πολλούς διαθέσιμους χώρους διεξαγωγής , να επιλεχθεί ο καταλληλότερος (δηλαδή αυτός που έχει το μεγαλύτερο score).Για την υλοποίηση του *AssignBestVenueForConcert* κατασκευάζουμε έναν CURSOR που αυτό που κάνει ουσιαστικά είναι να σαρώνει τον πίνακα για να βρίσκει διαθέσιμους χώρους (χώρους δηλαδή που έχουν πεδίο IsAvailable = 1) και capacity 10% μεγαλύτερο από το απαιτούμενο. Επίσης ο CURSOR ψάχνει τις εγγραφές που ΔΕΝ είναι 'SCHEDULED' και το VenueID ΔΕΝ είναι NULL. Όλο αυτό γίνεται μέσα σε ένα loop (ακριβώς με την ίδια λογική που κάναμε σε άλλα μαθήματα προγραμματισμού)
- Κατά την σάρωση των χώρων λειτουργεί μια μεταβλητή counter με όνομα **max_score** που έχει αρχικοποιηθεί με 0 hardcoded. Την max_score την χρησιμοποιούμε , επειδή θέλουμε να βρούμε ανάμεσα στους διαθέσιμους χώρους αυτόν που έχει το μεγαλύτερο score. Άρα , για να το πέτυχουμε αυτό, πρέπει συνεχώς να κάνουμε συγκρίσεις ανάμεσα στην τωρινή τιμή του max_score και την αμέσως προηγουμένη , ώστε να ξέρουμε ποια τιμή εν τέλει θα είναι η μεγαλύτερη και συνεπώς η καταλληλότερη για την συναυλία. Το max_score εννοείται υπολογίζεται όπως ορίζει ο Πίνακας 2 και όπως έχει υλοποιηθεί και στην **Απαίτηση 3.1.3.1 - Υπολογισμός Βαθμού Venue**

Αναλυτικός κώδικας SQL για την συγκεκριμένη απαίτηση :

```
-- Απαίτηση 3.1.3.3
-- Stored procedure στην οποία δίνεται ως ορίσματα εισόδου το id μιας
-- συναυλίας και απαιτούμενη χωρητικότητα για τη συναυλία και επιστρέφει σε
-- μια μεταβλητή εξόδου το id χώρου διεξαγωγής και σε μια δεύτερη, την
-- χωρητικότητα του.

DELIMITER $$
DROP PROCEDURE IF EXISTS GetConcertVenueInfo$$
CREATE PROCEDURE GetConcertVenueInfo(
    IN concert_id INT,
    IN required_capacity INT,
    OUT venue_id INT,
    OUT venue_capacity INT
)
BEGIN
    DECLARE concert_exists INT DEFAULT 0;
    DECLARE concert_status ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED');
    DECLARE assigned_venue_id INT DEFAULT NULL;

    -- Έλεγχος αν υπάρχει η συναυλία και η κατάστασή της
    SELECT COUNT(*), ConcertStatus, VenueID
    INTO concert_exists, concert_status, assigned_venue_id
    FROM concert
    WHERE ConcertID = concert_id;

    -- Αν δεν υπάρχει συναυλία με το συγκεκριμένο ID ή είναι ακυρωμένη
    IF concert_exists = 0 OR concert_status = 'CANCELLED' THEN
        SET venue_id = NULL;
        SET venue_capacity = 0;
    ELSE
        -- Αν έχει ήδη κλειστεί χώρος διεξαγωγής για τη συγκεκριμένη συναυλία
        IF assigned_venue_id IS NOT NULL THEN
            -- Επιστρέφουμε το ID και την χωρητικότητα του ήδη κλεισμένου χώρου
            SELECT assigned_venue_id, v.Capacity
            INTO venue_id, venue_capacity
            FROM venue v
            WHERE v.VenueID = assigned_venue_id;
        ELSE
            -- Αν δεν έχει κλειστεί χώρος, βρίσκουμε τον καλύτερο διαθέσιμο
            -- χρησιμοποιώντας την ίδια λογική με την AssignBestVenue
            CALL AssignBestVenueForConcert(concert_id, required_capacity, venue_id, venue_capacity);
        END IF;
    END IF;
END$$
```

```
-- Βοηθητική stored procedure για εύρεση καλύτερου χώρου
DROP PROCEDURE IF EXISTS AssignBestVenueForConcert$$
CREATE PROCEDURE AssignBestVenueForConcert(
    IN concert_id INT,
    IN required_capacity INT,
    OUT selected_venue_id INT,
    OUT selected_venue_capacity INT
)
BEGIN
    DECLARE max_score INT DEFAULT 0;
    DECLARE v_venue_id INT;
    DECLARE v_capacity INT;
    DECLARE v_completed_concerts INT;
    DECLARE v_years_operating INT;
    DECLARE v_current_score INT;
    DECLARE done INT DEFAULT FALSE;

    -- Cursor για εύρεση κατάλληλων χώρων
    DECLARE venue_cursor CURSOR FOR
        SELECT VenueID, Capacity, CompletedConcerts, YearsOperating
        FROM venue
        WHERE IsAvailable = 1
        AND Capacity >= (required_capacity * 1.1)  -- 10% περισσότερη χωρητικότητα
        AND VenueID NOT IN (
            SELECT VenueID FROM concert
            WHERE ConcertStatus = 'SCHEDULED'
            AND VenueID IS NOT NULL
            AND ConcertID != concert_id  -- Εξαιρούμε την τρέχουσα συναυλία
        );

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Αρχικοποίηση
    SET selected_venue_id = NULL;
    SET selected_venue_capacity = 0;
    SET max_score = 0;

    OPEN venue_cursor;

    venue_loop: LOOP
        FETCH venue_cursor INTO v_venue_id, v_capacity, v_completed_concerts, v_years_operating;

        IF done THEN
            LEAVE venue_loop;
        END IF;

        -- Υπολογισμός βαθμολογίας
        SET v_current_score =
            FLOOR(v_capacity / 1000) * 1 +
            FLOOR(v_completed_concerts / 100) * 3 +
            v_years_operating * 2;

        -- Επιλογή χώρου με μεγαλύτερη βαθμολογία
        IF v_current_score > max_score THEN
            SET max_score = v_current_score;
            SET selected_venue_id = v_venue_id;
            SET selected_venue_capacity = v_capacity;
        END IF;

    END LOOP;

    CLOSE venue_cursor;

    -- Αν δεν βρέθηκε κατάλληλος χώρος
    IF selected_venue_id IS NULL THEN
        SET selected_venue_capacity = 0;
    END IF;
END$$

DELIMITER ;
```

Testing και Screenshot από την εκτέλεση

Πριν ξεκινήσουμε το testing για αυτό το κομμάτι, καθαρίζουμε τους πίνακες concert και venue με την εντολή *DELETE FROM concert* και *DELETE FROM venue* αντίστοιχα , ώστε να έχουμε καθαρούς πίνακες για να δημιουργήσουμε τα κατάλληλα samples. Έπειτα προσθέτουμε μερικές εγγραφές στους πίνακες και ξεκινάμε την διαδικασία. Το script για τα παραπάνω είναι το εξής :

```
DELETE FROM concert;
DELETE FROM venue;
-- insert into dba (Username , Start_Date) values ('admin' , '2020-01-01');
SET @current_dba := 'admin';
-- Προσθήκη venues
INSERT INTO venue (VenueID, VenueName, Capacity, IsAvailable, CompletedConcerts, YearsOperating) VALUES
(1, 'Small Hall',     500, 1,  20,  2),   -- Μικρός χώρος
(2, 'Medium Hall',  2000, 1, 150,  5),   -- Μεσαίος χώρος
(3, 'Big Arena'   , 3000, 1, 300, 10),   -- Μεγάλος χώρος
```

```
(4, 'Closed Hall',   2500, 0,  50,  3);   -- Μη διαθέσιμος

-- Συναυλία Ακυρωμένη
INSERT INTO concert (ConcertID, ArtistID, VenueID, ConcertDate, ConcertStatus)
VALUES (101, 1, NULL, '2025-10-01',  'CANCELLED');

-- Συναυλία με ήδη κλεισμένο χώρο
INSERT INTO concert (ConcertID, ArtistID, VenueID, ConcertDate, ConcertStatus)
VALUES (102, 1, 2, '2025-10-05','SCHEDULED');

-- Συναυλία χωρίς χώρο (NULL στο VenueID)
INSERT INTO concert (ConcertID, ArtistID, VenueID, ConcertDate, ConcertStatus)
VALUES (103, 1, NULL, '2025-10-10', 'SCHEDULED');
```

- **Παράδειγμα 1 - Αν δεν υπάρχει συναυλία με το συγκεκριμένο id ή είναι ακυρωμένη**

Καλούμε την stored procedure με την εντολή:

```
CALL GetConcertVenueInfo(101, 1000, @venue_id, @venue_capacity);
SELECT @venue_id AS Venue_ID, @venue_capacity AS Venue_Capacity;
```

Και επιλέγουμε τα πεδία Venue_ID και Venue_Capacity. Η SQL επιστρέφει τις εξής τιμές

Venue_ID	Venue_Capacity
NULL	0

Φιγούρα 43: Αποτέλεσμα SQL

Το αποτέλεσμα είναι αναμενόμενο, καθώς η συναυλία με ID = 101 πρόκειται για μια ακυρωμένη (CANCELLED) άρα η απαίτησή μας επαληθεύεται.

- **Παράδειγμα 2 - Αν έχει κλειστεί χώρος διεξαγωγής για τη συγκεκριμένη συναυλία, επιστρέφει το id και την χωρητικότητά του χώρου διεξαγωγής.**

Καλούμε πάλι την stored procedure με την εντολή :

```
CALL GetConcertVenueInfo(102, 1000, @venue_id, @venue_capacity);
SELECT @venue_id AS Venue_ID, @venue_capacity AS Venue_Capacity;
```

Και το αποτέλεσμα που προκύπτει είναι:

Result Grid | Filter Rows:

Venue_ID	Venue_Capacity
2	2000

Φιγούρα 44: Αποτέλεσμα SQL

- **Παράδειγμα 3 - Αν δεν έχει κλειστεί χώρος διεξαγωγής για τη συγκεκριμένη συναυλία, τότε αναζητείται χώρος διεξαγωγής που είναι διαθέσιμος και με χωρητικότητα 10% μεγαλύτερης από την απαιτούμενη. Αν βρεθούν παραπάνω από ένας κατάλληλοι χώροι διεξαγωγής υπολογίζεται ο βαθμός για κάθε έναν από αυτούς με βάση τον Πίνακα 2 και επιστρέφεται το id και την χωρητικότητά του χώρου διεξαγωγής με το μεγαλύτερο βαθμό**

Καλούμε την stored procedure:

```
CALL GetConcertVenueInfo(103, 1500, @venue_id, @venue_capacity);
SELECT @venue_id AS Venue_ID, @venue_capacity AS Venue_Capacity;
```

Και το αποτέλεσμα είναι

Result Grid | Filter Rows:

Venue_ID	Venue_Capacity
3	3000

Φιγούρα 45: Αποτέλεσμα SQL

Το αποτέλεσμα είναι το ιδανικό και το επιθυμητό , μιας και έχει ο χώρος έχει capacity 10% μεγαλύτερο από τον απαιτούμενο (που είναι1500) και είναι διαθέσιμος χώρος (δηλαδή IsAvailable = 1).

Αν τώρα κάναμε μια μικρή τροποποίηση και είχαμε τους χώρους:

```
(2, 'Medium Hall',  3000, 1, 150,  5),   -- Μεσαίος χώρος
(3, 'Big Arena'   , 3000, 1, 300, 10),   -- Μεγάλος χώρος
```

Που ουσιαστικά έχουν την ίδια χωρητικότητα αλλά παρατηρούμε πως ο χώρος 3 έχει καλύτερο score από τον 2 με βάση τον Πίνακα 2. Επομένως και πάλι επιστρέφεται ο πίνακας 3 κατά την κλήση της stored procedure.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

VenueID	VenueName	Capacity	CompletedConcerts	YearsOperating	IsAvailable
1	Small Hall	500	20	2	1
2	Medium Hall	3000	150	5	1
3	Big Arena	3000	300	10	1
4	Closed Hall	2500	50	3	0

Φιγούρα 46: Νέες εγγραφές

```
33
34 •  CALL GetConcertVenueInfo(103, 1500, @venue_id, @venue_capacity);
35 •  SELECT @venue_id AS Venue_ID, @venue_capacity AS Venue_Capacity;
36
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Venue_ID	Venue_Capacity
3	3000

Φιγούρα 47: Αποτέλεσμα SQL

Απαίτηση 3.1.3.4 – Ευρετήρια (Indexes)

Για την απαίτηση αυτή , δημιουργήσαμε αρχικά ένα python script το οποίο έκανε generate 120000 εγγραφές (για να φανεί η διαφορά) σε μορφή JSON. Ύστερα , από το περιβάλλον του Workbench , κάνοντας δεξί κλικ στον πίνακα concert_history , κάναμε import[1] το JSON αρχείο που περιείχε τα δεδομένα

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

HistoricalID	First_Name	Last_Name	ConcertID	ArtistID	VenueID	ConcertDate	TicketsSold	ConcertStatus
1	David	Brown	1298	463	261	2014-11-20	19614	CANCELLED
2	Sarah	Brown	1242	108	157	2012-05-09	16743	CANCELLED
3	Emily	Johnson	124	338	80	2025-07-17	2387	COMPLETED
4	Sophia	Williams	1229	80	161	2020-05-21	26276	CANCELLED
5	Laura	Wilson	670	20	54	2008-12-26	1001	COMPLETED
6	Sarah	Brown	890	173	183	2015-03-11	35630	CANCELLED
7	Emily	Taylor	604	133	30	2010-07-31	48383	COMPLETED
8	James	Williams	1400	163	37	2010-08-20	17296	COMPLETED
9	Robert	Davis	1013	273	110	2024-12-26	9529	COMPLETED
10	Sarah	Miller	1337	292	192	2017-04-14	44463	COMPLETED
11	Michael	Miller	1900	441	109	2020-05-27	31692	CANCELLED
12	Michael	Miller	248	159	140	2005-12-06	7390	COMPLETED
13	Robert	Brown	163	198	159	2006-06-25	3673	COMPLETED
14	Emily	Miller	350	292	78	2013-01-19	40929	COMPLETED
15	Sophia	Johnson	897	90	148	2008-03-24	15173	COMPLETED
16	Laura	Miller	1075	169	74	2013-10-28	28869	CANCELLED
17	Emily	Brown	1510	33	25	2000-02-12	7003	COMPLETED
18	Sarah	Miller	1653	140	268	2024-12-25	9298	CANCELLED

Φιγούρα 48: Μερικά Δεδομένα από τον πίνακα concert_history

Επίσης δημιουργήσαμε τα παρακάτω Ευρετήρια (Indexes) για την εργαστηριακή άσκηση :

```
-- Index για την stored procedure GetArtistsByTicketRange (διάστημα εισιτηρίων)
CREATE INDEX idx_tickets_sold ON concert_history (TicketsSold);

-- Index για την stored procedure GetConcertDatesByVenue (χώρος διεξαγωγής)
```

[1] Το μόνο πρόβλημα που υπήρξε εδώ ήταν ότι η εφαρμογή έκανε hang και freezαρε αρκετές φορές παρόλο που δουλεύουμε σε υπολογιστή με CPU 4.5 GHz και 32GB RAM).

```
CREATE INDEX idx_venue_id ON concert_history (VenueID);

-- Σύνθετο index για βελτιστοποίηση JOINs με artist πίνακα
-- CREATE INDEX idx_artist_tickets ON concert_history (ArtistID, TicketsSold);

-- Index για ημερομηνία (χρήσιμο για χρονολογικές αναζητήσεις)
-- CREATE INDEX idx_concert_date ON concert_history (ConcertDate);
```

Α’ Ζητούμενο - Stored procedure στην οποία δίνονται ως παράμετροι εισόδου δύο διαφορετικά πλήθη εισιτηρίων (π.χ. 2.000 και 3.000) και εμφανίζει το ονόματα των καλλιτεχνών των οποίων οι συναυλίες είχαν πλήθος εισιτήριων ανάμεσα σε αυτό το διάστημα.

Κώδικας SQL :

```
-- Απαίτηση 3.1.3.4
-- α) Stored procedure για εύρεση καλλιτεχνών βάσει πλήθους εισιτηρίων
-- We use the index 'idx_tickets_sold' for quick search in the ticket range

DELIMITER $$

DROP PROCEDURE IF EXISTS GetArtistsByTicketRange$$
CREATE PROCEDURE GetArtistsByTicketRange(
    IN min_tickets INT,
    IN max_tickets INT
)
BEGIN
    SELECT First_Name, Last_Name, TicketsSold
    FROM concert_history
    WHERE TicketsSold BETWEEN min_tickets AND max_tickets
    ORDER BY First_Name, Last_Name;
END$$

DELIMITER ;
```

Δημιουργούμε το παρακάτω testing script στην SQL για να ελέγξουμε τον χρόνο που απαιτεί η εκτέλεση μιας αναζήτησης ανάμεσα στο 3000 και 4000

```
-- Test xronou me to index
SET profiling = 1;
CALL GetArtistsByTicketRange(100, 50000);
SHOW PROFILES;
SET profiling = 0;

-- Check index usage
EXPLAIN SELECT * FROM concert_history WHERE TicketsSold BETWEEN 3000 AND 4000;
```

Παρατηρούμε ότι προκύπτει:

0.00006850	SHOW WARNINGS
0.07117000	SELECT First_Name, Last_Name, Tick...

Φιγούρα 49: Εκτέλεση του SHOW PROFILES για αναζήτηση με Index

Από το παραπάνω screenshot παρατηρούμε πως ο χρόνος εκτέλεσης είναι 71 ms , κάτι που είναι σχετικά ικανοποιητικό για αναζήτηση σε πίνακα με 120000 εγγραφές.

Αν τρέξουμε την εντολή EXPLAIN , έχουμε το παρακάτω:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	concert_history	NULL	range	idx_tickets_sold	idx_tickets_sold	4	NULL	2434	100.00	Using index condition

Φιγούρα 50: Εκτέλεση του EXPLAIN για αναζήτηση με Index

Ουσιαστικά, αυτό που αναφέρει η παραπάνω φιγούρα είναι πως η SQL, κατά την εκτέλεση του ερωτήματος, χρησιμοποιεί το ευρετήριο (index) *idx_tickets_sold* για να φιλτράρει τα αποτελέσματα. Ο τύπος αναζήτησης εμφανίζεται ως range, που σημαίνει ότι το σύστημα δεν σαρώνει ολόκληρο τον πίνακα, αλλά μόνο τις εγγραφές που βρίσκονται μέσα στο ζητούμενο εύρος τιμών.

Το πεδίο rows = 2434 δείχνει ότι ο βελτιστοποιητής εκτιμά πως θα χρειαστεί να εξετάσει περίπου 2.434 γραμμές, ενώ η στήλη filtered = 100.00 υποδηλώνει ότι όλες οι γραμμές που ανακτώνται μέσω του index θεωρούνται ότι ικανοποιούν το φίλτρο του WHERE.

Τέλος, στο Extra = Using index condition εφαρμόζεται η τεχνική Index Condition Pushdown (ICP), δηλαδή η συνθήκη φιλτραρίσματος ελέγχεται ΚΑΤΕΥΘΕΙΑΝ κατά τη διάρκεια του index scan. Αυτό μειώνει την ανάγκη για επιπλέον scan στον πίνακα και οδηγεί σε πιο αποδοτική εκτέλεση. (πηγή https://dev.mysql.com/doc/refman/8.4/en/index-condition-pushdown-optimization.html)

Αν όμως τώρα **αφαιρέσουμε το index** και τρέξουμε πάλι το script :

```
ALTER TABLE concert_history DROP INDEX idx_tickets_sold;
-- Test xwris index
SET profiling = 1;
CALL GetArtistsByTicketRange(3000, 4000);
SHOW PROFILES;
SET profiling = 0;

EXPLAIN SELECT * FROM concert_history WHERE TicketsSold BETWEEN 3000 AND 4000;
```

Παρατηρούμε ότι προκύπτει:

0.00004550	SHOW WARNINGS
0.02204350	SELECT First_Name, Last_Name, Tick...

Φιγούρα 51: Εκτέλεση του SHOW PROFILES για αναζήτηση χωρίς Index

Εδώ όμως βλέπουμε κάτι που μας ξαφνιάζει. Η αναζήτηση χωρίς index διαρκεί μόλις 22 ms !!! που είναι σημαντικά μικρότερος χρόνος από ό,τι με Index. Γιατί όμως συμβαίνει αυτό ; Υποτίθεται ότι η χρήση των index κάνει κατά πολύ πιο γρήγορη την αναζήτηση σε έναν πίνακα.

Αν τρέξουμε και την εντολή explain βλέπουμε στην οθόνη μας:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	concert_history	NULL	ALL	NULL	NULL	NULL	NULL	114760	11.11	Using where

Φιγούρα 52: Εκτέλεση του EXPLAIN για αναζήτηση χωρίς Index

Από την ανάλυση του EXPLAIN και τα αποτελέσματα του profiling παρατηρούμε τα εξής:

Με τη χρήση του index idx_tickets_sold, η MySQL εκτελεί το query με τύπο πρόσβασης range, σαρώνοντας περίπου 2.434 εγγραφές και εφαρμόζοντας Index Condition Pushdown για πιο αποδοτικό filtering. Ωστόσο, ο χρόνος εκτέλεσης ήταν περίπου **71 ms (!!!)**

Χωρίς τη χρήση index, η MySQL κάνει πλήρη σάρωση του πίνακα (full table scan), διαβάζοντας όλες τις ~114.760 γραμμές του πίνακα, ενώ το φίλτρο WHERE εφαρμόζεται μετά τη σάρωση. Παρ' όλα αυτά, ο χρόνος εκτέλεσης ήταν μικρότερος, γύρω στα **22 ms**.

Αυτό το φαινόμενο οφείλεται στο γεγονός ότι το εύρος τιμών που χρησιμοποιείται στο WHERE επιστρέφει μεγάλο ποσοστό του πίνακα. Σε τέτοιες περιπτώσεις, το κόστος αναζήτησης μέσω του index και η επιπλέον ταξινόμηση (ORDER BY First_Name, Last_Name)[2] μπορεί να είναι μεγαλύτερο από μια απλή σειριακή ανάγνωση όλου του πίνακα. Συνεπώς, όταν το φίλτρο δεν περιορίζει αρκετά τα δεδομένα, η πλήρης σάρωση πίνακα αποδεικνύεται πιο αποδοτική από τη χρήση index.

[2] Η αναζήτηση με Index ενώ μπορεί να βρίσκει πιο γρήγορα τις κατάλληλες εγγραφές , δεν βοηθάει κάπως στο sorting.
Άρα είναι και αυτός ένας λόγος που αργεί η επιστροφή αποτελέσματος.

Κεφάλαιο 3 - Triggers

Απαίτηση 3.1.4.1

Trigger που θα ενημερώνουν το σχετικό πίνακα καταγραφής ενεργειών (log) για κάθε ενέργεια εισαγωγής, ενημέρωσης ή διαγραφής στους πίνακες person, band, album, concert, venue με την ημερομηνία και ώρα και το username του DBA που την εκτέλεσε

Όπως επεξηγήθηκε και στο Κεφάλαιο 1 (βλέπε **Επιπλέον πίνακες**) , για την απαίτηση 3.1.4.1 δημιουργήθηκαν οι πίνακες **dba** και **dba_log** που αναφέρονται στους διαχειριστές της ΒΔ. Για να μην εξηγούμε πάλι τα ίδια και κουράζουμε τον αναγνώστη με επαναλαμβανομένη πληροφορία, παραθέτουμε τον κώδικα που χρησιμοποιήθηκε για το συγκεκριμένο trigger:

```
-- Απαίτηση 3.1.4.1
-- Trigger που θα ενημερώνουν το σχετικό πίνακα καταγραφής ενεργειών
-- (log) για κάθε ενέργεια εισαγωγής, ενημέρωσης ή διαγραφής στους πίνακες
-- person, band, album, concert, venue με την ημερομηνία και ώρα και το
-- username του DBA που την εκτέλεσε.

SET @current_dba := 'DBA_USERNAME'; -- we set the current DBA manually

DELIMITER $$

-- Artist change log
DROP TRIGGER IF EXISTS artist_log_insert$$
CREATE TRIGGER artist_log_insert
AFTER INSERT ON artist
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: artist');
END$$

DROP TRIGGER IF EXISTS artist_log_update$$
CREATE TRIGGER artist_log_update
AFTER UPDATE ON artist
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: artist');
END$$

DROP TRIGGER IF EXISTS artist_log_delete$$
CREATE TRIGGER artist_log_delete
AFTER DELETE ON artist
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: artist');
END$$

-- Table 'person' changes log
DROP TRIGGER IF EXISTS person_log_insert$$
CREATE TRIGGER person_log_insert
AFTER INSERT ON person
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: person');
END$$

DROP TRIGGER IF EXISTS person_log_update$$
CREATE TRIGGER person_log_update
AFTER UPDATE ON person
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: person');
END$$

DROP TRIGGER IF EXISTS person_log_delete$$
CREATE TRIGGER person_log_delete
AFTER DELETE ON person
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: person');
END$$

-- Band changes log

DROP TRIGGER IF EXISTS band_log_insert$$
CREATE TRIGGER band_log_insert
AFTER INSERT ON band
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
```

```
    VALUES (NOW(), @current_dba, 'Inserted data in table: band');
END$$

DROP TRIGGER IF EXISTS band_log_update$$
CREATE TRIGGER band_log_update
AFTER UPDATE ON band
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: band');
END$$

DROP TRIGGER IF EXISTS band_log_delete$$
CREATE TRIGGER band_log_delete
AFTER DELETE ON band
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: band');
END$$

-- Album changes log

DROP TRIGGER IF EXISTS album_log_insert$$
CREATE TRIGGER album_log_insert
AFTER INSERT ON album
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: album');
END$$

DROP TRIGGER IF EXISTS album_log_update$$
CREATE TRIGGER album_log_update
AFTER UPDATE ON album
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: album');
END$$

DROP TRIGGER IF EXISTS album_log_delete$$
CREATE TRIGGER album_log_delete
AFTER DELETE ON album
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: album');
END$$

-- Concert changes log

DROP TRIGGER IF EXISTS concert_log_insert$$
CREATE TRIGGER concert_log_insert
AFTER INSERT ON concert
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: concert');
END$$

DROP TRIGGER IF EXISTS concert_log_update$$
CREATE TRIGGER concert_log_update
AFTER UPDATE ON concert
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: concert');
END$$

DROP TRIGGER IF EXISTS concert_log_delete$$
CREATE TRIGGER concert_log_delete
AFTER DELETE ON concert
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: concert');
END$$

-- Venue changes log

DROP TRIGGER IF EXISTS venue_log_insert$$
CREATE TRIGGER venue_log_insert
AFTER INSERT ON venue
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: venue');
END$$

DROP TRIGGER IF EXISTS venue_log_update$$
CREATE TRIGGER venue_log_update
AFTER UPDATE ON venue
FOR EACH ROW
```

```
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: venue');
END$$

DROP TRIGGER IF EXISTS venue_log_delete$$
CREATE TRIGGER venue_log_delete
AFTER DELETE ON venue
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: venue');
END$$

DELIMITER ;
```

Κάθε φορά που πραγματοποιείται μια αλλαγή σε έναν από τους συγκεκριμένους πίνακες:

- Χρησιμοποιούμε τη συνάρτηση NOW( )[3] ώστε να καταγράψουμε με ακρίβεια την ημερομηνία και ώρα της ενέργειας, η οποία αποθηκεύεται στο πεδίο ActionDate ,
- Στο πεδίο DBA_Username αποθηκεύεται το όνομα χρήστη του ενεργού DBA που εκτέλεσε τη συγκεκριμένη ενέργεια εκείνη τη στιγμή,
- Στο πεδίο log_action καταγράφεται ένα κατάλληλο μήνυμα, το οποίο περιγράφει τον τύπο της ενέργειας (Inserted/Modified/Deleted) καθώς και τον πίνακα στον οποίο αυτή εφαρμόστηκε.

Με αυτόν τον τρόπο διασφαλίζεται πλήρης και ξεκάθαρη ιχνηλασιμότητα όλων των αλλαγών που γίνονται στη ΒΔ ανά πάσα στιγμή και έτσι αποφεύγονται τα μπερδέματα.

Ο πίνακας dba_log μετά από updates/deletes/modifies από τον user με username = admin :

Result Grid | Filter Rows: | Edit: | Export/Import:

LogID	ActionDate	DBA_Username	log_action
79	2025-08-28 17:24:58	admin	Inserted data in table: album
80	2025-08-28 17:24:58	admin	Inserted data in table: album
81	2025-08-28 17:24:58	admin	Inserted data in table: album
82	2025-08-28 17:24:58	admin	Inserted data in table: album
83	2025-08-28 17:24:58	admin	Inserted data in table: album
84	2025-08-28 17:24:58	admin	Inserted data in table: album
85	2025-08-28 17:24:58	admin	Inserted data in table: album
86	2025-08-28 17:24:58	admin	Inserted data in table: album
87	2025-08-28 17:24:58	admin	Inserted data in table: album
88	2025-08-28 17:24:58	admin	Inserted data in table: album
89	2025-08-28 17:24:58	admin	Inserted data in table: album
90	2025-08-28 17:24:58	admin	Inserted data in table: album
91	2025-08-28 17:24:58	admin	Inserted data in table: album
92	2025-08-28 17:24:58	admin	Inserted data in table: album
93	2025-08-28 17:24:58	admin	Inserted data in table: album
94	2025-08-28 17:24:58	admin	Inserted data in table: album
95	2025-08-28 17:24:58	admin	Inserted data in table: album
96	2025-08-28 17:28:11	admin	Modified data in table: pers...
97	2025-08-28 17:30:35	admin	Modified data in table: pers...
98	2025-08-28 17:32:07	admin	Modified data in table: pers...
99	2025-08-28 17:33:39	admin	Deleted data in table: artist
NULL	NULL	NULL	NULL

Φιγούρα 53: Περιεχόμενα πίνακα dba_log μετά από αλλαγές

[3] Η συνάρτηση NOW() γενικά δεν προτείνεται για SQL server αλλά για ευκολία χρησιμοποιήθηκε εδώ.
https://stackoverflow.com/questions/385042/sql-server-equivalent-of-mysqls-now

Απαίτηση 3.1.4.2

Trigger ο οποίος θα αποτρέπει τον προγραμματισμό νέας συναυλίας, αν η ημερομηνία που γίνεται ο προγραμματισμός της συναυλίας απέχει λιγότερο από πέντε μέρες από την ημερομηνία διεξαγωγής της ή αν ο συγκεκριμένος καλλιτέχνης έχει ήδη τρεις προγραμματισμένες συναυλίες.

```
-- Απαίτηση 3.1.4.2
-- Trigger ο οποίος θα αποτρέπει τον προγραμματισμό νέας συναυλίας, αν η
-- ημερομηνία που γίνεται ο προγραμματισμός της συναυλίας απέχει λιγότερο
-- από πέντε μέρες από την ημερομηνία διεξαγωγής της ή αν ο συγκεκριμένος
-- καλλιτέχνης έχει ήδη τρεις προγραμματισμένες συναυλίες.

-- Prevent excessive concert scheduling
DELIMITER $$
DROP TRIGGER IF EXISTS prevent_too_many_concerts$$
CREATE TRIGGER prevent_too_many_concerts
BEFORE INSERT ON concert
FOR EACH ROW
BEGIN
    DECLARE scheduled_count INT;
    SELECT COUNT(*) INTO scheduled_count
    FROM concert
    WHERE ArtistID = NEW.ArtistID AND ConcertStatus = 'SCHEDULED';

    IF scheduled_count >= 3 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot schedule a new concert: Artist already has 3 scheduled concerts.';
    END IF;
END$$

-- Prevent scheduling less than 5 days in advance
DROP TRIGGER IF EXISTS prevent_last_minute_scheduling$$
CREATE TRIGGER prevent_last_minute_scheduling
BEFORE INSERT ON concert
FOR EACH ROW
BEGIN
    -- Error checking only when concert != cancelled or concert!= completed
    IF NEW.ConcertStatus IN ('SCHEDULED') THEN
        IF DATEDIFF(NEW.ConcertDate, CURDATE()) < 5 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot schedule a concert less than 5 days in advance.';
        END IF;
    END IF;
END$$
```

Η λογική του κώδικα είναι απλή και σύμφωνη με την εκφώνηση. Αρχικά, ορίζουμε μεταβλητή **scheduled_count**, η οποία μετράει πόσες συναυλίες βρίσκονται ήδη σε κατάσταση scheduled για έναν συγκεκριμένο καλλιτέχνη. Στη συνέχεια, με μια IF γίνεται έλεγχος: αν διαπιστωθεί ότι scheduled_count >= 3, τότε η SQL επιστρέφει σφάλμα 45000 με το αντίστοιχο μήνυμα. Με αυτόν τον τρόπο ο χειριστής ενημερώνεται για την παραβίαση του κανόνα και αποτρέπεται η εισαγωγή της νέας εγγραφής.

Αντίστοιχα, εφαρμόζεται η ίδια λογική και για τον περιορισμό που δεν επιτρέπει τον προγραμματισμό συναυλίας 5 μέρες νωρίτερα, εξασφαλίζοντας έτσι τη σωστή τήρηση των απαιτήσεων.

Πριν ξεκινήσουμε το testing , βάζουμε ενδεικτικά 3 Venues στον πίνακα με χρήση του παρακάτω snippet:

```
INSERT INTO venue (VenueName, Capacity , CompletedConcerts, YearsOperating , IsAvailable) VALUES ('Old Trafford' , 150000 , 150,
100 , 1);
INSERT INTO venue (VenueName, Capacity , CompletedConcerts, YearsOperating , IsAvailable) VALUES ('MSG' , 200000 , 500, 75 , 1);
INSERT INTO venue (VenueName, Capacity , CompletedConcerts, YearsOperating , IsAvailable) VALUES ('Manchester Apollo' , 3500 , 600
, 120,  1);
```

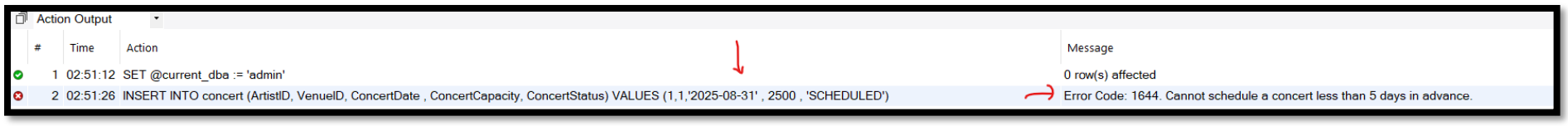
	VenueID	VenueName	Capacity	CompletedConcerts	YearsOperating	IsAvailable
▶	1	Old Trafford	150000	150	100	1
	2	MSG	200000	500	75	1
	3	Manchester Apollo	3500	600	120	1

Φιγούρα 54: 3 venues που μπήκαν στον πίνακα για να γίνει το τεστ

Παράδειγμα εισαγωγής εγγραφής λιγότερο από 5 μέρες πριν την συναυλία:

```
SET @current_dba := 'admin';
INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,1,'2025-
08-31' , 2500 , 'SCHEDULED');
```

Αποτέλεσμα κατά την εκτέλεση:



Φιγούρα 55: Αποτέλεσμα SQL

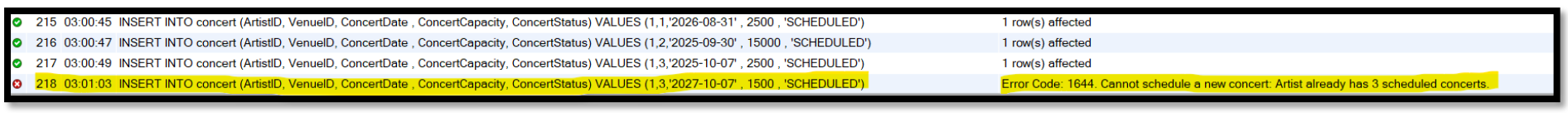
Αυτό συμβαίνει διότι η ημερομηνία που εκτελούμε(29.08) απέχει λιγότερο από 5 μέρες από τις 31.08. Άρα ορθώς και με βάση το trigger που έχουμε ορίσει , η SQL προειδοποιεί τον χρήστη και αποτρέπει την εισαγωγή εφόσον δεν πληρούνται τα κριτήρια.

Παράδειγμα εισαγωγής πάνω από 3 concert για έναν καλλιτέχνη:

Γράφουμε και εκτελούμε το παρακάτω script :

```
SET @current_dba := 'admin';
INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,1,'2026-08-31' , 2500 ,
'SCHEDULED');
INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,2,'2025-09-30' , 15000 ,
'SCHEDULED');
INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,3,'2025-10-07' , 2500 ,
'SCHEDULED');
INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,3,'2027-10-07' , 1500 ,
'SCHEDULED');
```

Προγραμματίζουμε συνολικά τέσσερις συναυλίες για τον καλλιτέχνη με ArtistID = 1 (Arctic Monkeys).
Η εισαγωγή των τριών πρώτων εγγραφών ολοκληρώνεται κανονικά. Ωστόσο, όταν προσπαθούμε να καταχωρήσουμε και 4η συναυλία, η SQL επιστρέφει το μήνυμα σφάλματος 45000 (σταματά το insert) που έχουμε ορίσει στον trigger. Με αυτόν τον τρόπο αποτρέπεται η εισαγωγή της νέας εγγραφής και διασφαλίζεται ότι δεν θα ξεπεραστεί το επιτρεπτό όριο των 3 συναυλιών/καλλιτέχνη.



Φιγούρα 56: Αποτέλεσμα SQL

Απαίτηση 3.1.4.3

Trigger ο οποίος θα αποτρέπει την ακύρωση μιας συναυλίας, αν η ημερομηνία που γίνεται η ακύρωση απέχει λιγότερο από τρεις μέρες από την ημερομηνία διεξαγωγής της. Επίσης, θα αποτρέπει τον επαναπρογραμματισμό ακυρωμένης συναυλίας αν ο συγκεκριμένος καλλιτέχνης έχει ήδη τρεις προγραμματισμένες συναυλίες.

Κώδικας SQL:

```
-- Απαίτηση 3.1.4.3
-- Trigger ο οποίος θα αποτρέπει την ακύρωση μιας συναυλίας, αν η
-- ημερομηνία που γίνεται η ακύρωση απέχει λιγότερο από τρεις μέρες από
-- την ημερομηνία διεξαγωγής της. Επίσης, θα αποτρέπει τον
-- επαναπρογραμματισμό ακυρωμένης συναυλίας αν ο συγκεκριμένος
-- καλλιτέχνης έχει ήδη τρεις προγραμματισμένες συναυλίες.
DELIMITER $$
DROP TRIGGER IF EXISTS prevent_late_cancellation_and_reschedule$$
CREATE TRIGGER prevent_late_cancellation_and_reschedule
BEFORE UPDATE ON concert
FOR EACH ROW
BEGIN
DECLARE scheduled_count INT;
    -- Cannot cancel 3 days before the concert , error checking
    IF NEW.ConcertStatus = 'CANCELLED' AND OLD.ConcertStatus != 'CANCELLED' THEN
        IF DATEDIFF(NEW.ConcertDate, CURDATE()) < 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot cancel a concert less than 3 days before the event.';
        END IF;
    END IF;

    -- Same as above with the excessive scheduling
    IF OLD.ConcertStatus = 'CANCELLED' AND NEW.ConcertStatus != 'CANCELLED' THEN

        SELECT COUNT(*)
        INTO scheduled_count
        FROM concert
        WHERE ArtistID = NEW.ArtistID
          AND ConcertStatus = 'SCHEDULED'
          AND ConcertID != NEW.ConcertID;

        -- If the artist already has 3 concerts scheduled.. you can't schedule a new one
        IF scheduled_count >= 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot reschedule a cancelled concert because the artist already has 3 scheduled concerts.';
        END IF;
    END IF;
```

```
END$$
DELIMITER ;
```

Παράδειγμα για cancel error δεν μπορεί να δοθεί σε screenshot την στιγμή που γράφεται η αναφορά, αλλά σε πιο παλιό testing δούλευε σωστά. Μόνο κατά την διάρκεια της εξέτασης ίσως μπορέσει να καλυφθεί αυτό το ερώτημα. Ευχαριστώ για την κατανόηση.

Για το 2ο test:

Snippet κώδικα για την εισαγωγή των δεδομένων:

```
-- eisagwgh concerts gia thn apaithsh 3.1.4.3
INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,3,'2027-10-07' , 1500 ,
'CANCELLED');
INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,1,'2025-09-03' , 2500 ,
'SCHEDULED');
INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,2,'2025-09-30' , 15000 ,
'SCHEDULED');
INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,3,'2025-10-17' , 2500 ,
'SCHEDULED');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ConcertID	ArtistID	VenueID	ConcertDate	ConcertCapacity	TicketsSold	ConcertStatus	IsArchived
6	1	3	2027-10-07	1500	0	CANCELLED	1
7	1	1	2025-09-03	2500	0	SCHEDULED	1
8	1	2	2025-09-30	15000	0	SCHEDULED	1
9	1	3	2025-10-17	2500	0	SCHEDULED	1

Φιγούρα 57: Δεδομένα πίνακα concert

Κώδικας για αλλαγή από ‘CANCELLED’ σε ‘SCHEDULED’:

```
UPDATE concert
SET
    ConcertStatus = 'SCHEDULED'
WHERE
    ConcertID = 6;
```

Αποτέλεσμα μετά την εκτέλεση του παραπάνω snippet:

```
14  -- eisagwgh concerts gia thn apaithsh 3.1.4.3
15  INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,3,'2027-10-07' , 1500 , 'CANCELLED');
16  INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,1,'2025-09-03' , 2500 , 'SCHEDULED');
17  INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,2,'2025-09-30' , 15000 , 'SCHEDULED');
18  INSERT INTO concert (ArtistID, VenueID, ConcertDate , ConcertCapacity, ConcertStatus) VALUES (1,3,'2025-10-17' , 2500 , 'SCHEDULED');
19
20  UPDATE concert
21  SET
22      ConcertStatus = 'SCHEDULED'
23  WHERE
24      ConcertID = 6;
25
Output
Action Output
#  Time      Action                                                                  Message
1  03:31:07  UPDATE concert  SET  ConcertStatus = 'SCHEDULED' WHERE  ConcertID = 6  Error Code: 1644. Cannot reschedule a cancelled concert because the artist already has 3 scheduled concerts.
```

Φιγούρα 58: Αποτέλεσμα SQL

Σωστά η SQL επέστρεψε το σφάλμα που έχουμε ορίσει στον trigger, καθώς παραβιάζονται οι κανόνες εισαγωγής δεδομένων στον πίνακα. Ο συγκεκριμένος καλλιτέχνης έχει ήδη τρεις προγραμματισμένες συναυλίες, και όταν επιχειρούμε να αλλάξουμε το status μιας συναυλίας από cancelled σε κάποια άλλη τιμή (δηλαδή σε κατάσταση που ΔΕΝ είναι cancelled (γι’ αυτό και ορίσαμε NEW.Status != ‘Cancelled’), ενεργοποιείται ο trigger και εμφανίζεται το αντίστοιχο μήνυμα σφάλματος. Με αυτόν τον τρόπο αποτρέπεται η υπέρβαση του επιτρεπτού ορίου συναυλιών.

Μέρος Β’ – Java GUI

Η γενική φιλοσοφία της εφαρμογής βασίζεται στη δημιουργία ενός περιβάλλοντος που διευκολύνει σημαντικά το maintenance και τις αλλαγές στην ΒΔ . Δόθηκε ιδιαίτερη έμφαση στην αποφυγή manual εισαγωγής δεδομένων, ώστε να μειωθούν τα πιθανά λάθη και να επιταχυνθεί η διαδικασία.

Περιληπτικά :

- Για τα πεδία τύπου ENUM έχουν υλοποιηθεί drop-down μενού, ώστε ο χρήστης να επιλέγει τιμές από λίστα και να αποτρέπεται η ελεύθερη (και πιθανώς λανθασμένη) πληκτρολόγηση.
- Για την επιλογή ημερομηνιών χρησιμοποιείται ενσωματωμένο ημερολόγιο, το οποίο απλοποιεί τη διαδικασία και εξασφαλίζει τη σωστή μορφοποίηση της ημερομηνίας.
- Κατά την εκκίνηση της εφαρμογής εμφανίζεται popup παραθύρου login, όπου ο χρήστης συνδέεται στον server με τα προσωπικά του credentials. Στη συνέχεια, ζητείται η εισαγωγή του username.
- Αν το συγκεκριμένο username δεν υπάρχει ήδη στη βάση, ο DBA δημιουργεί αυτόματα νέο χρήστη, αποθηκεύοντας όλα τα στοιχεία στη βάση project2024_gui, η οποία έχει σχεδιαστεί με μικρές διαφοροποιήσεις σε σχέση με τη βάση της Α’ Φάσης..

1. DatabaseUI()

Ξεκινήσαμε με την δημιουργία του παραθύρου της εφαρμογής. Δημιουργήσαμε την συνάρτηση *DatabaseUI*() μέσα στην οποία προσθέσαμε το μέγεθος του παραθύρου της εφαρμογής και την κλήση της συνάρτησης *connectToDB*(); Αυτή η συνάρτηση είναι το βασικό μας κλειδί, καθώς χάρη σε αυτή συνδεόμαστε στην ΒΔ μας .

```
public class DatabaseUI extends JFrame {
    private JComboBox<String> tableSelector;
    private JTable table;
    private DefaultTableModel tableModel;
    private Connection conn;

    // DBA Management fields
    private String currentDBAUsername;
    private Integer currentDBAId;
    private JLabel dbaStatusLabel;

    public DatabaseUI() {
        setTitle("Διαχείριση Βάσης Δεδομένων");
        setSize(2002, 1181);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        try {
            UIManager.setLookAndFeel("javax.swing.plaf.nimbus.NimbusLookAndFeel");
        } catch (Exception ignored) {
        }

        connectToDB();

        if (!authenticateDBA()) {
            System.exit(0);
            return;
        }

        setupGUI();
    }
```

Αν όλα ολοκληρωθούν με επιτυχία και κάνουμε authenticate του DBA (Μέσα από την κλήση του *authenticateDBA*()) που πρόκειται να διαχειριστεί την ΒΔ , τότε καλείται η αμέσως επόμενη συνάρτηση που είναι το *setupGUI*() και είναι η main μας εφαρμογή

2. SetupGUI()

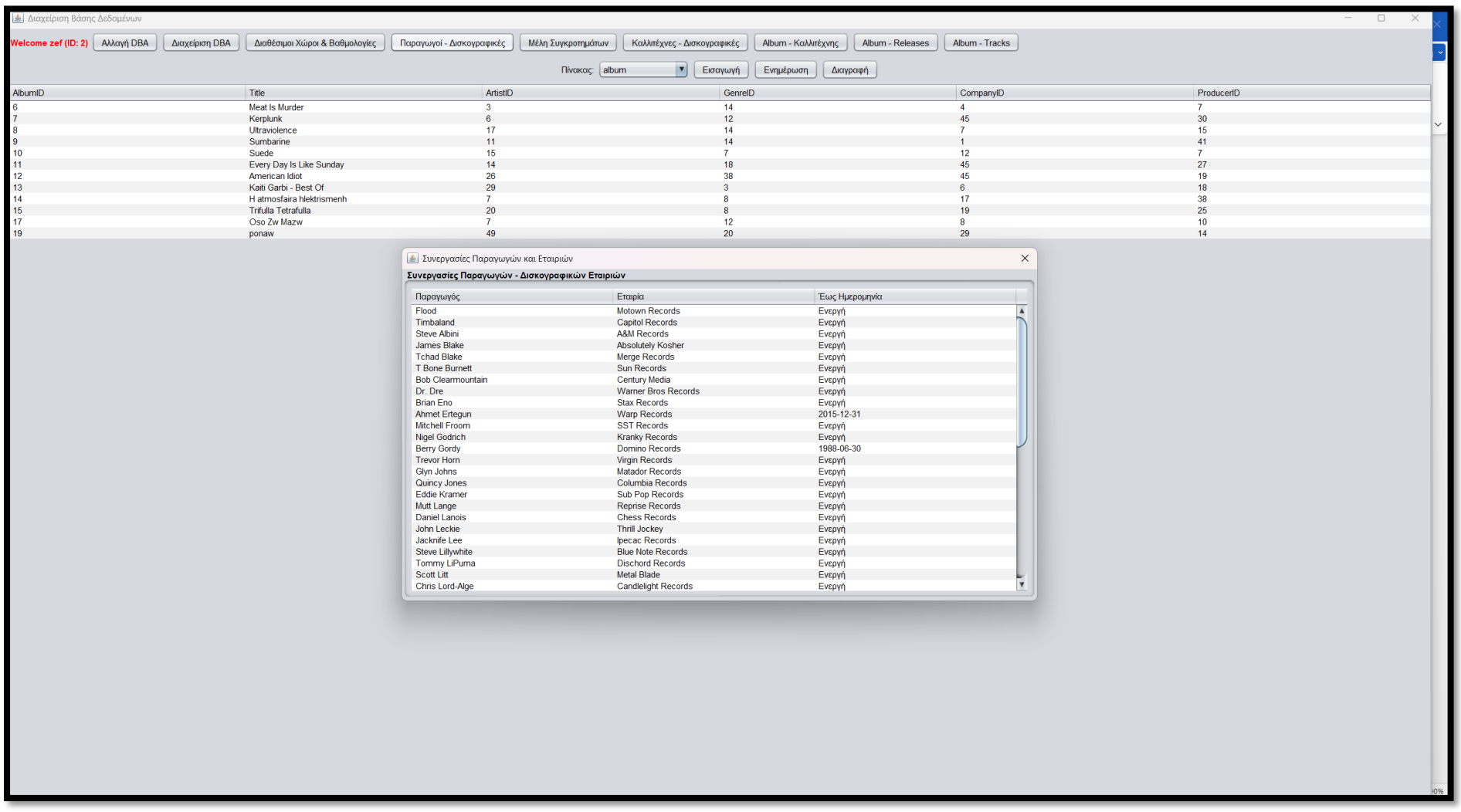
Η συνάρτηση αυτή ουσιαστικά είναι ό,τι βλέπουμε μπροστά μας όταν τρέξουμε την εφαρμογή .

Αρχικά, υπάρχει ένα welcome message που δείχνει το όνομα του DBA που είναι logged in καθώς και το ID του. Πιο δίπλα υπάρχουν κουμπιά για:

- Αλλαγή του DBA: Με αυτό το κουμπί κάνουμε ουσιαστικά logout από τον συνδεδεμένο μας DBA και επιλέγουμε άλλον . Αν δεν βρεθεί το username του νέου DBA που θέλουμε να κάνουμε login, τότε η εφαρμογή μάς πετάει εκτός.
- Διαχείριση DBA χρηστών : Αυτό το κουμπί φέρνει στην οθόνη ένα popup με μερικές επιλογές
    - την προσθήκη νέου DBA από την αρχή ,
    - την προβολή όλης της λίστας με DBA και

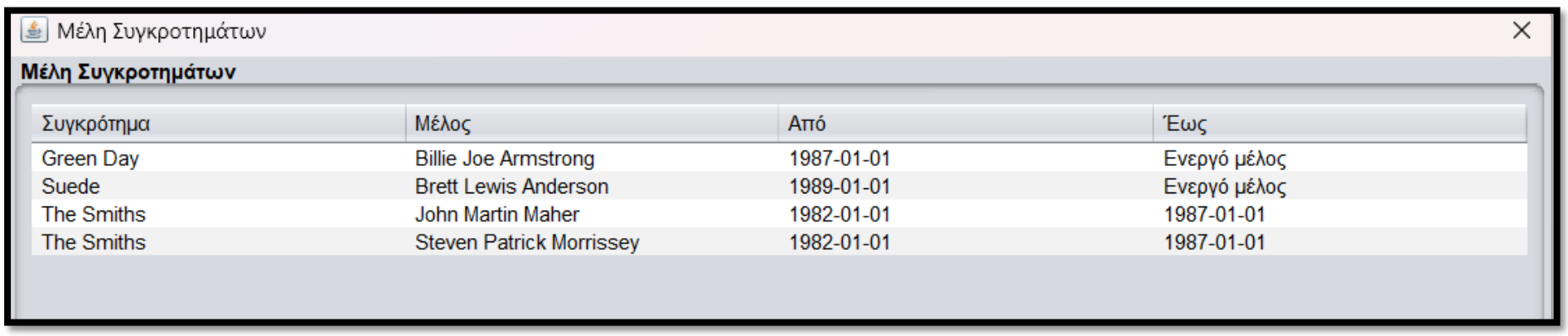
- την επιλογή για διαγραφή ενός DBA.

- <u>Διαθέσιμοι χώροι και Βαθμολογίες</u> : Με το πάτημα αυτού του κουμπιού εμφανίζονται οι διαθέσιμοι χώροι (αυτοί που ουσιαστικά έχουν οριστεί ως $IsAvailable = 1$) καθώς και η σχετική τους βαθμολογία σύμφωνα με τον Πίνακα 2
- <u>Παραγωγοί – Δισκογραφικές</u> : Εμφανίζονται οι παραγωγοί (τα ονόματα τους) και δίπλα σε ποια δισκογραφική έχουν υπογράψει και αν είναι ακόμα εκεί.



Φιγούρα 59: Στιγμιότυπο από την εφαρμογή

- <u>Μέλη συγκροτημάτων</u> : Μας εμφανίζει ποιοι artists από τον πίνακα person ανήκουν σε κάποιο συγκρότημα



Φιγούρα 60: Στιγμιότυπο από την εφαρμογή

- <u>Καλλιτέχνες – Δισκογραφικές</u> : Εμφανίζονται οι καλλιτέχνες (τα ονόματα τους) και δίπλα σε ποια δισκογραφική έχουν υπογράψει και αν είναι ακόμα εκεί.

Συνεργασίες Καλλιτεχνών - Δισκογραφικών Εταιριών

Συνεργασίες Καλλιτεχνών - Δισκογραφικών Εταιριών

Καλλιτέχνης	Εταιρία	Από	Έως
ABBA	Southern Lord	1972-05-31	1981-10-25
Adrianne Lenker	Stax Records	2014-01-01	Ενεργή συνεργασία
Agneta Fältskog	Southern Lord	1972-05-31	1981-10-25
Alexander Turner	Domino Records	2005-01-01	Ενεργή συνεργασία
Angela Trimble	Warp Records	2015-12-31	Ενεργή συνεργασία
Arctic Monkeys	Warner Bros Records	1982-01-01	1987-01-01
Billie Joe Armstrong	Reprise Records	1994-01-01	Ενεργή συνεργασία
Brett Lewis Anderson	Southern Lord	1994-01-01	1999-01-01
Damon Albarn	Domino Records	1988-06-30	1996-01-21
David Jones	Chess Records	1977-01-01	2015-01-01
Elizabeth Grant	Nuclear Blast	2011-01-01	Ενεργή συνεργασία
Green Day	Reprise Records	1994-01-01	Ενεργή συνεργασία
Ian Brown	Geffen Records	1991-01-01	2019-09-16
John Mellor	Rough Trade	1980-01-01	1990-08-13
John Martin Maher	Warner Bros Records	1982-01-01	1987-01-01
Madonna Turner	XL Recordings	1976-12-31	2002-08-05
Matthew Helders	Domino Records	2005-01-01	Ενεργή συνεργασία
Miles Kane	Sun Records	2011-01-01	2023-01-01
Nelly Kim Furtado	Ninja Tune	2005-03-07	Ενεργή συνεργασία
Peter Kinnes	Fat Wreck Chords	1965-09-03	1969-02-17
Pulp	Rough Trade	2024-12-12	Ενεργή συνεργασία
Radiohead	Interscope	1985-01-01	Ενεργή συνεργασία
Steven Patrick Morrissey	Warner Bros Records	1982-01-01	1987-01-01
Suede	Epitaph Records	1994-01-01	1999-01-01
The Kinks	Fat Wreck Chords	1965-09-03	1969-02-17

Φιγούρα 61: Στιγμιότυπο από την εφαρμογή

- <u>Album – Καλλιτέχνης</u> : Εμφανίζονται τα album που ανήκουν σε έναν καλλιτέχνη όλες οι σχετικές πληροφορίες

Album ↔ Καλλιτέχνης

Συσχετίσεις Album - Καλλιτέχνη

AlbumID	Title	Artist	Genre	Company	Producer
12	American Idiot	Madonna Turner	Swing	Roadrunner Records	Steve Lillywhite
11	Every Day Is Like Sunday	Steven Patrick Morrissey	Disco	Roadrunner Records	Ahmet Ertegun
14	H atmosfaira hlektrismenh	Pulp	Electronic	Virgin Records	Nigel Godrich
13	Kaiti Garbi - Best Of	Steve Thomas Lacy-Moya	Jazz	Capitol Records	Flood
7	Kerplunk	Green Day	Punk	Roadrunner Records	Eddie Kramer
6	Meat Is Murder	The Smiths	Alternative	Warner Bros Records	Pharrell Williams
6	Meat Is Murder	Arctic Monkeys	Alternative	Warner Bros Records	Pharrell Williams
17	Oso Zw Mazw	Pulp	Punk	Def Jam	Danger Mouse
19	ponaw		Techno	Matador Records	Alan Parsons
10	Suede	Brett Lewis Anderson	Blues	A&M Records	Pharrell Williams
9	Sumbarine	Alexander Turner	Alternative	Atlantic Records	Brendan O'Brien
15	Trifulla Tetrafulla	David Jones	Electronic	Blue Note Records	Berry Gordy
8	Ultraviolence	Elizabeth Grant	Alternative	RCA Records	Tony Visconti

Φιγούρα 62: Στιγμιότυπο από την εφαρμογή

- <u>Album Releases</u> : Εμφανίζονται τα είδη των release για album όλες οι σχετικές πληροφορίες

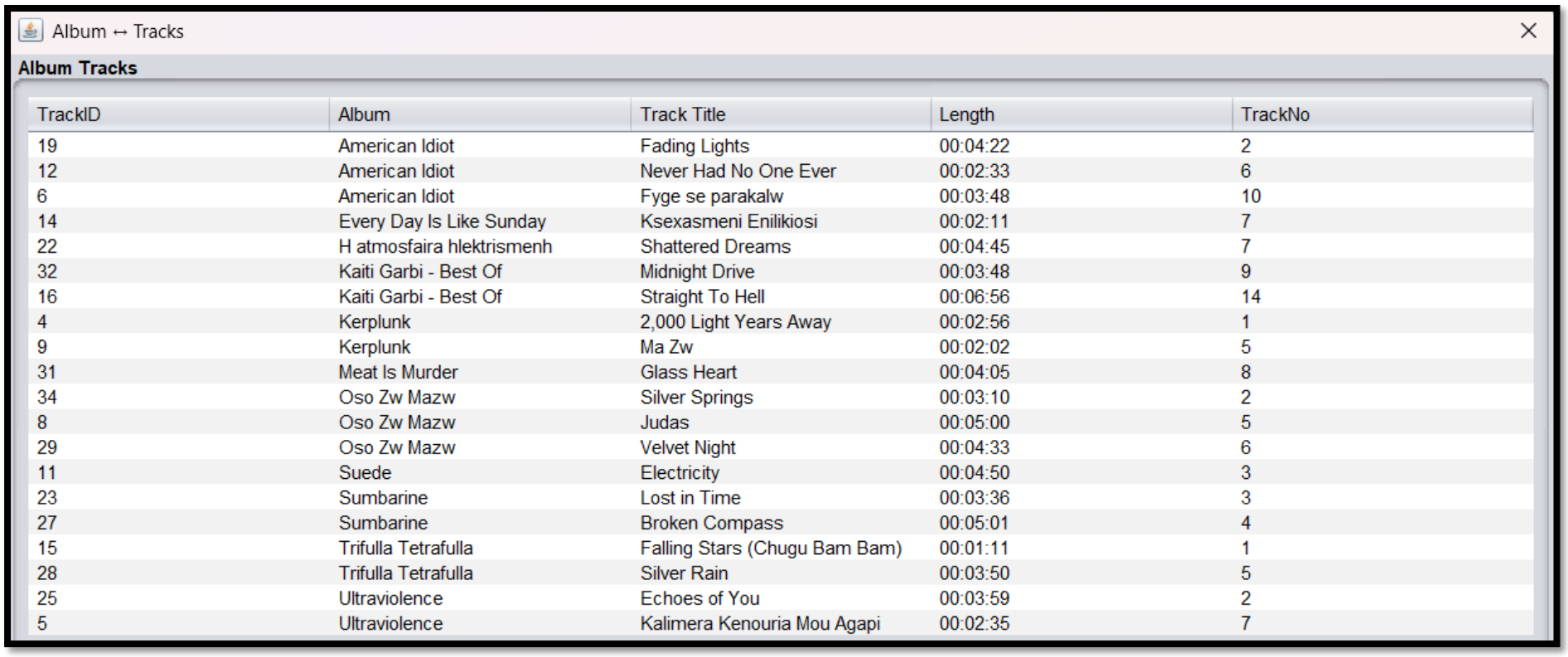
Album ↔ Releases

Album Releases

ReleaseID	Album	ReleaseDate	Type	Status	Packaging
7	Kerplunk	2022-07-30	CD	OFFICIAL	DIGIPAK
16	Meat Is Murder	2005-07-20	MP3	OFFICIAL	NOT APPLICABLE
6	Meat Is Murder	2020-12-01	MP3	CANCELLED	NOT APPLICABLE
10	Suede	2022-02-14	CD	WITHDRAWN	CARDBOARD SLEEVE
9	Sumbarine	2020-05-22	MP3	OFFICIAL	NOT APPLICABLE
8	Ultraviolence	2021-10-18	LP	PROMOTION	JEWEL CASE

Φιγούρα 63: Στιγμιότυπο από την εφαρμογή

- <u>Album – Tracks</u> : Εμφανίζονται τα tracks που ανήκουν σε album και όλες οι σχετικές πληροφορίες



TrackID	Album	Track Title	Length	TrackNo
19	American Idiot	Fading Lights	00:04:22	2
12	American Idiot	Never Had No One Ever	00:02:33	6
6	American Idiot	Fyge se parakalw	00:03:48	10
14	Every Day Is Like Sunday	Ksexasmeni Enilikiosi	00:02:11	7
22	H atmosfaira hlektrismenh	Shattered Dreams	00:04:45	7
32	Kaiti Garbi - Best Of	Midnight Drive	00:03:48	9
16	Kaiti Garbi - Best Of	Straight To Hell	00:06:56	14
4	Kerplunk	2,000 Light Years Away	00:02:56	1
9	Kerplunk	Ma Zw	00:02:02	5
31	Meat Is Murder	Glass Heart	00:04:05	8
34	Oso Zw Mazw	Silver Springs	00:03:10	2
8	Oso Zw Mazw	Judas	00:05:00	5
29	Oso Zw Mazw	Velvet Night	00:04:33	6
11	Suede	Electricity	00:04:50	3
23	Sumbarine	Lost in Time	00:03:36	3
27	Sumbarine	Broken Compass	00:05:01	4
15	Trifulla Tetrafulla	Falling Stars (Chugu Bam Bam)	00:01:11	1
28	Trifulla Tetrafulla	Silver Rain	00:03:50	5
25	Ultraviolence	Echoes of You	00:03:59	2
5	Ultraviolence	Kalimera Kenouria Mou Agapi	00:02:35	7

Φιγούρα 64: Στιγμιότυπο από την εφαρμογή

Δημιουργήθηκαν επίσης κουμπιά που κάνουν εισαγωγή , τροποποίηση και διαγραφή δεδομένων από τους πίνακες της ΒΔ καλώντας τις αντίστοιχες συναρτήσεις που εκτελούν SQL queries για αλλαγές πάνω στην ΒΔ

Κώδικας της συνάρτησης :

```
    private void setupGUI() {
        JPanel topPanel = new JPanel(new BorderLayout());

        JPanel dbaPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        dbaStatusLabel = new JLabel("Welcome \n" + currentDBAUsername + " (ID: " + currentDBAId + ")");
        dbaStatusLabel.setForeground(Color.RED);
        dbaStatusLabel.setFont(dbaStatusLabel.getFont().deriveFont(Font.BOLD));

        // Κουμπιά για διαχείριση DBA
        JButton logoutBtn = new JButton("Αλλαγή DBA");
        logoutBtn.addActionListener(e -> changeDBA());
        JButton manageDBABtn = new JButton("Διαχείριση DBA");
        manageDBABtn.addActionListener(e -> manageDBAUsers());
        // Εμφάνιση διαθέσιμων χώρων και βαθμολογίες τους
        JButton viewVenuesBtn = new JButton("Διαθέσιμοι Χώροι & Βαθμολογίες");
        viewVenuesBtn.addActionListener(e -> showVenuesWithScores());

        // Κουμπιά για εμφάνιση σχέσεων
        JButton viewProducerCompanyBtn = new JButton("Παραγωγοί - Δισκογραφικές");
        viewProducerCompanyBtn.addActionListener(e -> showProducerCompanyRelations());

        JButton viewBandMembersBtn = new JButton("Μέλη Συγκροτημάτων");
        viewBandMembersBtn.addActionListener(e -> showBandMembers());

        JButton viewArtistCompanyBtn = new JButton("Καλλιτέχνες - Δισκογραφικές");
        viewArtistCompanyBtn.addActionListener(e -> showArtistCompanyRelations());

        // Album relation buttons
        JButton viewAlbumArtistBtn = new JButton("Album - Καλλιτέχνης");
        viewAlbumArtistBtn.addActionListener(e -> showAlbumArtistRelations());

        JButton viewAlbumReleasesBtn = new JButton("Album - Releases");
        viewAlbumReleasesBtn.addActionListener(e -> showAlbumReleases());

        JButton viewAlbumTracksBtn = new JButton("Album - Tracks");
        viewAlbumTracksBtn.addActionListener(e -> showAlbumTracks());

        dbaPanel.add(dbaStatusLabel);
        dbaPanel.add(logoutBtn);
        dbaPanel.add(manageDBABtn);
        dbaPanel.add(viewVenuesBtn);
        dbaPanel.add(viewProducerCompanyBtn);
        dbaPanel.add(viewBandMembersBtn);
        dbaPanel.add(viewArtistCompanyBtn);
        dbaPanel.add(viewAlbumArtistBtn);
        dbaPanel.add(viewAlbumReleasesBtn);
        dbaPanel.add(viewAlbumTracksBtn);

        topPanel.add(dbaPanel, BorderLayout.NORTH);

        JPanel mainPanel = new JPanel(new BorderLayout());

        tableSelector = new JComboBox<>();
        tableSelector.addActionListener(e -> {
            String sel = (String) tableSelector.getSelectedItem();
            if (sel != null && !sel.startsWith("---")) {
                loadTableData(sel);
```

```
        }
    });

    tableModel = new DefaultTableModel();
    table = new JTable(tableModel);

    // Κουμπιά για διαχείριση των εγγραφών στους πίνακες της ΒΔ
    JPanel controls = new JPanel();
    JButton insertBtn = new JButton("Εισαγωγή");
    insertBtn.addActionListener(e -> insertRow()); // χρήση της συνάρτησης insertRow για να γίνει εισαγωγή των
                                                   // δεδομένων
    JButton updateBtn = new JButton("Ενημέρωση");
    updateBtn.addActionListener(e -> updateRow()); // χρήση της συνάρτησης updateRow για να γίνει ενημέρωση των
                                                   // δεδομένων
    JButton deleteBtn = new JButton("Διαγραφή");
    deleteBtn.addActionListener(e -> deleteRow()); // χρήση της συνάρτησης deleteRow για να γίνει διαγραφή των
                                                   // δεδομένων

    controls.add(new JLabel("Πίνακας:"));
    controls.add(tableSelector); // Εμφάνιση πινάκων που υπάρχουν στην ΒΔ
    controls.add(insertBtn); // κουμπί εισαγωγής
    controls.add(updateBtn); // κουμπί ενημέρωσης
    controls.add(deleteBtn); // κουμπί διαγραφής

    mainPanel.add(controls, BorderLayout.NORTH);
    mainPanel.add(new JScrollPane(table), BorderLayout.CENTER);

    getContentPane().add(topPanel, BorderLayout.NORTH);
    getContentPane().add(mainPanel, BorderLayout.CENTER);

    SwingUtilities.invokeLater(this::loadTables);
}
```

3. connectToDB()

Αυτή η συνάρτηση δημιουργήθηκε, ώστε να καταφέρουμε να κάνουμε establish την σύνδεση μας στην ΒΔ. Ζητείται από τον χρήση να εισάγει τα στοιχεία της βάσης (το port που λειτουργεί , το username και το password που έχει ορίσει κλπ).

Για προσωπική ευκολία έχω ήδη δηλώσει στο πεδίο όλες τις απαιτούμενες τιμές πέρα από το password. Προφανώς γίνεται και error handling , ώστε σε περίπτωση που δεν καταφέρουμε να επικοινωνήσουμε με την βάση , το πρόγραμμα να τερματίσει.

Κώδικας της συνάρτησης:

```
// Σύνδεση στη ΒΔ (εισαγωγή των credentials manually)
private void connectToDB() {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
    } catch (ClassNotFoundException e) {
        JOptionPane.showMessageDialog(this, "Ο MySQL JDBC driver δεν βρέθηκε: " + e.getMessage(), "Error",
                JOptionPane.ERROR_MESSAGE);
        System.exit(1);
    } // Αναγκαίο σε περίπτωση που ο χρήστης δεν εχει εγκατεστημένο τον JDBC driver

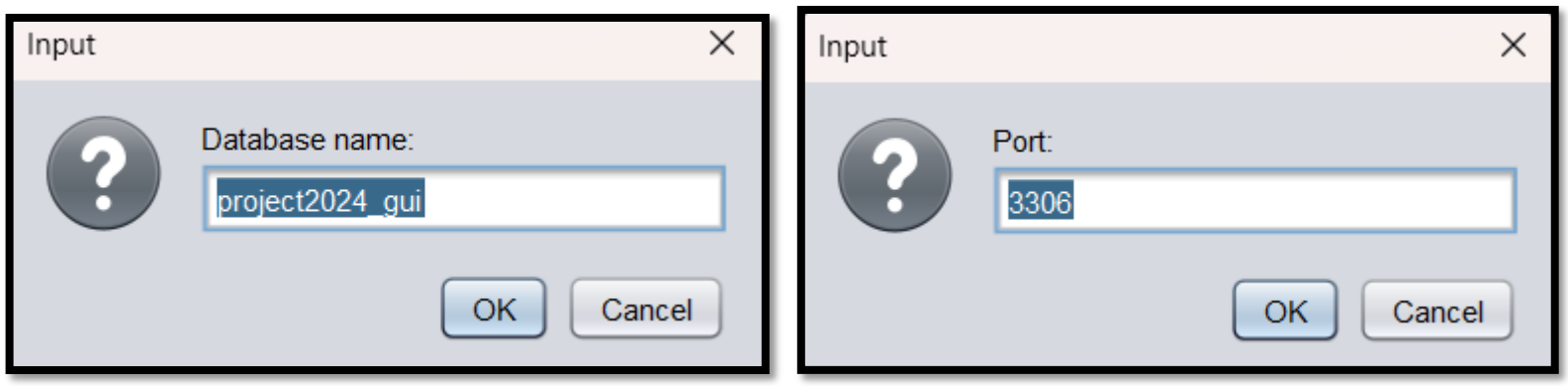
    // Εισαγωγή των credentials για σύνδεση στην DB (υπάρχουν ήδη προκαθορισμένα
    // αυτά που χρησιμοποιώ εγώ)
    while (true) {
        String host = JOptionPane.showInputDialog(this, "MySQL Host:", "localhost");
        if (host == null)
            System.exit(0);
        String port = JOptionPane.showInputDialog(this, "Port:", "3306");
        if (port == null)
            System.exit(0);
        String dbName = JOptionPane.showInputDialog(this, "Όνομα βάσης δεδομένων:", "project2024_gui");
        if (dbName == null)
            System.exit(0);
        String user = JOptionPane.showInputDialog(this, "Username:", "root");
        if (user == null)
            System.exit(0);
        JPasswordField pf = new JPasswordField();
        int okCxl = JOptionPane.showConfirmDialog(this, pf, "Εισάγετε password", JOptionPane.OK_CANCEL_OPTION,
                JOptionPane.PLAIN_MESSAGE);
        if (okCxl != JOptionPane.OK_OPTION)
            System.exit(0);
        String password = new String(pf.getPassword());

        // error handling
        String url = "jdbc:mysql://" + host + ":" + port + "/" + dbName
                + "?serverTimezone=UTC&allowPublicKeyRetrieval=true";
        try {
            conn = DriverManager.getConnection(url, user, password);
            break;
        } catch (SQLException e) {
            int retry = JOptionPane.showConfirmDialog(this,
                    "Σφάλμα σύνδεσης με τη βάση δεδομένων:\n" + e.getMessage() + "\n\nΘέλετε να δοκιμάσετε ξανά;",
```

```
                        "Σφάλμα σύνδεσης",
                        JOptionPane.YES_NO_OPTION,
                        JOptionPane.ERROR_MESSAGE);

                if (retry != JOptionPane.YES_OPTION) {
                    JOptionPane.showMessageDialog(this, "Το πρόγραμμα θα τερματίσει.", "Έξοδος",
                            JOptionPane.INFORMATION_MESSAGE);
                    System.exit(0);
                }
            }
        }
    }
```

Screenshot από τα παράθυρα διάλογου για εισαγωγή port και ονόματος της βάσης που χρησιμοποιούμε



Φιγούρα 65: Στιγμιότυπο από την εκκίνηση της εφαρμογής

4. setSessionDBA()

Η συνάρτηση αυτή δημιουργήθηκε , ώστε να μπορούμε να σεταρουμε εύκολα τον χρήστη που κάνει login και να χρησιμοποιήσουμε το username του για τις αλλαγές που πρόκειται να πραγματοποιηθούν επάνω στην ΒΔ. Δημιουργήθηκε σύμφωνα και με το $SET\ @current_dba := 'admin'$; που χρησιμοποιούσαμε στο Μέρος Α' κατά την εισαγωγή των δεδομένων (όπως αναφέρθηκε και στο μέρος Α' , ακόμα και in real life οι αλλαγές θέλουμε να γίνονται από authenticated χρήστες και όχι από κάποιον που δεν έχει εξουσιοδότηση).

Κώδικας συνάρτησης:

```
    private void setSessionDBA(String username) {
        if (conn == null || username == null)
            return;
        try (Statement s = conn.createStatement()) {
            String safe = username.replace("'", "''");
            s.execute("SET @current_dba = '" + safe + "'");
        } catch (SQLException e) {
            System.err.println("Δεν έγινε η ανάθεση της session variable: " + e.getMessage());
        }
    }
```

5. authenticateDBA ()

Όπως γράψαμε και για την συνάρτηση $setSessionDBA()$, έτσι και εδώ επαναλαμβάνουμε για να τονίσουμε πως **η διαχείριση των ΒΔ** είναι ένα πολύ σημαντικό ζήτημα στην πραγματική ζωή που **έρχεται με πολλές ευθύνες και πολλές καταστροφικές συνέπειες αν υπάρξουν λάθος κινήσεις**.

Επομένως, πρέπει πάντοτε εξουσιοδοτημένα πρόσωπα να δουλεύουν επάνω στην εκάστοτε ΒΔ, για να αποφευχθούν τεράστιες οικονομικές καταστροφές (φανταστείτε λ.χ. μέσα σε μια μέρα να σβηνόταν η ΒΔ του Instagram από ανθρώπινη απροσεξία, πόσα εκατομμύρια ζημιά θα ήταν αυτή για την Meta;).

Γι' αυτό λοιπόν, με αυτή την συνάρτηση ζητάμε από τον χρήστη να εισάγει το Username του. Αν δεν υπάρχει ΚΑΝΕΝΑ username στον πίνακα dba της ΒΔ (άρα μιλάμε για άδειο πίνακα dba), μπορούμε να δημιουργήσουμε εκείνη την στιγμή ένα εισάγοντας όλα τα απαραίτητα στοιχεία. Αν όμως υπάρχουν χρήστες και πληκτρολογήσουμε λάθος Username που δεν υπάρχει μέσα στον πίνακα , τότε ζητάμε την εισαγωγή του σωστού username ξανά.

Κώδικας συνάρτησης:

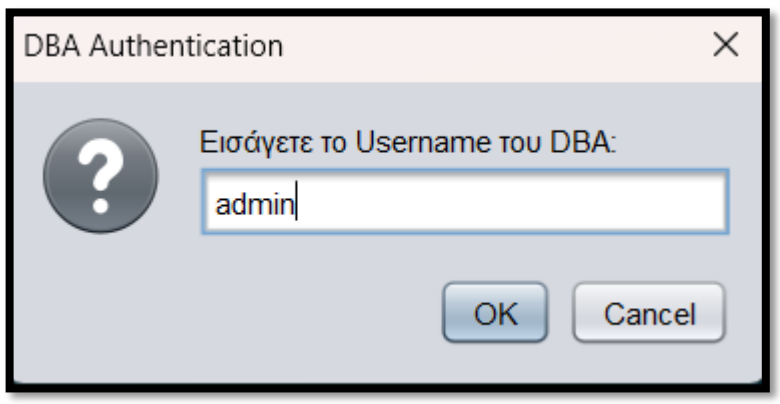
```
private boolean authenticateDBA() {
        while (true) {
            String username = JOptionPane.showInputDialog(this,
                    "Εισάγετε το Username του DBA:",
                    "Αυθεντικοποίηση DBA",
                    JOptionPane.QUESTION_MESSAGE);

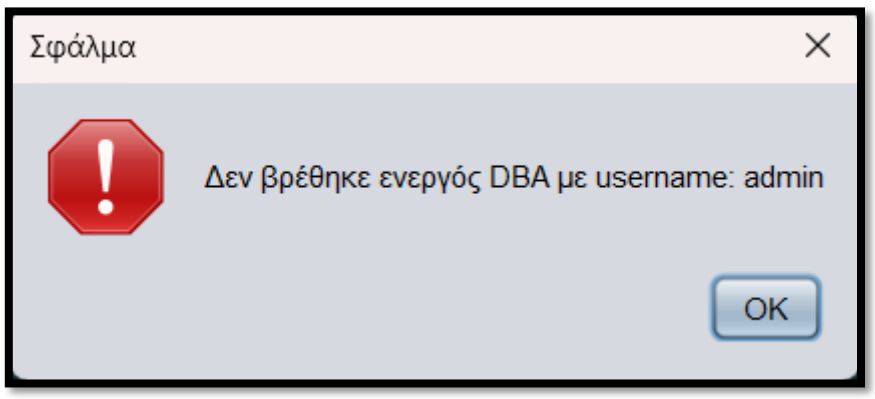
            if (username == null)
                return false;
            username = username.trim();
            if (username.isEmpty()) {
                JOptionPane.showMessageDialog(this,
                        "Παρακαλώ εισάγετε έγκυρο username.",
                        "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                continue;
            }

            try (PreparedStatement ps = conn
                    .prepareStatement("SELECT DBA_ID, Username FROM dba WHERE Username = ? AND End_Date IS NULL")) {
                ps.setString(1, username);
                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) {
                        currentDBAId = rs.getInt("DBA_ID");
                        currentDBAUsername = rs.getString("Username");
                        setSessionDBA(currentDBAUsername);
                        JOptionPane.showMessageDialog(this,
                                "Καλώς ήρθατε " + currentDBAUsername + "!\n",
                                "Επιτυχής είσοδος", JOptionPane.INFORMATION_MESSAGE);
                        return true;
                    }
                }

                // Σε περίπτωση που δεν υπάρχει ενεργός DBA , δημιουργούμε νέο και κάνουμε login
                // με αυτόν
                try (Statement s2 = conn.createStatement();
                        ResultSet cntRs = s2.executeQuery("SELECT COUNT(*) FROM dba WHERE End_Date IS NULL")) {
                    int activeCount = 0;
                    if (cntRs.next())
                        activeCount = cntRs.getInt(1);
                    if (activeCount == 0) {
                        int create = JOptionPane.showConfirmDialog(this,
                                "Δεν υπάρχει ενεργός DBA. Θέλετε να δημιουργήσετε έναν διαχειριστή τώρα?",
                                "Δημιουργία Διαχειριστή",
                                JOptionPane.YES_NO_OPTION);
                        if (create == JOptionPane.YES_OPTION) {
                            addNewDBA();
                            if (currentDBAUsername != null) {
                                JOptionPane.showMessageDialog(this,
                                        "Καλώς ήρθατε " + currentDBAUsername
                                                + "!",
                                        "Επιτυχής είσοδος", JOptionPane.INFORMATION_MESSAGE);
                                return true;
                            }
                        }
                    }
                    // Σε περίπτωση που δεν βρεθεί κάποιος ενεργός DBA, το εμφανίζουμε στον χρήστη
                    JOptionPane.showMessageDialog(this, "Δεν βρέθηκε ενεργός DBA με username: " + username, "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                } catch (SQLException e) {
                    JOptionPane.showMessageDialog(this, "Σφάλμα κατά την επαλήθευση: " + e.getMessage(), "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                }
            }
        }
    }
```

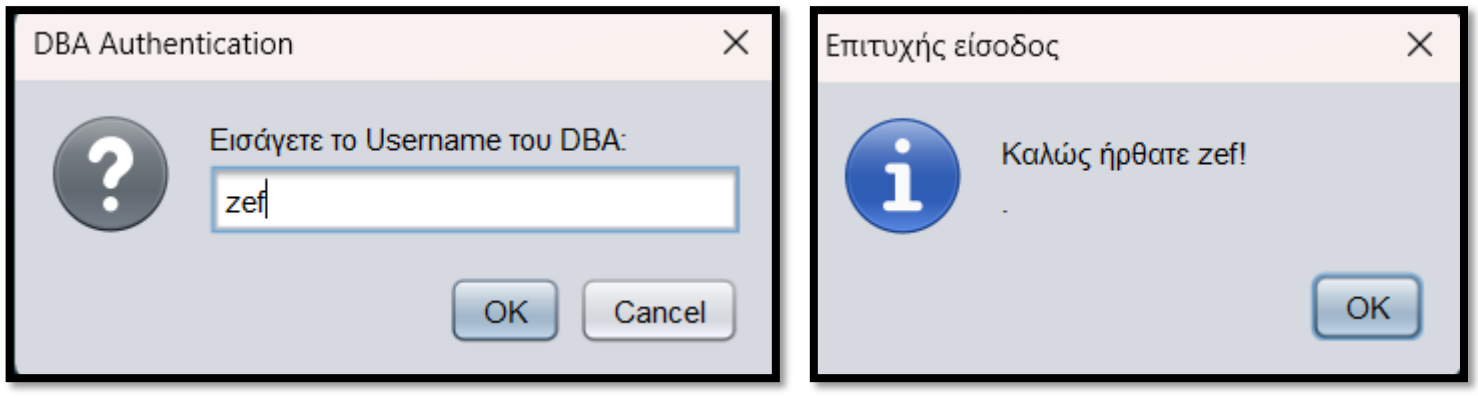
- Περίπτωση που ο DBA με Username ‘admin’ δεν υπάρχει στον πίνακα DBA





Φιγούρα 66: Στιγμιότυπο από την εφαρμογή

- Περίπτωση που κάνουμε επιτυχημένο Login



Φιγούρα 67: Στιγμιότυπο από την εφαρμογή

6. changeDBA()

Όπως αναφέρθηκε και παραπάνω (Βλ.**SetupGUI()**) , η συνάρτηση αυτή δημιουργήθηκε για να μπορέσουμε εύκολα να αλλάξουμε χρήστη όποτε το επιθυμούμε. Η λειτουργία είναι απλή… όταν είμαστε logged in με τον χρήστη X και θέλουμε να κάνουμε Login με τον χρήστη Y , τότε πατάμε το κουμπί και ζητείται να εισάγουμε το username του χρήστη που θέλουμε να κάνει Login . Αν το εισάγουμε σωστά , τότε όλα καλά. Αν όχι , η εφαρμογή μας κάνει Logout και κλείνει.

Κώδικας συνάρτησης :

```
// Συνάρτηση για αλλαγή του DBA
private void changeDBA() {
    int result = JOptionPane.showConfirmDialog(this,
            "Θέλετε να αλλάξετε DBA? ",
            "Αλλαγή DBA",
            JOptionPane.YES_NO_OPTION);

    if (result == JOptionPane.YES_OPTION) {
        currentDBAId = null; // μεταβλητή για να λάβει το ID του DBA που θέλει να συνδεθεί
        currentDBAUsername = null; // μεταβλητή για να λάβει το username του DBA που θέλει να συνδεθεί

        // αν έχουμε επιτυχή αυθεντικοποίηση
        if (authenticateDBA()) {
            dbaStatusLabel.setText("DBA: " + currentDBAUsername + " (ID: " + currentDBAId + ")");
            JOptionPane.showMessageDialog(this,
                    "Επιτυχής αλλαγή DBA σε: " + currentDBAUsername,
                    "Αλλαγή DBA",
                    JOptionPane.INFORMATION_MESSAGE);
        } else {
            System.exit(0);
        }
    }
}
```

7. manageDBAUsers()

Συγκεντρωτική συνάρτηση που περιέχει όλα τα κουμπιά που αναφέρθηκαν παραπάνω στο **SetupGUI()**. Δεν χρειάζεται ιδιαίτερη επεξήγηση , μιας και υπάρχουν σχόλια εσωτερικά του κώδικα.

Κώδικας συνάρτησης:

```
// Συναρτηση για την διαχειριση των DBA χρηστών
private void manageDBAUsers() {
    String[] options = { "Προσθήκη νέου DBA", "Προβολή όλων των DBA", "Διαγραφή DBA", "Ακύρωση" };
    int choice = JOptionPane.showOptionDialog(this,
            "Τι θέλετε να κάνετε;",
            "Διαχείριση DBA Χρηστών",
            JOptionPane.DEFAULT_OPTION,
            JOptionPane.QUESTION_MESSAGE,
            null,
            options,
            options[0]);

    switch (choice) {
        case 0:
            addNewDBA(); // Προσθήκη νέου DBA
            break;
        case 1:
            viewAllDBAs(); // Προβολή όλων των DBA
            break;
        case 2:
            deleteDBA(); // Διαγραφή DBA
            break;
        default:
            break;
    }
}
```

```
    // Προσθήκη νέου DBA
    private void addNewDBA() {
        JPanel panel = new JPanel(new GridLayout(5, 2));
        JTextField usernameField = new JTextField();
        JTextField firstNameField = new JTextField();
        JTextField lastNameField = new JTextField();
        JTextField emailField = new JTextField();
        // Δημιουργουμε πεδια για την εισαγωγη των δεδομενων του DBA .
        // Ο,τι γράφουμε , περνάει στην ΒΔ project2024_gui
        // στον πινακα 'dba' που περιεχει τα νεα μας πεδια ( αυτή είναι η διαφορά με την
        // δομή της βάσης της 1ης φάσης)
        panel.add(new JLabel("Username:"));
        panel.add(usernameField);
        panel.add(new JLabel("Όνομα:"));
        panel.add(firstNameField);
        panel.add(new JLabel("Επώνυμο:"));
        panel.add(lastNameField);
        panel.add(new JLabel("Email:"));
        panel.add(emailField);

        int result = JOptionPane.showConfirmDialog(this, panel,
                "Προσθήκη νέου DBA", JOptionPane.OK_CANCEL_OPTION);

        if (result != JOptionPane.OK_OPTION)
            return;

        String username = usernameField.getText().trim();
        String firstName = firstNameField.getText().trim();
        String lastName = lastNameField.getText().trim();
        String email = emailField.getText().trim();

        // Αν ο χρήστης παραλείψει να βάλει username εμφαν΄ίζουμε σφάλμα
        if (username.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Το username είναι υποχρεωτικό!",
                    "Σφάλμα", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Εδώ ελέγχουμε αν υπάρχει το username στον πίνακα DBA με την χρήση sql query.
        // Αν υπάρχει ήδη, τότε το εμφανίζουμε στον χρήστη και τον προτρέπουμε να
        // επιλέξει
        // άλλο
        try (PreparedStatement checkStmt = conn.prepareStatement("SELECT COUNT(*) FROM dba WHERE Username = ?")) {
            checkStmt.setString(1, username);
            try (ResultSet rs = checkStmt.executeQuery()) {
                if (rs.next() && rs.getInt(1) > 0) {
                    JOptionPane.showMessageDialog(this,
                            "Το username '" + username + "' υπάρχει ήδη!",
                            "Σφάλμα", JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα βάσης δεδομένων: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Προετοιμάζουμε την εισαγωγή των δεδομένων στον πίνακα της βάσης. Ωστόσο
        // κάνουμε και έναν έλεγχο αν έχουν προστεθεί σωστά τα πεδία ,
        // διότι άνθρωποι είμαστε. Αν υπάρχει κάτι κενό ,
        // τότε εμφανίζουμε σφάλμα και δεν εισάγουμε κάτι στον πίνακα
        try (PreparedStatement insertStmt = conn.prepareStatement(
                "INSERT INTO dba (Username, FirstName, LastName, Email, Start_Date) VALUES (?, ?, ?, ?, CURDATE())",
                Statement.RETURN_GENERATED_KEYS)) {

            insertStmt.setString(1, username);
            insertStmt.setString(2, firstName.isEmpty() ? null : firstName);
            insertStmt.setString(3, lastName.isEmpty() ? null : lastName);
            insertStmt.setString(4, email.isEmpty() ? null : email);

            int rowsAffected = insertStmt.executeUpdate();
            if (rowsAffected <= 0) {
                JOptionPane.showMessageDialog(this, "Σφάλμα κατά την προσθήκη του DBA!", "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }

            try (ResultSet gk = insertStmt.getGeneratedKeys()) {
                if (gk != null && gk.next())
                    currentDBAId = gk.getInt(1);
            }

            if (currentDBAId == null || currentDBAId == 0) {
                try (PreparedStatement q = conn.prepareStatement("SELECT DBA_ID FROM dba WHERE Username = ? LIMIT 1")) {
                    q.setString(1, username);
                    try (ResultSet r2 = q.executeQuery()) {
                        if (r2.next())
                            currentDBAId = r2.getInt(1);
                    }
                }
            }
            // Εφόσον έχουμε το currentDBAId, μπορούμε να το χρησιμοποιήσουμε τώρα γι' αυτό
            // και το θέτουμε στην μεταβλητή currentDBAUsername που δημιουργήσαμε νωρίτερα
```

```
            currentDBAUsername = username;
            setSessionDBA(currentDBAUsername);

            // Επιτυχία προσθήκης και σφάλμα αν υπήρξε πρόβλημα στην ΒΔ
            JOptionPane.showMessageDialog(this, "Ο DBA '" + username + "' προστέθηκε επιτυχώς!", "Επιτυχία",
                    JOptionPane.INFORMATION_MESSAGE);
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα βάσης δεδομένων: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    private void viewAllDBAs() {
        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(
                        "SELECT DBA_ID, Username, FirstName, LastName, Email, Start_Date, End_Date FROM dba ORDER BY Username")) {

            StringBuilder dbaList = new StringBuilder();
            dbaList.append("Λίστα όλων των DBA:\n\n");
            dbaList.append(String.format("%-5s %-15s %-15s %-15s %-25s %-12s %-12s\n",
                    "ID", "Username", "Όνομα", "Επώνυμο", "Email", "Start_Date", "End_Date"));
            for (int i = 0; i < 100; i++)
                dbaList.append("—");
            dbaList.append("\n");

            while (rs.next()) {
                dbaList.append(String.format("%-5d %-15s %-15s %-15s %-25s %-12s %-12s\n",
                        rs.getInt("DBA_ID"),
                        rs.getString("Username") != null ? rs.getString("Username") : "",
                        rs.getString("FirstName") != null ? rs.getString("FirstName") : "",
                        rs.getString("LastName") != null ? rs.getString("LastName") : "",
                        rs.getString("Email") != null ? rs.getString("Email") : "",
                        rs.getDate("Start_Date") != null ? rs.getDate("Start_Date").toString() : "",
                        rs.getDate("End_Date") != null ? rs.getDate("End_Date").toString() : "Ενεργός"));
            }

            JTextArea textArea = new JTextArea(dbaList.toString());
            textArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
            textArea.setEditable(false);
            JScrollPane scrollPane = new JScrollPane(textArea);
            scrollPane.setPreferredSize(new Dimension(800, 400));

            JOptionPane.showMessageDialog(this, scrollPane, "Λίστα DBA", JOptionPane.INFORMATION_MESSAGE);
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα κατά την ανάκτηση των DBA: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    // Διαγραφή ενός DBA από την ΒΔ
    private void deleteDBA() {
        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(
                        "SELECT DBA_ID, Username FROM dba WHERE DBA_ID != " + currentDBAId + " ORDER BY Username")) {

            ArrayList<String> dbaOptions = new ArrayList<>();
            java.util.Map<String, Integer> usernameToId = new java.util.HashMap<>();

            while (rs.next()) {
                String username = rs.getString("Username");
                int id = rs.getInt("DBA_ID");
                dbaOptions.add(username + " (ID: " + id + ")");
                usernameToId.put(username + " (ID: " + id + ")", id);
            }

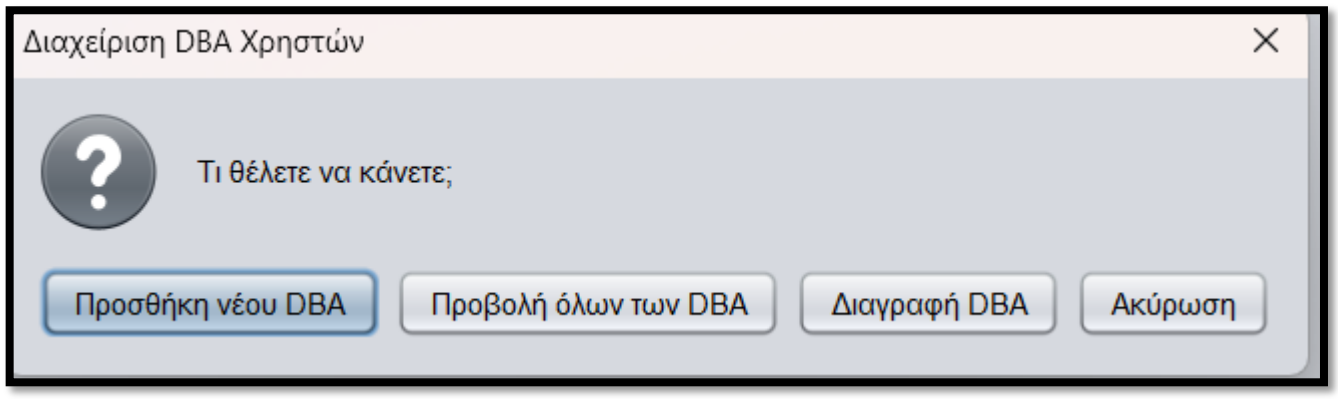
            // αν δεν υπαρχουν αλλοι χρηστες στον πινακα περα απο εμας που ειμαστε logged in
            if (dbaOptions.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Δεν υπάρχουν άλλοι DBA για διαγραφή!", "Πληροφορία",
                        JOptionPane.INFORMATION_MESSAGE);
                return;
            }
            // Εμφανιση παραθυρου διαλογου ωστε να επιλεξουμε τον χρηστη για διαγραφη
            String selected = (String) JOptionPane.showInputDialog(this,
                    "Επιλέξτε DBA για διαγραφή:\n(Δεν μπορείτε να διαγράψετε τον εαυτό σας)",
                    "Διαγραφή DBA",
                    JOptionPane.QUESTION_MESSAGE,
                    null,
                    dbaOptions.toArray(),
                    dbaOptions.get(0));

            if (selected != null) {
                int selectedId = usernameToId.get(selected);

                // παραθυρο επιβεβαιωσης
                int confirm = JOptionPane.showConfirmDialog(this,
                        "Είστε σίγουροι ότι θέλετε να διαγράψετε τον DBA: " + selected + "?\n" +
                                "Αυτή η ενέργεια θα ορίσει End_Date στον DBA και δεν μπορεί να αναιρεθεί!",
                        "Επιβεβαίωση Διαγραφής",
                        JOptionPane.YES_NO_OPTION,
                        JOptionPane.WARNING_MESSAGE);

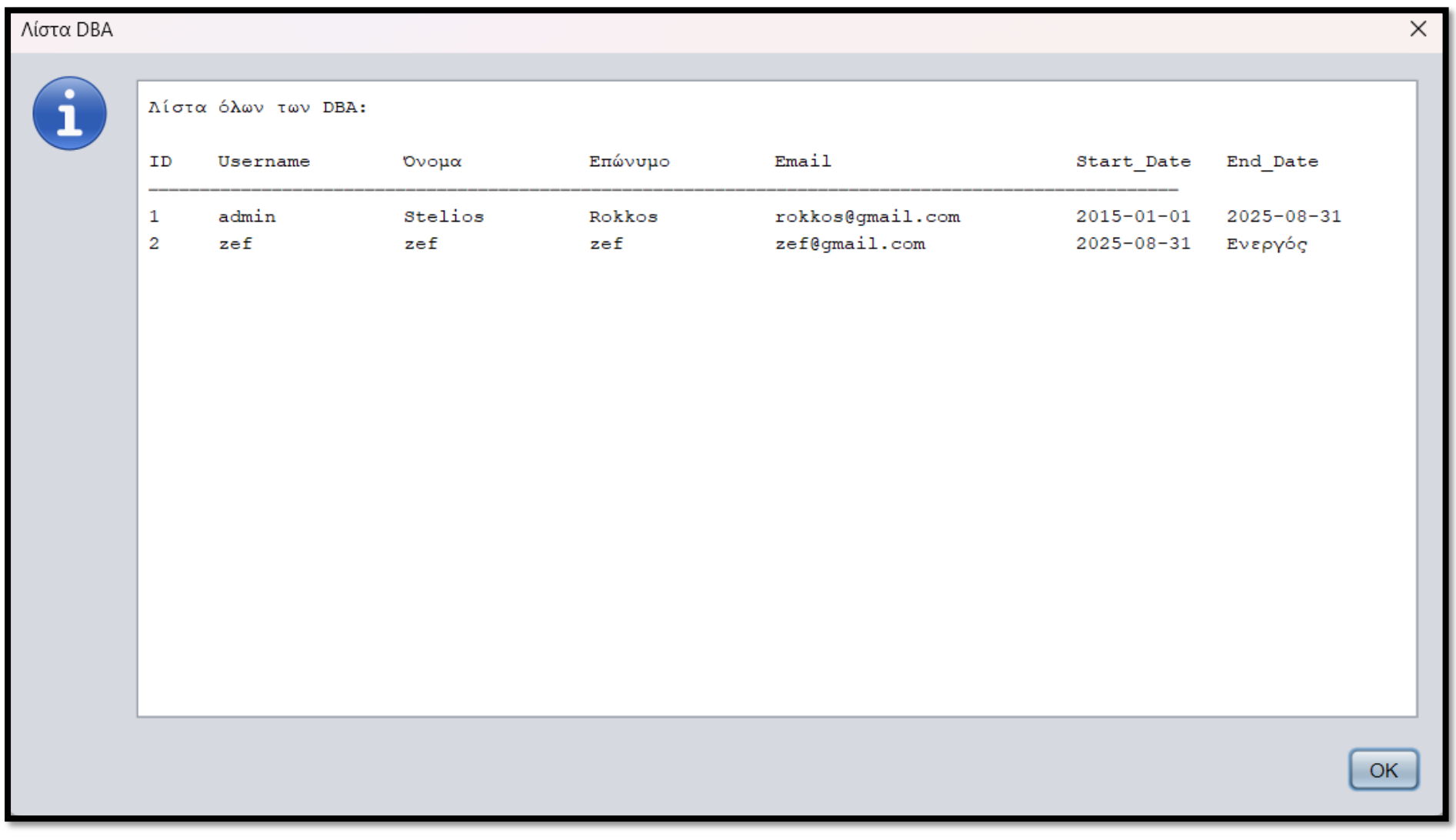
                // καλουμε το συγκεκριμενο sql query ωστε να κανουμε
                // ενημερωση του πινακα.
                // για την διαγραφη αυτο που κανουμε ουσιαστικα ειναι να ορισουμε ως end date
```

```
                // την τωρινη στιγμη με την συναρτηση NOW()
                // ετσι παυει να ειναι ενεργος ο χρηστης , αρα δεν μπορει να χρησιμοποιηθει
                // ξανα στην ΒΔ
                if (confirm == JOptionPane.YES_OPTION) {
                    try (PreparedStatement updateStmt = conn
                            .prepareStatement("UPDATE dba SET End_Date = CURDATE() WHERE DBA_ID = ?")) {
                        updateStmt.setInt(1, selectedId);
                        int rowsAffected = updateStmt.executeUpdate();
                        if (rowsAffected > 0) {
                            JOptionPane.showMessageDialog(this, "Ο DBA απενεργοποιήθηκε επιτυχώς!", "Επιτυχία",
                                    JOptionPane.INFORMATION_MESSAGE);
                        } else {
                            JOptionPane.showMessageDialog(this, "Σφάλμα κατά την απενεργοποίηση του DBA!", "Σφάλμα",
                                    JOptionPane.ERROR_MESSAGE);
                        }
                    }
                }
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα βάσης δεδομένων: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
```



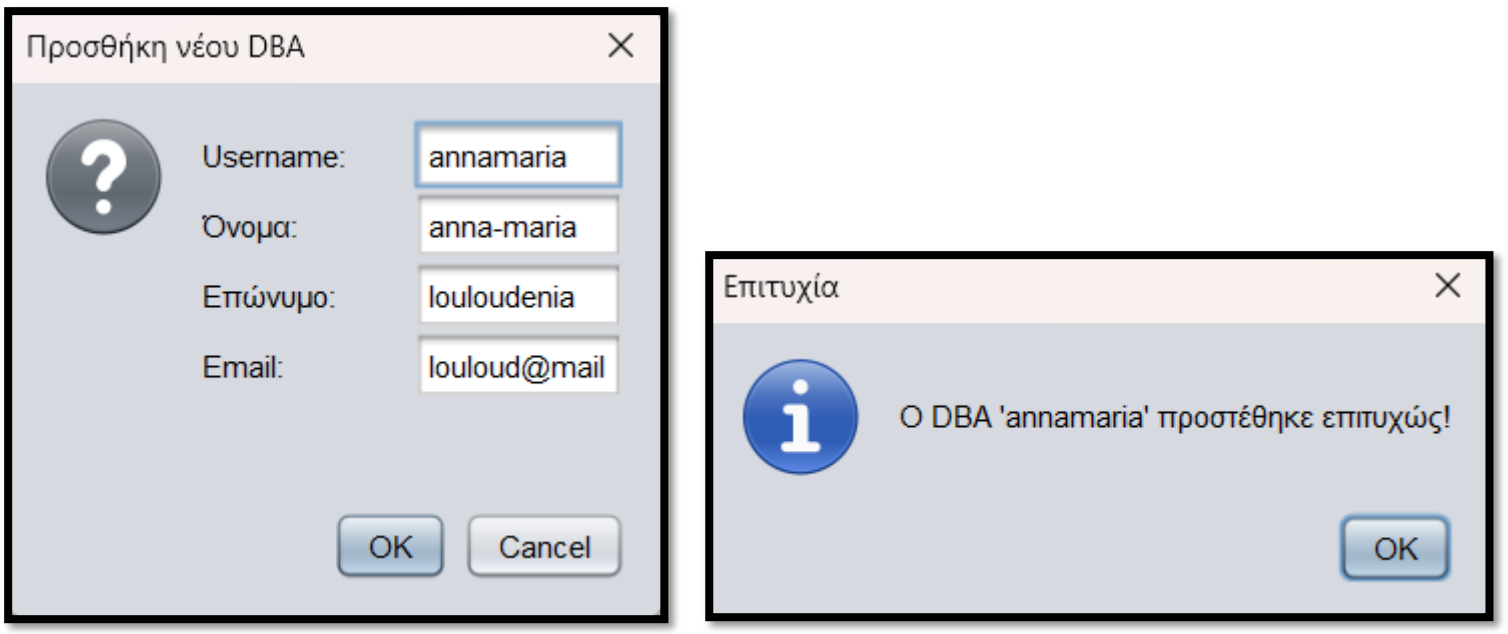
Φιγούρα 68: Στιγμιότυπο από το κουμπί 'Διαχείριση DBA'

- **Προβολή όλων των DBA**



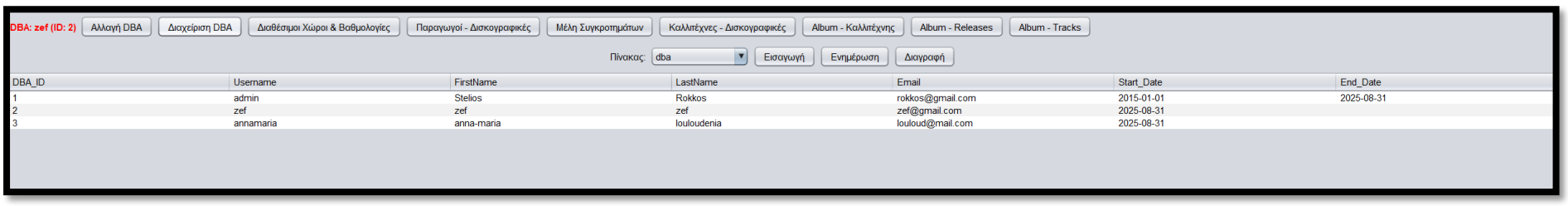
Φιγούρα 69: Χρήστες DBA

- **Προσθήκη νέου DBA User**

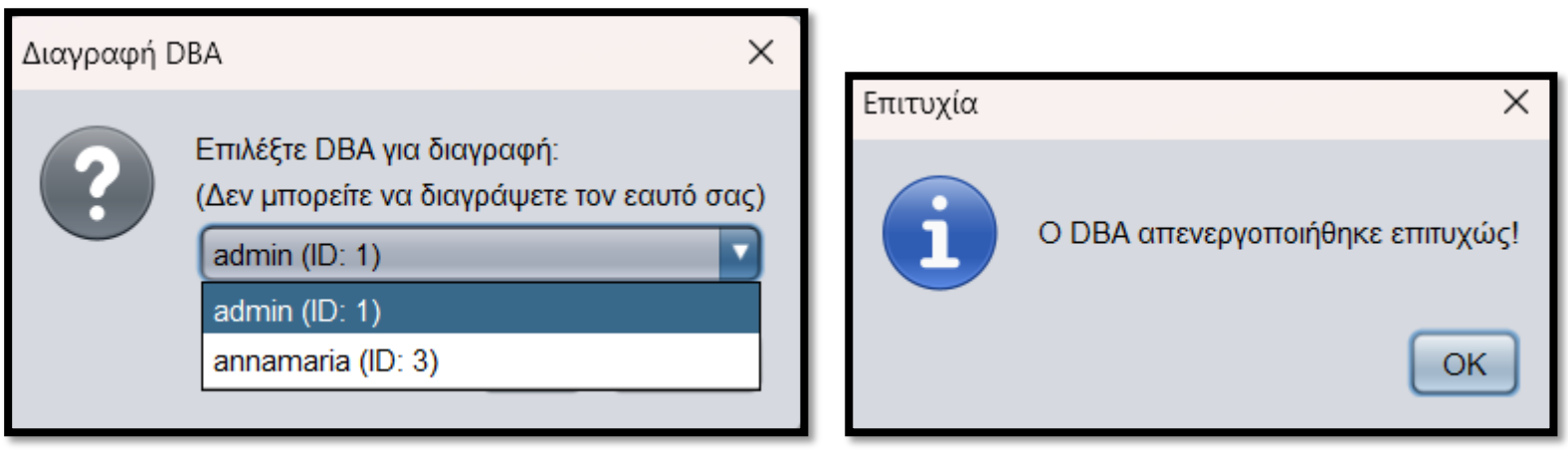


Φιγούρα 70 Στιγμιότυπο την εφαρμογή

- **Διαγραφή DBA**

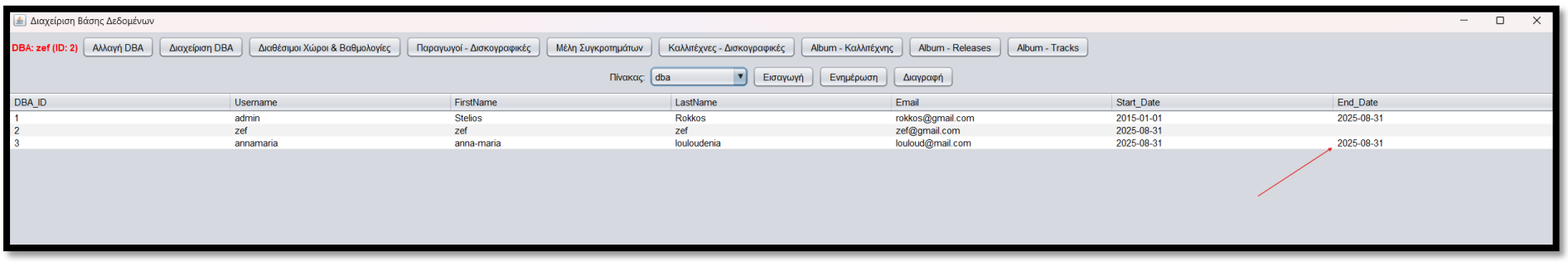


Φιγούρα 71: Στιγμιότυπο την εφαρμογή



Φιγούρα 72: Στιγμιότυπο την εφαρμογή

Φορτώνοντας πάλι τον πίνακα έχουμε :



Φιγούρα 73: Στιγμιότυπο την εφαρμογή

8. showVenuesWithScores()

Εδώ όπως αναγράφεται και στα σχόλια μέσα στον κώδικα, αξιοποιούμε την stored procedure που είχαμε δημιουργήσει με όνομα *CalculateVenueScore*. Εμφανίζουμε όλους τους χώρους που είναι διαθέσιμοι για να προγραμματίσουμε συναυλία (δηλαδή όλοι αυτοί που έχουν *IsAvailable* = 1) και δίπλα το score που τους έχει ανατεθεί χάρη στην stored procedure. Μπορούμε επίσης να επιλέξουμε έναν χώρο και να δούμε αναλυτικά τι συναυλίες είχαν πραγματοποιηθεί (δεδομένα από τον πίνακα concert_history)

Κώδικας συνάρτησης:

```
    // Κλήση ολων των διαθεσιμων χώρων (απο την SQL)
    private void showVenuesWithScores() {
        try (PreparedStatement ps = conn.prepareStatement(
                "SELECT VenueID, VenueName, Capacity, CompletedConcerts, YearsOperating FROM venue WHERE IsAvailable = 1")) {
            try (ResultSet rs = ps.executeQuery()) {
                DefaultTableModel model = new DefaultTableModel();
                model.addColumn("VenueID");
                model.addColumn("VenueName");
                model.addColumn("Capacity");
                model.addColumn("CompletedConcerts");
                model.addColumn("YearsOperating");
                model.addColumn("VenueScore");

                while (rs.next()) {
                    int id = rs.getInt("VenueID");
                    String name = rs.getString("VenueName");
                    int cap = rs.getInt("Capacity");
                    int comp = rs.getInt("CompletedConcerts");
                    int years = rs.getInt("YearsOperating");

                    Integer score = null;
                    // εδω αξιοποιουμε το stored procedure που ειχαμε δημιουργησει για την απαιτηση
                    // που αφορα το σκορ των χωρων
                    // το καλουμε και το θετουμε ως venue score και το εμφανιζουμε
                    try (CallableStatement cs = conn.prepareCall("{ CALL CalculateVenueScore(?) }")) {
                        cs.setInt(1, id);
                        boolean has = cs.execute();
                        if (has) {
                            try (ResultSet prs = cs.getResultSet()) {
                                if (prs.next())
                                    score = prs.getInt("VenueScore");
                            }
                        }
                    } catch (SQLException ex) {
                        score = (cap / 1000) * 1 + (comp / 100) * 3 + years * 2;
                    }

                    model.addRow(new Object[] { id, name, cap, comp, years, score });
                }
                // η εμφανιση των παραπανω γινεται σε table γιαυτο και χρησιμοποιουμε το JTable
                JTable vTable = new JTable(model);
                JScrollPane pane = new JScrollPane(vTable);
                vTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

                JPanel panel = new JPanel(new BorderLayout());
                panel.add(pane, BorderLayout.CENTER);

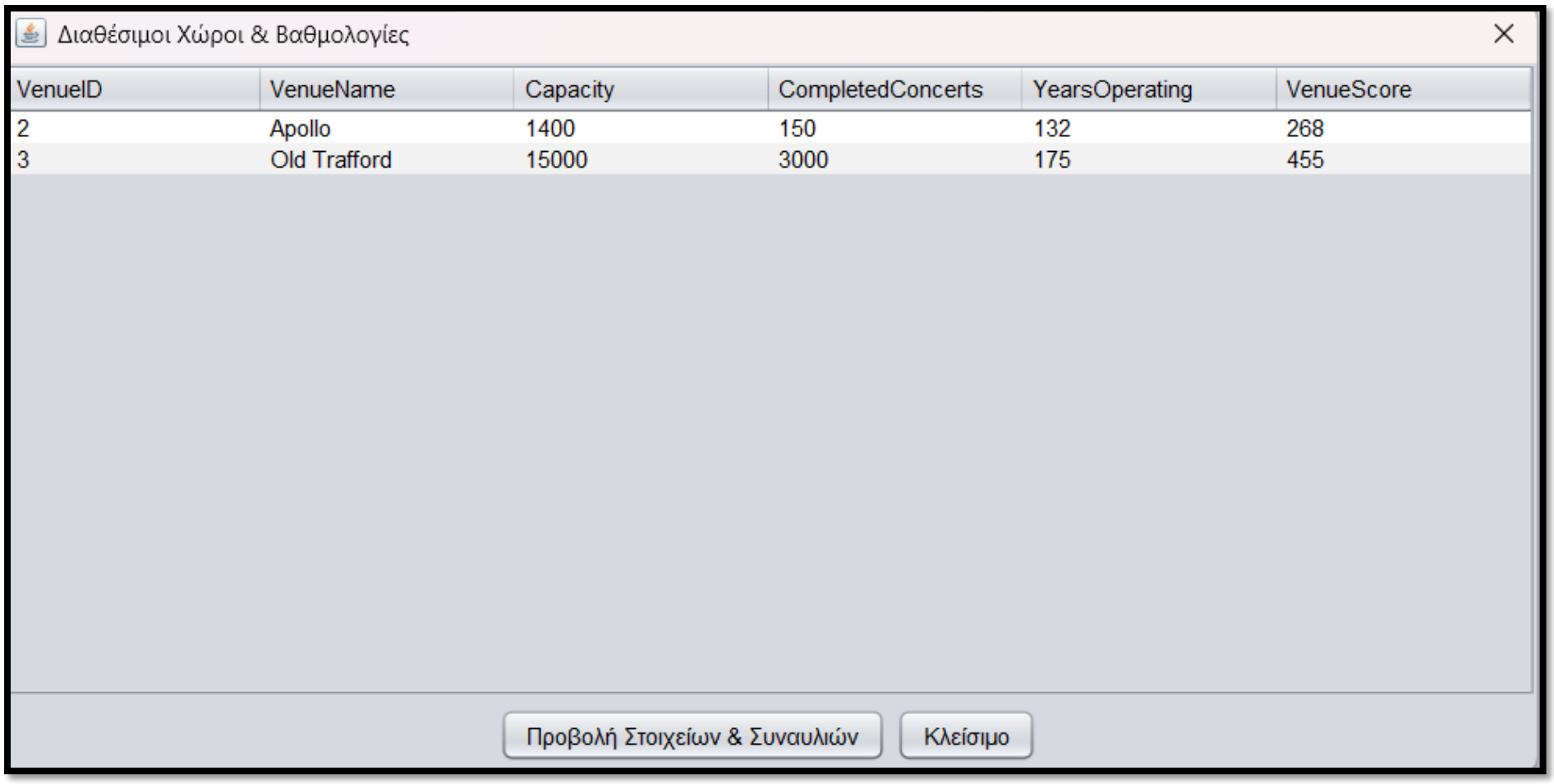
                // Νεα οθονη που εμφανιζει στοιχεια συναυλιων
                JPanel btns = new JPanel();
                JButton detailsBtn = new JButton("Προβολή Στοιχείων & Συναυλιών");
                JButton closeBtn = new JButton("Κλείσιμο");
                btns.add(detailsBtn);
                btns.add(closeBtn);
                panel.add(btns, BorderLayout.SOUTH);

                JDialog dialog = new JDialog(this, "Διαθέσιμοι Χώροι & Βαθμολογίες", true);
                dialog.getContentPane().add(panel);
                dialog.setSize(800, 400);
                dialog.setLocationRelativeTo(this);

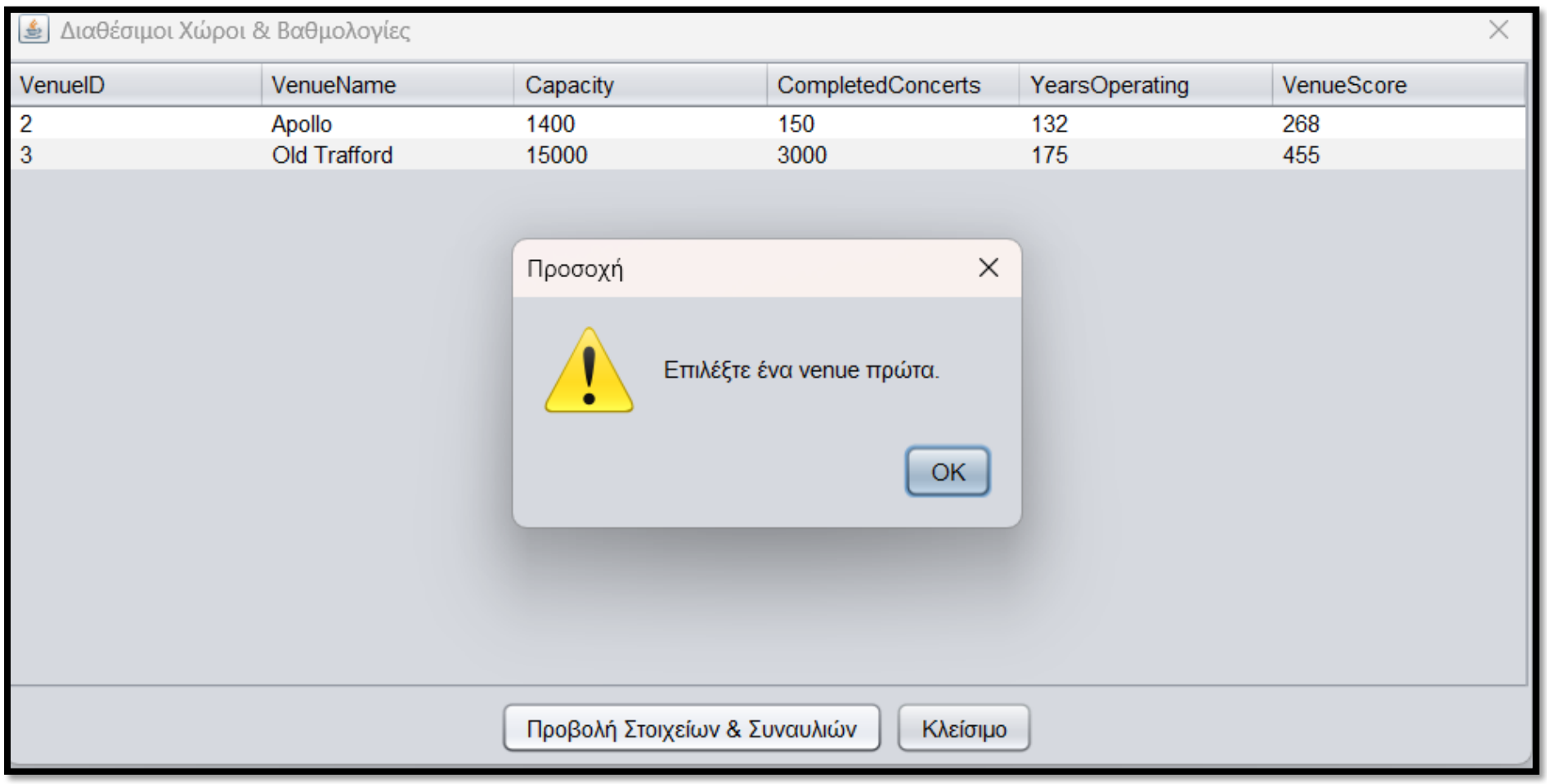
                detailsBtn.addActionListener(ev -> {
                    int sel = vTable.getSelectedRow();
                    if (sel == -1) { // σε περιπτωση που δεν επιλεγει ενας χωρος για να δουμε τι συναυλιες υπαρχουν
                                     // (απο το ιστορικο συναυλιων τις αντλουμε)
                        JOptionPane.showMessageDialog(dialog, "Επιλέξτε ένα venue πρώτα.", "Προσοχή",
                                JOptionPane.WARNING_MESSAGE);
                        return;
                    }
                    int venueId = (Integer) vTable.getValueAt(sel, 0);
                    showVenueDetails(venueId);
                });

                closeBtn.addActionListener(ev -> dialog.dispose());

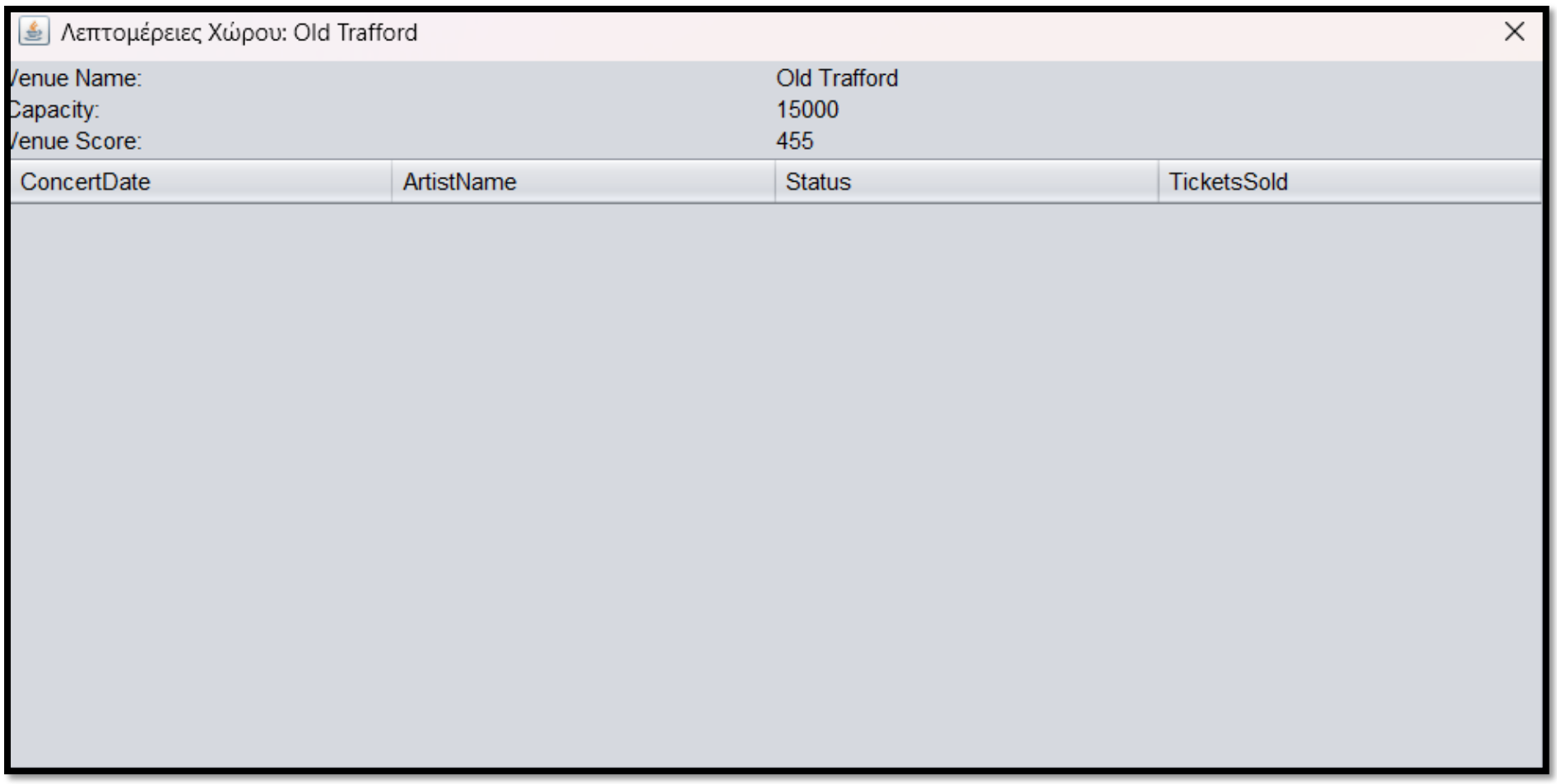
                dialog.setVisible(true);
            } // error handling
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα κατά την ανάκτηση χώρων: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
```



Φιγούρα 74: Στιγμιότυπο την εφαρμογή



Φιγούρα 75: Στιγμιότυπο την εφαρμογή



Φιγούρα 76: Στιγμιότυπο την εφαρμογή

Εμφάνιση πινάκων με συσχετίσεις

9. showBandmembers()

Το παρακάτω κομμάτι κώδικα ουσιαστικά δημιουργεί έναν πίνακα ώστε να βλέπουμε σε ένα συγκρότημα ποια μέλη της ΒΔ είναι σε αυτά. (Πιο αναλυτικά στο **SetupGUI()**)

Κώδικας συνάρτησης :

```
private void showBandMembers() {
    try {

        String checkTablesQuery = "SELECT TABLE_NAME FROM information_schema.TABLES " +
                "WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME IN ('band', 'bandmember', 'person')";

        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(checkTablesQuery)) {

            java.util.List<String> existingTables = new java.util.ArrayList<>();
            while (rs.next()) {
                existingTables.add(rs.getString("TABLE_NAME"));
            }

            if (!existingTables.contains("band") || !existingTables.contains("person")) {
                JOptionPane.showMessageDialog(this,
                        "Οι πίνακες band ή person δεν υπάρχουν στη βάση δεδομένων.",
                        "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }
        }

        // Query για να κανουμε fetch τα bandmembers
        String membersQuery = "SELECT b.BandName, CONCAT(p.FirstName, ' ', p.LastName) as MemberName, " +
                "bm.FromDate, bm.ToDate " +
                "FROM band b " +
                "INNER JOIN bandmember bm ON b.BandID = bm.BandID " +
                "INNER JOIN person p ON bm.PersonID = p.PersonID " +
                "ORDER BY b.BandName, bm.FromDate";

        // εμφανιζουμε τα αποτελεσματα σε table
        DefaultTableModel membersModel = new DefaultTableModel(
                new String[] { "Συγκρότημα", "Μέλος", "Από", "Έως" }, 0);

        // εισαγουμε τα δεδομενα που αντλησαμε απο τους πινακες
        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(membersQuery)) {

            while (rs.next()) {
                String bandName = rs.getString("BandName");
                String memberName = rs.getString("MemberName");
                java.sql.Date fromDate = rs.getDate("FromDate");
                java.sql.Date toDate = rs.getDate("ToDate");
```

```
                String fromDateStr = fromDate != null ? fromDate.toString() : "";
                String toDateStr = toDate != null ? toDate.toString() : "Ενεργό μέλος";

                membersModel.addRow(new Object[] {
                        bandName,
                        memberName,
                        fromDateStr,
                        toDateStr
                });
            }
        }

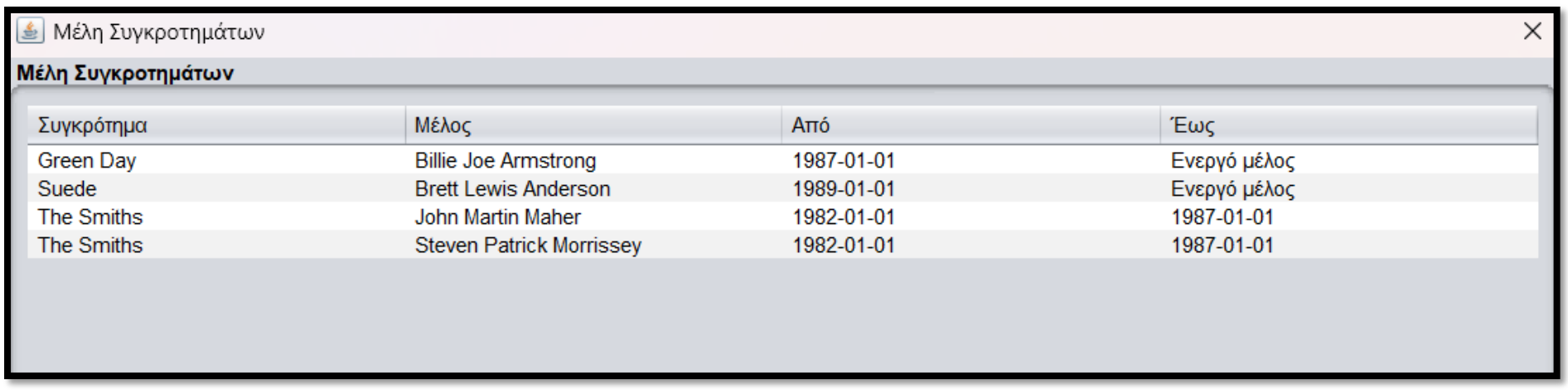
        // Δημιουργια JTable για εμφανιση
        JTable membersTable = new JTable(membersModel);

        // σκρολ
        JScrollPane scrollPane = new JScrollPane(membersTable);
        scrollPane.setPreferredSize(new Dimension(800, 400));

        scrollPane.setBorder(BorderFactory.createTitledBorder("Μέλη Συγκροτημάτων"));

        JDialog dialog = new JDialog(this, "Μέλη Συγκροτημάτων", true);
        dialog.getContentPane().add(scrollPane);
        dialog.setSize(900, 500);
        dialog.setLocationRelativeTo(this);
        dialog.setVisible(true);

    } catch (SQLException e) {
            JOptionPane.showMessageDialog(this,
                    "Σφάλμα κατά την ανάκτηση των μελών: " + e.getMessage(),
                    "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
```



Φιγούρα 77: Στιγμιότυπο την εφαρμογή

10. showArtistCompanyRelations()

Δεν χρειάζεται περαιτέρω ανάλυση καθώς έχει ήδη εξηγηθεί στο **SetupGUI()**. Η προγραμματιστική λογική είναι ακριβώς ίδια με το **showBandmembers()**.

Κώδικας συνάρτησης :

```
    // ΙΔΙΑ ΛΟΓΙΚΗ ΜΕ showBandMembers
    private void showArtistCompanyRelations() {
        try {

            String checkTablesQuery = "SELECT TABLE_NAME FROM information_schema.TABLES " +
                    "WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME IN ('artist', 'artistcompany', 'recordcompany')";

            try (Statement stmt = conn.createStatement();
                    ResultSet rs = stmt.executeQuery(checkTablesQuery)) {

                java.util.List<String> existingTables = new java.util.ArrayList<>();
                while (rs.next()) {
                    existingTables.add(rs.getString("TABLE_NAME"));
                }

                if (!existingTables.contains("artist") || !existingTables.contains("recordcompany")) {
                    JOptionPane.showMessageDialog(this,
                            "Οι πίνακες artist ή recordcompany δεν υπάρχουν στη βάση δεδομένων.",
                            "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }
```

```
String relationshipQuery = "SELECT " +
        "CASE " +
        "  WHEN a.ArtistType = 'PERSON' THEN CONCAT(p.FirstName, ' ', p.LastName) " +
        "  WHEN a.ArtistType = 'BAND' THEN b.BandName " +
        "  ELSE a.ArtistType " +
        "END AS ArtistName, " +
        "rc.CompanyName, ac.FromDate, ac.ToDate " +
        "FROM artist a " +
        "INNER JOIN artistcompany ac ON a.ArtistID = ac.ArtistID " +
        "INNER JOIN recordcompany rc ON ac.CompanyID = rc.CompanyID " +
        "LEFT JOIN person p ON a.ArtistID = p.ArtistID AND a.ArtistType = 'PERSON' " +
        "LEFT JOIN band b ON a.ArtistID = b.ArtistID AND a.ArtistType = 'BAND' " +
        "ORDER BY ArtistName, rc.CompanyName";

DefaultTableModel relationshipModel = new DefaultTableModel(
        new String[] { "Καλλιτέχνης", "Εταιρία", "Από", "Έως" }, 0);

try (Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(relationshipQuery)) {

    while (rs.next()) {
        String artistName = rs.getString("ArtistName");
        String companyName = rs.getString("CompanyName");
        java.sql.Date fromDate = rs.getDate("FromDate");
        java.sql.Date toDate = rs.getDate("ToDate");

        String fromDateStr = fromDate != null ? fromDate.toString() : "";
        String toDateStr = toDate != null ? toDate.toString() : "Ενεργή συνεργασία";

                relationshipModel.addRow(new Object[] {
                        artistName,
                        companyName,
                        fromDateStr,
                        toDateStr
                });
            }
        }

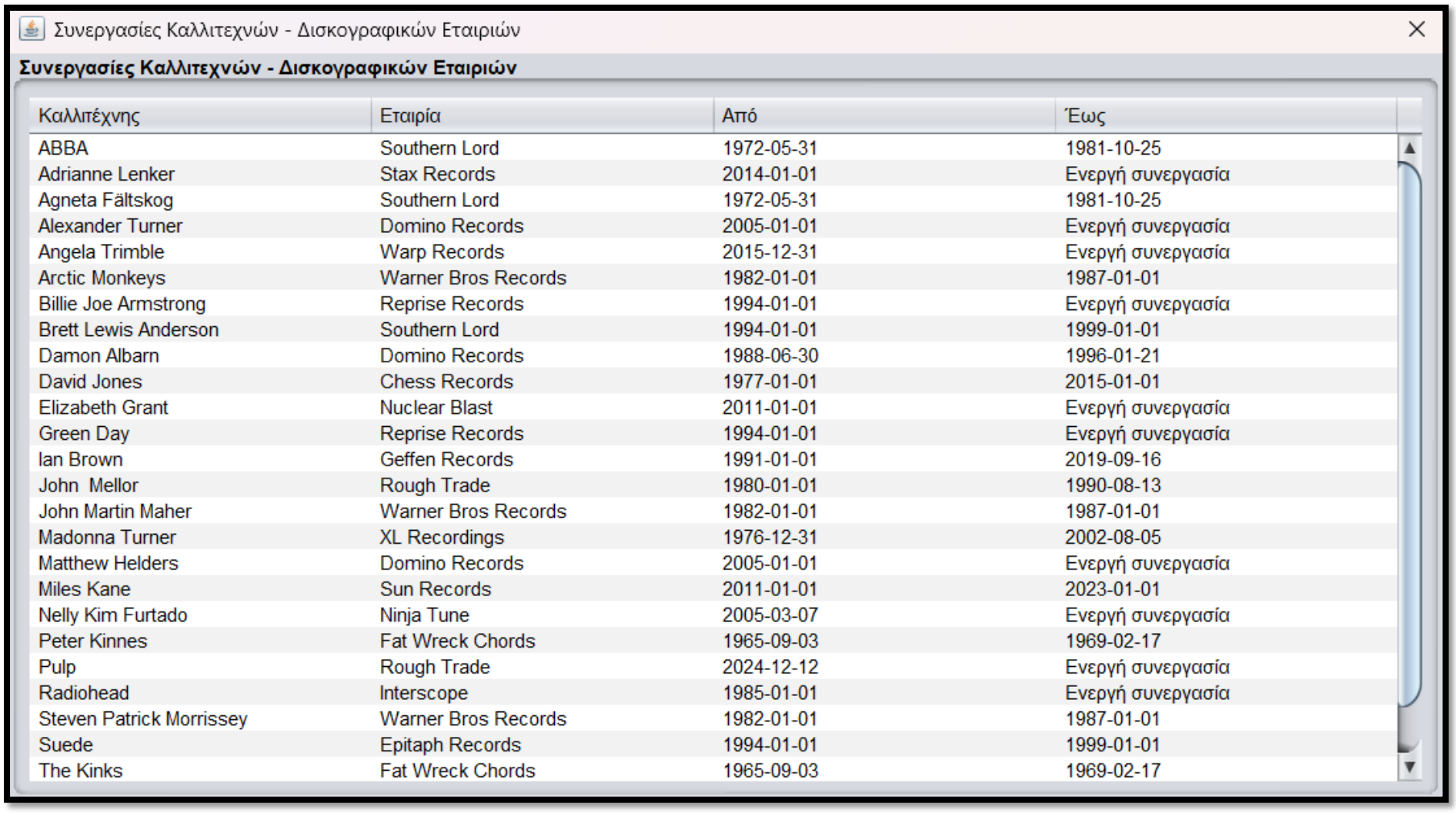
        JTable relationshipTable = new JTable(relationshipModel);

        JScrollPane scrollPane = new JScrollPane(relationshipTable);
        scrollPane.setPreferredSize(new Dimension(800, 400));

        scrollPane.setBorder(BorderFactory.createTitledBorder("Συνεργασίες Καλλιτεχνών - Δισκογραφικών Εταιριών"));

        JDialog dialog = new JDialog(this, "Συνεργασίες Καλλιτεχνών - Δισκογραφικών Εταιριών", true);
        dialog.getContentPane().add(scrollPane);
        dialog.setSize(900, 500);
        dialog.setLocationRelativeTo(this);
        dialog.setVisible(true);

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this,
                "Σφάλμα κατά την ανάκτηση των συνεργασιών: " + e.getMessage(),
                "Σφάλμα",
                JOptionPane.ERROR_MESSAGE);
    }
}
```



Φιγούρα 78: Στιγμιότυπο την εφαρμογή

11. showAlbumArtistRelations ()

Δεν χρειάζεται περαιτέρω ανάλυση καθώς έχει ήδη εξηγηθεί στο **SetupGUI()**. Η προγραμματιστική λογική είναι ακριβώς ίδια με το **showBandmembers()**.

Κώδικας συνάρτησης :

```
// ΙΔΙΑ ΛΟΓΙΚΗ ΜΕ showBandMembers
    private void showAlbumArtistRelations() {
        try {
            String check = "SELECT TABLE_NAME FROM information_schema.TABLES WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME
IN ('album','artist','person','band','genre')";
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(check)) {
                java.util.Set<String> existing = new java.util.HashSet<>();
                while (rs.next())
                    existing.add(rs.getString("TABLE_NAME"));
                if (!existing.contains("album") || !existing.contains("artist")) {
                    JOptionPane.showMessageDialog(this, "Οι πίνακες album ή artist δεν υπάρχουν.", "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }

            String q = "SELECT a.AlbumID, a.Title, " +
                    "CASE WHEN ar.ArtistType='PERSON' THEN CONCAT(p.FirstName,' ',p.LastName) WHEN ar.ArtistType='BAND' THEN
b.BandName ELSE ar.ArtistType END AS ArtistName, "
                    +
                    "g.GenreName, rc.CompanyName, CONCAT(pr.FirstName,' ',pr.LastName) AS ProducerName " +
                    "FROM album a " +
                    "INNER JOIN artist ar ON a.ArtistID = ar.ArtistID " +
                    "LEFT JOIN person p ON ar.ArtistID = p.ArtistID AND ar.ArtistType='PERSON' " +
                    "LEFT JOIN band b ON ar.ArtistID = b.ArtistID AND ar.ArtistType='BAND' " +
                    "LEFT JOIN genre g ON a.GenreID = g.GenreID " +
                    "LEFT JOIN recordcompany rc ON a.CompanyID = rc.CompanyID " +
                    "LEFT JOIN producer pr ON a.ProducerID = pr.ProducerID " +
                    "ORDER BY a.Title";

            DefaultTableModel model = new DefaultTableModel(
                    new String[] { "AlbumID", "Title", "Artist", "Genre", "Company", "Producer" }, 0);
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(q)) {
                while (rs.next()) {
                    model.addRow(new Object[] { rs.getInt("AlbumID"), rs.getString("Title"), rs.getString("ArtistName"),
                            rs.getString("GenreName"), rs.getString("CompanyName"), rs.getString("ProducerName") });
                }
            }

            JTable t = new JTable(model);
            JScrollPane sp = new JScrollPane(t);
            sp.setPreferredSize(new Dimension(900, 400));
            sp.setBorder(BorderFactory.createTitledBorder("Συσχετίσεις Album - Καλλιτέχνη"));
            JDialog d = new JDialog(this, "Album ↔ Καλλιτέχνης", true);
```

```
            d.getContentPane().add(sp);
            d.setSize(1000, 500);
            d.setLocationRelativeTo(this);
            d.setVisible(true);

        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα ανάκτησης album-artist: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
```

Συσχετίσεις Album - Καλλιτέχνη

AlbumID	Title	Artist	Genre	Company	Producer
12	American Idiot	Madonna Turner	Swing	Roadrunner Records	Steve Lillywhite
11	Every Day Is Like Sunday	Steven Patrick Morrissey	Disco	Roadrunner Records	Ahmet Ertegun
14	H atmosfaira hlektrismenh	Pulp	Electronic	Virgin Records	Nigel Godrich
13	Kaiti Garbi - Best Of	Steve Thomas Lacy-Moya	Jazz	Capitol Records	Flood
7	Kerplunk	Green Day	Punk	Roadrunner Records	Eddie Kramer
6	Meat Is Murder	The Smiths	Alternative	Warner Bros Records	Pharrell Williams
6	Meat Is Murder	Arctic Monkeys	Alternative	Warner Bros Records	Pharrell Williams
17	Oso Zw Mazw	Pulp	Punk	Def Jam	Danger Mouse
19	ponaw		Techno	Matador Records	Alan Parsons
10	Suede	Brett Lewis Anderson	Blues	A&M Records	Pharrell Williams
9	Sumbarine	Alexander Turner	Alternative	Atlantic Records	Brendan O'Brien
15	Trifulla Tetrafulla	David Jones	Electronic	Blue Note Records	Berry Gordy
8	Ultraviolence	Elizabeth Grant	Alternative	RCA Records	Tony Visconti

Φιγούρα 79: Στιγμιότυπο την εφαρμογή

12. showAlbumReleases()

Ομοίως με τα παραπάνω.

Κώδικας συνάρτησης:

```
private void showAlbumReleases() {
        try {
            String check = "SELECT TABLE_NAME FROM information_schema.TABLES WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME
IN ('album','albumrelease')";
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(check)) {
                java.util.Set<String> existing = new java.util.HashSet<>();
                while (rs.next())
                    existing.add(rs.getString("TABLE_NAME"));
                if (!existing.contains("album") || !existing.contains("albumrelease")) {
                    JOptionPane.showMessageDialog(this, "Οι πίνακες album ή albumrelease δεν υπάρχουν.", "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }

            String q = "SELECT ar.ReleaseID, a.Title AS AlbumTitle, ar.ReleaseDate, ar.ReleaseType, ar.ReleaseStatus, ar.Packaging
"
                    +
                    "FROM albumrelease ar INNER JOIN album a ON ar.AlbumID = a.AlbumID ORDER BY a.Title, ar.ReleaseDate";

            DefaultTableModel model = new DefaultTableModel(
                    new String[] { "ReleaseID", "Album", "ReleaseDate", "Type", "Status", "Packaging" }, 0);
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(q)) {
                while (rs.next()) {
                    model.addRow(new Object[] { rs.getInt("ReleaseID"), rs.getString("AlbumTitle"),
                            rs.getDate("ReleaseDate"), rs.getString("ReleaseType"), rs.getString("ReleaseStatus"),
                            rs.getString("Packaging") });
                }
            }

            JTable t = new JTable(model);
            JScrollPane sp = new JScrollPane(t);
            sp.setPreferredSize(new Dimension(900, 400));
            sp.setBorder(BorderFactory.createTitledBorder("Album Releases"));
            JDialog d = new JDialog(this, "Album ⇔ Releases", true);
            d.getContentPane().add(sp);
            d.setSize(1000, 500);
            d.setLocationRelativeTo(this);
            d.setVisible(true);

        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα ανάκτησης album-releases: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
```

Album Releases

ReleaseID	Album	ReleaseDate	Type	Status	Packaging
7	Kerplunk	2022-07-30	CD	OFFICIAL	DIGIPAK
16	Meat Is Murder	2005-07-20	MP3	OFFICIAL	NOT APPLICABLE
6	Meat Is Murder	2020-12-01	MP3	CANCELLED	NOT APPLICABLE
10	Suede	2022-02-14	CD	WITHDRAWN	CARDBOARD SLEEVE
9	Sumbarine	2020-05-22	MP3	OFFICIAL	NOT APPLICABLE
8	Ultraviolence	2021-10-18	LP	PROMOTION	JEWEL CASE

Φιγούρα 80: Στιγμιότυπο την εφαρμογή

13. showAlbumTracks ()

Ομοίως με τα παραπάνω.

Κώδικας συνάρτησης:

```
private void showAlbumTracks() {
        try {
            String check = "SELECT TABLE_NAME FROM information_schema.TABLES WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME
IN ('album','track')";
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(check)) {
                java.util.Set<String> existing = new java.util.HashSet<>();
                while (rs.next())
                    existing.add(rs.getString("TABLE_NAME"));
                if (!existing.contains("album") || !existing.contains("track")) {
                    JOptionPane.showMessageDialog(this, "Οι πίνακες album ή track δεν υπάρχουν.", "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }
```

```
            }

            String q = "SELECT t.TrackID, a.Title AS AlbumTitle, t.Title AS TrackTitle, t.TrackLength, t.TrackNo " +
                    "FROM track t INNER JOIN album a ON t.AlbumID = a.AlbumID ORDER BY a.Title, t.TrackNo";

            DefaultTableModel model = new DefaultTableModel(
                    new String[] { "TrackID", "Album", "Track Title", "Length", "TrackNo" }, 0);
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(q)) {
                while (rs.next()) {
                    model.addRow(new Object[] { rs.getInt("TrackID"), rs.getString("AlbumTitle"),
                            rs.getString("TrackTitle"), rs.getTime("TrackLength"), rs.getInt("TrackNo") });
                }
            }

            JTable t = new JTable(model);
            JScrollPane sp = new JScrollPane(t);
            sp.setPreferredSize(new Dimension(900, 400));
            sp.setBorder(BorderFactory.createTitledBorder("Album Tracks"));
            JDialog d = new JDialog(this, "Album ↔ Tracks", true);
            d.getContentPane().add(sp);
            d.setSize(1000, 500);
            d.setLocationRelativeTo(this);
            d.setVisible(true);

        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα ανάκτησης album-tracks: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
```

Album Tracks

TrackID	Album	Track Title	Length	TrackNo
19	American Idiot	Fading Lights	00:04:22	2
12	American Idiot	Never Had No One Ever	00:02:33	6
6	American Idiot	Fyge se parakalw	00:03:48	10
14	Every Day Is Like Sunday	Ksexasmeni Enilikiosi	00:02:11	7
22	H atmosfaira hlektrismenh	Shattered Dreams	00:04:45	7
32	Kaiti Garbi - Best Of	Midnight Drive	00:03:48	9
16	Kaiti Garbi - Best Of	Straight To Hell	00:06:56	14
4	Kerplunk	2,000 Light Years Away	00:02:56	1
9	Kerplunk	Ma Zw	00:02:02	5
31	Meat Is Murder	Glass Heart	00:04:05	8
34	Oso Zw Mazw	Silver Springs	00:03:10	2
8	Oso Zw Mazw	Judas	00:05:00	5
29	Oso Zw Mazw	Velvet Night	00:04:33	6
11	Suede	Electricity	00:04:50	3
23	Sumbarine	Lost in Time	00:03:36	3
27	Sumbarine	Broken Compass	00:05:01	4
15	Trifulla Tetrafulla	Falling Stars (Chugu Bam Bam)	00:01:11	1
28	Trifulla Tetrafulla	Silver Rain	00:03:50	5
25	Ultraviolence	Echoes of You	00:03:59	2
5	Ultraviolence	Kalimera Kenouria Mou Agapi	00:02:35	7

Φιγούρα 81: Στιγμιότυπο την εφαρμογή

14. showProducerCompanyRelations ()

Ομοίως με τα παραπάνω.

Κώδικας συνάρτησης:

```
// ΙΔΙΑ ΛΟΓΙΚΗ ΜΕ showBandMembers
    private void showProducerCompanyRelations() {
        try {
            // Check if tables exist
            String checkTablesQuery = "SELECT TABLE_NAME FROM information_schema.TABLES " +
                    "WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME IN ('producer', 'producercompany', 'recordcompany')";

            try (Statement stmt = conn.createStatement();
                    ResultSet rs = stmt.executeQuery(checkTablesQuery)) {

                java.util.List<String> existingTables = new java.util.ArrayList<>();
                while (rs.next()) {
                    existingTables.add(rs.getString("TABLE_NAME"));
                }

                if (!existingTables.contains("producer") || !existingTables.contains("recordcompany")) {
                    JOptionPane.showMessageDialog(this,
                            "Οι πίνακες producer ή recordcompany δεν υπάρχουν στη βάση δεδομένων.",
                            "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }

            String collaborationQuery = "SELECT p.ProducerID, CONCAT(p.FirstName, ' ', p.LastName) as ProducerName, " +
```

```
                "rc.CompanyName, pc.ToDate " +
                "FROM producer p " +
                "INNER JOIN producercompany pc ON p.ProducerID = pc.ProducerID " +
                "INNER JOIN recordcompany rc ON pc.CompanyID = rc.CompanyID " +
                "ORDER BY p.LastName, p.FirstName, rc.CompanyName";

        DefaultTableModel collaborationModel = new DefaultTableModel(
                new String[] { "Παραγωγός", "Εταιρία", "Έως Ημερομηνία" }, 0);

        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(collaborationQuery)) {

            while (rs.next()) {
                String producerName = rs.getString("ProducerName");
                String companyName = rs.getString("CompanyName");
                java.sql.Date toDate = rs.getDate("ToDate");
                String toDateStr = toDate != null ? toDate.toString() : "Ενεργή";

                collaborationModel.addRow(new Object[] {
                        producerName,
                        companyName,
                        toDateStr
                });
            }
        }

        JTable collaborationTable = new JTable(collaborationModel);

        JScrollPane scrollPane = new JScrollPane(collaborationTable);
                scrollPane.setPreferredSize(new Dimension(800, 400));

                scrollPane.setBorder(BorderFactory.createTitledBorder("Συνεργασίες Παραγωγών - Δισκογραφικών Εταιριών"));

                JDialog dialog = new JDialog(this, "Συνεργασίες Παραγωγών και Εταιριών", true);
                dialog.getContentPane().add(scrollPane);
                dialog.setSize(900, 500);
                dialog.setLocationRelativeTo(this);
                dialog.setVisible(true);

            } catch (SQLException e) {
                JOptionPane.showMessageDialog(this,
                        "Σφάλμα κατά την ανάκτηση των συνεργασιών: " + e.getMessage(),
                        "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
            }
        }
```

Συνεργασίες Παραγωγών - Δισκογραφικών Εταιριών

Παραγωγός	Εταιρία	Έως Ημερομηνία
Flood	Motown Records	Ενεργή
Timbaland	Capitol Records	Ενεργή
Steve Albini	A&M Records	Ενεργή
James Blake	Absolutely Kosher	Ενεργή
Tchad Blake	Merge Records	Ενεργή
T Bone Burnett	Sun Records	Ενεργή
Bob Clearmountain	Century Media	Ενεργή
Dr. Dre	Warner Bros Records	Ενεργή
Brian Eno	Stax Records	Ενεργή
Ahmet Ertegun	Warp Records	2015-12-31
Mitchell Froom	SST Records	Ενεργή
Nigel Godrich	Kranky Records	Ενεργή
Berry Gordy	Domino Records	1988-06-30
Trevor Horn	Virgin Records	Ενεργή
Glyn Johns	Matador Records	Ενεργή
Quincy Jones	Columbia Records	Ενεργή
Eddie Kramer	Sub Pop Records	Ενεργή
Mutt Lange	Reprise Records	Ενεργή
Daniel Lanois	Chess Records	Ενεργή
John Leckie	Thrill Jockey	Ενεργή
Jacknife Lee	Ipecac Records	Ενεργή
Steve Lillywhite	Blue Note Records	Ενεργή
Tommy LiPuma	Dischord Records	Ενεργή
Scott Litt	Metal Blade	Ενεργή
Chris Lord-Alge	Candlelight Records	Ενεργή

Φιγούρα 82: Στιγμιότυπο την εφαρμογή

Συναρτήσεις που αφορούν τους τύπους δεδομένων :

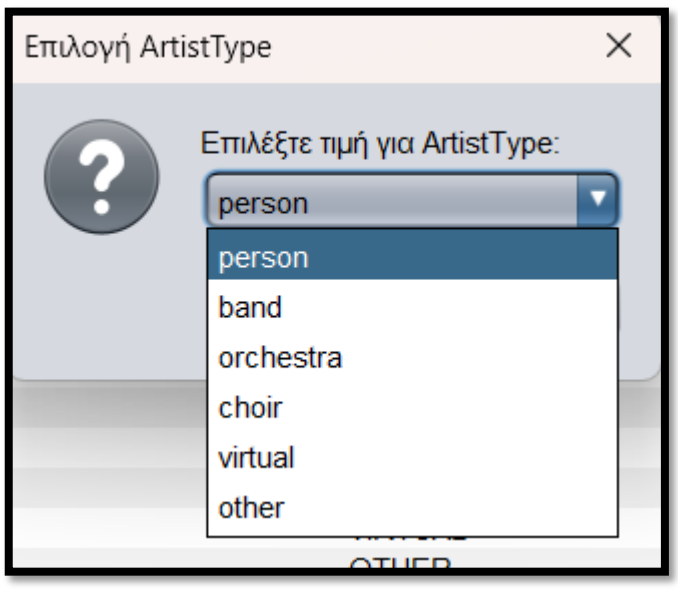
15. getEnumValues(String tableName, String columnName)

Η συνάρτηση αυτή σχεδιάστηκε επειδή θέλουμε να αποφεύγονται όσο το δυνατόν γίνεται τα λάθη κατά την εισαγωγή δεδομένων. Γι' αυτό λοιπόν κάθε φορά που η εφαρμογή εντοπίζει κάποιο ENUM πεδίο στην SQL , το μετατρέπει σε drop down menu . Έτσι ο χρήστης αντί να πληκτρολογήσει κάποια τιμή (κάτι που επιφέρει και μεγάλο κίνδυνο να προκληθεί σφάλμα) , επιλεγεί την τιμή που θέλει από το drop down menu . Αυτή η λογική ενσωματώθηκε και για μεταβλητές ελέγχου όπως IsAvailable και IsSoloArtist με το 1 να αντιστοιχίζεται σε true τιμή και το 0 σε false.

Κώδικας συνάρτησης :

```
    private String[] getEnumValues(String tableName, String columnName) {
        try {
            ArrayList<String> values = new ArrayList<>();
            try (Statement stmt = conn.createStatement();
                    ResultSet rs = stmt
                            .executeQuery("SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + columnName + "'")) {
                if (rs.next()) {
                    String type = rs.getString("Type").toLowerCase();

                    // Ειδική μεταχείριση για boolean/tinyint(1) πεδία
                    if (type.equals("tinyint(1)")) {
                        values.add("1 - Ναι");
                        values.add("0 - Όχι");
                    }
                    // Κανονική μεταχείριση για enum πεδία
                    else if (type.startsWith("enum(")) {
                        type = type.substring(5, type.length() - 1);
                        String[] splits = type.split(",");
                        for (String s : splits) {
                            String cleanValue = s.trim().replace("'", "");
                            // Μετατροπή boolean τιμών σε Ναι/Όχι
                            if (cleanValue.equalsIgnoreCase("true")) {
                                values.add("1 - Ναι");
                            } else if (cleanValue.equalsIgnoreCase("false")) {
                                values.add("0 - Όχι");
                            } else {
                                values.add(cleanValue);
                            }
                        }
                    }
                }
            }
            return values.toArray(new String[0]);
        } catch (SQLException e) {
            return new String[0];
        }
    }
```



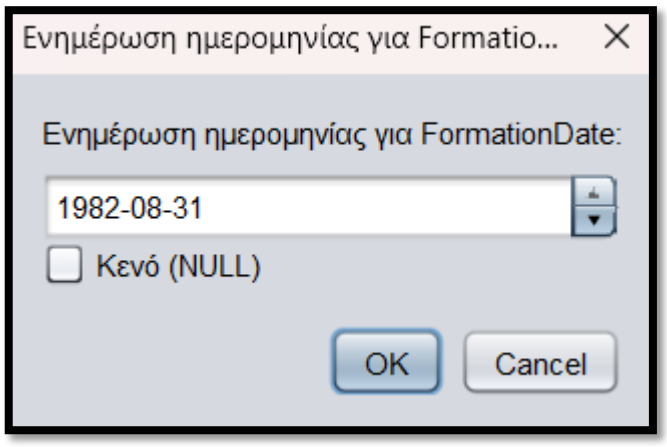
Φιγούρα 83: Στιγμιότυπο την εφαρμογή

16. isDateColumn(String tableName, String columnName)

Αυτή η συνάρτηση δημιουργήθηκε έτσι ώστε όταν συναντάμε κάποιο πεδίο με Date στην SQL (όπως From_Date, To_Date) να μπορούμε να επιλέγουμε με βελάκια την επιθυμητή τιμή ώστε να αποφεύγονται λάθη κατά την εισαγωγή του date sting (κάτι που κάνουμε με την εκτέλεση SQL script)

Κώδικας συνάρτησης :

```
private boolean isDateColumn(String tableName, String columnName) {
        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt
                        .executeQuery("SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + columnName + "'")) {
            if (rs.next()) {
                String type = rs.getString("Type").toLowerCase();
                return type.contains("date") || type.contains("timestamp");
            }
        } catch (SQLException e) {
            return false;
        }
        return false;
    }
```



Φιγούρα 84: Στιγμιότυπο την εφαρμογή

17. isAutoIncrement(String tableName, String columnName)

Αυτή η συνάρτηση δημιουργήθηκε έτσι ώστε όταν συναντάμε κάποιο πεδίο με AUTO INCREMENT χαρακτηριστικό στην SQL (όπως τα ID κλειδιά στους πίνακες μας) να μην μπορούμε σαν χρήστες να εισάγουμε κάποια τιμή και η διαδικασία να γίνεται αυτόματα. Έτσι, αποφεύγονται τα λάθη και είμαστε ήσυχοι.

Κώδικας συνάρτησης :

```
private boolean isAutoIncrement(String tableName, String columnName) {
        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt
                        .executeQuery("SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + columnName + "'")) {
            if (rs.next()) {
                return "auto_increment".equalsIgnoreCase(rs.getString("Extra"));
            }
        } catch (SQLException e) {
            return false;
        }
        return false;
    }
```

Συναρτήσεις που εμφανίζουν τους πίνακες της ΒΔ :

18. loadTableData(String tableName)

Η συνάρτηση αυτή εμφανίζει τους πίνακες και όλα τα δεδομένα που περιέχονται μέσα σε αυτούς σε στήλες και γραμμές.

19. extractTableName(String Selection)

Επιλογή του ονόματος που 'κουβαλάει' ο πίνακας που επιλέγουμε από το drop down menu.

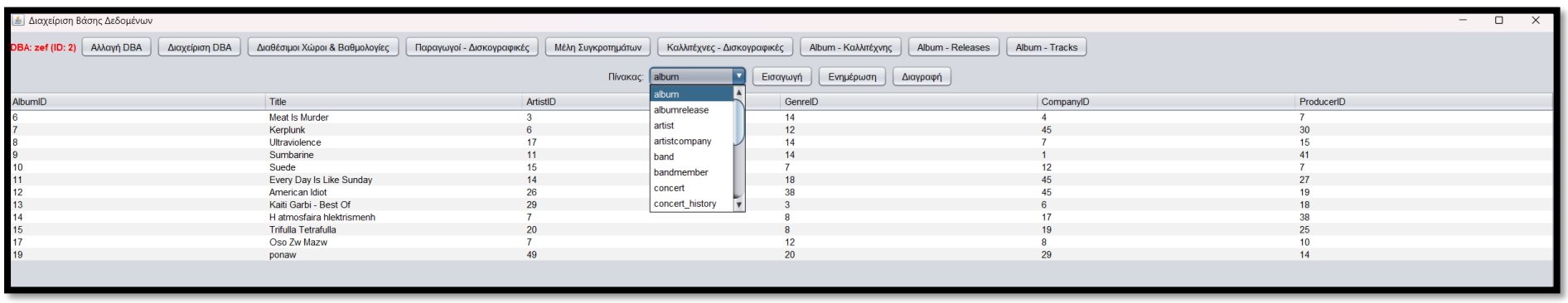
Συγκεντρωτικός κώδικας:

```
    private void loadTableData(String tableName) {
         try {
             tableModel.setRowCount(0);
             tableModel.setColumnCount(0);

             // Εξάγουμε το πραγματικό όνομα του πίνακα/view από την επιλογή
             String actualTableName = extractTableName(tableName);

             try (Statement stmt = conn.createStatement();
                     ResultSet rs = stmt.executeQuery("SELECT * FROM " + actualTableName)) {
                 ResultSetMetaData metaData = rs.getMetaData();
                 int columnCount = metaData.getColumnCount();
                 for (int i = 1; i <= columnCount; i++)
                     tableModel.addColumn(metaData.getColumnName(i));
                 while (rs.next()) {
                     Vector<Object> row = new Vector<>();
                     for (int i = 1; i <= columnCount; i++)
                         row.add(rs.getObject(i));
                     tableModel.addRow(row);
                 }
             }
         } catch (SQLException e) {
             JOptionPane.showMessageDialog(this, "Σφάλμα φόρτωσης δεδομένων: " + e.getMessage());
         }
    }

    private String extractTableName(String selection) {
         if (selection == null)
             return null;
         if (selection.contains("("))
             return selection.substring(0, selection.indexOf("(")).trim();
         if (selection.startsWith("---"))
             return "artist";
         return selection;
    }
```



Φιγούρα 85: Στιγμιότυπο την εφαρμογή

Συναρτήσεις insert , update , delete

20. insertRow()

Όπως εξηγήσαμε και παραπάνω (Βλ. **SetupGUI()** , η *insertRow*() είναι μια συνάρτηση που επιτρέπει στον χρήστη να εισάγει μια νέα γραμμή δεδομένων στον επιλεγμένο πίνακα. Καλείται όταν ο χρήστης βρίσκεται στον επιθυμητό πίνακα και πατήσει το κουμπί 'Εισαγωγή' που βρίσκεται στο πάνω σημείο της εφαρμογής. Όταν πατηθεί το κουμπί , η συνάρτηση φέρνει τα πεδία του ενεργού πίνακα στην οθόνη , ένα-ένα την φορά, και περιμένει από τον χρήστη να τα συμπληρώσει,. Γίνεται χρήση βοηθητικών συναρτήσεων ώστε,

1. Να αποφεύγεται η εισαγωγή AUTO INCREMENT τιμών (βλ. **isAutoIncrement(String tableName, String columnName)**) ,
2. τα ENUMS να επιλέγονται από drop down menu (βλ. **getEnumValues(String tableName, String columnName)**)
3. οι ημερομηνίες να επιλέγονται από spinner ώστε να μην βάζουμε manually όλο το string της ημερομηνίας και προκαλούνται λάθη.(βλ. **isDateColumn(String tableName, String columnName)**)

Όλα τα στοιχεία που πληκτρολογούμε/επιλέγουμε , αποθηκεύονται ύστερα στον αντίστοιχο πίνακα, μέσα από SQL queries και έτσι εμφανίζονται στην οθόνη μας.

Κώδικας συνάρτησης :

```
// εισαγωγη δεδομενων σε πινακα
    private void insertRow() {
        String tableName = (String) tableSelector.getSelectedItem();
        if (tableName == null)
            return;
        if (isViewSelection(tableName)) {
            JOptionPane.showMessageDialog(this, "Δεν μπορείτε να προσθέσετε δεδομένα σε view.", "Μη έγκυρη επιλογή",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        String actualTableName = extractTableName(tableName);

        ArrayList<String> reqFields = new ArrayList<>();
        ArrayList<String> dateFields = new ArrayList<>();
        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery("SHOW COLUMNS FROM " + actualTableName)) {
            while (rs.next()) {
                String field = rs.getString("Field");
                String type = rs.getString("Type");
                String nullable = rs.getString("Null");
                if (nullable.equals("NO")) {
                    reqFields.add(field);
                }
                if (type.toLowerCase().contains("date")) {
                    dateFields.add(field);
                }
            }
        } catch (SQLException e) {

            System.err.println("Warning: Could not get column metadata: " + e.getMessage());
        }

        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery("SELECT * FROM " + actualTableName + " LIMIT 1")) {

            ResultSetMetaData meta = rs.getMetaData();
            int colCount = meta.getColumnCount();

            ArrayList<String> columnNames = new ArrayList<>();
            ArrayList<String> values = new ArrayList<>();

            for (int i = 1; i <= colCount; i++) {
                String col = meta.getColumnName(i);

                // Προσπέραση AUTO_INCREMENT πεδίων
                if (isAutoIncrement(actualTableName, col)) {
                    continue;
                }

                columnNames.add(col);
                String val = null;

                if (isForeignKeyField(actualTableName, col)) {
                    String[] options = getForeignKeyOptions(actualTableName, col);
                    if (options != null && options.length > 0) {
                        String selected = (String) JOptionPane.showInputDialog(this,
                                "Επιλέξτε τιμή για " + col + ":",
                                "Επιλογή " + col,
                                JOptionPane.QUESTION_MESSAGE,
                                null,
```

```
                    options,
                    options[0]);

            // Αν πατήσει Cancel , στοπ την διαδικασία
            if (selected == null) {
                return;
            }
            val = extractIdFromSelection(selected);
        }
    }
    // Επιλογή ημερομηνίας
    else if (isDateColumn(actualTableName, col)) {
        val = showDatePicker("Επιλέξτε ημερομηνία για " + col, null);
        // Αν πατήσει Cancel , στοπ την διαδικασία
        if (val == null && !isNullableColumn(actualTableName, col)) {
            return;
        }
    }
    // Επιλογή Enum με dropdown menu
    else {
        String[] enumVals = getEnumValues(actualTableName, col);
        if (enumVals != null && enumVals.length > 0) {
            val = (String) JOptionPane.showInputDialog(this,
                    "Επιλέξτε τιμή για " + col + ":",
                    "Επιλογή " + col,
                    JOptionPane.QUESTION_MESSAGE,
                    null,
                    enumVals,
                    enumVals[0]);
            }

            else {
                // Κανονικό text input
                val = JOptionPane.showInputDialog(this, "Τιμή για " + col + ":");
                // Αν πατήσει Cancel , στοπ την διαδικασία
                if (val == null && !isNullableColumn(actualTableName, col)) {
                    return;
                }
            }
        }

        // Καθάρισμα κενών τιμών
        if (val != null && val.trim().isEmpty()) {
            val = null;
        }

        // Έλεγχος για υποχρεωτικά πεδία μετά το καθάρισμα
        if (val == null && !isNullableColumn(actualTableName, col)) {
            JOptionPane.showMessageDialog(this, "Το πεδίο " + col + " είναι υποχρεωτικό!", "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
            return;
        }

        values.add(val);
    }

    if (columnNames.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Δεν υπάρχουν πεδία για εισαγωγή!", "Σφάλμα",
                JOptionPane.ERROR_MESSAGE);
        return;
    }

    String columnsList = String.join(",", columnNames);
    String placeholders = String.join(",", Collections.nCopies(values.size(), "?"));
    String sql = "INSERT INTO " + actualTableName + " (" + columnsList + ") VALUES (" + placeholders + ")";

    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        for (int i = 0; i < values.size(); i++) {
            String v = values.get(i);
            String columnName = columnNames.get(i);

            // Get column metadata to handle types properly
            String columnType = "";
            try (Statement s = conn.createStatement();
                    ResultSet r = s.executeQuery(
                            "SHOW COLUMNS FROM " + actualTableName + " WHERE Field = '" + columnName + "'")) {
                if (r.next()) {
                    columnType = r.getString("Type").toLowerCase();
                }
            }

            if (v == null || v.trim().isEmpty()) {
                ps.setNull(i + 1, Types.NULL);
                continue;
            }

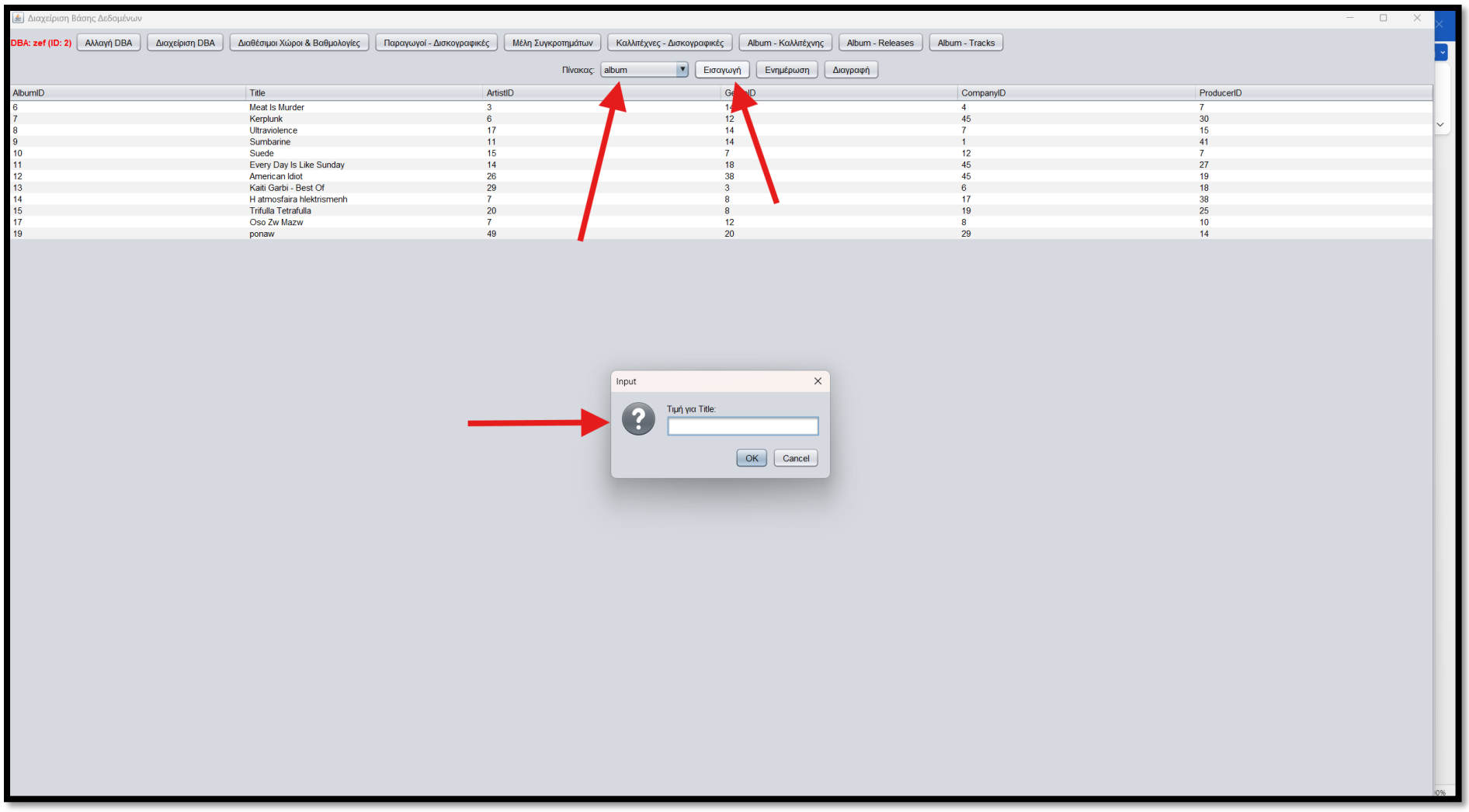
            // Διαχείριση boolean τιμών (το TINYINT(1) χρησιμοποιείται για boolean στο
            // MySQL)
            if (columnType.equals("tinyint(1)")) {

                // Για 1 -> true , για 0 -> false
                if (v.contains(" - ")) {
                    v = v.substring(0, v.indexOf(" - ")).trim();
                }
```

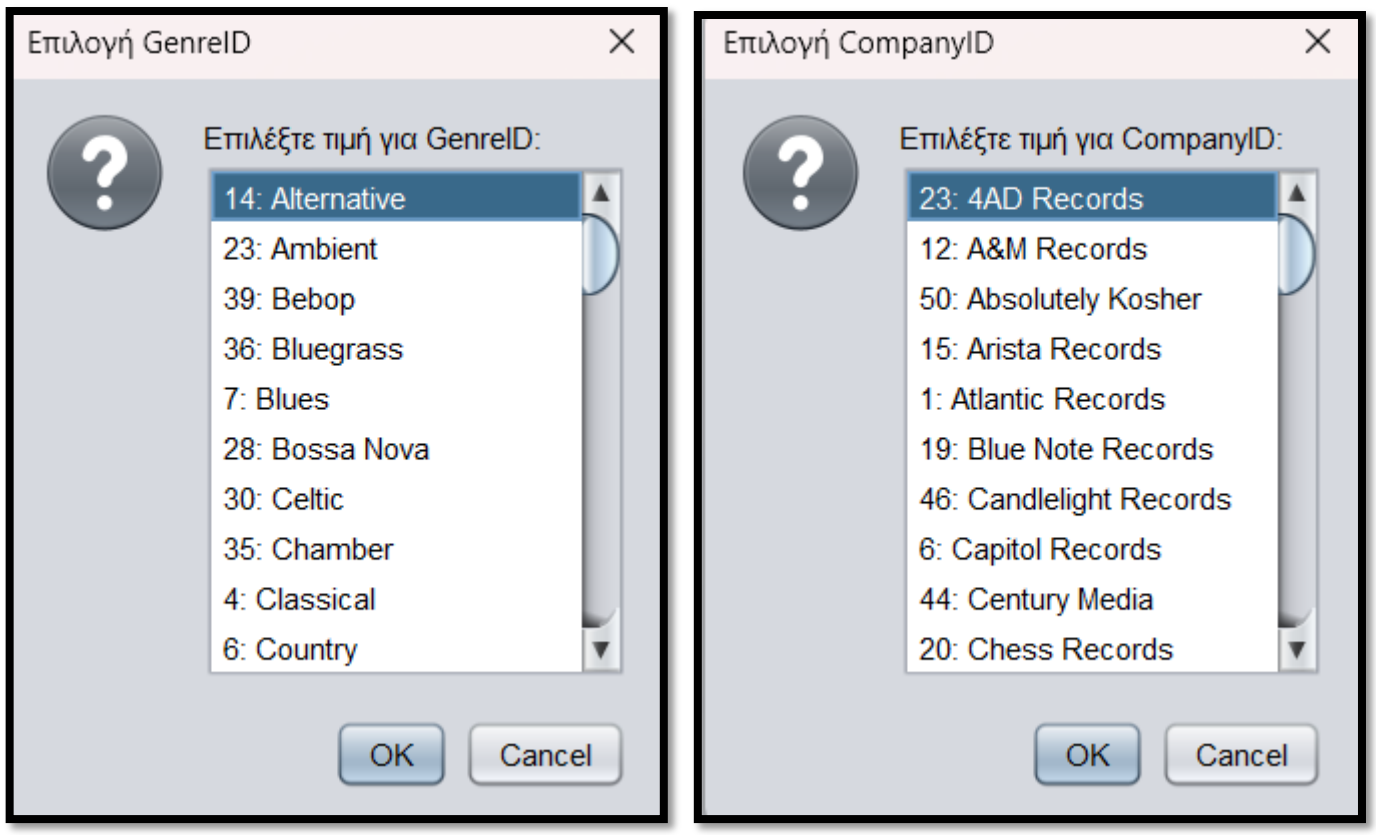
```
                    ps.setInt(i + 1, v.equals("1") || v.equalsIgnoreCase("true") ? 1 : 0);
                } else if (columnType.contains("int")) {
                    // Extract ID from "ID: Description" format if needed
                    if (v.contains(":")) {
                        v = v.substring(0, v.indexOf(":")).trim();
                    }
                    ps.setInt(i + 1, Integer.parseInt(v));
                }

                else if (columnType.contains("date")) {
                    ps.setDate(i + 1, java.sql.Date.valueOf(v));
                }

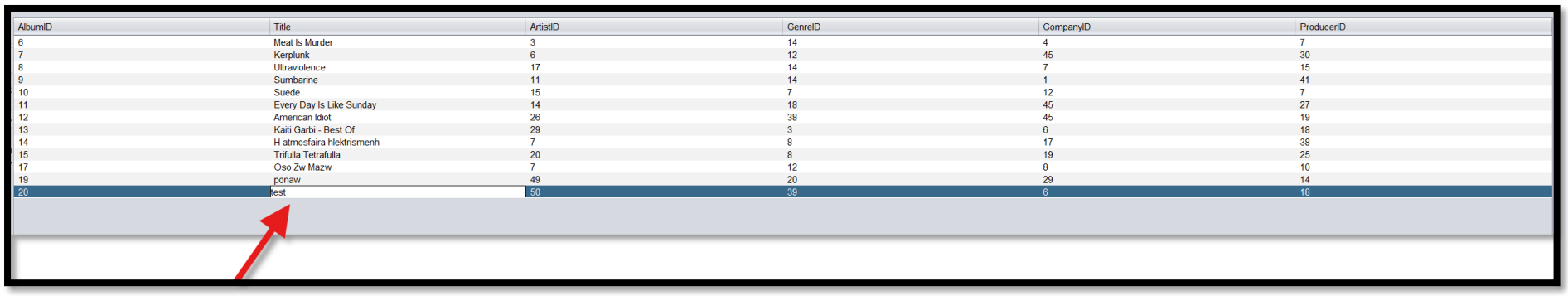
                else if (columnType.contains("timestamp")) {
                    ps.setTimestamp(i + 1, java.sql.Timestamp.valueOf(v));
                }
                // Διαχείριση δεκαδικών/αριθμητικών τύπων
                else if (columnType.contains("decimal") || columnType.contains("numeric")) {
                    ps.setBigDecimal(i + 1, new java.math.BigDecimal(v));
                }
                // Διαχείριση τύπων float/double
                else if (columnType.contains("float") || columnType.contains("double")) {
                    ps.setDouble(i + 1, Double.parseDouble(v));
                }
                // Default τιμή για όλους τους άλλους τύπους
                else {
                    ps.setString(i + 1, v);
                }
            }
            ps.executeUpdate();
            logDBAAction("INSERT", actualTableName, "Inserted values");
            loadTableData(tableName);
            JOptionPane.showMessageDialog(this, "Εισαγωγή επιτυχής!");
        }
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, "Σφάλμα εισαγωγής: " + e.getMessage());
    }
}
```



Φιγούρα 86: Στιγμιότυπο την εφαρμογή



Φιγούρα 87: Στιγμιότυπο την εφαρμογή



AlbumID	Title	ArtistID	GenreID	CompanyID	ProducerID
6	Meat Is Murder	3	14	4	7
7	Kerplunk	6	12	45	30
8	Ultraviolence	17	14	7	15
9	Sumbarine	11	14	1	41
10	Suede	15	7	12	7
11	Every Day Is Like Sunday	14	18	45	27
12	American Idiot	26	38	45	19
13	Kaiti Garbi - Best Of	29	3	6	18
14	H atmosfaira hlektrismenh	7	8	17	38
15	Trifulla Tetrafulla	20	8	19	25
17	Oso Zw Mazw	7	12	8	10
19	ponaw	49	20	29	14
20	test	50	39	6	18

Φιγούρα 88: Στιγμιότυπο την εφαρμογή

21. updateRow()

Όμοια με την **insertRow()** , δημιουργούμε την *updateRow*() που αυτό που κάνει είναι να επιτρέπει στον χρήστη να τροποποιήσει μια επιλεγμένη εγγραφή . Ουσιαστικά , ο χρήστης επιλέγει την εγγραφή με το να επιλέξει την σειρά που ανήκει αυτή , πατάει το κουμπί 'Ενημέρωση' και υστέρα η ΒΔ επιστρέφει τα πεδία που μπορεί να τροποποιήσει ο χρήστης. Λειτουργεί με τον ίδιο τρόπο που λειτουργεί και η *insertRow*() και έχει σχεδιαστεί όπως και εκείνη.

Κώδικας συνάρτησης:

```
// Ενημέρωση εγγραφής
    private void updateRow() {
        String tableName = (String) tableSelector.getSelectedItem();
        if (tableName == null)
            return;

        int selectedRow = table.getSelectedRow();
        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Παρακαλώ επιλέξτε μια γραμμή για ενημέρωση.", "Προσοχή",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery("SELECT * FROM " + tableName + " LIMIT 1")) {

            ResultSetMetaData meta = rs.getMetaData();
            int colCount = meta.getColumnCount();

            // Παιρνουμε πρωτευον κλειδι
            DatabaseMetaData dbMeta = conn.getMetaData();
            ResultSet pkeys = dbMeta.getPrimaryKeys(null, null, tableName);
            if (!pkeys.next()) {
                JOptionPane.showMessageDialog(this, "Ο πίνακας δεν έχει primary key!", "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }
            String pkColumn = pkeys.getString("COLUMN_NAME");
            String pkValue = table.getValueAt(selectedRow, findColumnIndex(pkColumn)).toString();

            ArrayList<String> columnNames = new ArrayList<>();
            ArrayList<String> values = new ArrayList<>();

            for (int i = 1; i <= colCount; i++) {
                String col = meta.getColumnName(i);
                if (col.equals(pkColumn))
                    continue;

                Object currentValue = table.getValueAt(selectedRow, i - 1);
                columnNames.add(col);
                String val = null;

                // Διαχείριση foreign κλειδιών
                if (isForeignKeyField(tableName, col)) {
                    String[] options = getForeignKeyOptions(tableName, col);
                    if (options != null && options.length > 0) {
                        String selected = (String) JOptionPane.showInputDialog(this,
                                "Ενημέρωση " + col + ":",
                                "Επεξεργασία " + col,
                                JOptionPane.QUESTION_MESSAGE,
                                null,
                                options,
                                findCurrentOption(options, currentValue != null ? currentValue.toString() : null));

                        if (selected != null) {
                            val = extractIdFromSelection(selected);
                        }
                    }
                }
                // Διαχείριση ημερομηνιών
                else if (isDateColumn(tableName, col)) {
                    val = showDatePicker("Ενημέρωση ημερομηνίας για " + col,
                            currentValue != null ? currentValue.toString() : null);
                }
                // Διαχείριση enums με dropdown
                else {
                    String[] enumVals = getEnumValues(tableName, col);
                    if (enumVals != null && enumVals.length > 0) {
                        val = (String) JOptionPane.showInputDialog(this,
                                "Ενημέρωση " + col + ":",
                                "Επεξεργασία " + col,
                                JOptionPane.QUESTION_MESSAGE,
                                null,
                                enumVals,
                                currentValue);
                    }

                    else {
                        val = JOptionPane.showInputDialog(this,
                                "Ενημέρωση " + col + ":",
```

```
                        currentValue != null ? currentValue.toString() : "");
            }
        }

        if (val == null && !isNullableColumn(tableName, col)) {
            JOptionPane.showMessageDialog(this, "Το πεδίο " + col + " είναι υποχρεωτικό!", "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
            return;
        }

        values.add(val);
    }

    if (columnNames.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Δεν υπάρχουν πεδία για ενημέρωση!", "Σφάλμα",
                JOptionPane.ERROR_MESSAGE);
        return;
    }

    StringBuilder sql = new StringBuilder("UPDATE ").append(tableName).append(" SET ");
    for (int i = 0; i < columnNames.size(); i++) {
        if (i > 0)
            sql.append(", ");
        sql.append(columnNames.get(i)).append(" = ?");
    }
    sql.append(" WHERE ").append(pkColumn).append(" = ?");

    try (PreparedStatement ps = conn.prepareStatement(sql.toString())) {
        for (int i = 0; i < values.size(); i++) {
            String v = values.get(i);
            String columnName = columnNames.get(i);

            String columnType = "";
            try (Statement s = conn.createStatement();
                    ResultSet r = s.executeQuery(
                            "SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + columnName + "'")) {
                if (r.next()) {
                    columnType = r.getString("Type").toLowerCase();
                }
            }

            if (v == null || v.trim().isEmpty()) {
                ps.setNull(i + 1, Types.NULL);
                continue;
            }

            // διαχειριση boolean
            if (columnType.equals("tinyint(1)")) {
                // 1 -> true, 0 -> false
                if (v.contains(" - ")) {
                    v = v.substring(0, v.indexOf(" - ")).trim();
                }
                ps.setInt(i + 1, v.equals("1") || v.equalsIgnoreCase("true") ? 1 : 0);
            }
            // διαχειριση integer
            else if (columnType.contains("int")) {

                if (v.contains(":")) {
                    v = v.substring(0, v.indexOf(":")).trim();
                }
                ps.setInt(i + 1, Integer.parseInt(v));
            }
            // διαχείριση ημερομηνίας
            else if (columnType.contains("date")) {
                ps.setDate(i + 1, java.sql.Date.valueOf(v));
            }
            // διαχείριση timestamp
            else if (columnType.contains("timestamp")) {
                ps.setTimestamp(i + 1, java.sql.Timestamp.valueOf(v));
            }
            // διαχείριση decimal/numeric
            else if (columnType.contains("decimal") || columnType.contains("numeric")) {
                ps.setBigDecimal(i + 1, new java.math.BigDecimal(v));
            }
            // διαχείριση floating point
            else if (columnType.contains("float") || columnType.contains("double")) {
                ps.setDouble(i + 1, Double.parseDouble(v));
            }
            // Default για άλλους τύπους
            else {
                ps.setString(i + 1, v);
            }
        }

        // διαχείριση του πρωτεύοντος κλειδιού
        String pkColumnType = "";
        try (Statement s = conn.createStatement();
                ResultSet r = s
                        .executeQuery("SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + pkColumn + "'")) {
            if (r.next()) {
                pkColumnType = r.getString("Type").toLowerCase();
            }
        }
```

```
            if (pkColumnType.contains("int")) {
                ps.setInt(values.size() + 1, Integer.parseInt(pkValue));
            } else {
                ps.setString(values.size() + 1, pkValue);
            }

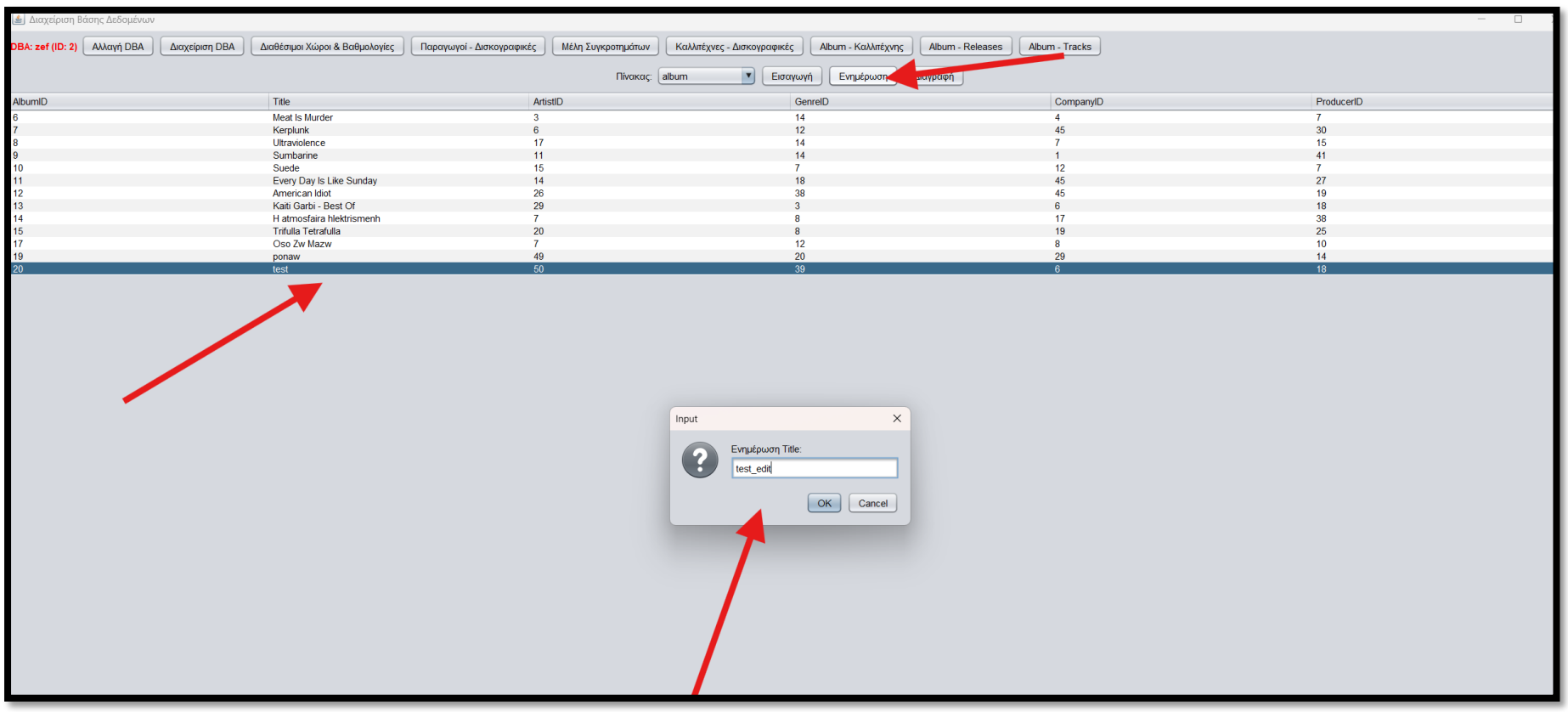
            int affected = ps.executeUpdate();
            if (affected > 0) {
                logDBAAction("UPDATE", tableName, "Updated record with " + pkColumn + "=" + pkValue);
                loadTableData(tableName);
                JOptionPane.showMessageDialog(this, "Η ενημέρωση ολοκληρώθηκε επιτυχώς!");
            } else {
                JOptionPane.showMessageDialog(this, "Δεν έγινε ενημέρωση. Η εγγραφή μπορεί να μην υπάρχει πλέον.",
                        "Προσοχή", JOptionPane.WARNING_MESSAGE);
            }
        }
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, "Σφάλμα ενημέρωσης: " + e.getMessage());
    }
}

private int findColumnIndex(String columnName) {
    for (int i = 0; i < table.getColumnCount(); i++) {
        if (table.getColumnName(i).equals(columnName)) {
            return i;
        }
    }
    return -1;
}

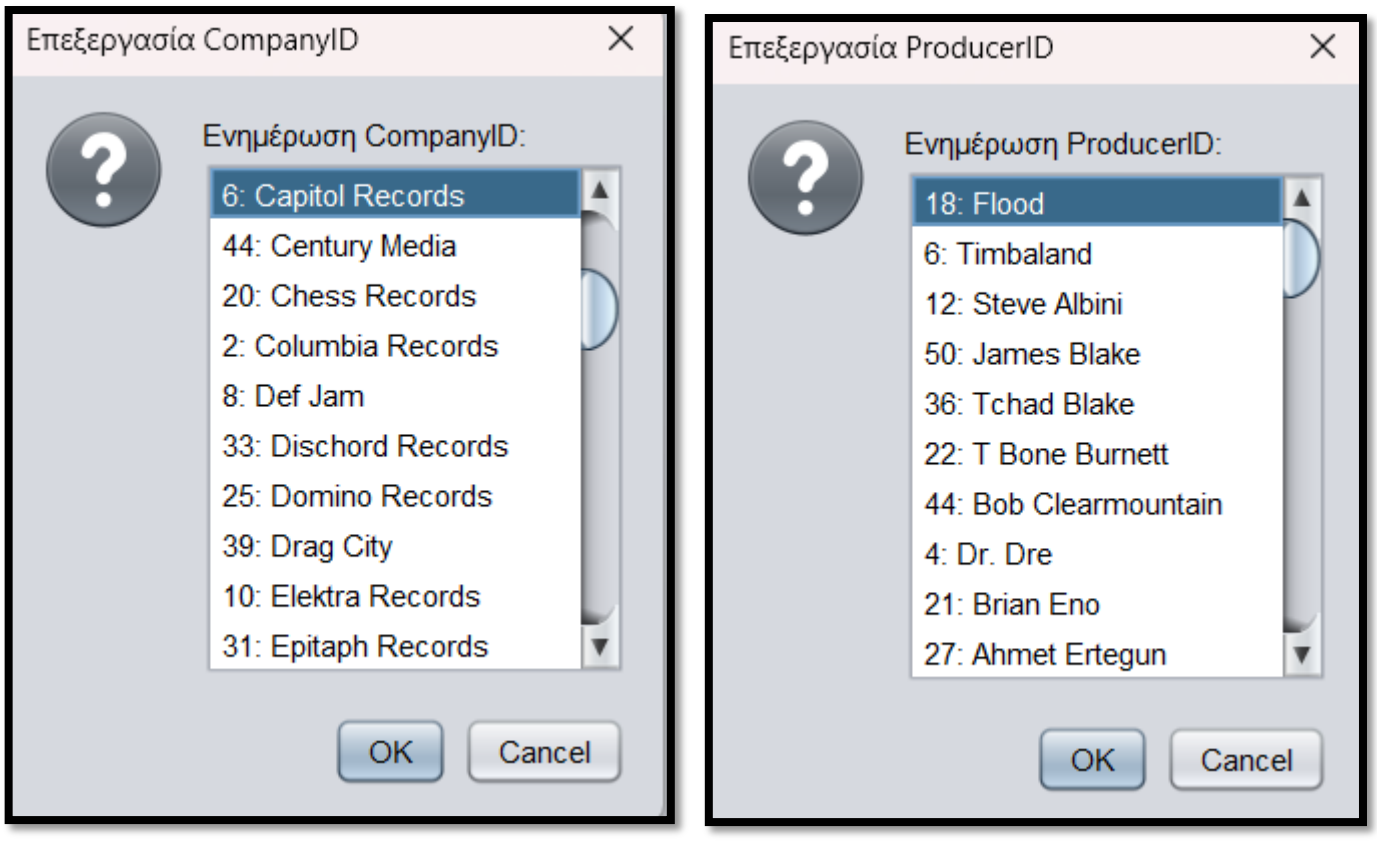
private String findCurrentOption(String[] options, String currentValue) {
    if (currentValue == null)
        return options[0];

    // Για boolean πεδία (1/0)
    if (currentValue.equals("1") || currentValue.equals("0")) {
        for (String option : options) {
            if (option.startsWith(currentValue + " - ")) {
                return option;
            }
        }
    }

    // Για foreign keys (id: description)
    for (String option : options) {
        if (option.startsWith(currentValue + ":") || option.equals(currentValue)) {
            return option;
        }
    }
    return options[0];
}
```

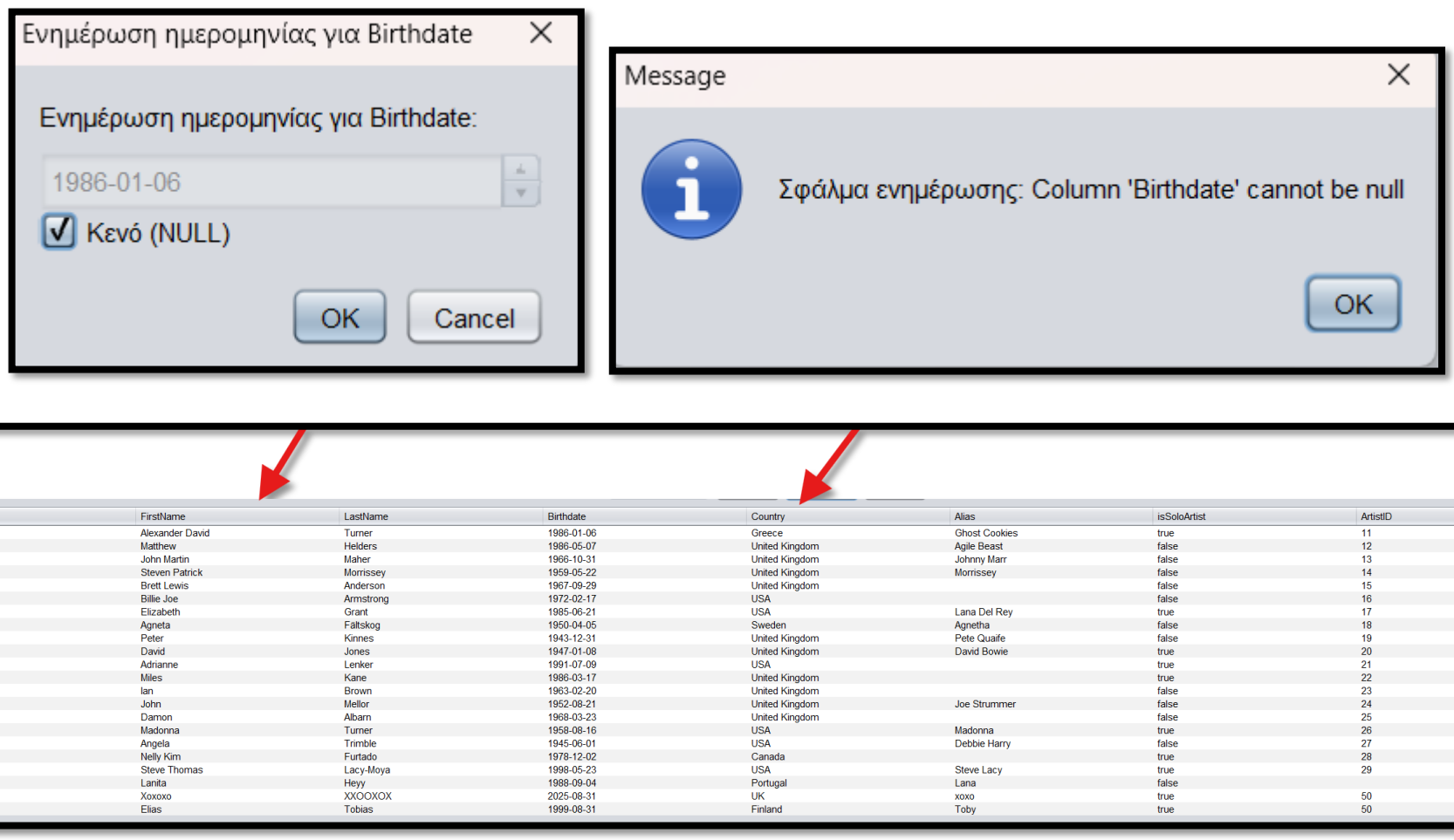


Φιγούρα 89: Στιγμιότυπο την εφαρμογή



Φιγούρα 90: Στιγμιότυπο την εφαρμογή

Ενημέρωση person όταν βάζουμε NULL σε πεδίο που είναι NOT NULL:



PersonID	FirstName	LastName	Birthdate	Country	Alias	isSoloArtist	ArtistID
1	Alexander David	Turner	1986-01-06	Greece	Ghost Cookies	true	11
2	Matthew	Helders	1986-05-07	United Kingdom	Agile Beast	false	12
3	John Martin	Maher	1966-10-31	United Kingdom	Johnny Marr	false	13
4	Steven Patrick	Morrissey	1959-05-22	United Kingdom	Morrissey	false	14
5	Brett Lewis	Anderson	1967-09-29	United Kingdom		false	15
6	Billie Joe	Armstrong	1972-02-17	USA		false	16
7	Elizabeth	Grant	1985-06-21	USA	Lana Del Rey	true	17
8	Agneta	Fältskog	1950-04-05	Sweden	Agnetha	false	18
9	Peter	Kinnes	1943-12-31	United Kingdom	Pete Quaife	false	19
10	David	Jones	1947-01-08	United Kingdom	David Bowie	true	20
11	Adrianne	Lenker	1991-07-09	USA		true	21
12	Miles	Kane	1986-03-17	United Kingdom		true	22
13	Ian	Brown	1963-02-20	United Kingdom		false	23
14	John	Mellor	1952-08-21	United Kingdom	Joe Strummer	false	24
15	Damon	Albarn	1968-03-23	United Kingdom		false	25
16	Madonna	Turner	1958-08-16	USA	Madonna	true	26
17	Angela	Trimble	1945-06-01	USA	Debbie Harry	false	27
18	Nelly Kim	Furtado	1978-12-02	Canada		true	28
19	Steve Thomas	Lacy-Moya	1998-05-23	USA	Steve Lacy	true	29
20	Lanita	Heyy	1988-09-04	Portugal	Lana	false	
21	Xoxoxo	XXOOXOX	2025-08-31	UK	xoxo	true	50
22	Elias	Tobias	1999-08-31	Finland	Toby	true	50

Φιγούρα 91: Στιγμιότυπο την εφαρμογή

22. deleteRow()

Λειτουργεί ακριβώς όπως οι άλλες συναρτήσεις μόνο που τώρα κατά την επιλογή διαγράφουμε πλήρως την εγγραφή από την ΒΔ.

Κώδικας συνάρτησης:

```
// διαγραφη εγγραφης απο πινακα
    private void deleteRow() {
        String tableName = (String) tableSelector.getSelectedItem();
        if (tableName == null)
            return;

        int selectedRow = table.getSelectedRow();
        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Παρακαλώ επιλέξτε μια γραμμή για διαγραφή.", "Προσοχή",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        try {
            // παιρνουμε πρωτευον κλειδι πινακα
            DatabaseMetaData dbMeta = conn.getMetaData();
            ResultSet pkeys = dbMeta.getPrimaryKeys(null, null, tableName);
            if (!pkeys.next()) {
                JOptionPane.showMessageDialog(this, "Ο πίνακας δεν έχει primary key!", "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }
            String pkColumn = pkeys.getString("COLUMN_NAME");

            // παιρνουμε πρωτευον κλειδι απο την γραμμη που μας ενδιαφερει
            int pkColIndex = findColumnIndex(pkColumn);
            if (pkColIndex == -1) {
                JOptionPane.showMessageDialog(this, "Δεν βρέθηκε η στήλη " + pkColumn, "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }

            Object pkValue = table.getValueAt(selectedRow, pkColIndex);
            if (pkValue == null) {
                JOptionPane.showMessageDialog(this, "Η επιλεγμένη γραμμή δεν έχει τιμή primary key!", "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }

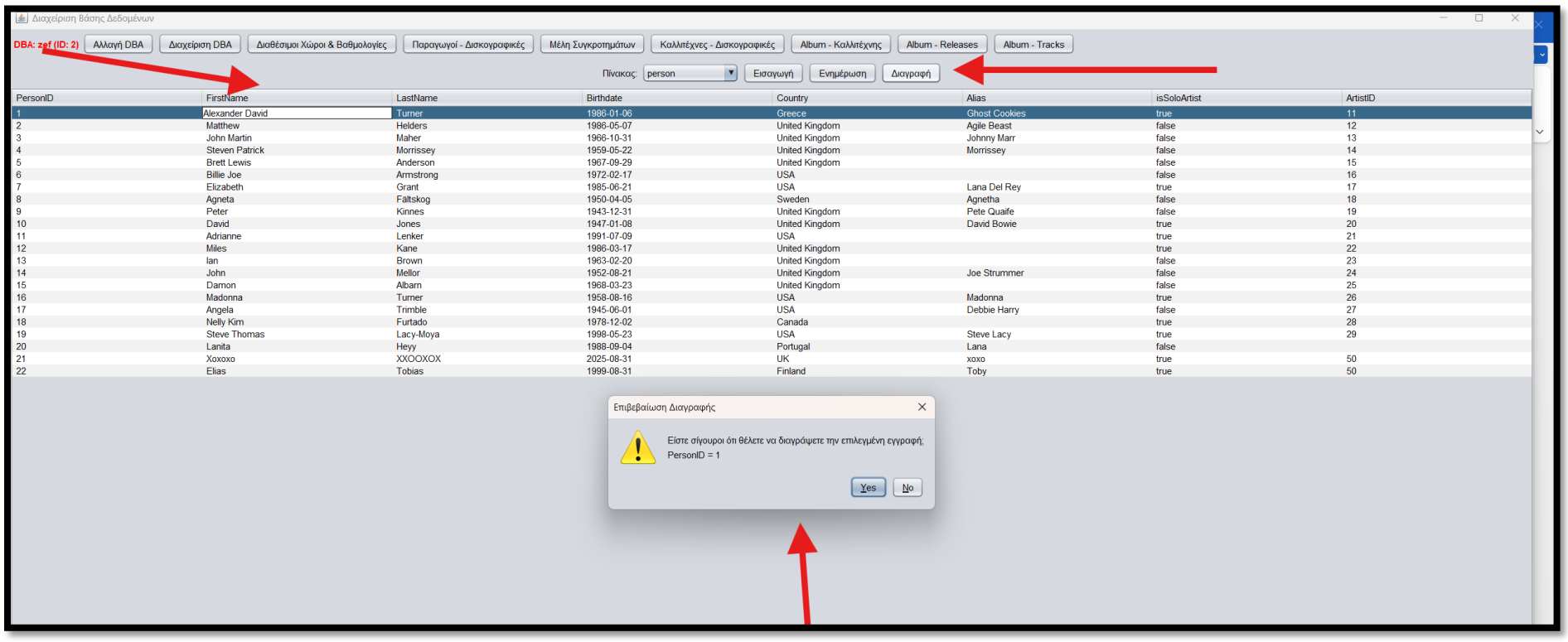
            // επιβεβαίωση διαγραφής
            int confirm = JOptionPane.showConfirmDialog(this,
                    "Είστε σίγουροι ότι θέλετε να διαγράψετε την επιλεγμένη εγγραφή;\n" +
                            pkColumn + " = " + pkValue,
                    "Επιβεβαίωση Διαγραφής",
                    JOptionPane.YES_NO_OPTION,
                    JOptionPane.WARNING_MESSAGE);

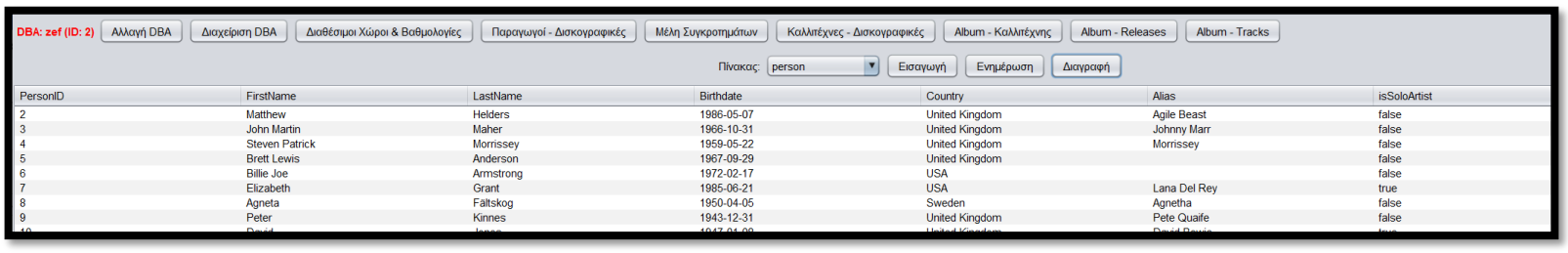
            if (confirm != JOptionPane.YES_OPTION)
                return;

            // Διαγραφή από τον πίνακα με χρήση sql
            String sql = "DELETE FROM " + tableName + " WHERE " + pkColumn + " = ?";
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setString(1, pkValue.toString());

                int affected = ps.executeUpdate();
                if (affected > 0) {
                    logDBAAction("DELETE", tableName, "Deleted record with " + pkColumn + "=" + pkValue);
                    loadTableData(tableName);
                    JOptionPane.showMessageDialog(this, "Η διαγραφή ολοκληρώθηκε επιτυχώς!");
                } else {
                    JOptionPane.showMessageDialog(this, "Δεν έγινε διαγραφή. Η εγγραφή μπορεί να μην υπάρχει πλέον.",
                            "Προσοχή", JOptionPane.WARNING_MESSAGE);
                }
            }
        } catch (SQLException e) {
            if (e.getMessage().toLowerCase().contains("foreign key")
                    || e.getMessage().toLowerCase().contains("reference")) {
                JOptionPane.showMessageDialog(this,
                        "Δεν μπορεί να γίνει διαγραφή γιατί η εγγραφή χρησιμοποιείται από άλλους πίνακες!",
                        "Σφάλμα Foreign Key",
                        JOptionPane.ERROR_MESSAGE);
            } else {
                JOptionPane.showMessageDialog(this, "Σφάλμα διαγραφής: " + e.getMessage(), "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
            }
        }
    }
```

- Διαγραφή εγγραφής ‘Alexander David Turner’





Φιγούρα 92: Σιγμιότυπο την εφαρμογή

Μέρος Γ’ – Παράρτημα

Συγκεντρωτικός κώδικας μέρους Α’

```
DROP DATABASE IF EXISTS project2024;
CREATE DATABASE project2024;
USE project2024;

-- Table: artist
CREATE TABLE artist (
    ArtistID INT AUTO_INCREMENT PRIMARY KEY,
    ArtistType ENUM('PERSON','BAND','ORCHESTRA','CHOIR','VIRTUAL','OTHER') NOT NULL
);

-- Table: person
CREATE TABLE person (
    PersonID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(80) NOT NULL,
    Birthdate DATE NOT NULL,
    Country VARCHAR(30) NOT NULL,
    Alias VARCHAR(50),
    isSoloArtist TINYINT(1) NOT NULL,
    ArtistID INT,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: band
CREATE TABLE band (
    BandID INT AUTO_INCREMENT PRIMARY KEY,
    BandName VARCHAR(50) NOT NULL,
    FormationDate DATE NOT NULL,
    DisbandDate DATE,
    ArtistID INT,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: bandmember
CREATE TABLE bandmember (
    BandID INT,
    PersonID INT,
    FromDate DATE NOT NULL,
    ToDate DATE,
    PRIMARY KEY (BandID , PersonID),
    FOREIGN KEY (BandID)
        REFERENCES band (BandID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (PersonID)
        REFERENCES person (PersonID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: recordcompany
CREATE TABLE recordcompany (
    CompanyID INT AUTO_INCREMENT PRIMARY KEY,
    CompanyName VARCHAR(80) NOT NULL,
    Address VARCHAR(150),
    Phone VARCHAR(20),
    Email VARCHAR(40) NOT NULL,
    BeginDate DATE NOT NULL,
    EndDate DATE
);

-- Table: artistcompany
-- This table right here explains to which record company an artist is assigned
CREATE TABLE artistcompany (
    ArtistID INT NOT NULL,
    CompanyID INT NOT NULL,
    FromDate DATE NOT NULL,
    ToDate DATE,
    PRIMARY KEY (ArtistID , CompanyID),
    FOREIGN KEY (ArtistID) REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CompanyID) REFERENCES recordcompany (CompanyID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: producer
CREATE TABLE producer (
    ProducerID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(80) NOT NULL,
    NumberOfProductions INT
);

-- Table: producercompany
-- This table right here explains to which record company an artist is assigned
CREATE TABLE producercompany (
    ProducerID INT NOT NULL,
    CompanyID INT NOT NULL,
    ToDate DATE,
    PRIMARY KEY (ProducerID , CompanyID),
    FOREIGN KEY (ProducerID)
```

```
        REFERENCES producer (ProducerID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CompanyID)
        REFERENCES recordcompany (CompanyID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: genre
CREATE TABLE genre (
    GenreID INT AUTO_INCREMENT PRIMARY KEY,
    GenreName VARCHAR(30) NOT NULL
);

-- Table: album
CREATE TABLE album (
    AlbumID INT AUTO_INCREMENT PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    ArtistID INT NOT NULL,
    GenreID INT NOT NULL,
    CompanyID INT NULL,
    ProducerID INT NOT NULL,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (GenreID)
        REFERENCES genre (GenreID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CompanyID)
        REFERENCES recordcompany (CompanyID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (ProducerID)
        REFERENCES producer (ProducerID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: track
CREATE TABLE track (
    TrackID INT AUTO_INCREMENT PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    AlbumID INT NOT NULL,
    TrackLength TIME NOT NULL,
    TrackNo INT,
    Lyrics TEXT CHARACTER SET utf8mb4,
    FOREIGN KEY (AlbumID)
        REFERENCES album (AlbumID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: albumrelease
CREATE TABLE albumrelease (
    ReleaseID INT AUTO_INCREMENT PRIMARY KEY,
    AlbumID INT NOT NULL,
    ReleaseDate DATE NOT NULL,
    ReleaseType ENUM('LP', 'CD', 'MP3') NOT NULL,
    ReleaseStatus ENUM('OFFICIAL', 'PROMOTION', 'BOOTLEG', 'WITHDRAWN', 'CANCELLED') NOT NULL,
    Packaging ENUM('BOOK', 'CARDBOARD SLEEVE', 'DIGIPAK', 'JEWEL CASE', 'NOT APPLICABLE') NOT NULL,
    FOREIGN KEY (AlbumID)
        REFERENCES album (AlbumID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Apart from the main DB , we create additional tables which are useful and needed for the next questions
CREATE TABLE venue (
    VenueID INT AUTO_INCREMENT PRIMARY KEY,
    VenueName VARCHAR(100) NOT NULL,
    Capacity INT NOT NULL,
    CompletedConcerts INT,
    YearsOperating INT,
    IsAvailable TINYINT(1) DEFAULT 1
);

-- Table: concert
CREATE TABLE concert (
    ConcertID INT AUTO_INCREMENT PRIMARY KEY,
    ArtistID INT NOT NULL,
    VenueID INT NULL,
    ConcertDate DATE NOT NULL,
    ConcertCapacity INT,
    TicketsSold INT DEFAULT 0,
    ConcertStatus ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED') NOT NULL,
    IsArchived TINYINT(1) DEFAULT 1,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (VenueID)
        REFERENCES venue (VenueID)
        ON DELETE CASCADE ON UPDATE CASCADE
);


-- Table: concert_history
CREATE TABLE concert_history (
    HistoricalID INT AUTO_INCREMENT PRIMARY KEY,
    First_Name VARCHAR (50) NOT NULL,
    Last_Name VARCHAR (50) NOT NULL ,
    ConcertID INT NOT NULL,
    ArtistID INT NOT NULL,
    VenueID INT NOT NULL,
    ConcertDate DATE NOT NULL,
```

```
    TicketsSold INT NOT NULL,
    ConcertStatus ENUM('COMPLETED', 'CANCELLED') NOT NULL
);

-- Table: dba
-- This table is used to keep track of who has access to the DB and manages tables and records
CREATE TABLE dba (
    DBA_ID INT AUTO_INCREMENT PRIMARY KEY,
    Username CHAR(15) UNIQUE NOT NULL,
    Start_Date DATE NOT NULL,
    End_Date DATE DEFAULT NULL
);


-- Table: dba_log
-- This is used to keep a log of all the actions made on the DB
-- We keep a track of the logged in user and log exactly what they change , where they make that change and the time they do.
-- (afto to erwthma einai akoma kalutero sto Java implementation)
CREATE TABLE dba_log (
    LogID INT AUTO_INCREMENT PRIMARY KEY,
    ActionDate DATETIME NOT NULL DEFAULT NOW(),
    DBA_Username CHAR(15) NOT NULL,
    log_action VARCHAR(255),
    FOREIGN KEY (DBA_Username)
        REFERENCES dba (Username)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Ευρετήρια (Indexes)
-- Index για την GetArtistsByTicketRange (διάστημα εισιτηρίων)
CREATE INDEX idx_tickets_sold ON concert_history (TicketsSold);

-- Index για την GetConcertDatesByVenue (χώρος διεξαγωγής)
CREATE INDEX idx_venue_id ON concert_history (VenueID);

-- Σύνθετο index για βελτιστοποίηση JOINs με artist πίνακα (afto tha xreiastei sto gui)
-- CREATE INDEX idx_artist_tickets ON concert_history (ArtistID, TicketsSold);
-- ALTER TABLE concert_history DROP INDEX idx_artist_tickets;

-- Index για ημερομηνία (χρήσιμο για χρονολογικές αναζητήσεις)
-- CREATE INDEX idx_concert_date ON concert_history (ConcertDate);
-- ALTER TABLE concert_history DROP INDEX idx_concert_date;


-- Απαίτηση 3.1.2.2
/*Αν βρεθούν παραπάνω από ένας κατάλληλοι χώροι διεξαγωγής για την
συναυλία θα επιλέγεται αυτός με τη μεγαλύτερη βαθμολογία, η οποία θα
υπολογίζεται σύμφωνα με τα κριτήρια, όπως φαίνονται στον παρακάτω
πίνακα.
*/

DELIMITER $

DROP PROCEDURE IF EXISTS AssignBestVenue$
CREATE PROCEDURE AssignBestVenue(IN concert_id INT, IN required_capacity INT)
BEGIN
    DECLARE selected_venue INT DEFAULT NULL;
    DECLARE venue_score INT DEFAULT 0;
    DECLARE max_score INT DEFAULT 0;

    -- Declare variables to use for our cursor
    DECLARE v_venue_id INT;
    DECLARE v_capacity INT;
    DECLARE v_completed_concerts INT;
    DECLARE v_years_operating INT;
    DECLARE v_current_score INT;
    DECLARE done INT DEFAULT FALSE;

    -- This cursor scans every free venue , calculates each one's score  (depending on what the exercise wants)
    -- and then assigns the best one (the one with the best score) for the concert
    DECLARE venue_cursor CURSOR FOR
        SELECT VenueID, Capacity, CompletedConcerts, YearsOperating
        FROM venue
        WHERE IsAvailable = 1
        AND Capacity >= (required_capacity * 1.1)  -- 10% more capacity
        AND VenueID NOT IN (
            SELECT VenueID FROM concert
            WHERE ConcertStatus = 'SCHEDULED'
            AND VenueID IS NOT NULL
        ); -- Exclude venues with scheduled concerts , we need free venues to have our concerts

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Initialize max score (simple iteration to find the best score)
    SET max_score = 0;

    OPEN venue_cursor;

    venue_loop: LOOP
        FETCH venue_cursor INTO v_venue_id, v_capacity, v_completed_concerts, v_years_operating;

        IF done THEN
            LEAVE venue_loop;
        END IF;

        -- Calculate score according to the table:
        -- For every 1,000 people capacity -> 1 point
        -- For every 100 completed concerts -> 3 points
        -- For every year of operation -> 2 points
```

```
        SET v_current_score =
            FLOOR(v_capacity / 1000) * 1 +
            FLOOR(v_completed_concerts / 100) * 3 +
            v_years_operating * 2;

        -- Select venue with highest score
        IF v_current_score > max_score THEN
            SET max_score = v_current_score;
            SET selected_venue = v_venue_id;
        END IF;

    END LOOP;

    CLOSE venue_cursor;

    -- If no venue is found, print an error so the user can know what went wrong and search again
    IF selected_venue IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'No suitable venue found for the concert with required capacity.';
    END IF;

    -- Update the concert with the selected venue
    UPDATE concert
    SET VenueID = selected_venue
    WHERE ConcertID = concert_id;

    -- Mark the venue as unavailable
    UPDATE venue
    SET IsAvailable = 0
    WHERE VenueID = selected_venue;

    -- Return selected venue info for confirmation
    SELECT
        selected_venue as SelectedVenueID,
        max_score as VenueScore,
        v.VenueName,
        v.Capacity
    FROM venue v
    WHERE v.VenueID = selected_venue;

END$



-- trigger το οποιο ειναι χρησιμο για να εχω το ιστορικο συναυλιων σε περιπτωση που αλλαξει κατι στο status
DELIMITER $$

CREATE PROCEDURE archive_concerts_proc()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_ConcertID INT;
    DECLARE v_ArtistID INT;
    DECLARE v_VenueID INT;
    DECLARE v_ConcertDate DATE;
    DECLARE v_TicketsSold INT;
    DECLARE v_ConcertStatus VARCHAR(20);

    -- Cursor για όλες τις συναυλίες που πρέπει να αρχειοθετηθούν
    DECLARE cur CURSOR FOR
        SELECT ConcertID, ArtistID, VenueID, ConcertDate, TicketsSold, ConcertStatus
        FROM concert
        WHERE ConcertStatus IN ('COMPLETED', 'CANCELLED')
          AND IsArchived = FALSE;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;

  read_loop: LOOP
    FETCH cur INTO v_ConcertID, v_ArtistID, v_VenueID, v_ConcertDate, v_TicketsSold, v_ConcertStatus;
    IF done THEN
        LEAVE read_loop;
    END IF;

    -- Εισαγωγή στο ιστορικό
    INSERT INTO concert_history (
        ConcertID,
        ArtistID,
        VenueID,
        ConcertDate,
        TicketsSold,
        ConcertStatus
    )
    VALUES (
        v_ConcertID,
        v_ArtistID,
        v_VenueID,
        v_ConcertDate,
        v_TicketsSold,
        v_ConcertStatus
    );

    -- Κάνε το venue διαθέσιμο
    UPDATE venue
    SET IsAvailable = 1
    WHERE VenueID = v_VenueID;

    -- Αν ήταν COMPLETED, αύξησε completed count
    IF v_ConcertStatus = 'COMPLETED' THEN
```

```
            UPDATE venue
            SET CompletedConcerts = CompletedConcerts + 1
            WHERE VenueID = v_VenueID;
        END IF;

        -- Μαρκάρισμα ως αρχειοθετημένο
        UPDATE concert
        SET IsArchived = TRUE
        WHERE ConcertID = v_ConcertID;
    END LOOP;

    CLOSE cur;
END$$

DELIMITER ;



-- Απαίτηση 3.1.3.1
-- Stored procedure η οποία παίρνει ως όρισμα το id ενός χώρου διεξαγωγής
-- και θα υπολογίζει και θα επιστρέφει το βαθμό του με βάση τον Πίνακα 2 της
-- Απαίτησης 3.1.2.2.

DROP PROCEDURE IF EXISTS CalculateVenueScore$
DELIMITER $
CREATE PROCEDURE CalculateVenueScore(IN venue_id INT) -- here, we use a variable named 'venue_id' , we type in the exact id we wanna
search for
                                                                                                        -- and then we fetch the results
where venue_id = VenueID from the venue table
BEGIN -- fetch the exact info the venue table holds for this specific 'venue_id' we are searching for
    SELECT
        VenueID,
        VenueName,
        Capacity,
        CompletedConcerts,
        YearsOperating,
        IsAvailable,

        -- Calculate the exact score as the table explains
        (FLOOR(Capacity / 1000) * 1 +
         FLOOR(CompletedConcerts / 100) * 3 +
         YearsOperating * 2) AS VenueScore
    FROM venue
    WHERE VenueID = venue_id;
END$
DELIMITER ;



-- Απαίτηση 3.1.3.2
-- Stored procedure στην οποία θα δίνονται ως ορίσματα εισόδου το id ενός
-- καλλιτέχνη, μια ημερομηνία και ένας χαρακτήρας (‘i’, ‘c’ ή ‘a’). (πιο αναλυτικά στην εκφώνηση)

DELIMITER //

CREATE PROCEDURE ConcertManagement (
    IN p_ArtistID INT,
    IN p_ConcertDate DATE,
    IN p_Action CHAR(1)
)
BEGIN
    DECLARE v_ConcertID INT;
    DECLARE v_Status ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED');

SELECT
    ConcertID, ConcertStatus
INTO v_ConcertID , v_Status FROM
    concert
WHERE
    ArtistID = p_ArtistID
        AND ConcertDate = p_ConcertDate
LIMIT 1;

    -- Actions based on our needs
    IF p_Action = 'i' THEN
        IF v_ConcertID IS NOT NULL THEN
            IF v_Status = 'SCHEDULED' THEN
                SELECT 'There is a concert already scheduled for this date' AS Message;
            ELSEIF v_Status = 'CANCELLED' THEN
                SELECT 'This date\'s concert has already been cancelled before' AS Message;
            ELSE
                SELECT 'Concert is completed' AS Message;
            END IF;
        ELSE
            -- Schedule new concert (VenueID = 1, ConcertCapacity = 1000)
            -- Check if venue exists and is available
            IF EXISTS(SELECT 1 FROM venue WHERE VenueID = 1 AND IsAvailable = 1) THEN
                INSERT INTO concert (ArtistID, VenueID, ConcertDate, ConcertCapacity, ConcertStatus)
                VALUES (p_ArtistID, 1, p_ConcertDate, 1000, 'SCHEDULED');

                SELECT 'Concert has been scheduled' AS Message;
            ELSE
                SELECT 'Venue is not available for scheduling' AS Message;
            END IF;
        END IF;

    ELSEIF p_Action = 'c' THEN
        IF v_ConcertID IS NULL THEN
```

```
            SELECT 'No concert found for this date.' AS Message;
        ELSE
            IF v_Status = 'CANCELLED' THEN
                SELECT 'Concert already cancelled' AS Message;
            ELSEIF v_Status = 'SCHEDULED' THEN
                UPDATE concert
                SET ConcertStatus = 'CANCELLED'
                WHERE ConcertID = v_ConcertID;
SELECT 'Concert cancelled successfully' AS Message;
            ELSE
                SELECT 'Concert already completed and can\'t be cancelled.' AS Message;
            END IF;
        END IF;

    ELSEIF p_Action = 'a' THEN
        IF v_ConcertID IS NULL THEN
            SELECT 'There is no cancelled concert' AS Message;
        ELSE
            IF v_Status = 'CANCELLED' THEN
                UPDATE concert
                SET ConcertStatus = 'SCHEDULED'
                WHERE ConcertID = v_ConcertID;
SELECT 'Cancelled concert is back up again' AS Message;
            ELSEIF v_Status = 'SCHEDULED' THEN
                SELECT 'Concert is active' AS Message;
            ELSE
                SELECT 'Concert is already completed' AS Message;
            END IF;
        END IF;

    ELSE
        SELECT 'Invalid type of action. Please only use:  ''i'', ''c'' or ''a''.' AS Message;
    END IF;
END //

DELIMITER ;




-- Απαίτηση 3.1.3.3
-- Stored procedure στην οποία δίνεται ως ορίσματα εισόδου το id μιας
-- συναυλίας και απαιτούμενη χωρητικότητα για τη συναυλία και επιστρέφει σε
-- μια μεταβλητή εξόδου το id χώρου διεξαγωγής και σε μια δεύτερη, την
-- χωρητικότητα του.

DELIMITER $$
DROP PROCEDURE IF EXISTS GetConcertVenueInfo$$
CREATE PROCEDURE GetConcertVenueInfo(
    IN concert_id INT,
    IN required_capacity INT,
    OUT venue_id INT,
    OUT venue_capacity INT
)
BEGIN
    DECLARE concert_exists INT DEFAULT 0;
    DECLARE concert_status ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED');
    DECLARE assigned_venue_id INT DEFAULT NULL;

    -- Έλεγχος αν υπάρχει η συναυλία και η κατάστασή της
    SELECT COUNT(*), ConcertStatus, VenueID
    INTO concert_exists, concert_status, assigned_venue_id
    FROM concert
    WHERE ConcertID = concert_id;

    -- Αν δεν υπάρχει συναυλία με το συγκεκριμένο ID ή είναι ακυρωμένη
    IF concert_exists = 0 OR concert_status = 'CANCELLED' THEN
        SET venue_id = NULL;
        SET venue_capacity = 0;
    ELSE
        -- Αν έχει ήδη κλειστεί χώρος διεξαγωγής για τη συγκεκριμένη συναυλία
        IF assigned_venue_id IS NOT NULL THEN
            -- Επιστρέφουμε το ID και την χωρητικότητα του ήδη κλεισμένου χώρου
            SELECT assigned_venue_id, v.Capacity
            INTO venue_id, venue_capacity
            FROM venue v
            WHERE v.VenueID = assigned_venue_id;
        ELSE
            -- Αν δεν έχει κλειστεί χώρος, βρίσκουμε τον καλύτερο διαθέσιμο
            -- χρησιμοποιώντας την ίδια λογική με την AssignBestVenue
            CALL AssignBestVenueForConcert(concert_id, required_capacity, venue_id, venue_capacity);
        END IF;
    END IF;
END$$

-- Βοηθητική stored procedure για εύρεση καλύτερου χώρου
DROP PROCEDURE IF EXISTS AssignBestVenueForConcert$$
CREATE PROCEDURE AssignBestVenueForConcert(
    IN concert_id INT,
    IN required_capacity INT,
    OUT selected_venue_id INT,
    OUT selected_venue_capacity INT
)
BEGIN
    DECLARE max_score INT DEFAULT 0;
    DECLARE v_venue_id INT;
    DECLARE v_capacity INT;
    DECLARE v_completed_concerts INT;
    DECLARE v_years_operating INT;
```

```
    DECLARE v_current_score INT;
    DECLARE done INT DEFAULT FALSE;

    -- Cursor για εύρεση κατάλληλων χώρων
    DECLARE venue_cursor CURSOR FOR
        SELECT VenueID, Capacity, CompletedConcerts, YearsOperating
        FROM venue
        WHERE IsAvailable = 1
        AND Capacity >= (required_capacity * 1.1)  -- 10% περισσότερη χωρητικότητα
        AND VenueID NOT IN (
            SELECT VenueID FROM concert
            WHERE ConcertStatus = 'SCHEDULED'
            AND VenueID IS NOT NULL
            AND ConcertID != concert_id  -- Εξαιρούμε την τρέχουσα συναυλία
        );

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Αρχικοποίηση
    SET selected_venue_id = NULL;
    SET selected_venue_capacity = 0;
    SET max_score = 0;

    OPEN venue_cursor;

    venue_loop: LOOP
        FETCH venue_cursor INTO v_venue_id, v_capacity, v_completed_concerts, v_years_operating;

        IF done THEN
            LEAVE venue_loop;
        END IF;

        -- Υπολογισμός βαθμολογίας
        SET v_current_score =
            FLOOR(v_capacity / 1000) * 1 +
            FLOOR(v_completed_concerts / 100) * 3 +
            v_years_operating * 2;

        -- Επιλογή χώρου με μεγαλύτερη βαθμολογία
        IF v_current_score > max_score THEN
            SET max_score = v_current_score;
            SET selected_venue_id = v_venue_id;
            SET selected_venue_capacity = v_capacity;
        END IF;

    END LOOP;

    CLOSE venue_cursor;

    -- Αν δεν βρέθηκε κατάλληλος χώρος
    IF selected_venue_id IS NULL THEN
        SET selected_venue_capacity = 0;
    END IF;
END$$

DELIMITER ;



-- Απαίτηση 3.1.3.4
-- α) Stored procedure για εύρεση καλλιτεχνών βάσει πλήθους εισιτηρίων
-- We use the index 'idx_tickets_sold' for quick search in the ticket range

DELIMITER $$

DROP PROCEDURE IF EXISTS GetArtistsByTicketRange$$
CREATE PROCEDURE GetArtistsByTicketRange(
    IN min_tickets INT,
    IN max_tickets INT
)
BEGIN
    SELECT First_Name, Last_Name, TicketsSold
    FROM concert_history
    WHERE TicketsSold BETWEEN min_tickets AND max_tickets
    ORDER BY First_Name, Last_Name;
END$$

DELIMITER ;



-- ΤΟ ΠΑΡΑΚΑΤΩ STORED PROCEDURE ΔΗΜΙΟΥΡΓΗΘΗΚΕ ΑΛΛΑ ΔΕΝ ΜΠΟΡΟΥΣΑ ΝΑ ΤΟ ΕΚΤΕΛΕΣΩ
-- ΓΙΑ ΤΟΝ ΕΛΕΓΧΟ ΤΟΥ INDEX ΚΑΘΩΣ ΑΠΑΙΤΟΥΣΕ ΤΗΝ ΕΙΣΑΓΩΓΗ ΠΟΛΛΩΝ ΕΓΓΡΑΦΩΝ ΚΑΙ ΣΤΟΝ ΠΙΝΑΚΑ VENUES
-- ΚΑΤΙ ΠΟΥ ΗΤΑΝ ΔΥΣΚΟΛΟ ΕΠΕΙΔΗ ΤΟ ΠΕΡΙΒΑΛΛΟΝ ΤΗΣ SQL ΕΚΑΝΕ HANG ΣΥΝΕΧΩΣ ΚΑΤΑ ΤΗΝ ΕΙΣΑΓΩΓΗ ΠΟΛΛΩΝ ΕΓΓΡΑΦΩΝ

/* β) Stored procedure στην οποία δίνεται ως παράμετρος εισόδου το όνομα
-- ενός χώρου διεξαγωγής συναυλιών και εμφανίζει τις ημερομηνίες που έγιναν
-- συναυλίες στον συγκεκριμένο χώρο
-- We use the previously created index named 'idx_venue_id' for a quick search by venue
-- DELIMITER $$
-- DROP PROCEDURE IF EXISTS GetConcertDatesByVenue$$
CREATE PROCEDURE GetConcertDatesByVenue(
    IN venue_name VARCHAR(100)
)
BEGIN
    -- Αναζήτηση στον πίνακα ιστορικού με χρήση του index idx_venue_id
    SELECT
        v.VenueName,
        ch.ConcertDate,
```

```
        CASE
            WHEN a.ArtistType = 'PERSON' THEN CONCAT(p.FirstName, ' ', p.LastName)
            WHEN a.ArtistType = 'BAND' THEN b.BandName
        END AS ArtistName,
        ch.ConcertStatus,
        ch.TicketsSold,
        'HISTORICAL' AS Source
    FROM concert_history ch
    INNER JOIN venue v ON ch.VenueID = v.VenueID
    INNER JOIN artist a ON ch.ArtistID = a.ArtistID
    LEFT JOIN person p ON a.ArtistID = p.ArtistID AND a.ArtistType = 'PERSON'
    LEFT JOIN band b ON a.ArtistID = b.ArtistID AND a.ArtistType = 'BAND'
    WHERE v.VenueName = venue_name


    ORDER BY ConcertDate;
END$$

DELIMITER ;

*/

-- Απαίτηση 3.1.4.1
-- Trigger που θα ενημερώνουν το σχετικό πίνακα καταγραφής ενεργειών
-- (log) για κάθε ενέργεια εισαγωγής, ενημέρωσης ή διαγραφής στους πίνακες
-- person, band, album, concert, venue με την ημερομηνία και ώρα και το
-- username του DBA που την εκτέλεσε.

SET @current_dba := 'DBA_USERNAME'; -- we set the current DBA manually

DELIMITER $$

-- Artist change log
DROP TRIGGER IF EXISTS artist_log_insert$$
CREATE TRIGGER artist_log_insert
AFTER INSERT ON artist
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: artist');
END$$

DROP TRIGGER IF EXISTS artist_log_update$$
CREATE TRIGGER artist_log_update
AFTER UPDATE ON artist
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: artist');
END$$

DROP TRIGGER IF EXISTS artist_log_delete$$
CREATE TRIGGER artist_log_delete
AFTER DELETE ON artist
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: artist');
END$$

-- Table 'person' changes log
DROP TRIGGER IF EXISTS person_log_insert$$
CREATE TRIGGER person_log_insert
AFTER INSERT ON person
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: person');
END$$

DROP TRIGGER IF EXISTS person_log_update$$
CREATE TRIGGER person_log_update
AFTER UPDATE ON person
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: person');
END$$

DROP TRIGGER IF EXISTS person_log_delete$$
CREATE TRIGGER person_log_delete
AFTER DELETE ON person
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: person');
END$$

-- Band changes log

DROP TRIGGER IF EXISTS band_log_insert$$
CREATE TRIGGER band_log_insert
AFTER INSERT ON band
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: band');
END$$
```

```
DROP TRIGGER IF EXISTS band_log_update$$
CREATE TRIGGER band_log_update
AFTER UPDATE ON band
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: band');
END$$

DROP TRIGGER IF EXISTS band_log_delete$$
CREATE TRIGGER band_log_delete
AFTER DELETE ON band
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: band');
END$$

-- Album changes log

DROP TRIGGER IF EXISTS album_log_insert$$
CREATE TRIGGER album_log_insert
AFTER INSERT ON album
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: album');
END$$

DROP TRIGGER IF EXISTS album_log_update$$
CREATE TRIGGER album_log_update
AFTER UPDATE ON album
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: album');
END$$

DROP TRIGGER IF EXISTS album_log_delete$$
CREATE TRIGGER album_log_delete
AFTER DELETE ON album
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: album');
END$$

-- Concert changes log

DROP TRIGGER IF EXISTS concert_log_insert$$
CREATE TRIGGER concert_log_insert
AFTER INSERT ON concert
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: concert');
END$$

DROP TRIGGER IF EXISTS concert_log_update$$
CREATE TRIGGER concert_log_update
AFTER UPDATE ON concert
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: concert');
END$$

DROP TRIGGER IF EXISTS concert_log_delete$$
CREATE TRIGGER concert_log_delete
AFTER DELETE ON concert
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: concert');
END$$

-- Venue changes log

DROP TRIGGER IF EXISTS venue_log_insert$$
CREATE TRIGGER venue_log_insert
AFTER INSERT ON venue
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: venue');
END$$

DROP TRIGGER IF EXISTS venue_log_update$$
CREATE TRIGGER venue_log_update
AFTER UPDATE ON venue
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: venue');
END$$

DROP TRIGGER IF EXISTS venue_log_delete$$
CREATE TRIGGER venue_log_delete
AFTER DELETE ON venue
```

```
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: venue');
END$$

DELIMITER ;



-- Απαίτηση 3.1.4.2
-- Trigger ο οποίος θα αποτρέπει τον προγραμματισμό νέας συναυλίας, αν η
-- ημερομηνία που γίνεται ο προγραμματισμός της συναυλίας απέχει λιγότερο
-- από πέντε μέρες από την ημερομηνία διεξαγωγής της ή αν ο συγκεκριμένος
-- καλλιτέχνης έχει ήδη τρεις προγραμματισμένες συναυλίες.

-- Prevent excessive concert scheduling
DELIMITER $$
DROP TRIGGER IF EXISTS prevent_too_many_concerts$$
CREATE TRIGGER prevent_too_many_concerts
BEFORE INSERT ON concert
FOR EACH ROW
BEGIN
    DECLARE scheduled_count INT;
    SELECT COUNT(*) INTO scheduled_count
    FROM concert
    WHERE ArtistID = NEW.ArtistID AND ConcertStatus = 'SCHEDULED';

    IF scheduled_count >= 3 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot schedule a new concert: Artist already has 3 scheduled concerts.';
    END IF;
END$$

-- Prevent scheduling less than 5 days in advance
DROP TRIGGER IF EXISTS prevent_last_minute_scheduling$$
CREATE TRIGGER prevent_last_minute_scheduling
BEFORE INSERT ON concert
FOR EACH ROW
BEGIN
    -- Error checking only when concert != cancelled or concert!= completed
    IF NEW.ConcertStatus IN ('SCHEDULED') THEN
        IF DATEDIFF(NEW.ConcertDate, CURDATE()) < 5 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot schedule a concert less than 5 days in advance.';
        END IF;
    END IF;
END$$



-- Απαίτηση 3.1.4.3
-- Trigger ο οποίος θα αποτρέπει την ακύρωση μιας συναυλίας, αν η
-- ημερομηνία που γίνεται η ακύρωση απέχει λιγότερο από τρεις μέρες από
-- την ημερομηνία διεξαγωγής της. Επίσης, θα αποτρέπει τον
-- επαναπρογραμματισμό ακυρωμένης συναυλίας αν ο συγκεκριμένος
-- καλλιτέχνης έχει ήδη τρεις προγραμματισμένες συναυλίες.
DELIMITER $$
DROP TRIGGER IF EXISTS prevent_late_cancellation_and_reschedule$$
CREATE TRIGGER prevent_late_cancellation_and_reschedule
BEFORE UPDATE ON concert
FOR EACH ROW
BEGIN
DECLARE scheduled_count INT;
    -- Cannot cancel 3 days before the concert , error checking
    IF NEW.ConcertStatus = 'CANCELLED' AND OLD.ConcertStatus != 'CANCELLED' THEN
        IF DATEDIFF(NEW.ConcertDate, CURDATE()) < 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot cancel a concert less than 3 days before the event.';
        END IF;
    END IF;

    -- Same as above with the excessive scheduling
    IF OLD.ConcertStatus = 'CANCELLED' AND NEW.ConcertStatus != 'CANCELLED' THEN

        SELECT COUNT(*)
        INTO scheduled_count
        FROM concert
        WHERE ArtistID = NEW.ArtistID
          AND ConcertStatus = 'SCHEDULED'
          AND ConcertID != NEW.ConcertID;

        -- If the artist already has 3 concerts scheduled.. you can't schedule a new one
        IF scheduled_count >= 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot reschedule a cancelled concert because the artist already has 3 scheduled concerts.';
        END IF;
    END IF;

END$$

DELIMITER ;



-- Miscellaneous
-- Just a quick check on whether the DBA we want to use exists or not
```

```
DELIMITER $$
CREATE PROCEDURE set_dba_user(IN input_username CHAR(15))
BEGIN
    DECLARE end_date_val DATE;
    DECLARE user_exists INT DEFAULT 0;

    SELECT COUNT(*) INTO user_exists
    FROM dba
    WHERE Username = input_username;

    IF user_exists = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'User does not exist in DBA table.';
    ELSE
        SELECT End_Date INTO end_date_val
        FROM dba
        WHERE Username = input_username;

        IF end_date_val IS NOT NULL THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'User is no longer affiliated with us (End_Date: NOT NULL). Cannot use this DBA.';
        ELSE
            SELECT 'User is active. Continue' AS Message;
        END IF;
    END IF;
END$$

DELIMITER ;
```

Συγκεντρωτικός κώδικας μέρους Β'

Βάση Δεδομένων για το GUI

```
DROP DATABASE IF EXISTS project2024_gui;
CREATE DATABASE project2024_gui;
USE project2024_gui;

-- Table: artist
CREATE TABLE artist (
    ArtistID INT AUTO_INCREMENT PRIMARY KEY,
    ArtistType ENUM('PERSON','BAND','ORCHESTRA','CHOIR','VIRTUAL','OTHER') NOT NULL
);

-- Table: person
CREATE TABLE person (
    PersonID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(80) NOT NULL,
    Birthdate DATE NOT NULL,
    Country VARCHAR(30) NOT NULL,
    Alias VARCHAR(50),
    isSoloArtist TINYINT(1) NOT NULL,
    ArtistID INT,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: band
CREATE TABLE band (
    BandID INT AUTO_INCREMENT PRIMARY KEY,
    BandName VARCHAR(50) NOT NULL,
    FormationDate DATE NOT NULL,
    DisbandDate DATE,
    ArtistID INT,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: bandmember
CREATE TABLE bandmember (
    BandID INT,
    PersonID INT,
    FromDate DATE NOT NULL,
    ToDate DATE,
    PRIMARY KEY (BandID , PersonID),
    FOREIGN KEY (BandID)
        REFERENCES band (BandID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (PersonID)
        REFERENCES person (PersonID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: recordcompany
CREATE TABLE recordcompany (
    CompanyID INT AUTO_INCREMENT PRIMARY KEY,
    CompanyName VARCHAR(80) NOT NULL,
    Address VARCHAR(150),
    Phone VARCHAR(20),
    Email VARCHAR(40) NOT NULL,
    BeginDate DATE NOT NULL,
    EndDate DATE
);
```

```
-- Table: artistcompany
-- This table right here explains to which record company an artist is assigned
CREATE TABLE artistcompany (
    ArtistID INT NOT NULL,
    CompanyID INT NOT NULL,
    FromDate DATE NOT NULL,
    ToDate DATE,
    PRIMARY KEY (ArtistID , CompanyID),
    FOREIGN KEY (ArtistID) REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CompanyID) REFERENCES recordcompany (CompanyID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: producer
CREATE TABLE producer (
    ProducerID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(80) NOT NULL,
    NumberOfProductions INT
);

-- Table: producercompany
-- This table right here explains to which record company an artist is assigned
CREATE TABLE producercompany (
    ProducerID INT NOT NULL,
    CompanyID INT NOT NULL,
    ToDate DATE,
    PRIMARY KEY (ProducerID , CompanyID),
    FOREIGN KEY (ProducerID)
        REFERENCES producer (ProducerID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CompanyID)
        REFERENCES recordcompany (CompanyID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: genre
CREATE TABLE genre (
    GenreID INT AUTO_INCREMENT PRIMARY KEY,
    GenreName VARCHAR(30) NOT NULL
);

-- Table: album
CREATE TABLE album (
    AlbumID INT AUTO_INCREMENT PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    ArtistID INT NOT NULL,
    GenreID INT NOT NULL,
    CompanyID INT NULL,
    ProducerID INT NOT NULL,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (GenreID)
        REFERENCES genre (GenreID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (CompanyID)
        REFERENCES recordcompany (CompanyID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (ProducerID)
        REFERENCES producer (ProducerID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: track
CREATE TABLE track (
    TrackID INT AUTO_INCREMENT PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    AlbumID INT NOT NULL,
    TrackLength TIME NOT NULL,
    TrackNo INT,
    Lyrics TEXT CHARACTER SET utf8mb4,
    FOREIGN KEY (AlbumID)
        REFERENCES album (AlbumID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table: albumrelease
CREATE TABLE albumrelease (
    ReleaseID INT AUTO_INCREMENT PRIMARY KEY,
    AlbumID INT NOT NULL,
    ReleaseDate DATE NOT NULL,
    ReleaseType ENUM('LP', 'CD', 'MP3') NOT NULL,
    ReleaseStatus ENUM('OFFICIAL', 'PROMOTION', 'BOOTLEG', 'WITHDRAWN', 'CANCELLED') NOT NULL,
    Packaging ENUM('BOOK', 'CARDBOARD SLEEVE', 'DIGIPAK', 'JEWEL CASE', 'NOT APPLICABLE') NOT NULL,
    FOREIGN KEY (AlbumID)
        REFERENCES album (AlbumID)
        ON DELETE CASCADE ON UPDATE CASCADE
);
-- Apart from the main DB , we create additional tables which are useful and needed for the next questions

CREATE TABLE venue (
```

```
    VenueID INT AUTO_INCREMENT PRIMARY KEY,
    VenueName VARCHAR(100) NOT NULL,
    Capacity INT NOT NULL,
    CompletedConcerts INT,
    YearsOperating INT,
    IsAvailable TINYINT(1) DEFAULT 1
);

-- Table: concert
CREATE TABLE concert (
    ConcertID INT AUTO_INCREMENT PRIMARY KEY,
    ArtistID INT NOT NULL,
    VenueID INT NULL,
    ConcertDate DATE NOT NULL,
    ConcertCapacity INT,
    TicketsSold INT DEFAULT 0,
    ConcertStatus ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED') NOT NULL,
    IsArchived TINYINT(1) DEFAULT 1,
    FOREIGN KEY (ArtistID)
        REFERENCES artist (ArtistID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (VenueID)
        REFERENCES venue (VenueID)
        ON DELETE CASCADE ON UPDATE CASCADE
);


-- Table: concert_history
CREATE TABLE concert_history (
    HistoricalID INT AUTO_INCREMENT PRIMARY KEY,
    First_Name VARCHAR (50) NOT NULL,
    Last_Name VARCHAR (50) NOT NULL ,
    ConcertID INT NOT NULL,
    ArtistID INT NOT NULL,
    VenueID INT NOT NULL,
    ConcertDate DATE NOT NULL,
    TicketsSold INT NOT NULL,
    ConcertStatus ENUM('COMPLETED', 'CANCELLED') NOT NULL
);

-- Table: dba
-- This table is used to keep track of who has access to the DB and manages tables and records
CREATE TABLE dba (
    DBA_ID INT AUTO_INCREMENT PRIMARY KEY,
    Username CHAR(15) UNIQUE NOT NULL,
    FirstName VARCHAR(50),
    LastName VARCHAR(80),
    Email VARCHAR(100),
    Start_Date DATE NOT NULL DEFAULT (CURRENT_DATE()),
    End_Date DATE DEFAULT NULL
);


-- Table: dba_log
-- This is used to keep a log of all the actions made on the DB
-- We keep a track of the logged in user and log exactly what they change , where they make that change and the time they do.

CREATE TABLE dba_log (
    LogID INT AUTO_INCREMENT PRIMARY KEY,
    ActionDate DATETIME NOT NULL DEFAULT NOW(),
    DBA_Username CHAR(15) NOT NULL,
    log_action VARCHAR(255),
    FOREIGN KEY (DBA_Username)
        REFERENCES dba (Username)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Ευρετήρια (Indexes) για βελτιστοποίηση απόδοσης των stored procedures
-- Index για την stored procedure GetArtistsByTicketRange (διάστημα εισιτηρίων)
CREATE INDEX idx_tickets_sold ON concert_history (TicketsSold);

-- Index για την stored procedure GetConcertDatesByVenue (χώρος διεξαγωγής)
CREATE INDEX idx_venue_id ON concert_history (VenueID);

-- Σύνθετο index για βελτιστοποίηση JOINs με artist πίνακα
CREATE INDEX idx_artist_tickets ON concert_history (ArtistID, TicketsSold);

-- Index για ημερομηνία (χρήσιμο για χρονολογικές αναζητήσεις)
CREATE INDEX idx_concert_date ON concert_history (ConcertDate);




-- Απαίτηση 3.1.2.2
/*Αν βρεθούν παραπάνω από ένας κατάλληλοι χώροι διεξαγωγής για την
συναυλία θα επιλέγεται αυτός με τη μεγαλύτερη βαθμολογία, η οποία θα
υπολογίζεται σύμφωνα με τα κριτήρια, όπως φαίνονται στον παρακάτω
πίνακα.
*/

DELIMITER $

DROP PROCEDURE IF EXISTS AssignBestVenue$
```

```
CREATE PROCEDURE AssignBestVenue(IN concert_id INT, IN required_capacity INT)
BEGIN
    DECLARE selected_venue INT DEFAULT NULL;
    DECLARE venue_score INT DEFAULT 0;
    DECLARE max_score INT DEFAULT 0;

    -- Declare variables to use for our cursor
    DECLARE v_venue_id INT;
    DECLARE v_capacity INT;
    DECLARE v_completed_concerts INT;
    DECLARE v_years_operating INT;
    DECLARE v_current_score INT;
    DECLARE done INT DEFAULT FALSE;

    -- This cursor scans every free venue , calculates each one's score  (depending on what the exercise wants)
    -- and then assigns the best one (the one with the best score) for the concert
    DECLARE venue_cursor CURSOR FOR
        SELECT VenueID, Capacity, CompletedConcerts, YearsOperating
        FROM venue
        WHERE IsAvailable = 1
        AND Capacity >= (required_capacity * 1.1)  -- 10% more capacity
        AND VenueID NOT IN (
            SELECT VenueID FROM concert
            WHERE ConcertStatus = 'SCHEDULED'
            AND VenueID IS NOT NULL
        ); -- Exclude venues with scheduled concerts , we need free venues to have our concerts

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Initialize max score (simple iteration to find the best score)
    SET max_score = 0;

    OPEN venue_cursor;

    venue_loop: LOOP
        FETCH venue_cursor INTO v_venue_id, v_capacity, v_completed_concerts, v_years_operating;

        IF done THEN
            LEAVE venue_loop;
        END IF;

        -- Calculate score according to the table:
        -- For every 1,000 people capacity -> 1 point
        -- For every 100 completed concerts -> 3 points
        -- For every year of operation -> 2 points
        SET v_current_score =
            FLOOR(v_capacity / 1000) * 1 +
            FLOOR(v_completed_concerts / 100) * 3 +
            v_years_operating * 2;

        -- Select venue with highest score
        IF v_current_score > max_score THEN
            SET max_score = v_current_score;
            SET selected_venue = v_venue_id;
        END IF;

    END LOOP;

    CLOSE venue_cursor;

    -- If no venue is found, print an error so the user can know what went wrong and search again
    IF selected_venue IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'No suitable venue found for the concert with required capacity.';
    END IF;

    -- Update the concert with the selected venue
    UPDATE concert
    SET VenueID = selected_venue
    WHERE ConcertID = concert_id;

    -- Mark the venue as unavailable
    UPDATE venue
    SET IsAvailable = 0
    WHERE VenueID = selected_venue;

    -- Return selected venue info for confirmation
    SELECT
        selected_venue as SelectedVenueID,
        max_score as VenueScore,
        v.VenueName,
        v.Capacity
    FROM venue v
    WHERE v.VenueID = selected_venue;

END$



-- trigger το οποιο ειναι χρησιμο για να εχω το ιστορικο συναυλιων σε περιπτωση που αλλαξει κατι στο status
DELIMITER $$

CREATE PROCEDURE archive_concerts_proc()
BEGIN
```

```
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_ConcertID INT;
    DECLARE v_ArtistID INT;
    DECLARE v_VenueID INT;
    DECLARE v_ConcertDate DATE;
    DECLARE v_TicketsSold INT;
    DECLARE v_ConcertStatus VARCHAR(20);

    -- Cursor για όλες τις συναυλίες που πρέπει να αρχειοθετηθούν
    DECLARE cur CURSOR FOR
        SELECT ConcertID, ArtistID, VenueID, ConcertDate, TicketsSold, ConcertStatus
        FROM concert
        WHERE ConcertStatus IN ('COMPLETED', 'CANCELLED')
          AND IsArchived = FALSE;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO v_ConcertID, v_ArtistID, v_VenueID, v_ConcertDate, v_TicketsSold, v_ConcertStatus;
        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Εισαγωγή στο ιστορικό
        INSERT INTO concert_history (
            ConcertID,
            ArtistID,
            VenueID,
            ConcertDate,
            TicketsSold,
            ConcertStatus
        )
        VALUES (
            v_ConcertID,
            v_ArtistID,
            v_VenueID,
            v_ConcertDate,
            v_TicketsSold,
            v_ConcertStatus
        );

        -- Κάνε το venue διαθέσιμο
        UPDATE venue
        SET IsAvailable = 1
        WHERE VenueID = v_VenueID;

        -- Αν ήταν COMPLETED, αύξησε completed count
        IF v_ConcertStatus = 'COMPLETED' THEN
            UPDATE venue
            SET CompletedConcerts = CompletedConcerts + 1
            WHERE VenueID = v_VenueID;
        END IF;

        -- Μαρκάρισμα ως αρχειοθετημένο
        UPDATE concert
        SET IsArchived = TRUE
        WHERE ConcertID = v_ConcertID;
    END LOOP;

    CLOSE cur;
END$$

DELIMITER ;



-- Απαίτηση 3.1.3.1
-- Stored procedure η οποία παίρνει ως όρισμα το id ενός χώρου διεξαγωγής
-- και θα υπολογίζει και θα επιστρέφει το βαθμό του με βάση τον Πίνακα 2 της
-- Απαίτησης 3.1.2.2.

DROP PROCEDURE IF EXISTS CalculateVenueScore$
DELIMITER $
CREATE PROCEDURE CalculateVenueScore(IN venue_id INT) -- here, we use a variable named 'venue_id' , we type in the exact id we
wanna search for
                                                                                                    -- and then we fetch the results
where venue_id = VenueID from the venue table
BEGIN -- fetch the exact info the venue table holds for this specific 'venue_id' we are searching for
    SELECT
        VenueID,
        VenueName,
        Capacity,
        CompletedConcerts,
        YearsOperating,
        IsAvailable,

        -- Calculate the exact score as the table explains
        (FLOOR(Capacity / 1000) * 1 +
         FLOOR(CompletedConcerts / 100) * 3 +
         YearsOperating * 2) AS VenueScore
    FROM venue
    WHERE VenueID = venue_id;
```

```
END$
DELIMITER ;




-- Απαίτηση 3.1.3.2
-- Stored procedure στην οποία θα δίνονται ως ορίσματα εισόδου το id ενός
-- καλλιτέχνη, μια ημερομηνία και ένας χαρακτήρας (‘i’, ‘c’ ή ‘a’). (πιο αναλυτικά στην εκφώνηση)

DELIMITER //

CREATE PROCEDURE ConcertManagement (
    IN p_ArtistID INT,
    IN p_ConcertDate DATE,
    IN p_Action CHAR(1)
)
BEGIN
    DECLARE v_ConcertID INT;
    DECLARE v_Status ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED');

SELECT
    ConcertID, ConcertStatus
INTO v_ConcertID , v_Status FROM
    concert
WHERE
    ArtistID = p_ArtistID
        AND ConcertDate = p_ConcertDate
LIMIT 1;

    -- Actions based on our needs
    IF p_Action = 'i' THEN
        IF v_ConcertID IS NOT NULL THEN
            IF v_Status = 'SCHEDULED' THEN
                SELECT 'There is a concert already scheduled for this date' AS Message;
            ELSEIF v_Status = 'CANCELLED' THEN
                SELECT 'This date\'s concert has already been cancelled before' AS Message;
            ELSE
                SELECT 'Concert is completed' AS Message;
            END IF;
        ELSE
            -- Schedule new concert (VenueID = 1, ConcertCapacity = 1000)
            -- Check if venue exists and is available
            IF EXISTS(SELECT 1 FROM venue WHERE VenueID = 1 AND IsAvailable = 1) THEN
                INSERT INTO concert (ArtistID, VenueID, ConcertDate, ConcertCapacity, ConcertStatus)
                VALUES (p_ArtistID, 1, p_ConcertDate, 1000, 'SCHEDULED');

                SELECT 'Concert has been scheduled' AS Message;
            ELSE
                SELECT 'Venue is not available for scheduling' AS Message;
            END IF;
        END IF;

    ELSEIF p_Action = 'c' THEN
        IF v_ConcertID IS NULL THEN
            SELECT 'No concert found for this date.' AS Message;
        ELSE
            IF v_Status = 'CANCELLED' THEN
                SELECT 'Concert already cancelled' AS Message;
            ELSEIF v_Status = 'SCHEDULED' THEN
                UPDATE concert
                SET ConcertStatus = 'CANCELLED'
                WHERE ConcertID = v_ConcertID;
SELECT 'Concert cancelled successfully' AS Message;
            ELSE
                SELECT 'Concert already completed and can\'t be cancelled.' AS Message;
            END IF;
        END IF;

    ELSEIF p_Action = 'a' THEN
        IF v_ConcertID IS NULL THEN
            SELECT 'There is no cancelled concert' AS Message;
        ELSE
            IF v_Status = 'CANCELLED' THEN
                UPDATE concert
                SET ConcertStatus = 'SCHEDULED'
                WHERE ConcertID = v_ConcertID;
SELECT 'Cancelled concert is back up again' AS Message;
            ELSEIF v_Status = 'SCHEDULED' THEN
                SELECT 'Concert is active' AS Message;
            ELSE
                SELECT 'Concert is already completed' AS Message;
            END IF;
        END IF;

    ELSE
        SELECT 'Invalid type of action. Please only use:  ''i'', ''c'' or ''a''.' AS Message;
    END IF;
END //

DELIMITER ;
```

```
-- Απαίτηση 3.1.3.3
-- Stored procedure στην οποία δίνεται ως ορίσματα εισόδου το id μιας
-- συναυλίας και απαιτούμενη χωρητικότητα για τη συναυλία και επιστρέφει σε
-- μια μεταβλητή εξόδου το id χώρου διεξαγωγής και σε μια δεύτερη, την
-- χωρητικότητα του.

DELIMITER $$
DROP PROCEDURE IF EXISTS GetConcertVenueInfo$$
CREATE PROCEDURE GetConcertVenueInfo(
    IN concert_id INT,
    IN required_capacity INT,
    OUT venue_id INT,
    OUT venue_capacity INT
)
BEGIN
    DECLARE concert_exists INT DEFAULT 0;
    DECLARE concert_status ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED');
    DECLARE assigned_venue_id INT DEFAULT NULL;

    -- Έλεγχος αν υπάρχει η συναυλία και η κατάστασή της
    SELECT COUNT(*), ConcertStatus, VenueID
    INTO concert_exists, concert_status, assigned_venue_id
    FROM concert
    WHERE ConcertID = concert_id;

    -- Αν δεν υπάρχει συναυλία με το συγκεκριμένο ID ή είναι ακυρωμένη
    IF concert_exists = 0 OR concert_status = 'CANCELLED' THEN
        SET venue_id = NULL;
        SET venue_capacity = 0;
    ELSE
        -- Αν έχει ήδη κλειστεί χώρος διεξαγωγής για τη συγκεκριμένη συναυλία
        IF assigned_venue_id IS NOT NULL THEN
            -- Επιστρέφουμε το ID και την χωρητικότητα του ήδη κλεισμένου χώρου
            SELECT assigned_venue_id, v.Capacity
            INTO venue_id, venue_capacity
            FROM venue v
            WHERE v.VenueID = assigned_venue_id;
        ELSE
            -- Αν δεν έχει κλειστεί χώρος, βρίσκουμε τον καλύτερο διαθέσιμο
            -- χρησιμοποιώντας την ίδια λογική με την AssignBestVenue
            CALL AssignBestVenueForConcert(concert_id, required_capacity, venue_id, venue_capacity);
        END IF;
    END IF;
END$$

-- Βοηθητική stored procedure για εύρεση καλύτερου χώρου
DROP PROCEDURE IF EXISTS AssignBestVenueForConcert$$
CREATE PROCEDURE AssignBestVenueForConcert(
    IN concert_id INT,
    IN required_capacity INT,
    OUT selected_venue_id INT,
    OUT selected_venue_capacity INT
)
BEGIN
    DECLARE max_score INT DEFAULT 0;
    DECLARE v_venue_id INT;
    DECLARE v_capacity INT;
    DECLARE v_completed_concerts INT;
    DECLARE v_years_operating INT;
    DECLARE v_current_score INT;
    DECLARE done INT DEFAULT FALSE;

    -- Cursor για εύρεση κατάλληλων χώρων
    DECLARE venue_cursor CURSOR FOR
        SELECT VenueID, Capacity, CompletedConcerts, YearsOperating
        FROM venue
        WHERE IsAvailable = 1
        AND Capacity >= (required_capacity * 1.1)  -- 10% περισσότερη χωρητικότητα
        AND VenueID NOT IN (
            SELECT VenueID FROM concert
            WHERE ConcertStatus = 'SCHEDULED'
            AND VenueID IS NOT NULL
            AND ConcertID != concert_id  -- Εξαιρούμε την τρέχουσα συναυλία
        );

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Αρχικοποίηση
    SET selected_venue_id = NULL;
    SET selected_venue_capacity = 0;
    SET max_score = 0;

    OPEN venue_cursor;

    venue_loop: LOOP
        FETCH venue_cursor INTO v_venue_id, v_capacity, v_completed_concerts, v_years_operating;

        IF done THEN
            LEAVE venue_loop;
        END IF;

        -- Υπολογισμός βαθμολογίας
        SET v_current_score =
```

```
            FLOOR(v_capacity / 1000) * 1 +
            FLOOR(v_completed_concerts / 100) * 3 +
            v_years_operating * 2;

        -- Επιλογή χώρου με μεγαλύτερη βαθμολογία
        IF v_current_score > max_score THEN
            SET max_score = v_current_score;
            SET selected_venue_id = v_venue_id;
            SET selected_venue_capacity = v_capacity;
        END IF;

    END LOOP;

    CLOSE venue_cursor;

    -- Αν δεν βρέθηκε κατάλληλος χώρος
    IF selected_venue_id IS NULL THEN
        SET selected_venue_capacity = 0;
    END IF;
END$$

DELIMITER ;



-- Απαίτηση 3.1.3.4
-- α) Stored procedure για εύρεση καλλιτεχνών βάσει πλήθους εισιτηρίων
-- We use the index 'idx_tickets_sold' for quick search in the ticket range

DELIMITER $$

DROP PROCEDURE IF EXISTS GetArtistsByTicketRange$$
CREATE PROCEDURE GetArtistsByTicketRange(
    IN min_tickets INT,
    IN max_tickets INT
)
BEGIN
    SELECT First_Name, Last_Name, TicketsSold
    FROM concert_history
    WHERE TicketsSold BETWEEN min_tickets AND max_tickets
    ORDER BY First_Name, Last_Name;
END$$

DELIMITER ;




-- β) Stored procedure στην οποία δίνεται ως παράμετρος εισόδου το όνομα
-- ενός χώρου διεξαγωγής συναυλιών και εμφανίζει τις ημερομηνίες που έγιναν
-- συναυλίες στον συγκεκριμένο χώρο
-- We use the previously created index named 'idx_venue_id' for a quick search by venue

DELIMITER $$
DROP PROCEDURE IF EXISTS GetConcertDatesByVenue$$
CREATE PROCEDURE GetConcertDatesByVenue(
    IN venue_name VARCHAR(100)
)
BEGIN
    -- Αναζήτηση στον πίνακα ιστορικού με χρήση του index idx_venue_id
    SELECT
        v.VenueName,
        ch.ConcertDate,
        CASE
            WHEN a.ArtistType = 'PERSON' THEN CONCAT(p.FirstName, ' ', p.LastName)
            WHEN a.ArtistType = 'BAND' THEN b.BandName
    END AS ArtistName,
    ch.ConcertStatus,
    ch.TicketsSold,
    'HISTORICAL' AS Source
FROM concert_history ch
INNER JOIN venue v ON ch.VenueID = v.VenueID
INNER JOIN artist a ON ch.ArtistID = a.ArtistID
LEFT JOIN person p ON a.ArtistID = p.ArtistID AND a.ArtistType = 'PERSON'
LEFT JOIN band b ON a.ArtistID = b.ArtistID AND a.ArtistType = 'BAND'
WHERE v.VenueName = venue_name

UNION

-- Επίσης αναζήτηση στον ενεργό πίνακα συναυλιών
SELECT
    v.VenueName,
    c.ConcertDate,
    CASE
        WHEN a.ArtistType = 'PERSON' THEN CONCAT(p.FirstName, ' ', p.LastName)
        WHEN a.ArtistType = 'BAND' THEN b.BandName
    END AS ArtistName,
    c.ConcertStatus,
    c.TicketsSold,
    'ACTIVE' AS Source
FROM concert c
INNER JOIN venue v ON c.VenueID = v.VenueID
INNER JOIN artist a ON c.ArtistID = a.ArtistID
```

```
    LEFT JOIN person p ON a.ArtistID = p.ArtistID AND a.ArtistType = 'PERSON'
    LEFT JOIN band b ON a.ArtistID = b.ArtistID AND a.ArtistType = 'BAND'
    WHERE v.VenueName = venue_name

    ORDER BY ConcertDate;
END$$

DELIMITER ;



-- Απαίτηση 3.1.4.1
-- Trigger που θα ενημερώνουν το σχετικό πίνακα καταγραφής ενεργειών
-- (log) για κάθε ενέργεια εισαγωγής, ενημέρωσης ή διαγραφής στους πίνακες
-- person, band, album, concert, venue με την ημερομηνία και ώρα και το
-- username του DBA που την εκτέλεσε.

SET @current_dba := 'DBA_USERNAME'; -- we set the current DBA manually

DELIMITER $$

-- Artist change log
DROP TRIGGER IF EXISTS artist_log_insert$$
CREATE TRIGGER artist_log_insert
AFTER INSERT ON artist
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: artist');
END$$

DROP TRIGGER IF EXISTS artist_log_update$$
CREATE TRIGGER artist_log_update
AFTER UPDATE ON artist
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: artist');
END$$

DROP TRIGGER IF EXISTS artist_log_delete$$
CREATE TRIGGER artist_log_delete
AFTER DELETE ON artist
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: artist');
END$$

-- Table 'person' changes log
DROP TRIGGER IF EXISTS person_log_insert$$
CREATE TRIGGER person_log_insert
AFTER INSERT ON person
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: person');
END$$

DROP TRIGGER IF EXISTS person_log_update$$
CREATE TRIGGER person_log_update
AFTER UPDATE ON person
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: person');
END$$

DROP TRIGGER IF EXISTS person_log_delete$$
CREATE TRIGGER person_log_delete
AFTER DELETE ON person
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: person');
END$$

-- Band changes log

DROP TRIGGER IF EXISTS band_log_insert$$
CREATE TRIGGER band_log_insert
AFTER INSERT ON band
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: band');
END$$

DROP TRIGGER IF EXISTS band_log_update$$
CREATE TRIGGER band_log_update
AFTER UPDATE ON band
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
```

```
    VALUES (NOW(), @current_dba, 'Modified data in table: band');
END$$

DROP TRIGGER IF EXISTS band_log_delete$$
CREATE TRIGGER band_log_delete
AFTER DELETE ON band
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: band');
END$$

-- Album changes log

DROP TRIGGER IF EXISTS album_log_insert$$
CREATE TRIGGER album_log_insert
AFTER INSERT ON album
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: album');
END$$

DROP TRIGGER IF EXISTS album_log_update$$
CREATE TRIGGER album_log_update
AFTER UPDATE ON album
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: album');
END$$

DROP TRIGGER IF EXISTS album_log_delete$$
CREATE TRIGGER album_log_delete
AFTER DELETE ON album
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: album');
END$$

-- Concert changes log

DROP TRIGGER IF EXISTS concert_log_insert$$
CREATE TRIGGER concert_log_insert
AFTER INSERT ON concert
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: concert');
END$$

DROP TRIGGER IF EXISTS concert_log_update$$
CREATE TRIGGER concert_log_update
AFTER UPDATE ON concert
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: concert');
END$$

DROP TRIGGER IF EXISTS concert_log_delete$$
CREATE TRIGGER concert_log_delete
AFTER DELETE ON concert
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: concert');
END$$

-- Venue changes log

DROP TRIGGER IF EXISTS venue_log_insert$$
CREATE TRIGGER venue_log_insert
AFTER INSERT ON venue
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Inserted data in table: venue');
END$$

DROP TRIGGER IF EXISTS venue_log_update$$
CREATE TRIGGER venue_log_update
AFTER UPDATE ON venue
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Modified data in table: venue');
END$$

DROP TRIGGER IF EXISTS venue_log_delete$$
CREATE TRIGGER venue_log_delete
AFTER DELETE ON venue
FOR EACH ROW
```

```
BEGIN
    INSERT INTO dba_log (ActionDate, DBA_Username, log_action)
    VALUES (NOW(), @current_dba, 'Deleted data in table: venue');
END$$

DELIMITER ;



-- Απαίτηση 3.1.4.2
-- Trigger ο οποίος θα αποτρέπει τον προγραμματισμό νέας συναυλίας, αν η
-- ημερομηνία που γίνεται ο προγραμματισμός της συναυλίας απέχει λιγότερο
-- από πέντε μέρες από την ημερομηνία διεξαγωγής της ή αν ο συγκεκριμένος
-- καλλιτέχνης έχει ήδη τρεις προγραμματισμένες συναυλίες.

-- Prevent excessive concert scheduling
DELIMITER $$
DROP TRIGGER IF EXISTS prevent_too_many_concerts$$
CREATE TRIGGER prevent_too_many_concerts
BEFORE INSERT ON concert
FOR EACH ROW
BEGIN
    DECLARE scheduled_count INT;
    SELECT COUNT(*) INTO scheduled_count
    FROM concert
    WHERE ArtistID = NEW.ArtistID AND ConcertStatus = 'SCHEDULED';

    IF scheduled_count >= 3 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot schedule a new concert: Artist already has 3 scheduled concerts.';
    END IF;
END$$

-- Prevent scheduling less than 5 days in advance
DROP TRIGGER IF EXISTS prevent_last_minute_scheduling$$
CREATE TRIGGER prevent_last_minute_scheduling
BEFORE INSERT ON concert
FOR EACH ROW
BEGIN
    -- Error checking only when concert != cancelled or concert!= completed
    IF NEW.ConcertStatus IN ('SCHEDULED') THEN
        IF DATEDIFF(NEW.ConcertDate, CURDATE()) < 5 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot schedule a concert less than 5 days in advance.';
        END IF;
    END IF;
END$$



-- Απαίτηση 3.1.4.3
-- Trigger ο οποίος θα αποτρέπει την ακύρωση μιας συναυλίας, αν η
-- ημερομηνία που γίνεται η ακύρωση απέχει λιγότερο από τρεις μέρες από
-- την ημερομηνία διεξαγωγής της. Επίσης, θα αποτρέπει τον
-- επαναπρογραμματισμό ακυρωμένης συναυλίας αν ο συγκεκριμένος
-- καλλιτέχνης έχει ήδη τρεις προγραμματισμένες συναυλίες.
DELIMITER $$
DROP TRIGGER IF EXISTS prevent_late_cancellation_and_reschedule$$
CREATE TRIGGER prevent_late_cancellation_and_reschedule
BEFORE UPDATE ON concert
FOR EACH ROW
BEGIN
DECLARE scheduled_count INT;
    -- Cannot cancel 3 days before the concert , error checking
    IF NEW.ConcertStatus = 'CANCELLED' AND OLD.ConcertStatus != 'CANCELLED' THEN
        IF DATEDIFF(NEW.ConcertDate, CURDATE()) < 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot cancel a concert less than 3 days before the event.';
        END IF;
    END IF;

    -- Same as above with the excessive scheduling
    IF OLD.ConcertStatus = 'CANCELLED' AND NEW.ConcertStatus != 'CANCELLED' THEN

        SELECT COUNT(*)
        INTO scheduled_count
        FROM concert
        WHERE ArtistID = NEW.ArtistID
          AND ConcertStatus = 'SCHEDULED'
          AND ConcertID != NEW.ConcertID;

        -- If the artist already has 3 concerts scheduled.. you can't schedule a new one
        IF scheduled_count >= 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot reschedule a cancelled concert because the artist already has 3 scheduled concerts.';
        END IF;
    END IF;

END$$

DELIMITER ;
```

```
-- Miscellaneous
-- Just a quick check on whether the DBA we want to use exists or not

DELIMITER $$
CREATE PROCEDURE set_dba_user(IN input_username CHAR(15))
BEGIN
    DECLARE end_date_val DATE;
    DECLARE user_exists INT DEFAULT 0;

    SELECT COUNT(*) INTO user_exists
    FROM dba
    WHERE Username = input_username;

    IF user_exists = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'User does not exist in DBA table.';
    ELSE
        SELECT End_Date INTO end_date_val
        FROM dba
        WHERE Username = input_username;

        IF end_date_val IS NOT NULL THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'User is no longer affiliated with us (End_Date: NOT NULL). Cannot use this
DBA.';
        ELSE
            SELECT 'User is active. Continue' AS Message;
        END IF;
    END IF;
END$$

DELIMITER ;
```

Κώδικας Java

```
package dbmanager;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.sql.*;
import java.util.*;
import java.util.Map;
import java.sql.Date;

public class DatabaseUI extends JFrame {
    private JComboBox<String> tableSelector;
    private JTable table;
    private DefaultTableModel tableModel;
    private Connection conn;

    // DBA Management fields
    private String currentDBAUsername;
    private Integer currentDBAId;
    private JLabel dbaStatusLabel;

    public DatabaseUI() {
        setTitle("Διαχείριση Βάσης Δεδομένων");
        setSize(2002, 1181);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        try {
            UIManager.setLookAndFeel("javax.swing.plaf.nimbus.NimbusLookAndFeel");
        } catch (Exception ignored) {
        }

        connectToDB();

        if (!authenticateDBA()) {
            System.exit(0);
            return;
        }

        setupGUI();
    }

    private void setupGUI() {
        JPanel topPanel = new JPanel(new BorderLayout());

        JPanel dbaPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        dbaStatusLabel = new JLabel("Welcome \n" + currentDBAUsername + " (ID: " + currentDBAId + ")");
        dbaStatusLabel.setForeground(Color.RED);
        dbaStatusLabel.setFont(dbaStatusLabel.getFont().deriveFont(Font.BOLD));

        // Κουμπιά για διαχείριση DBA
        JButton logoutBtn = new JButton("Αλλαγή DBA");
        logoutBtn.addActionListener(e -> changeDBA());
        JButton manageDBABtn = new JButton("Διαχείριση DBA");
        manageDBABtn.addActionListener(e -> manageDBAUsers());
        // Εμφάνιση διαθέσιμων χώρων και βαθμολογίες τους
        JButton viewVenuesBtn = new JButton("Διαθέσιμοι Χώροι & Βαθμολογίες");
        viewVenuesBtn.addActionListener(e -> showVenuesWithScores());

        // Κουμπιά για εμφάνιση σχέσεων
        JButton viewProducerCompanyBtn = new JButton("Παραγωγοί - Δισκογραφικές");
        viewProducerCompanyBtn.addActionListener(e -> showProducerCompanyRelations());

        JButton viewBandMembersBtn = new JButton("Μέλη Συγκροτημάτων");
        viewBandMembersBtn.addActionListener(e -> showBandMembers());
```

```
        JButton viewArtistCompanyBtn = new JButton("Καλλιτέχνες - Δισκογραφικές");
        viewArtistCompanyBtn.addActionListener(e -> showArtistCompanyRelations());

        // Album relation buttons
        JButton viewAlbumArtistBtn = new JButton("Album - Καλλιτέχνης");
        viewAlbumArtistBtn.addActionListener(e -> showAlbumArtistRelations());

        JButton viewAlbumReleasesBtn = new JButton("Album - Releases");
        viewAlbumReleasesBtn.addActionListener(e -> showAlbumReleases());

        JButton viewAlbumTracksBtn = new JButton("Album - Tracks");
        viewAlbumTracksBtn.addActionListener(e -> showAlbumTracks());

        dbaPanel.add(dbaStatusLabel);
        dbaPanel.add(logoutBtn);
        dbaPanel.add(manageDBABtn);
        dbaPanel.add(viewVenuesBtn);
        dbaPanel.add(viewProducerCompanyBtn);
        dbaPanel.add(viewBandMembersBtn);
        dbaPanel.add(viewArtistCompanyBtn);
        dbaPanel.add(viewAlbumArtistBtn);
        dbaPanel.add(viewAlbumReleasesBtn);
        dbaPanel.add(viewAlbumTracksBtn);

        topPanel.add(dbaPanel, BorderLayout.NORTH);

        JPanel mainPanel = new JPanel(new BorderLayout());

        tableSelector = new JComboBox<>();
        tableSelector.addActionListener(e -> {
            String sel = (String) tableSelector.getSelectedItem();
            if (sel != null && !sel.startsWith("---")) {
                loadTableData(sel);
            }
        });

        tableModel = new DefaultTableModel();
        table = new JTable(tableModel);

        // Κουμπιά για διαχείριση των εγγραφών στους πίνακες της ΒΔ
        JPanel controls = new JPanel();
        JButton insertBtn = new JButton("Εισαγωγή");
        insertBtn.addActionListener(e -> insertRow()); // χρήση της συνάρτησης insertRow για να γίνει εισαγωγή των
                                                       // δεδομένων
        JButton updateBtn = new JButton("Ενημέρωση");
        updateBtn.addActionListener(e -> updateRow()); // χρήση της συνάρτησης updateRow για να γίνει ενημέρωση των
                                                       // δεδομένων
        JButton deleteBtn = new JButton("Διαγραφή");
        deleteBtn.addActionListener(e -> deleteRow()); // χρήση της συνάρτησης deleteRow για να γίνει διαγραφή των
                                                       // δεδομένων

        controls.add(new JLabel("Πίνακας:"));
        controls.add(tableSelector); // Εμφάνιση πινάκων που υπάρχουν στην ΒΔ
        controls.add(insertBtn); // κουμπί εισαγωγής
        controls.add(updateBtn); // κουμπί ενημέρωσης
        controls.add(deleteBtn); // κουμπί διαγραφής

        mainPanel.add(controls, BorderLayout.NORTH);
        mainPanel.add(new JScrollPane(table), BorderLayout.CENTER);

        getContentPane().add(topPanel, BorderLayout.NORTH);
        getContentPane().add(mainPanel, BorderLayout.CENTER);

        SwingUtilities.invokeLater(this::loadTables);
    }

    // Σύνδεση στη ΒΔ (εισαγωγή των credentials manually)
    private void connectToDB() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            JOptionPane.showMessageDialog(this, "Ο MySQL JDBC driver δεν βρέθηκε: " + e.getMessage(), "Error",
                    JOptionPane.ERROR_MESSAGE);
            System.exit(1);
        } // Αναγκαίο σε περίπτωση που ο χρήστης δεν εχει εγκατεστημένο τον JDBC driver

        // Εισαγωγή των credentials για σύνδεση στην DB (υπάρχουν ήδη προκαθορισμένα
        // αυτά που χρησιμοποιώ εγώ)
        while (true) {
            String host = JOptionPane.showInputDialog(this, "MySQL Host:", "localhost");
            if (host == null)
                System.exit(0);
            String port = JOptionPane.showInputDialog(this, "Port:", "3306");
            if (port == null)
                System.exit(0);
            String dbName = JOptionPane.showInputDialog(this, "Όνομα βάσης δεδομένων:", "project2024_gui");
            if (dbName == null)
                System.exit(0);
            String user = JOptionPane.showInputDialog(this, "Username:", "root");
            if (user == null)
                System.exit(0);
            JPasswordField pf = new JPasswordField();
            int okCxl = JOptionPane.showConfirmDialog(this, pf, "Εισάγετε password", JOptionPane.OK_CANCEL_OPTION,
                    JOptionPane.PLAIN_MESSAGE);
            if (okCxl != JOptionPane.OK_OPTION)
                System.exit(0);
            String password = new String(pf.getPassword());

            // error handling
            String url = "jdbc:mysql://" + host + ":" + port + "/" + dbName
                    + "?serverTimezone=UTC&allowPublicKeyRetrieval=true";
            try {
                conn = DriverManager.getConnection(url, user, password);
                break;
            } catch (SQLException e) {
```

```
            int retry = JOptionPane.showConfirmDialog(this,
                    "Σφάλμα σύνδεσης με τη βάση δεδομένων:\n" + e.getMessage() + "\n\nΘέλετε να δοκιμάσετε ξανά;",
                    "Σφάλμα σύνδεσης",
                    JOptionPane.YES_NO_OPTION,
                    JOptionPane.ERROR_MESSAGE);

            if (retry != JOptionPane.YES_OPTION) {
                JOptionPane.showMessageDialog(this, "Το πρόγραμμα θα τερματίσει.", "Έξοδος",
                        JOptionPane.INFORMATION_MESSAGE);
                System.exit(0);
            }
        }
    }
}

// Θέτουμε session variable για τον τρέχοντα DBA ώστε να μπορούν τα triggers να
// το διαβάσουν
// Αυτό είχε αναφερθεί και στο μέρος Α όπου δουλεύαμε στο MySQL Workbench
private void setSessionDBA(String username) {
    if (conn == null || username == null)
        return;
    try (Statement s = conn.createStatement()) {
        String safe = username.replace("'", "''");
        s.execute("SET @current_dba = '" + safe + "'");
    } catch (SQLException e) {
        System.err.println("Δεν έγινε η ανάθεση της session variable: " + e.getMessage());
    }
}

// Αυθεντικοποίηση DBA
private boolean authenticateDBA() {
    while (true) {
        String username = JOptionPane.showInputDialog(this,
        "Εισάγετε το Username του DBA:",
        "Αυθεντικοποίηση DBA",
        JOptionPane.QUESTION_MESSAGE);

    if (username == null)
        return false;
    username = username.trim();
    if (username.isEmpty()) {
        JOptionPane.showMessageDialog(this,
                "Παρακαλώ εισάγετε έγκυρο username.",
                "Σφάλμα",
                JOptionPane.ERROR_MESSAGE);
        continue;
    }

    try (PreparedStatement ps = conn
            .prepareStatement("SELECT DBA_ID, Username FROM dba WHERE Username = ? AND End_Date IS NULL")) {
        ps.setString(1, username);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                currentDBAId = rs.getInt("DBA_ID");
                currentDBAUsername = rs.getString("Username");
                setSessionDBA(currentDBAUsername);
                JOptionPane.showMessageDialog(this,
                        "Καλώς ήρθατε " + currentDBAUsername + "!\n",
                        "Επιτυχής είσοδος", JOptionPane.INFORMATION_MESSAGE);
                return true;
            }
        }

        // Σε περίπτωση που δεν υπάρχει ενεργός DBA , δημιουργούμε νέο και κάνουμε login
        // με αυτόν
        try (Statement s2 = conn.createStatement();
                ResultSet cntRs = s2.executeQuery("SELECT COUNT(*) FROM dba WHERE End_Date IS NULL")) {
            int activeCount = 0;
            if (cntRs.next())
                activeCount = cntRs.getInt(1);
            if (activeCount == 0) {
                int create = JOptionPane.showConfirmDialog(this,
                        "Δεν υπάρχει ενεργός DBA. Θέλετε να δημιουργήσετε έναν διαχειριστή τώρα?",
                        "Δημιουργία Διαχειριστή",
                        JOptionPane.YES_NO_OPTION);
                if (create == JOptionPane.YES_OPTION) {
                    addNewDBA();
                    if (currentDBAUsername != null) {
                        JOptionPane.showMessageDialog(this,
                                "Καλώς ήρθατε " + currentDBAUsername
                                        + "!",
                                "Επιτυχής είσοδος", JOptionPane.INFORMATION_MESSAGE);
                        return true;
                    }
                }
            }
        }
        // Σε περίπτωση που δεν βρεθεί κάποιος ενεργός DBA, το εμφανίζουμε στον χρήστη
        JOptionPane.showMessageDialog(this, "Δεν βρέθηκε ενεργός DBA με username: " + username, "Σφάλμα",
                JOptionPane.ERROR_MESSAGE);
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, "Σφάλμα κατά την επαλήθευση: " + e.getMessage(), "Σφάλμα",
                JOptionPane.ERROR_MESSAGE);
    }
    }
}

// Συνάρτηση για αλλαγή του DBA
private void changeDBA() {
    int result = JOptionPane.showConfirmDialog(this,
            "Θέλετε να αλλάξετε DBA? ",
            "Αλλαγή DBA",
            JOptionPane.YES_NO_OPTION);

    if (result == JOptionPane.YES_OPTION) {
```

```
            currentDBAId = null; // μεταβλητή για να λάβει το ID του DBA που θέλει να συνδεθεί
            currentDBAUsername = null; // μεταβλητή για να λάβει το username του DBA που θέλει να συνδεθεί

            // αν έχουμε επιτυχή αυθεντικοποίηση
            if (authenticateDBA()) {
                dbaStatusLabel.setText("DBA: " + currentDBAUsername + " (ID: " + currentDBAId + ")");
                JOptionPane.showMessageDialog(this,
                        "Επιτυχής αλλαγή DBA σε: " + currentDBAUsername,
                        "Αλλαγή DBA",
                        JOptionPane.INFORMATION_MESSAGE);
            } else {
                System.exit(0);
            }
        }
    }

    // Συναρτηση για την διαχειριση των DBA χρηστών
    private void manageDBAUsers() {
        String[] options = { "Προσθήκη νέου DBA", "Προβολή όλων των DBA", "Διαγραφή DBA", "Ακύρωση" };
        int choice = JOptionPane.showOptionDialog(this,
                "Τι θέλετε να κάνετε;",
                "Διαχείριση DBA Χρηστών",
                JOptionPane.DEFAULT_OPTION,
                JOptionPane.QUESTION_MESSAGE,
                null,
                options,
                options[0]);

        switch (choice) {
            case 0:
                addNewDBA(); // Προσθήκη νέου DBA
                break;
            case 1:
                viewAllDBAs(); // Προβολή όλων των DBA
                break;
            case 2:
                deleteDBA(); // Διαγραφή DBA
                break;
            default:
                break;
        }
    }

    // Προσθήκη νέου DBA
    private void addNewDBA() {
        JPanel panel = new JPanel(new GridLayout(5, 2));
        JTextField usernameField = new JTextField();
        JTextField firstNameField = new JTextField();
        JTextField lastNameField = new JTextField();
        JTextField emailField = new JTextField();
        // Δημιουργουμε πεδια για την εισαγωγη των δεδομενων του DBA .
        // Ο,τι γράφουμε , περνάει στην ΒΔ project2024_gui
        // στον πινακα 'dba' που περιεχει τα νεα μας πεδια ( αυτή είναι η διαφορά με την
        // δομή της βάσης της 1ης φάσης)
        panel.add(new JLabel("Username:"));
        panel.add(usernameField);
        panel.add(new JLabel("Όνομα:"));
        panel.add(firstNameField);
        panel.add(new JLabel("Επώνυμο:"));
        panel.add(lastNameField);
        panel.add(new JLabel("Email:"));
        panel.add(emailField);

        int result = JOptionPane.showConfirmDialog(this, panel,
                "Προσθήκη νέου DBA", JOptionPane.OK_CANCEL_OPTION);

        if (result != JOptionPane.OK_OPTION)
            return;

        String username = usernameField.getText().trim();
        String firstName = firstNameField.getText().trim();
        String lastName = lastNameField.getText().trim();
        String email = emailField.getText().trim();

        // Αν ο χρήστης παραλείψει να βάλει username εμφαν΄ίζουμε σφάλμα
        if (username.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Το username είναι υποχρεωτικό!",
                    "Σφάλμα", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Εδώ ελέγχουμε αν υπάρχει το username στον πίνακα DBA με την χρήση sql query.
        // Αν υπάρχει ήδη, τότε το εμφανίζουμε στον χρήστη και τον προτρέπουμε να
        // επιλέξει
        // άλλο
        try (PreparedStatement checkStmt = conn.prepareStatement("SELECT COUNT(*) FROM dba WHERE Username = ?")) {
            checkStmt.setString(1, username);
            try (ResultSet rs = checkStmt.executeQuery()) {
                if (rs.next() && rs.getInt(1) > 0) {
                    JOptionPane.showMessageDialog(this,
                            "Το username '" + username + "' υπάρχει ήδη!",
                            "Σφάλμα", JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα βάσης δεδομένων: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Προετοιμάζουμε την εισαγωγή των δεδομένων στον πίνακα της βάσης. Ωστόσο
        // κάνουμε και έναν έλεγχο αν έχουν προστεθεί σωστά τα πεδία ,
        // διότι άνθρωποι είμαστε. Αν υπάρχει κάτι κενό ,
        // τότε εμφανίζουμε σφάλμα και δεν εισάγουμε κάτι στον πίνακα
```

```
        try (PreparedStatement insertStmt = conn.prepareStatement(
                "INSERT INTO dba (Username, FirstName, LastName, Email, Start_Date) VALUES (?, ?, ?, ?, CURDATE())",
                Statement.RETURN_GENERATED_KEYS)) {

            insertStmt.setString(1, username);
            insertStmt.setString(2, firstName.isEmpty() ? null : firstName);
            insertStmt.setString(3, lastName.isEmpty() ? null : lastName);
            insertStmt.setString(4, email.isEmpty() ? null : email);

            int rowsAffected = insertStmt.executeUpdate();
            if (rowsAffected <= 0) {
                JOptionPane.showMessageDialog(this, "Σφάλμα κατά την προσθήκη του DBA!", "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }

            try (ResultSet gk = insertStmt.getGeneratedKeys()) {
                if (gk != null && gk.next())
                    currentDBAId = gk.getInt(1);
            }

            if (currentDBAId == null || currentDBAId == 0) {
                try (PreparedStatement q = conn.prepareStatement("SELECT DBA_ID FROM dba WHERE Username = ? LIMIT 1")) {
                    q.setString(1, username);
                    try (ResultSet r2 = q.executeQuery()) {
                        if (r2.next())
                            currentDBAId = r2.getInt(1);
                    }
                }
            }
            // Εφόσον έχουμε το currentDBAId, μπορούμε να το χρησιμοποιήσουμε τώρα γι' αυτό
            // και το θέτουμε στην μεταβλητή currentDBAUsername που δημιουργήσαμε νωρίτερα
            currentDBAUsername = username;
            setSessionDBA(currentDBAUsername);

            // Επιτυχία προσθήκης και σφάλμα αν υπήρξε πρόβλημα στην ΒΔ
            JOptionPane.showMessageDialog(this, "Ο DBA '" + username + "' προστέθηκε επιτυχώς!", "Επιτυχία",
                    JOptionPane.INFORMATION_MESSAGE);
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα βάσης δεδομένων: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    private void viewAllDBAs() {
        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(
                        "SELECT DBA_ID, Username, FirstName, LastName, Email, Start_Date, End_Date FROM dba ORDER BY Username")) {

            StringBuilder dbaList = new StringBuilder();
            dbaList.append("Λίστα όλων των DBA:\n\n");
            dbaList.append(String.format("%-5s %-15s %-15s %-15s %-25s %-12s %-12s\n",
                    "ID", "Username", "Όνομα", "Επώνυμο", "Email", "Start_Date", "End_Date"));
            for (int i = 0; i < 100; i++)
                dbaList.append("—");
            dbaList.append("\n");

            while (rs.next()) {
                dbaList.append(String.format("%-5d %-15s %-15s %-15s %-25s %-12s %-12s\n",
                        rs.getInt("DBA_ID"),
                        rs.getString("Username") != null ? rs.getString("Username") : "",
                        rs.getString("FirstName") != null ? rs.getString("FirstName") : "",
                        rs.getString("LastName") != null ? rs.getString("LastName") : "",
                        rs.getString("Email") != null ? rs.getString("Email") : "",
                        rs.getDate("Start_Date") != null ? rs.getDate("Start_Date").toString() : "",
                        rs.getDate("End_Date") != null ? rs.getDate("End_Date").toString() : "Ενεργός"));
            }

            JTextArea textArea = new JTextArea(dbaList.toString());
            textArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
            textArea.setEditable(false);
            JScrollPane scrollPane = new JScrollPane(textArea);
            scrollPane.setPreferredSize(new Dimension(800, 400));

            JOptionPane.showMessageDialog(this, scrollPane, "Λίστα DBA", JOptionPane.INFORMATION_MESSAGE);
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα κατά την ανάκτηση των DBA: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    // Διαγραφή ενός DBA από την ΒΔ
    private void deleteDBA() {
        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(
                        "SELECT DBA_ID, Username FROM dba WHERE DBA_ID != " + currentDBAId + " ORDER BY Username")) {

            ArrayList<String> dbaOptions = new ArrayList<>();
            java.util.Map<String, Integer> usernameToId = new java.util.HashMap<>();

            while (rs.next()) {
                String username = rs.getString("Username");
                int id = rs.getInt("DBA_ID");
                dbaOptions.add(username + " (ID: " + id + ")");
                usernameToId.put(username + " (ID: " + id + ")", id);
            }

            // αν δεν υπαρχουν αλλοι χρηστες στον πινακα περα απο εμας που ειμαστε logged in
            if (dbaOptions.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Δεν υπάρχουν άλλοι DBA για διαγραφή!", "Πληροφορία",
                        JOptionPane.INFORMATION_MESSAGE);
                return;
            }
            // Εμφανιση παραθυρου διαλογου ωστε να επιλεξουμε τον χρηστη για διαγραφη
            String selected = (String) JOptionPane.showInputDialog(this,
```

```
                "Επιλέξτε DBA για διαγραφή:\n(Δεν μπορείτε να διαγράψετε τον εαυτό σας)",
                "Διαγραφή DBA",
                JOptionPane.QUESTION_MESSAGE,
                null,
                dbaOptions.toArray(),
                dbaOptions.get(0));

        if (selected != null) {
            int selectedId = usernameToId.get(selected);

            // παραθυρο επιβεβαιωσης
            int confirm = JOptionPane.showConfirmDialog(this,
                    "Είστε σίγουροι ότι θέλετε να διαγράψετε τον DBA: " + selected + "?\n" +
                            "Αυτή η ενέργεια θα ορίσει End_Date στον DBA και δεν μπορεί να αναιρεθεί!",
                    "Επιβεβαίωση Διαγραφής",
                    JOptionPane.YES_NO_OPTION,
                    JOptionPane.WARNING_MESSAGE);

            // καλουμε το συγκεκριμενο sql query ωστε να κανουμε
            // ενημερωση του πινακα.
            // για την διαγραφη αυτο που κανουμε ουσιαστικα ειναι να ορισουμε ως end date
            // την τωρινη στιγμη με την συναρτηση NOW()
            // ετσι παυει να ειναι ενεργος ο χρηστης , αρα δεν μπορει να χρησιμοποιηθει
            // ξανα στην ΒΔ
            if (confirm == JOptionPane.YES_OPTION) {
                try (PreparedStatement updateStmt = conn
                        .prepareStatement("UPDATE dba SET End_Date = CURDATE() WHERE DBA_ID = ?")) {
                    updateStmt.setInt(1, selectedId);
                    int rowsAffected = updateStmt.executeUpdate();
                    if (rowsAffected > 0) {
                        JOptionPane.showMessageDialog(this, "Ο DBA απενεργοποιήθηκε επιτυχώς!", "Επιτυχία",
                                JOptionPane.INFORMATION_MESSAGE);
                    } else {
                        JOptionPane.showMessageDialog(this, "Σφάλμα κατά την απενεργοποίηση του DBA!", "Σφάλμα",
                                JOptionPane.ERROR_MESSAGE);
                    }
                }
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα βάσης δεδομένων: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    // Simple logging helper (keeps previous behavior)
    private void logDBAAction(String action, String tableName, String details) {
        if (currentDBAId == null || currentDBAUsername == null)
            return;
        try (PreparedStatement ps = conn
                .prepareStatement("INSERT INTO dba_log (ActionDate, dba_username, log_action) VALUES (NOW(), ?, ?)")) {
            ps.setString(1, currentDBAUsername);
            ps.setString(2, action + " on " + tableName + ": " + details);
            ps.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Failed to log DBA action: " + e.getMessage());
        }
    }

    // Φορτωση ολων των πινακων απο την ΒΔ
    private void loadTables() {
        try {
            if (conn == null)
                return;
            tableSelector.removeAllItems();

            try (Statement stmt = conn.createStatement();
                    ResultSet rs = stmt.executeQuery(
                            "SELECT table_name FROM information_schema.tables " +
                                    "WHERE table_schema = 'project2024_gui' AND table_type = 'BASE TABLE' " +
                                    "ORDER BY table_name")) {
                while (rs.next()) {
                    tableSelector.addItem(rs.getString("table_name"));
                }
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα φόρτωσης πινάκων: " + e.getMessage());
        }
    }

    // Κλήση ολων των διαθεσιμων χώρων (απο την SQL)
    private void showVenuesWithScores() {
        try (PreparedStatement ps = conn.prepareStatement(
                "SELECT VenueID, VenueName, Capacity, CompletedConcerts, YearsOperating FROM venue WHERE IsAvailable = 1")) {
            try (ResultSet rs = ps.executeQuery()) {
                DefaultTableModel model = new DefaultTableModel();
                model.addColumn("VenueID");
                model.addColumn("VenueName");
                model.addColumn("Capacity");
                model.addColumn("CompletedConcerts");
                model.addColumn("YearsOperating");
                model.addColumn("VenueScore");

                while (rs.next()) {
                    int id = rs.getInt("VenueID");
                    String name = rs.getString("VenueName");
                    int cap = rs.getInt("Capacity");
                    int comp = rs.getInt("CompletedConcerts");
                    int years = rs.getInt("YearsOperating");

                    Integer score = null;
                    // εδω αξιοποιουμε το stored procedure που ειχαμε δημιουργησει για την απαιτηση
                    // που αφορα το σκορ των χωρων
                    // το καλουμε και το θετουμε ως venue score και το εμφανιζουμε
                    try (CallableStatement cs = conn.prepareCall("{ CALL CalculateVenueScore(?) }")) {
```

```
                        cs.setInt(1, id);
                        boolean has = cs.execute();
                        if (has) {
                            try (ResultSet prs = cs.getResultSet()) {
                                if (prs.next())
                                    score = prs.getInt("VenueScore");
                            }
                        }
                    } catch (SQLException ex) {
                        score = (cap / 1000) * 1 + (comp / 100) * 3 + years * 2;
                    }

                    model.addRow(new Object[] { id, name, cap, comp, years, score });
                }
                // η εμφανιση των παραπανω γινεται σε table γιαυτο και χρησιμοποιουμε το JTable
                JTable vTable = new JTable(model);
                JScrollPane pane = new JScrollPane(vTable);
                vTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

                JPanel panel = new JPanel(new BorderLayout());
                panel.add(pane, BorderLayout.CENTER);

                // Νεα οθονη που εμφανιζει στοιχεια συναυλιων
                JPanel btns = new JPanel();
                JButton detailsBtn = new JButton("Προβολή Στοιχείων & Συναυλιών");
                JButton closeBtn = new JButton("Κλείσιμο");
                btns.add(detailsBtn);
                btns.add(closeBtn);
                panel.add(btns, BorderLayout.SOUTH);

                JDialog dialog = new JDialog(this, "Διαθέσιμοι Χώροι & Βαθμολογίες", true);
                dialog.getContentPane().add(panel);
                dialog.setSize(800, 400);
                dialog.setLocationRelativeTo(this);

                detailsBtn.addActionListener(ev -> {
                    int sel = vTable.getSelectedRow();
                    if (sel == -1) { // σε περιπτωση που δεν επιλεγει ενας χωρος για να δουμε τι συναυλιες υπαρχουν
                                     // (απο το ιστορικο συναυλιων τις αντλουμε)
                        JOptionPane.showMessageDialog(dialog, "Επιλέξτε ένα venue πρώτα.", "Προσοχή",
                                JOptionPane.WARNING_MESSAGE);
                        return;
                    }
                    int venueId = (Integer) vTable.getValueAt(sel, 0);
                    showVenueDetails(venueId);
                });

                closeBtn.addActionListener(ev -> dialog.dispose());

                dialog.setVisible(true);
            } // error handling
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα κατά την ανάκτηση χώρων: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    private void showBandMembers() {
        try {

            String checkTablesQuery = "SELECT TABLE_NAME FROM information_schema.TABLES " +
                    "WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME IN ('band', 'bandmember', 'person')";

            try (Statement stmt = conn.createStatement();
                    ResultSet rs = stmt.executeQuery(checkTablesQuery)) {

                java.util.List<String> existingTables = new java.util.ArrayList<>();
                while (rs.next()) {
                    existingTables.add(rs.getString("TABLE_NAME"));
                }

                if (!existingTables.contains("band") || !existingTables.contains("person")) {
                    JOptionPane.showMessageDialog(this,
                            "Οι πίνακες band ή person δεν υπάρχουν στη βάση δεδομένων.",
                            "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }

            // Query για να κανουμε fetch τα bandmembers
            String membersQuery = "SELECT b.BandName, CONCAT(p.FirstName, ' ', p.LastName) as MemberName, " +
                    "bm.FromDate, bm.ToDate " +
                    "FROM band b " +
                    "INNER JOIN bandmember bm ON b.BandID = bm.BandID " +
                    "INNER JOIN person p ON bm.PersonID = p.PersonID " +
                    "ORDER BY b.BandName, bm.FromDate";

            // εμφανιζουμε τα αποτελεσματα σε table
            DefaultTableModel membersModel = new DefaultTableModel(
                    new String[] { "Συγκρότημα", "Μέλος", "Από", "Έως" }, 0);

            // εισαγουμε τα δεδομενα που αντλησαμε απο τους πινακες
            try (Statement stmt = conn.createStatement();
                    ResultSet rs = stmt.executeQuery(membersQuery)) {

                while (rs.next()) {
                    String bandName = rs.getString("BandName");
                    String memberName = rs.getString("MemberName");
                    java.sql.Date fromDate = rs.getDate("FromDate");
                    java.sql.Date toDate = rs.getDate("ToDate");

                    String fromDateStr = fromDate != null ? fromDate.toString() : "";
                    String toDateStr = toDate != null ? toDate.toString() : "Ενεργό μέλος";
```

```
                membersModel.addRow(new Object[] {
                        bandName,
                        memberName,
                        fromDateStr,
                        toDateStr
                });
            }
        }

        // Δημιουργια JTable για εμφανιση
        JTable membersTable = new JTable(membersModel);

        // σκρολ
        JScrollPane scrollPane = new JScrollPane(membersTable);
        scrollPane.setPreferredSize(new Dimension(800, 400));

        scrollPane.setBorder(BorderFactory.createTitledBorder("Μέλη Συγκροτημάτων"));

        JDialog dialog = new JDialog(this, "Μέλη Συγκροτημάτων", true);
        dialog.getContentPane().add(scrollPane);
        dialog.setSize(900, 500);
        dialog.setLocationRelativeTo(this);
        dialog.setVisible(true);

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this,
                "Σφάλμα κατά την ανάκτηση των μελών: " + e.getMessage(),
                "Σφάλμα",
                JOptionPane.ERROR_MESSAGE);
    }
}

// ΙΔΙΑ ΛΟΓΙΚΗ ΜΕ showBandMembers
private void showArtistCompanyRelations() {
    try {

        String checkTablesQuery = "SELECT TABLE_NAME FROM information_schema.TABLES " +
                "WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME IN ('artist', 'artistcompany', 'recordcompany')";

        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(checkTablesQuery)) {

            java.util.List<String> existingTables = new java.util.ArrayList<>();
            while (rs.next()) {
                existingTables.add(rs.getString("TABLE_NAME"));
            }

            if (!existingTables.contains("artist") || !existingTables.contains("recordcompany")) {
                JOptionPane.showMessageDialog(this,
                        "Οι πίνακες artist ή recordcompany δεν υπάρχουν στη βάση δεδομένων.",
                        "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }
        }

        String relationshipQuery = "SELECT " +
                "CASE " +
                "  WHEN a.ArtistType = 'PERSON' THEN CONCAT(p.FirstName, ' ', p.LastName) " +
                "  WHEN a.ArtistType = 'BAND' THEN b.BandName " +
                "  ELSE a.ArtistType " +
                "END AS ArtistName, " +
                "rc.CompanyName, ac.FromDate, ac.ToDate " +
                "FROM artist a " +
                "INNER JOIN artistcompany ac ON a.ArtistID = ac.ArtistID " +
                "INNER JOIN recordcompany rc ON ac.CompanyID = rc.CompanyID " +
                "LEFT JOIN person p ON a.ArtistID = p.ArtistID AND a.ArtistType = 'PERSON' " +
                "LEFT JOIN band b ON a.ArtistID = b.ArtistID AND a.ArtistType = 'BAND' " +
                "ORDER BY ArtistName, rc.CompanyName";

        DefaultTableModel relationshipModel = new DefaultTableModel(
                new String[] { "Καλλιτέχνης", "Εταιρία", "Από", "Έως" }, 0);

        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(relationshipQuery)) {

            while (rs.next()) {
                String artistName = rs.getString("ArtistName");
                String companyName = rs.getString("CompanyName");
                java.sql.Date fromDate = rs.getDate("FromDate");
                java.sql.Date toDate = rs.getDate("ToDate");

                String fromDateStr = fromDate != null ? fromDate.toString() : "";
                String toDateStr = toDate != null ? toDate.toString() : "Ενεργή συνεργασία";

                relationshipModel.addRow(new Object[] {
                        artistName,
                        companyName,
                        fromDateStr,
                        toDateStr
                });
            }
        }

        JTable relationshipTable = new JTable(relationshipModel);

        JScrollPane scrollPane = new JScrollPane(relationshipTable);
        scrollPane.setPreferredSize(new Dimension(800, 400));

        scrollPane.setBorder(BorderFactory.createTitledBorder("Συνεργασίες Καλλιτεχνών - Δισκογραφικών Εταιριών"));

        JDialog dialog = new JDialog(this, "Συνεργασίες Καλλιτεχνών - Δισκογραφικών Εταιριών", true);
        dialog.getContentPane().add(scrollPane);
        dialog.setSize(900, 500);
        dialog.setLocationRelativeTo(this);
```

```
            dialog.setVisible(true);

        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this,
                    "Σφάλμα κατά την ανάκτηση των συνεργασιών: " + e.getMessage(),
                    "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    // ΙΔΙΑ ΛΟΓΙΚΗ ΜΕ showBandMembers
    private void showAlbumArtistRelations() {
        try {
            String check = "SELECT TABLE_NAME FROM information_schema.TABLES WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME IN
('album','artist','person','band','genre')";
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(check)) {
                java.util.Set<String> existing = new java.util.HashSet<>();
                while (rs.next())
                    existing.add(rs.getString("TABLE_NAME"));
                if (!existing.contains("album") || !existing.contains("artist")) {
                    JOptionPane.showMessageDialog(this, "Οι πίνακες album ή artist δεν υπάρχουν.", "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }

            String q = "SELECT a.AlbumID, a.Title, " +
                    "CASE WHEN ar.ArtistType='PERSON' THEN CONCAT(p.FirstName,' ',p.LastName) WHEN ar.ArtistType='BAND' THEN b.BandName ELSE
ar.ArtistType END AS ArtistName, "
                    +
                    "g.GenreName, rc.CompanyName, CONCAT(pr.FirstName,' ',pr.LastName) AS ProducerName " +
                    "FROM album a " +
                    "INNER JOIN artist ar ON a.ArtistID = ar.ArtistID " +
                    "LEFT JOIN person p ON ar.ArtistID = p.ArtistID AND ar.ArtistType='PERSON' " +
                    "LEFT JOIN band b ON ar.ArtistID = b.ArtistID AND ar.ArtistType='BAND' " +
                    "LEFT JOIN genre g ON a.GenreID = g.GenreID " +
                    "LEFT JOIN recordcompany rc ON a.CompanyID = rc.CompanyID " +
                    "LEFT JOIN producer pr ON a.ProducerID = pr.ProducerID " +
                    "ORDER BY a.Title";

            DefaultTableModel model = new DefaultTableModel(
                    new String[] { "AlbumID", "Title", "Artist", "Genre", "Company", "Producer" }, 0);
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(q)) {
                while (rs.next()) {
                    model.addRow(new Object[] { rs.getInt("AlbumID"), rs.getString("Title"), rs.getString("ArtistName"),
                            rs.getString("GenreName"), rs.getString("CompanyName"), rs.getString("ProducerName") });
                }
            }

            JTable t = new JTable(model);
            JScrollPane sp = new JScrollPane(t);
            sp.setPreferredSize(new Dimension(900, 400));
            sp.setBorder(BorderFactory.createTitledBorder("Συσχετίσεις Album - Καλλιτέχνη"));
            JDialog d = new JDialog(this, "Album ⇔ Καλλιτέχνης", true);
            d.getContentPane().add(sp);
            d.setSize(1000, 500);
            d.setLocationRelativeTo(this);
            d.setVisible(true);

        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα ανάκτησης album-artist: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    // ΙΔΙΑ ΛΟΓΙΚΗ ΜΕ showBandMembers
    private void showAlbumReleases() {
        try {
            String check = "SELECT TABLE_NAME FROM information_schema.TABLES WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME IN
('album','albumrelease')";
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(check)) {
                java.util.Set<String> existing = new java.util.HashSet<>();
                while (rs.next())
                    existing.add(rs.getString("TABLE_NAME"));
                if (!existing.contains("album") || !existing.contains("albumrelease")) {
                    JOptionPane.showMessageDialog(this, "Οι πίνακες album ή albumrelease δεν υπάρχουν.", "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }

            String q = "SELECT ar.ReleaseID, a.Title AS AlbumTitle, ar.ReleaseDate, ar.ReleaseType, ar.ReleaseStatus, ar.Packaging "
                    +
                    "FROM albumrelease ar INNER JOIN album a ON ar.AlbumID = a.AlbumID ORDER BY a.Title, ar.ReleaseDate";

            DefaultTableModel model = new DefaultTableModel(
                    new String[] { "ReleaseID", "Album", "ReleaseDate", "Type", "Status", "Packaging" }, 0);
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(q)) {
                while (rs.next()) {
                    model.addRow(new Object[] { rs.getInt("ReleaseID"), rs.getString("AlbumTitle"),
                            rs.getDate("ReleaseDate"), rs.getString("ReleaseType"), rs.getString("ReleaseStatus"),
                            rs.getString("Packaging") });
                }
            }

            JTable t = new JTable(model);
            JScrollPane sp = new JScrollPane(t);
            sp.setPreferredSize(new Dimension(900, 400));
            sp.setBorder(BorderFactory.createTitledBorder("Album Releases"));
            JDialog d = new JDialog(this, "Album ⇔ Releases", true);
            d.getContentPane().add(sp);
            d.setSize(1000, 500);
            d.setLocationRelativeTo(this);
            d.setVisible(true);
```

```
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα ανάκτησης album-releases: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    // ΙΔΙΑ ΛΟΓΙΚΗ ΜΕ showBandMembers
    private void showAlbumTracks() {
        try {
            String check = "SELECT TABLE_NAME FROM information_schema.TABLES WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME IN
('album','track')";
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(check)) {
                java.util.Set<String> existing = new java.util.HashSet<>();
                while (rs.next())
                    existing.add(rs.getString("TABLE_NAME"));
                if (!existing.contains("album") || !existing.contains("track")) {
                    JOptionPane.showMessageDialog(this, "Οι πίνακες album ή track δεν υπάρχουν.", "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }

            String q = "SELECT t.TrackID, a.Title AS AlbumTitle, t.Title AS TrackTitle, t.TrackLength, t.TrackNo " +
                    "FROM track t INNER JOIN album a ON t.AlbumID = a.AlbumID ORDER BY a.Title, t.TrackNo";

            DefaultTableModel model = new DefaultTableModel(
                    new String[] { "TrackID", "Album", "Track Title", "Length", "TrackNo" }, 0);
            try (Statement s = conn.createStatement(); ResultSet rs = s.executeQuery(q)) {
                while (rs.next()) {
                    model.addRow(new Object[] { rs.getInt("TrackID"), rs.getString("AlbumTitle"),
                            rs.getString("TrackTitle"), rs.getTime("TrackLength"), rs.getInt("TrackNo") });
                }
            }

            JTable t = new JTable(model);
            JScrollPane sp = new JScrollPane(t);
            sp.setPreferredSize(new Dimension(900, 400));
            sp.setBorder(BorderFactory.createTitledBorder("Album Tracks"));
            JDialog d = new JDialog(this, "Album ↔ Tracks", true);
            d.getContentPane().add(sp);
            d.setSize(1000, 500);
            d.setLocationRelativeTo(this);
            d.setVisible(true);

        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα ανάκτησης album-tracks: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    // ΙΔΙΑ ΛΟΓΙΚΗ ΜΕ showBandMembers
    private void showProducerCompanyRelations() {
        try {
            // Check if tables exist
            String checkTablesQuery = "SELECT TABLE_NAME FROM information_schema.TABLES " +
                    "WHERE TABLE_SCHEMA = 'project2024_gui' AND TABLE_NAME IN ('producer', 'producercompany', 'recordcompany')";

            try (Statement stmt = conn.createStatement();
                    ResultSet rs = stmt.executeQuery(checkTablesQuery)) {

                java.util.List<String> existingTables = new java.util.ArrayList<>();
                while (rs.next()) {
                    existingTables.add(rs.getString("TABLE_NAME"));
                }

                if (!existingTables.contains("producer") || !existingTables.contains("recordcompany")) {
                    JOptionPane.showMessageDialog(this,
                            "Οι πίνακες producer ή recordcompany δεν υπάρχουν στη βάση δεδομένων.",
                            "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }
            }

            String collaborationQuery = "SELECT p.ProducerID, CONCAT(p.FirstName, ' ', p.LastName) as ProducerName, " +
                    "rc.CompanyName, pc.ToDate " +
                    "FROM producer p " +
                    "INNER JOIN producercompany pc ON p.ProducerID = pc.ProducerID " +
                    "INNER JOIN recordcompany rc ON pc.CompanyID = rc.CompanyID " +
                    "ORDER BY p.LastName, p.FirstName, rc.CompanyName";

            DefaultTableModel collaborationModel = new DefaultTableModel(
                    new String[] { "Παραγωγός", "Εταιρία", "Έως Ημερομηνία" }, 0);

            try (Statement stmt = conn.createStatement();
                    ResultSet rs = stmt.executeQuery(collaborationQuery)) {

                while (rs.next()) {
                    String producerName = rs.getString("ProducerName");
                    String companyName = rs.getString("CompanyName");
                    java.sql.Date toDate = rs.getDate("ToDate");
                    String toDateStr = toDate != null ? toDate.toString() : "Ενεργή";

                    collaborationModel.addRow(new Object[] {
                            producerName,
                            companyName,
                            toDateStr
                    });
                }
            }

            JTable collaborationTable = new JTable(collaborationModel);

            JScrollPane scrollPane = new JScrollPane(collaborationTable);
            scrollPane.setPreferredSize(new Dimension(800, 400));
```

```
        scrollPane.setBorder(BorderFactory.createTitledBorder("Συνεργασίες Παραγωγών - Δισκογραφικών Εταιριών"));

        JDialog dialog = new JDialog(this, "Συνεργασίες Παραγωγών και Εταιριών", true);
        dialog.getContentPane().add(scrollPane);
        dialog.setSize(900, 500);
        dialog.setLocationRelativeTo(this);
        dialog.setVisible(true);

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this,
                "Σφάλμα κατά την ανάκτηση των συνεργασιών: " + e.getMessage(),
                "Σφάλμα",
                JOptionPane.ERROR_MESSAGE);
    }
}

private void showVenueDetails(int venueId) {
    try (PreparedStatement ps = conn.prepareStatement(
            "SELECT VenueID, VenueName, Capacity, CompletedConcerts, YearsOperating, IsAvailable FROM venue WHERE VenueID = ?")) {
        ps.setInt(1, venueId);
        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) {
                JOptionPane.showMessageDialog(this, "Το venue δεν βρέθηκε.", "Πληροφορία",
                        JOptionPane.INFORMATION_MESSAGE);
                return;
            }

            String venueName = rs.getString("VenueName");
            int capacity = rs.getInt("Capacity");
            int completed = rs.getInt("CompletedConcerts");
            int years = rs.getInt("YearsOperating");

                Integer score = null;
                try (CallableStatement cs = conn.prepareCall("{ CALL CalculateVenueScore(?) }")) {
                    cs.setInt(1, venueId);
                    boolean has = cs.execute();
                    if (has) {
                        try (ResultSet prs = cs.getResultSet()) {
                            if (prs.next())
                                score = prs.getInt("VenueScore");
                        }
                    }
                } catch (SQLException ex) {
                    score = (capacity / 1000) * 1 + (completed / 100) * 3 + years * 2;
                }

                DefaultTableModel concertsModel = new DefaultTableModel();
                concertsModel.addColumn("ConcertDate");
                concertsModel.addColumn("ArtistName");
                concertsModel.addColumn("Status");
                concertsModel.addColumn("TicketsSold");

                try (CallableStatement cs2 = conn.prepareCall("{ CALL GetConcertDatesByVenue(?) }")) {
                    cs2.setString(1, venueName);
                    boolean has = cs2.execute();
                    if (has) {
                        try (ResultSet crs = cs2.getResultSet()) {
                            while (crs.next()) {
                                Date d = crs.getDate("ConcertDate");
                                String artist = crs.getString("ArtistName");
                                String status = crs.getString("ConcertStatus");
                                int sold = crs.getInt("TicketsSold");
                                concertsModel
                                        .addRow(new Object[] { d != null ? d.toString() : "", artist, status, sold });
                            }
                        }
                    }
                } catch (SQLException ex) {
                    try (PreparedStatement q = conn.prepareStatement(
                            "SELECT c.ConcertDate, CASE WHEN a.ArtistType='PERSON' THEN CONCAT(p.FirstName,' ',p.LastName) WHEN
a.ArtistType='BAND' THEN b.BandName END AS ArtistName, c.ConcertStatus, c.TicketsSold "
                                    +
                                    "FROM concert c INNER JOIN venue v ON c.VenueID = v.VenueID INNER JOIN artist a ON c.ArtistID = a.ArtistID "
                                    +
                                    "LEFT JOIN person p ON a.ArtistID = p.ArtistID AND a.ArtistType='PERSON' LEFT JOIN band b ON a.ArtistID =
b.ArtistID AND a.ArtistType='BAND' WHERE v.VenueID = ?")) {
                        q.setInt(1, venueId);
                        try (ResultSet crs = q.executeQuery()) {
                            while (crs.next()) {
                                Date d = crs.getDate("ConcertDate");
                                String artist = crs.getString("ArtistName");
                                String status = crs.getString("ConcertStatus");
                                int sold = crs.getInt("TicketsSold");
                                concertsModel
                                        .addRow(new Object[] { d != null ? d.toString() : "", artist, status, sold });
                            }
                        }
                    }
                }

                JPanel info = new JPanel(new BorderLayout());
                JPanel top = new JPanel(new GridLayout(3, 2));
                top.add(new JLabel("Venue Name:"));
                top.add(new JLabel(venueName));
                top.add(new JLabel("Capacity:"));
                top.add(new JLabel(String.valueOf(capacity)));
                top.add(new JLabel("Venue Score:"));
                top.add(new JLabel(String.valueOf(score)));
                info.add(top, BorderLayout.NORTH);

                JTable concertsTable = new JTable(concertsModel);
                info.add(new JScrollPane(concertsTable), BorderLayout.CENTER);

                JDialog dialog = new JDialog(this, "Λεπτομέρειες Χώρου: " + venueName, true);
```

```
                dialog.getContentPane().add(info);
                dialog.setSize(800, 400);
                dialog.setLocationRelativeTo(this);
                dialog.setVisible(true);
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα ανάκτησης στοιχείων venue: " + e.getMessage(), "Σφάλμα",
                    JOptionPane.ERROR_MESSAGE);
        }
    }

    private void loadTableData(String tableName) {
        try {
            tableModel.setRowCount(0);
            tableModel.setColumnCount(0);

            // Εξάγουμε το πραγματικό όνομα του πίνακα/view από την επιλογή
            String actualTableName = extractTableName(tableName);

            try (Statement stmt = conn.createStatement();
                    ResultSet rs = stmt.executeQuery("SELECT * FROM " + actualTableName)) {
                ResultSetMetaData metaData = rs.getMetaData();
                int columnCount = metaData.getColumnCount();
                for (int i = 1; i <= columnCount; i++)
                    tableModel.addColumn(metaData.getColumnName(i));
                while (rs.next()) {
                    Vector<Object> row = new Vector<>();
                    for (int i = 1; i <= columnCount; i++)
                        row.add(rs.getObject(i));
                    tableModel.addRow(row);
                }
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα φόρτωσης δεδομένων: " + e.getMessage());
        }
    }

    private String extractTableName(String selection) {
        if (selection == null)
            return null;
        if (selection.contains("("))
            return selection.substring(0, selection.indexOf("(")).trim();
        if (selection.startsWith("---"))
            return "artist";
        return selection;
    }

    private boolean isViewSelection(String selection) {
        return selection.contains("_view") || selection.contains("(");
    }

    private String[] getEnumValues(String tableName, String columnName) {
        try {
            ArrayList<String> values = new ArrayList<>();
            try (Statement stmt = conn.createStatement();
                    ResultSet rs = stmt
                            .executeQuery("SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + columnName + "'")) {
                if (rs.next()) {
                    String type = rs.getString("Type").toLowerCase();

                    // Ειδική μεταχείριση για boolean/tinyint(1) πεδία
                    if (type.equals("tinyint(1)")) {
                        values.add("1 - Ναι");
                        values.add("0 - Όχι");
                    }
                    // Κανονική μεταχείριση για enum πεδία
                    else if (type.startsWith("enum(")) {
                        type = type.substring(5, type.length() - 1);
                        String[] splits = type.split(",");
                        for (String s : splits) {
                            String cleanValue = s.trim().replace("'", "");
                            // Μετατροπή boolean τιμών σε Ναι/Όχι
                            if (cleanValue.equalsIgnoreCase("true")) {
                                values.add("1 - Ναι");
                            } else if (cleanValue.equalsIgnoreCase("false")) {
                                values.add("0 - Όχι");
                            } else {
                                values.add(cleanValue);
                            }
                        }
                    }
                }
            }
            return values.toArray(new String[0]);
        } catch (SQLException e) {
            return new String[0];
        }
    }

    private boolean isDateColumn(String tableName, String columnName) {
        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt
                        .executeQuery("SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + columnName + "'")) {
            if (rs.next()) {
                String type = rs.getString("Type").toLowerCase();
                return type.contains("date") || type.contains("timestamp");
            }
        } catch (SQLException e) {
            return false;
        }
        return false;
    }

    private boolean isAutoIncrement(String tableName, String columnName) {
        try (Statement stmt = conn.createStatement();
```

```
            ResultSet rs = stmt
                    .executeQuery("SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + columnName + "'")) {
        if (rs.next()) {
            return "auto_increment".equalsIgnoreCase(rs.getString("Extra"));
        }
    } catch (SQLException e) {
        return false;
    }
    return false;
}

private boolean isNullableColumn(String tableName, String columnName) {
    try (Statement stmt = conn.createStatement();
            ResultSet rs = stmt
                    .executeQuery("SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + columnName + "'")) {
        if (rs.next()) {
            return "YES".equalsIgnoreCase(rs.getString("Null"));
        }
    } catch (SQLException e) {
        return true;
    }
    return true;
}

// Εμφάνιση του πίνακα παραγωγών
private String[] getProducers() {
    ArrayList<String> producers = new ArrayList<>();
    try (Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(
                    "SELECT ProducerID, FirstName, LastName FROM producer ORDER BY LastName, FirstName")) {
        while (rs.next()) {
            producers.add(
                    rs.getInt("ProducerID") + ": " + rs.getString("FirstName") + " " + rs.getString("LastName"));
        }
    } catch (SQLException e) {
        System.err.println("Error loading producers: " + e.getMessage());
    }
    return producers.toArray(new String[0]);
}

// Εμφάνιση του πίνακα δισκογραφικών εταιριών
private String[] getRecordCompanies() {
    ArrayList<String> companies = new ArrayList<>();
    try (Statement stmt = conn.createStatement();
            ResultSet rs = stmt
                    .executeQuery("SELECT CompanyID, CompanyName FROM recordcompany ORDER BY CompanyName")) {
        while (rs.next()) {
            companies.add(rs.getInt("CompanyID") + ": " + rs.getString("CompanyName"));
        }
    } catch (SQLException e) {
        System.err.println("Error loading record companies: " + e.getMessage());
    }
    return companies.toArray(new String[0]);
}

private String extractIdFromSelection(String selection) {
    if (selection == null)
        return null;

    // Για τα boolean πεδία (1 - Ναι, 0 - Όχι)
    if (selection.contains(" - ")) {
        return selection.substring(0, selection.indexOf(" - ")).trim();
    }

    // Για τα foreign key πεδία (ID: Description)
    int colonIdx = selection.indexOf(':');
    if (colonIdx > 0) {
        try {
            return selection.substring(0, colonIdx).trim();
        } catch (Exception e) {
            return null;
        }
    }
    return selection;
}

private boolean isForeignKeyField(String tableName, String columnName) {
    try (ResultSet rs = conn.getMetaData().getImportedKeys(null, null, tableName)) {
        while (rs.next()) {
            if (columnName.equalsIgnoreCase(rs.getString("FKCOLUMN_NAME"))) {
                return true;
            }
        }
    } catch (SQLException e) {
        return false;
    }
    return false;
}

private String[] getForeignKeyOptions(String tableName, String columnName) {
    ArrayList<String> options = new ArrayList<>();
    try {
        ResultSet fks = conn.getMetaData().getImportedKeys(null, null, tableName);
        while (fks.next()) {
            if (columnName.equalsIgnoreCase(fks.getString("FKCOLUMN_NAME"))) {
                String pkTable = fks.getString("PKTABLE_NAME");
                String pkColumn = fks.getString("PKCOLUMN_NAME");

                // Special handling for common reference tables
                if (pkTable.equalsIgnoreCase("producer")) {
                    return getProducers();
                }
                if (pkTable.equalsIgnoreCase("recordcompany")) {
                    return getRecordCompanies();
                }
```

```
                // Default handling: show ID and first VARCHAR column
                try (Statement stmt = conn.createStatement()) {
                    ResultSet cols = stmt.executeQuery("SHOW COLUMNS FROM " + pkTable
                            + " WHERE Type LIKE '%varchar%' OR Type LIKE '%char%'");
                    String displayCol = pkColumn;
                    if (cols.next()) {
                        displayCol = cols.getString("Field");
                    }

                    try (ResultSet rs = stmt.executeQuery("SELECT " + pkColumn + ", " + displayCol
                            + " FROM " + pkTable + " ORDER BY " + displayCol)) {
                        while (rs.next()) {
                            String id = rs.getString(1);
                            String display = rs.getString(2);
                            if (display == null)
                                display = id;
                            options.add(id + ": " + display);
                        }
                    }
                }
                break;
            }
        }
    } catch (SQLException e) {
        System.err.println("Error getting FK options: " + e.getMessage());
    }
    return options.toArray(new String[0]);
}

// επιλογέας ημερομηνίας με βελάκια
private String showDatePicker(String title, String currentValue) {
    // αν θελω να εχω μια τιμη null βαζω ενα tickbox που μπορω να το επιλεξω και να
    // κανει assign NULL
    JPanel panel = new JPanel(new BorderLayout(8, 8));

    JSpinner dateSpinner = new JSpinner(new SpinnerDateModel());
    JSpinner.DateEditor dateEditor = new JSpinner.DateEditor(dateSpinner, "yyyy-MM-dd"); // μορφή ημερομηνίας
    dateSpinner.setEditor(dateEditor);

    JCheckBox nullCheck = new JCheckBox("Κενό (NULL)");

    // αν ειναι κενο ή εχει επιλεχθει NULL , τοτε θετω NULL στην ημερομηνια
    if (currentValue == null || currentValue.isEmpty()) {
        nullCheck.setSelected(true);
        dateSpinner.setEnabled(false);
    } else {
        try {
            java.util.Date date = new java.text.SimpleDateFormat("yyyy-MM-dd").parse(currentValue);
            dateSpinner.setValue(date);
        } catch (Exception ignored) {
        }
    }

    nullCheck.addActionListener(ev -> dateSpinner.setEnabled(!nullCheck.isSelected()));

    JPanel center = new JPanel(new BorderLayout());
    center.add(dateSpinner, BorderLayout.CENTER);
    center.add(nullCheck, BorderLayout.SOUTH);

    panel.add(new JLabel(title + ":"), BorderLayout.NORTH);
    panel.add(center, BorderLayout.CENTER);

    int result = JOptionPane.showConfirmDialog(null, panel, title,
            JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);

    if (result == JOptionPane.OK_OPTION) {
        if (nullCheck.isSelected()) {

            return "";
        }
        java.util.Date date = (java.util.Date) dateSpinner.getValue();
            return new java.text.SimpleDateFormat("yyyy-MM-dd").format(date);
        }

        return null;
    }

    // εισαγωγη δεδομενων σε πινακα
    private void insertRow() {
        String tableName = (String) tableSelector.getSelectedItem();
        if (tableName == null)
            return;
        if (isViewSelection(tableName)) {
            JOptionPane.showMessageDialog(this, "Δεν μπορείτε να προσθέσετε δεδομένα σε view.", "Μη έγκυρη επιλογή",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        String actualTableName = extractTableName(tableName);

        ArrayList<String> reqFields = new ArrayList<>();
        ArrayList<String> dateFields = new ArrayList<>();
        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery("SHOW COLUMNS FROM " + actualTableName)) {
            while (rs.next()) {
                String field = rs.getString("Field");
                String type = rs.getString("Type");
                String nullable = rs.getString("Null");
                if (nullable.equals("NO")) {
                    reqFields.add(field);
                }
                if (type.toLowerCase().contains("date")) {
                    dateFields.add(field);
                }
```

```
            }
        } catch (SQLException e) {

            System.err.println("Warning: Could not get column metadata: " + e.getMessage());
        }

        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery("SELECT * FROM " + actualTableName + " LIMIT 1")) {

            ResultSetMetaData meta = rs.getMetaData();
            int colCount = meta.getColumnCount();

            ArrayList<String> columnNames = new ArrayList<>();
            ArrayList<String> values = new ArrayList<>();

            for (int i = 1; i <= colCount; i++) {
                String col = meta.getColumnName(i);

                // Προσπέραση AUTO_INCREMENT πεδίων
                if (isAutoIncrement(actualTableName, col)) {
                    continue;
                }

                columnNames.add(col);
                String val = null;

                if (isForeignKeyField(actualTableName, col)) {
                    String[] options = getForeignKeyOptions(actualTableName, col);
                    if (options != null && options.length > 0) {
                        String selected = (String) JOptionPane.showInputDialog(this,
                                "Επιλέξτε τιμή για " + col + ":",
                                "Επιλογή " + col,
                                JOptionPane.QUESTION_MESSAGE,
                                null,
                                options,
                                options[0]);

                        // Αν πατήσει Cancel , στοπ την διαδικασία
                        if (selected == null) {
                            return;
                        }
                        val = extractIdFromSelection(selected);
                    }
                }
                // Επιλογή ημερομηνίας
                else if (isDateColumn(actualTableName, col)) {
                    val = showDatePicker("Επιλέξτε ημερομηνία για " + col, null);
                    // Αν πατήσει Cancel , στοπ την διαδικασία
                    if (val == null && !isNullableColumn(actualTableName, col)) {
                        return;
                    }
                }
                // Επιλογή Enum με dropdown menu
                else {
                    String[] enumVals = getEnumValues(actualTableName, col);
                    if (enumVals != null && enumVals.length > 0) {
                        val = (String) JOptionPane.showInputDialog(this,
                                "Επιλέξτε τιμή για " + col + ":",
                                "Επιλογή " + col,
                                JOptionPane.QUESTION_MESSAGE,
                                null,
                                enumVals,
                                enumVals[0]);
                    }

                    else {
                        // Κανονικό text input
                        val = JOptionPane.showInputDialog(this, "Τιμή για " + col + ":");
                        // Αν πατήσει Cancel , στοπ την διαδικασία
                        if (val == null && !isNullableColumn(actualTableName, col)) {
                            return;
                        }
                    }

                    // Καθάρισμα κενών τιμών
                    if (val != null && val.trim().isEmpty()) {
                        val = null;
                    }

                    // Έλεγχος για υποχρεωτικά πεδία μετά το καθάρισμα
                    if (val == null && !isNullableColumn(actualTableName, col)) {
                        JOptionPane.showMessageDialog(this, "Το πεδίο " + col + " είναι υποχρεωτικό!", "Σφάλμα",
                                JOptionPane.ERROR_MESSAGE);
                        return;
                    }

                    values.add(val);
                }

                if (columnNames.isEmpty()) {
                    JOptionPane.showMessageDialog(this, "Δεν υπάρχουν πεδία για εισαγωγή!", "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }

                String columnsList = String.join(",", columnNames);
                String placeholders = String.join(",", Collections.nCopies(values.size(), "?"));
                String sql = "INSERT INTO " + actualTableName + " (" + columnsList + ") VALUES (" + placeholders + ")";

                try (PreparedStatement ps = conn.prepareStatement(sql)) {
                    for (int i = 0; i < values.size(); i++) {
                        String v = values.get(i);
                        String columnName = columnNames.get(i);
```

```
                    // Get column metadata to handle types properly
                    String columnType = "";
                    try (Statement s = conn.createStatement();
                            ResultSet r = s.executeQuery(
                                    "SHOW COLUMNS FROM " + actualTableName + " WHERE Field = '" + columnName + "'")) {
                        if (r.next()) {
                            columnType = r.getString("Type").toLowerCase();
                        }
                    }

                    if (v == null || v.trim().isEmpty()) {
                        ps.setNull(i + 1, Types.NULL);
                        continue;
                    }

                    // Διαχείριση boolean τιμών (το TINYINT(1) χρησιμοποιείται για boolean στο
                    // MySQL)
                    if (columnType.equals("tinyint(1)")) {

                        // Για 1 -> true , για 0 -> false
                        if (v.contains(" - ")) {
                            v = v.substring(0, v.indexOf(" - ")).trim();
                        }
                        ps.setInt(i + 1, v.equals("1") || v.equalsIgnoreCase("true") ? 1 : 0);
                    } else if (columnType.contains("int")) {
                        // Extract ID from "ID: Description" format if needed
                        if (v.contains(":")) {
                            v = v.substring(0, v.indexOf(":")).trim();
                        }
                        ps.setInt(i + 1, Integer.parseInt(v));
                    }

                    else if (columnType.contains("date")) {
                        ps.setDate(i + 1, java.sql.Date.valueOf(v));
                    }

                    else if (columnType.contains("timestamp")) {
                        ps.setTimestamp(i + 1, java.sql.Timestamp.valueOf(v));
                    }
                    // Διαχείριση δεκαδικών/αριθμητικών τύπων
                    else if (columnType.contains("decimal") || columnType.contains("numeric")) {
                        ps.setBigDecimal(i + 1, new java.math.BigDecimal(v));
                    }
                    // Διαχείριση τύπων float/double
                    else if (columnType.contains("float") || columnType.contains("double")) {
                        ps.setDouble(i + 1, Double.parseDouble(v));
                    }
                    // Default τιμή για όλους τους άλλους τύπους
                    else {
                        ps.setString(i + 1, v);
                    }
                }
                ps.executeUpdate();
                logDBAAction("INSERT", actualTableName, "Inserted values");
                loadTableData(tableName);
                JOptionPane.showMessageDialog(this, "Εισαγωγή επιτυχής!");
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα εισαγωγής: " + e.getMessage());
        }
    }

    // Ενημέρωση εγγραφής
    private void updateRow() {
        String tableName = (String) tableSelector.getSelectedItem();
        if (tableName == null)
            return;

        int selectedRow = table.getSelectedRow();
        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Παρακαλώ επιλέξτε μια γραμμή για ενημέρωση.", "Προσοχή",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery("SELECT * FROM " + tableName + " LIMIT 1")) {

            ResultSetMetaData meta = rs.getMetaData();
            int colCount = meta.getColumnCount();

            // Παίρνουμε πρωτευον κλειδι
            DatabaseMetaData dbMeta = conn.getMetaData();
            ResultSet pkeys = dbMeta.getPrimaryKeys(null, null, tableName);
            if (!pkeys.next()) {
                JOptionPane.showMessageDialog(this, "Ο πίνακας δεν έχει primary key!", "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }
            String pkColumn = pkeys.getString("COLUMN_NAME");
            String pkValue = table.getValueAt(selectedRow, findColumnIndex(pkColumn)).toString();

            ArrayList<String> columnNames = new ArrayList<>();
            ArrayList<String> values = new ArrayList<>();

            for (int i = 1; i <= colCount; i++) {
                String col = meta.getColumnName(i);
                if (col.equals(pkColumn))
                    continue;

                Object currentValue = table.getValueAt(selectedRow, i - 1);
                columnNames.add(col);
                String val = null;

                // Διαχείριση foreign κλειδιών
```

```
                if (isForeignKeyField(tableName, col)) {
                    String[] options = getForeignKeyOptions(tableName, col);
                    if (options != null && options.length > 0) {
                        String selected = (String) JOptionPane.showInputDialog(this,
                                "Ενημέρωση " + col + ":",
                                "Επεξεργασία " + col,
                                JOptionPane.QUESTION_MESSAGE,
                                null,
                                options,
                                findCurrentOption(options, currentValue != null ? currentValue.toString() : null));

                        if (selected != null) {
                            val = extractIdFromSelection(selected);
                        }
                    }
                }
                // Διαχείριση ημερομηνιών
                else if (isDateColumn(tableName, col)) {
                    val = showDatePicker("Ενημέρωση ημερομηνίας για " + col,
                            currentValue != null ? currentValue.toString() : null);
                }
                // Διαχείριση enums με dropdown
                else {
                    String[] enumVals = getEnumValues(tableName, col);
                    if (enumVals != null && enumVals.length > 0) {
                        val = (String) JOptionPane.showInputDialog(this,
                                "Ενημέρωση " + col + ":",
                                "Επεξεργασία " + col,
                                JOptionPane.QUESTION_MESSAGE,
                                null,
                                enumVals,
                                currentValue);
                    }
                    else {
                        val = JOptionPane.showInputDialog(this,
                                "Ενημέρωση " + col + ":",
                                currentValue != null ? currentValue.toString() : "");
                    }
                }

                if (val == null && !isNullableColumn(tableName, col)) {
                    JOptionPane.showMessageDialog(this, "Το πεδίο " + col + " είναι υποχρεωτικό!", "Σφάλμα",
                            JOptionPane.ERROR_MESSAGE);
                    return;
                }

                values.add(val);
            }

            if (columnNames.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Δεν υπάρχουν πεδία για ενημέρωση!", "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }

            StringBuilder sql = new StringBuilder("UPDATE ").append(tableName).append(" SET ");
            for (int i = 0; i < columnNames.size(); i++) {
                if (i > 0)
                    sql.append(", ");
                sql.append(columnNames.get(i)).append(" = ?");
            }
            sql.append(" WHERE ").append(pkColumn).append(" = ?");

            try (PreparedStatement ps = conn.prepareStatement(sql.toString())) {
                for (int i = 0; i < values.size(); i++) {
                    String v = values.get(i);
                    String columnName = columnNames.get(i);

                    String columnType = "";
                    try (Statement s = conn.createStatement();
                            ResultSet r = s.executeQuery(
                                    "SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + columnName + "'")) {
                        if (r.next()) {
                            columnType = r.getString("Type").toLowerCase();
                        }
                    }

                    if (v == null || v.trim().isEmpty()) {
                        ps.setNull(i + 1, Types.NULL);
                        continue;
                    }

                    // διαχειριση boolean
                    if (columnType.equals("tinyint(1)")) {
                        // 1 -> true, 0 -> false
                        if (v.contains(" - ")) {
                            v = v.substring(0, v.indexOf(" - ")).trim();
                        }
                        ps.setInt(i + 1, v.equals("1") || v.equalsIgnoreCase("true") ? 1 : 0);
                    }
                    // διαχειριση integer
                    else if (columnType.contains("int")) {

                        if (v.contains(":")) {
                            v = v.substring(0, v.indexOf(":")).trim();
                        }
                        ps.setInt(i + 1, Integer.parseInt(v));
                    }
                    // διαχείριση ημερομηνίας
                    else if (columnType.contains("date")) {
                        ps.setDate(i + 1, java.sql.Date.valueOf(v));
                    }
                    // διαχείριση timestamp
                    else if (columnType.contains("timestamp")) {
```

```
                        ps.setTimestamp(i + 1, java.sql.Timestamp.valueOf(v));
                    }
                    // διαχείριση decimal/numeric
                    else if (columnType.contains("decimal") || columnType.contains("numeric")) {
                        ps.setBigDecimal(i + 1, new java.math.BigDecimal(v));
                    }
                    // διαχείριση floating point
                    else if (columnType.contains("float") || columnType.contains("double")) {
                        ps.setDouble(i + 1, Double.parseDouble(v));
                    }
                    // Default για άλλους τύπους
                    else {
                        ps.setString(i + 1, v);
                    }
                }

                // διαχείριση του πρωτεύοντος κλειδιού
                String pkColumnType = "";
                try (Statement s = conn.createStatement();
                        ResultSet r = s
                                .executeQuery("SHOW COLUMNS FROM " + tableName + " WHERE Field = '" + pkColumn + "'")) {
                    if (r.next()) {
                        pkColumnType = r.getString("Type").toLowerCase();
                    }
                }

                if (pkColumnType.contains("int")) {
                    ps.setInt(values.size() + 1, Integer.parseInt(pkValue));
                } else {
                    ps.setString(values.size() + 1, pkValue);
                }

                int affected = ps.executeUpdate();
                if (affected > 0) {
                    logDBAAction("UPDATE", tableName, "Updated record with " + pkColumn + "=" + pkValue);
                    loadTableData(tableName);
                    JOptionPane.showMessageDialog(this, "Η ενημέρωση ολοκληρώθηκε επιτυχώς!");
                } else {
                    JOptionPane.showMessageDialog(this, "Δεν έγινε ενημέρωση. Η εγγραφή μπορεί να μην υπάρχει πλέον.",
                            "Προσοχή", JOptionPane.WARNING_MESSAGE);
                }
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Σφάλμα ενημέρωσης: " + e.getMessage());
        }
    }

    private int findColumnIndex(String columnName) {
        for (int i = 0; i < table.getColumnCount(); i++) {
            if (table.getColumnName(i).equals(columnName)) {
                return i;
            }
        }
        return -1;
    }

    private String findCurrentOption(String[] options, String currentValue) {
        if (currentValue == null)
            return options[0];

        // Για boolean πεδία (1/0)
        if (currentValue.equals("1") || currentValue.equals("0")) {
            for (String option : options) {
                if (option.startsWith(currentValue + " - ")) {
                    return option;
                }
            }
        }

        // Για foreign keys (id: description)
        for (String option : options) {
            if (option.startsWith(currentValue + ":") || option.equals(currentValue)) {
                return option;
            }
        }
        return options[0];
    }

    // διαγραφη εγγραφης απο πινακα
    private void deleteRow() {
        String tableName = (String) tableSelector.getSelectedItem();
        if (tableName == null)
            return;

        int selectedRow = table.getSelectedRow();
        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Παρακαλώ επιλέξτε μια γραμμή για διαγραφή.", "Προσοχή",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        try {
            // παιρνουμε πρωτευον κλειδι πινακα
            DatabaseMetaData dbMeta = conn.getMetaData();
            ResultSet pkeys = dbMeta.getPrimaryKeys(null, null, tableName);
            if (!pkeys.next()) {
                JOptionPane.showMessageDialog(this, "Ο πίνακας δεν έχει primary key!", "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }
            String pkColumn = pkeys.getString("COLUMN_NAME");

            // παιρνουμε πρωτευον κλειδι απο την γραμμη που μας ενδιαφερει
            int pkColIndex = findColumnIndex(pkColumn);
            if (pkColIndex == -1) {
```

```
                JOptionPane.showMessageDialog(this, "Δεν βρέθηκε η στήλη " + pkColumn, "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }

            Object pkValue = table.getValueAt(selectedRow, pkColIndex);
            if (pkValue == null) {
                JOptionPane.showMessageDialog(this, "Η επιλεγμένη γραμμή δεν έχει τιμή primary key!", "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
                return;
            }

            // επιβεβαίωση διαγραφής
            int confirm = JOptionPane.showConfirmDialog(this,
                    "Είστε σίγουροι ότι θέλετε να διαγράψετε την επιλεγμένη εγγραφή;\n" +
                            pkColumn + " = " + pkValue,
                    "Επιβεβαίωση Διαγραφής",
                    JOptionPane.YES_NO_OPTION,
                    JOptionPane.WARNING_MESSAGE);

            if (confirm != JOptionPane.YES_OPTION)
                return;

            // Διαγραφή από τον πίνακα με χρήση sql
            String sql = "DELETE FROM " + tableName + " WHERE " + pkColumn + " = ?";
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setString(1, pkValue.toString());

                int affected = ps.executeUpdate();
                if (affected > 0) {
                    logDBAAction("DELETE", tableName, "Deleted record with " + pkColumn + "=" + pkValue);
                    loadTableData(tableName);
                    JOptionPane.showMessageDialog(this, "Η διαγραφή ολοκληρώθηκε επιτυχώς!");
                } else {
                    JOptionPane.showMessageDialog(this, "Δεν έγινε διαγραφή. Η εγγραφή μπορεί να μην υπάρχει πλέον.",
                            "Προσοχή", JOptionPane.WARNING_MESSAGE);
                }
            }
        } catch (SQLException e) {
            if (e.getMessage().toLowerCase().contains("foreign key")
                    || e.getMessage().toLowerCase().contains("reference")) {
                JOptionPane.showMessageDialog(this,
                        "Δεν μπορεί να γίνει διαγραφή γιατί η εγγραφή χρησιμοποιείται από άλλους πίνακες!",
                        "Σφάλμα Foreign Key",
                        JOptionPane.ERROR_MESSAGE);
            } else {
                JOptionPane.showMessageDialog(this, "Σφάλμα διαγραφής: " + e.getMessage(), "Σφάλμα",
                        JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new DatabaseUI().setVisible(true));
    }
}
```